

Sıfırdan İleri Seviyeye WEB PROGRAMLAMA

Temel Kavramlar, Sunucu ve İstemci Mimarileri



Elektronik Yük. Müh.

İlhan ÖZKAN

WEB PROGRAMLAMA

Yazar: Elektronik Yük. Müh. İlhan ÖZKAN

© Eylül 2026, İlhan ÖZKAN. Tüm hakları saklıdır.

Bu kitap kâr amacı güdülmenden hazırlanmıştır ve **referans gösterilerek** her yerde kullanılabilir.

Bu kitabın bir kısmı veya tamamı, **yazarın önceden yazılı izni alınarak** elektronik, mekanik, fotokopi veya herhangi bir kayıt sistemi ile çoğaltılabilir, yayımlanabilir veya depolanabilir.

ISBN: ***_***_**_****_*

Baskı Tarihi: Eylül 2026

Baskı Yeri: Ankara, Türkiye

Önsöz

Bu kitabın güncel sürümüne <https://github.com/HoydaBre/WebProgramlama> üzerinden erişilebilir. Anlaşılır, sade ve yapısal programlama disiplinine uygun bir üslup hedeflenmiştir. Katkı ve geri bildirimleriniz kitabınızın gelecekteki sürümleri için son derece kıymetlidir.

Burada verilen kavramlar web programlama sırasında en çok karşılaşacağınız kavramlardır. Bu kavramların nasıl ortaya çıktığı ve nasıl uygulandığı üzerinde durulacaktır.

Başarılar ve iyi çalışmalar dilerim!

Elektronik Yüksek Mühendisi, İlhan ÖZKAN
ilhanozkan[at]outlook.com
[linkedin.com/in/ilhanozkan](https://www.linkedin.com/in/ilhanozkan)

İçindekiler

Önsöz	i
1 İnternet ve Web	1
1.1 İnternet Nedir?	1
1.1.1 İstemci-Sunucu Modeli: En Basit Hâliyle	2
1.1.2 Tarihsel Gelişimi ve Evrimi	2
1.2 TCP/IP Protokol Kümesi ve Çalışma Mimarisi	3
1.2.1 TCP/IP Katmanlı Mimari Yapısı ve Veri Kapsülleme	3
1.2.2 TCP Üçlü El Sıkışması	4
1.3 Alan Adı Sistemi ve Alan Adı Çözümleme	4
1.3.1 DNS Ağacının Hiyerarşik Yapısı	4
1.3.2 DNS Adres Çözümleme Adımları	5
1.4 Web Nedir?	5
1.4.1 Nasıl Ortaya Çıktı?	5
1.4.2 Web Sunucusu Nedir ve Nasıl Davranır?	6
1.5 HTML Nedir?	6
1.6 Tekbiçim Kaynak Bulucu ve Sunucuların Konumlandırılması	7
1.7 Tekbiçim Kaynak Kimliği	7
1.8 Web Tarayıcısı Ne Yapar?	8
1.8.1 DOM Ağacı	8
1.8.2 Java Script	8
1.8.3 En Çok Kullanılan Web Tarayıcıları	9
1.9 HTTP Nedir?	9
1.9.1 HTTP Protokolünün Kronolojik Evrimi	9
1.9.2 Web Tarayıcı ve Sunucu İletişimi	9
1.9.3 HTTP Yöntemleri	11
1.9.4 HTTP Durum Kodları	13
1.10 Bölüm Özeti	13
Bölüm Özeti	13
1.11 Düşün ve Çöz	13
2 HTML	14
2.1 HTML Nedir?	14
2.2 Etiketler	14
2.3 Etiketlerde Hiyerarşi	15
2.4 Etiket Nitelikleri	15
2.5 Diğer Metin Biçimlendirme Etiketleri	16
2.6 Listeler	16
2.6.1 Sırasız Liste	16
2.6.2 Sıralı Liste	17
2.6.3 Tanım Listesi	17
2.7 Görseller	17
2.8 Tablolar	18
2.9 Blok ve Satır İçi Elemanlar	19
2.10 HTML5 Semantik Etiketleri	19
2.11 Formlar	19
2.12 Yorum Satırları	20
2.13 Kendi Kendini Kapanan Etiketler	20
2.13.1 Bölüm Özeti	21
2.14 Düşün ve Çöz	21
2.15 Uygulamalı Projeler	21
2.15.1 Blog Ana Sayfa Örneği	21
3 CSS	23

3.1	CSS Nedir ve Neden Kullanılır?	23
3.2	HTML Belgelerine CSS Uygulama Yöntemleri	23
3.2.1	Satır İçi Stil Kullanımı	23
3.2.2	Dahili Stil Kullanımı	23
3.2.3	Harici Stil Kullanımı	24
3.3	CSS Sözdizimi	24
3.3.1	Yorum Satırları	25
3.4	CSS Seçicileri	25
3.4.1	Öge Seçicisi	25
3.4.2	Sınıf Seçicisi	25
3.4.3	Kimlik Seçicisi	25
3.4.4	Evrensel Seçici	25
3.4.5	Gruplama Seçicisi	25
3.4.6	Torun ve Çocuk Seçicileri	26
3.4.7	Nitelik Seçicisi	26
3.4.8	Sözde Sınıflar ve Sözde Öğeler	26
3.5	CSS'te Kademeli Yapı ve Kalıtım	26
3.5.1	Kalıtım Mantığı	26
3.5.2	Seçici Özgüllüğü ve Öncelik Sırası	27
3.6	Renkler ve Ölçü Birimleri	27
3.6.1	Renk Belirtme Yöntemleri	27
3.6.2	Ölçü Birimleri	27
3.7	Kutu Modeli	28
3.7.1	Kutu Boyutlandırma Hesabı	28
3.7.2	Temel Konumlandırma Yöntemleri	28
3.7.3	Uygulamalı Düzen Senaryoları	29
3.7.4	Bir kutuyu yatayda ve dikeyde ortalama	29
	Bölüm Özeti	31
3.8	Düşün ve Çöz	31
3.9	Uygulamalı Projeler	31
3.9.1	Blog Ana Sayfa CSS Örneği	32
4	Program ve Programlama	35
4.1	Programlama Dillerinin Tarihsel Gelişimi	35
4.1.1	Düşük Düzey Programlama	35
4.1.2	Yüksek Düzey Diller	36
4.1.3	Arapsaçı Kod	36
4.2	Temel Programlama Kavramları: Değişken ve Fonksiyon	37
4.2.1	Değişken Nedir?	37
4.2.2	Fonksiyon Nedir?	38
4.3	Programlama Paradigmaları	39
4.3.1	Yapısal Programlama	39
4.3.2	Nesne Yönelimli Programlama	40
4.4	Kod Nasıl Çalıştırılır? Derleme ve Yorumlama	41
4.4.1	Derleyici ve Yorumlayıcı	41
4.4.2	Derleyici	41
4.4.3	Yorumlayıcı	42
4.4.4	Script Nedir?	42
	Bölüm Özeti	42
4.5	Düşün ve Çöz	43
5	JavaScript	44
5.1	JavaScript Tarihçesi	44
5.2	JavaScript Kullanım Alanları	44
5.3	JavaScript Çalışma Mantığı	44
5.4	JavaScript'e Giriş	45
5.4.1	Sözdizimi (Syntax) Kuralları	45
5.4.2	Değişken Tanımlama	45
5.4.3	Değişken Kimliklendirme (Identifiers)	45
5.4.4	Değişmezler (Literals)	45
5.4.5	İşleçler (Operators)	45

5.4.6	İfadeler (Expressions)	46
5.4.7	Değer Atanamaz Değişken Kullanımı ve Referans Sabitliği	46
5.4.8	İlkel Veri Tipleri	46
5.5	JavaScript Kontrol Akışını Değiştiren Talimatlar	48
5.5.1	Duruma Göre Seçim Talimatları	48
5.5.2	İlişkisel Döngüler	48
5.5.3	Fonksiyonlar	48
5.6	JavaScript Nesneler	48
5.6.1	Nesne Nedir?	48
5.6.2	Nesne Özellikleri	48
5.6.3	Nesne Yöntemleri	49
5.6.4	this Kelimesi	49
5.6.5	Nesnenin Gösterilmesi	49
5.6.6	Nesne Yapıcıları (Constructors)	49
5.6.7	Nesne Parçalama (Object Destructuring)	49
5.7	JavaScript Kapsam	49
5.7.1	Değişkenlerin Kapsamı	49
5.7.2	Kod Blokları	50
5.7.3	Yükseltme Nedir?	50
5.7.4	Katı Veri Tipi Kontrolü	50
5.8	JavaScript Dahili Nesneleri	50
5.8.1	Sayılar (Numbers)	50
5.8.2	Metinler (Strings)	50
5.8.3	Zamanlayıcılar (Timers)	50
5.8.4	Tarihler (Dates)	50
5.8.5	Diziler (Arrays)	50
5.8.6	Kümeler (Sets)	50
5.8.7	Haritalar (Maps)	50
5.8.8	Yineleyiciler (Iterators)	50
5.8.9	Matematik (Math)	51
5.8.10	Düzenli İfadeler (RegExp)	51
5.9	JavaScript Hata Yönetimi	51
5.9.1	Hatalar ve Yakalama (try...catch)	51
5.9.2	Özel Hata Fırlatma (throw)	51
	Bölüm Özeti	51
5.10	Düşün ve Çöz	51
6	Tarayıcı Tarafında Programlama	53
6.1	Dinamik Web Sayfası Nedir?	53
6.2	JavaScript ve HTML DOM	53
6.2.1	HTML DOM API	53
6.2.2	HTML Öğelerini Seçme	53
6.2.3	HTML Değiştirme	53
6.2.4	CSS Değiştirme	54
6.2.5	Form Verilerini Doğrulama	54
6.2.6	DOM Animasyonu	54
6.2.7	Document Properties and Methods	54
6.2.8	Element Properties and Methods	54
6.3	XML	55
6.4	AJAX	55
6.4.1	AJAX Nedir?	55
6.4.2	AJAX XMLHttpRequest	55
6.4.3	AJAX Request	55
6.4.4	AJAX Response	56
6.4.5	AJAX ile XML Dosyası Uygulaması	56
6.5	JQUERY	56
6.5.1	jQuery Selectors	56
6.5.2	jQuery HTML	56
6.5.3	jQuery CSS	56
6.5.4	jQuery DOM	57
6.6	Tek Sayfa Uygulaması	57

6.6.1	Geleneksel Çok Sayfalı Uygulama ile SPA Karşılaştırması	57
6.6.2	SPA Nasıl Çalışır?	57
6.6.3	SPA Kütüphaneleri ve Çatıları	59
6.7	Uygulamalı Projeler	59
6.8	Yapılacaklar Listesi Örneği	59
8	Web Sunucusu Tarafında Programlama	94
8.1	Common Gateway Interface	94
8.2	Sunucu Tarafı Betik	94
8.3	Apache Web Sunucusu	95
8.3.1	Apache2 Paketinin Kurulması	95
8.3.2	Servis Durumunun ve Portların Kontrolü	95
8.3.3	Apache Dizin Yapısı ve Önemli Dosyalar	96
8.3.4	Apache Modülleri ve PHP Entegrasyonu	96
8.4	PHP	97
8.4.1	Linux PHP Kurulumu	97
8.4.2	PHP Hello Uygulaması	97
8.4.3	PHP Değişkenleri ve Veri Tipleri	97
8.4.4	PHP Dizileri	100
8.4.5	PHP Kontrol Akışı ve Karar Yapıları	100
8.4.6	PHP Fonksiyonları	103
8.4.7	PHP Nesne Yönelimli Programlama (OOP) ve Nesneler	105
8.4.8	PHP Dahili Nesneleri	108
8.4.9	PHP Modülleri ve Eklentileri	110
8.4.10	PHP ile Çıktı HTML Dokümanını Değiştirme	112
8.4.11	HTML Form Verilerini HTTP GET Yöntemi ile Tarayıcıdan Alma	112
8.4.12	HTML Form Verilerini HTTP POST Yöntemi ile Tarayıcıdan Alma	113
8.5	MySQL	113
8.5.1	Linux MySQL Kurulumu	113
8.5.2	Veritabanı Ana Sayfa	114
8.5.3	Veritabanı Oluşturan PHP	114
8.5.4	Veritabanında Tablo Oluşturan PHP	115
8.5.5	Veritabanında Tabloyu Veri İle Dolduran PHP	116
8.5.6	Veritabanından Veri Okuyan PHP	117
8.5.7	PHP ile Veritabanında CRUD İşlemleri	118
8.5.8	PHP ile Form Verilerini Veritabanına Kaydetme	119
8.5.9	Hazırlıklı İfadeler	119
8.5.10	Veritabanının Güvenli Olan ve Olmayan Kullanımı	119
8.6	Oturum ve Çerez Yönetimi	120
8.6.1	Çerezler (Cookies)	120
8.6.2	Oturumlar	120
8.6.3	PHP ve AJAX Örnekleri	122
	Bölüm Özeti	123
8.7	Düşün ve Çöz	123
8.8	Uygulamalı Projeler	123
8.8.1	Ziyaretçi Defteri Örneği	124
8	Web Sunucusu Tarafında Programlama	94
8.1	Common Gateway Interface	94
8.2	Sunucu Tarafı Betik	94
8.3	Apache Web Sunucusu	95
8.3.1	Apache2 Paketinin Kurulması	95
8.3.2	Servis Durumunun ve Portların Kontrolü	95
8.3.3	Apache Dizin Yapısı ve Önemli Dosyalar	96
8.3.4	Apache Modülleri ve PHP Entegrasyonu	96
8.4	PHP	97
8.4.1	Linux PHP Kurulumu	97
8.4.2	PHP Hello Uygulaması	97
8.4.3	PHP Değişkenleri ve Veri Tipleri	97
8.4.4	PHP Dizileri	100
8.4.5	PHP Kontrol Akışı ve Karar Yapıları	100

8.4.6	PHP Fonksiyonları	103
8.4.7	PHP Nesne Yönelimli Programlama (OOP) ve Nesneler	105
8.4.8	PHP Dahili Nesneleri	108
8.4.9	PHP Modülleri ve Eklentileri	110
8.4.10	PHP ile Çıktı HTML Dokümanını Değiştirme	112
8.4.11	HTML Form Verilerini HTTP GET Yöntemi ile Tarayıcıdan Alma	112
8.4.12	HTML Form Verilerini HTTP POST Yöntemi ile Tarayıcıdan Alma	113
8.5	MySQL	113
8.5.1	Linux MySQL Kurulumu	113
8.5.2	Veritabanı Ana Sayfa	114
8.5.3	Veritabanı Oluşturan PHP	114
8.5.4	Veritabanında Tablo Oluşturan PHP	115
8.5.5	Veritabanında Tabloyu Veri İle Dolduran PHP	116
8.5.6	Veritabanından Veri Okuyan PHP	117
8.5.7	PHP ile Veritabanında CRUD İşlemleri	118
8.5.8	PHP ile Form Verilerini Veritabanına Kaydetme	119
8.5.9	Hazırlıklı İfadeler	119
8.5.10	Veritabanının Güvenli Olan ve Olmayan Kullanımı	119
8.6	Oturum ve Çerez Yönetimi	120
8.6.1	Çerezler (Cookies)	120
8.6.2	Oturumlar	120
8.6.3	PHP ve AJAX Örnekleri	122
	Bölüm Özeti	123
8.7	Düşün ve Çöz	123
8.8	Uygulamalı Projeler	123
8.8.1	Ziyaretçi Defteri Örneği	124
9	RESTful Mimari ve Web Servis Tasarımı	126
9.0.1	RESTful Mimariye Giriş	126
9.0.2	RESTful Mimarinin Yapı Taşları	126
9.0.3	PHP (Laravel) ile RESTful API Geliştirme Örneği	129
9.0.4	ASP.NET Core ile RESTful API Geliştirme Örneği	129
9.0.5	Özet ve Karşılaştırma	130
10	Modern Web Mimarisi	131
10.0.1	Ön Yüz ve Arka Yüz Kavramlarına Giriş	131
10.0.2	Ön Yüz Teknolojileri ve Mimarisi	131
10.0.3	Arka Yüz Teknolojileri ve Mimarisi	132
10.0.4	Ön Yüz ve Arka Yüz Entegrasyonu	132
10.0.5	Bölüm Özeti ve Değerlendirme Soruları	133
11	Node.js (Express.js) - React ile Öğrenci Bilgi Sistemi	134
11.1	Proje Mimarisi	134
11.2	Linux Üzerinde Geliştirme Ortamının Kurulumu	134
11.2.1	Node.js Kurulumu	134
11.2.2	MySQL Kurulumu	134
11.3	Veritabanı Hazırlığı	135
11.4	Back-End: Node.js ve Express.js ile REST API Geliştirme	135
11.4.1	Proje Kurulumu ve Bağımlılıklar	135
11.4.2	Veritabanı Bağlantısı (db.js)	135
11.4.3	RESTful API (server.js)	135
11.4.4	API'yi Çalıştırma ve Test Etme	136
11.5	Front-End: React ile Arayüz Geliştirme	136
11.5.1	Proje Kurulumu	136
11.5.2	Öğrenci Listesi ve Güncelleme Bileşeni (src/App.jsx)	136
11.5.3	Uygulamayı Çalıştırma	138
11.6	Uygulamayı Bütün Olarak Çalıştırma	138
	Bölüm Özeti	138
11.7	Düşün ve Çöz	138
11.8	Uygulamalı Projeler	139

12 C# (ASP.NET Core) - Vue.js ile Öğrenci Bilgi Sistemi	140
12.1 Proje Mimarisi	140
12.2 Linux Üzerinde Geliştirme Ortamının Kurulumu	140
12.2.1 .NET SDK Kurulumu	140
12.2.2 Node.js Kurulumu (Vue Araçları İçin)	140
12.2.3 MySQL Kurulumu	140
12.3 Veritabanı Hazırlığı	141
12.4 Back-End: ASP.NET Core ile REST API Geliştirme	141
12.4.1 Proje Kurulumu ve Bağımlılıklar	141
12.4.2 Öğrenci Modeli (Models/Ogrenci.cs)	141
12.4.3 Veritabanı Servisi (Services/OgrenciServisi.cs)	141
12.4.4 RESTful Denetleyici (Controllers/OgrencilerController.cs)	142
12.4.5 CORS Yapılandırması (Program.cs)	143
12.4.6 API'yi Çalıştırma	143
12.5 Front-End: Vue.js ile Arayüz Geliştirme	143
12.5.1 Proje Kurulumu	143
12.5.2 Öğrenci Listesi ve Güncelleme Bileşeni (src/App.vue)	143
12.5.3 Uygulamayı Çalıştırma	144
12.6 Uygulamayı Bütün Olarak Çalıştırma	145
Bölüm Özeti	145
12.7 Düşün ve Çöz	145
12.8 Uygulamalı Projeler	145
13 Python (FastAPI) - Angular ile Öğrenci Bilgi Sistemi	146
13.1 Proje Mimarisi	146
13.2 Linux Üzerinde Geliştirme Ortamının Kurulumu	146
13.2.1 Python ve Sanal Ortam Kurulumu	146
13.2.2 Node.js ve Angular CLI Kurulumu	146
13.2.3 MySQL Kurulumu	146
13.3 Veritabanı Hazırlığı	147
13.4 Back-End: Python ve FastAPI ile REST API Geliştirme	147
13.4.1 Proje Kurulumu ve Bağımlılıklar	147
13.4.2 Veri Modeli ve API (main.py)	147
13.4.3 API'yi Çalıştırma ve Test Etme	148
13.5 Front-End: Angular ile Arayüz Geliştirme	149
13.5.1 Proje Kurulumu	149
13.5.2 API Servisi (src/app/ogrenci.service.ts)	149
13.5.3 Ana Bileşen (src/app/app.component.ts)	149
13.5.4 Şablon (src/app/app.component.html)	150
13.5.5 HttpClient'ı Etkinleştirme (src/app/app.config.ts)	150
13.5.6 Uygulamayı Çalıştırma	151
13.6 Uygulamayı Bütün Olarak Çalıştırma	151
Bölüm Özeti	151
13.7 Düşün ve Çöz	151
13.8 Uygulamalı Projeler	151

1. Bölüm

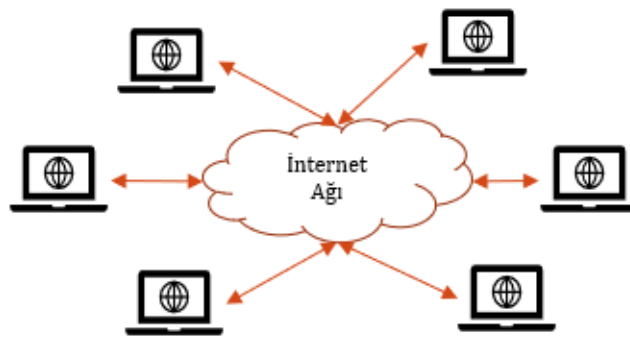
İnternet ve Web

Bir web programcısı olarak yazacağınız her satır kod, aslında görünmez ama katı kurallarla işleyen dev bir sistemin üzerine inşa edilir. Bir tarayıcıya adres yazıp Enter'a bastığımız anda; adres bir yerlerde bir isimden bir sayıya dönüşür, iki bilgisayar önce "tanışır", sonra veri alışverişine başlar ve nihayetinde ekranınızda bir sayfa belirir. Bu bölümde, ileride yazacağınız her web uygulamasının temelini oluşturan bu görünmez katmanları – İnternet'in ne olduğundan HTTP protokolüne kadar – adım adım, en basit fikirden başlayıp giderek teknik ayrıntıya inerek açacağız. Amacımız sadece tanımları ezberlemek değil, bir tarayıcı isteği gönderdiğinde perde arkasında gerçekte ne olup bittiğini kavramaktır; çünkü ileriki bölümlerde karşılaşacağınız pek çok kavram (oturum yönetimi, API tasarımı, performans optimizasyonu) doğrudan bu temellere dayanacaktır. Bu bölümü hiç ön bilgisi olmayan biri için yazdık; teknik bir terimle karşılaştığınızda önce günlük hayattan bir benzetme, ardından da terimin resmî tanımı verilecektir.

1.1 İnternet Nedir?

En basit hâliyle başlayalım. Elinizdeki bilgisayar, tek başına yalnızca kendi diskinde ne varsa onu gösterebilen bir kutudur. İki bilgisayarı bir kabloyla birbirine bağlarsanız, aralarında dosya alışverişi yapabilen küçük bir **ağ** (*network*) kurmuş olursunuz. Şimdi bu fikri milyonlarca kez büyütün: dünyanın dört bir yanındaki evler, okullar, şirketler ve devlet kurumları, kendi küçük ağlarını – tıpkı sizin iki bilgisayarınız gibi – birbirine bağlamış durumda. İşte bu ağların ağına **İnternet** diyoruz.

Daha resmî bir tanımla **internet**, dünya genelinde standart iletişim protokolleri aracılığıyla birbiriyle haberleşen sayısız bilgisayar ağının bir araya gelmesiyle oluşan bir **ağların ağı** (*network of networks*) sistemidir. Ancak İnternet'i yalnızca teknik bir altyapı olarak görmek eksik bir tanım olur; o, aynı zamanda bilginin sınır tanımadan serbestçe dolaşmasını sağlayan küresel ve dağıtık bir dijital otobandır.



Şekil 1.1: İnternet

Bu tanımı biraz daha somutlaştıralım. İnternet'i şu şekilde özetleyebiliriz:

- ◇ Dünya genelindeki bilgisayar ağlarını ve kurumsal bilgisayar sistemlerini birbirine bağlayan küresel bir elektronik iletişim ağıdır.
- ◇ Çok protokollü bir ağıdır; birbirinden farklı mimarideki bilgisayar ağları, ortak kurallar çerçevesinde bir araya gelerek bu yapıyı oluşturur.
- ◇ Binlerce akademik, askeri, ticari ve devlet ağının heterojen bir bütün hâlinde birleşmesinden meydana gelir.
- ◇ Bilgisayarlar arası veri alışverişi, belirli standart kurallara (protokollere) göre küçük **paketler** (*packets*) hâlinde gerçekleşir.
- ◇ Üzerinde e-posta, dosya aktarımı, uzaktan erişim ve birbirine köprülerle bağlı web sayfaları gibi pek çok farklı servis ve hizmet barındırır.

§1.1.1 İstemci-Sunucu Modeli: En Basit Hâliyle

İnternet üzerindeki neredeyse her etkileşim, aslında tek bir basit fikrin binlerce kez tekrarlanmasıdır: birisi **ister**, birisi de o isteği **karşılar**. Bu kitap boyunca sürekli karşınıza çıkacağı için bu fikri en baştan, teknik hiçbir terim kullanmadan netleştirelim.

Restoran Analojisi

Bir restorana gittiğinizi düşünün. Siz (**müşteri**) garsona ne yemek istediğinizi söylersiniz; garson bu isteği mutfağa (**sunucu**) iletir; mutfak yemeği hazırlar ve garson aracılığıyla size (yani isteği yapan tarafa) geri gönderir. Siz mutfak nasıl çalıştığını bilmek zorunda değilsiniz; sadece isteğinizi doğru şekilde iletmeniz ve cevabı almanız yeterlidir.

Bilgisayar dünyasında da işleyiş aynıdır:

- ◊ **İstemci (Client):** İsteği başlatan taraftır. Web dünyasında bu, sizin bilgisayarınızdaki tarayıcıdır (*browser*). En bilinen tarayıcılar Chrome, Firefox ve Safari tarayıcılarıdır.
- ◊ **Sunucu (Server):** İstemcilerden gelen isteği karşılayan, gerekli veriyi hazırlayıp geri gönderen taraftır. Genellikle uzak bir veri merkezinde 7/24 çalışan güçlü bir bilgisayardır. Web dünyasında bu, web sunucusudur (*web server*).

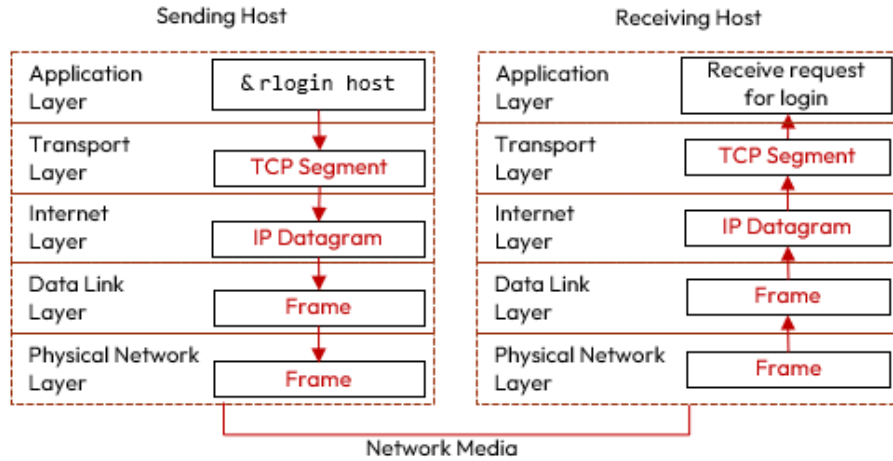
Bu bölümde göreceğiniz İnternet, TCP/IP, DNS, Web ve HTTP kavramlarının hepsi, aslında bu basit istemci-sunucu ilişkisinin farklı katmanlarda nasıl gerçekleştiğini anlatır. Şimdi bu ilişkinin üzerinde çalıştığı tarihsel altyapıya bakalım.

§1.1.2 Tarihsel Gelişimi ve Evrimi

İnternet bir gecede ortaya çıkmadı; askeri bir ihtiyacın çözümü olarak doğdu ve zamanla tüm insanlığın ortak altyapısına dönüştü. Kökeni, 1960'ların sonlarında Amerika Birleşik Devletleri Savunma Bakanlığı bünyesindeki **ARPA** (*Advanced Research Projects Agency*) tarafından başlatılan **ARPANET** projesine dayanır. Projenin temel amacı, olası bir nükleer saldırı veya fiziksel iletişim kesintisi durumunda dahi ayakta kalabilen, merkezi olmayan (dağıtık) bir iletişim altyapısı kurmaktır. Bu hikâyenin dönüm noktalarını kronolojik olarak izleyelim:

- ◊ **1965 – İlk Uzak Mesafe Bağlantısı:** MIT (*Massachusetts Institute of Technology*) bünyesindeki *Lawrence Roberts*, çevirmeli telefon hatları üzerinden Kaliforniya'daki bir bilgisayar ile Massachusetts'teki bir bilgisayarı konuşturarak ilk veri aktarımını gerçekleştirdi.
- ◊ **1969 – İlk ARPANET Mesajı:** Los Angeles Kaliforniya Üniversitesi ile Stanford Üniversitesi arasında ilk başarılı veri iletimi sağlandı. Gönderilmeye çalışılan "LOGIN" kelimesinin yalnızca ilk iki harfi ("LO") iletebilmiş ve sistem çökmüştür; kusurlu da olsa bu küçük deneme, İnternet tarihinin başlangıcı kabul edilir.
- ◊ **1970'ler – TCP/IP'nin Doğuşu:** *Vinton Cerf* ve *Robert Kahn*, birbirinden farklı ağ yapılarını tek bir ortak dil altında birleştiren **TCP/IP** (*Transmission Control Protocol/Internet Protocol*) protokol kümesini geliştirdi.
- ◊ **1 Ocak 1983 – Standartlaşma ve Doğuş:** ARPANET, bünyesindeki tüm sistemlerde TCP/IP protokolüne resmi olarak geçiş yaptı. Bu tarih, modern İnternet'in doğum günü olarak kabul edilir.
- ◊ **1990'lar – Halka Açılma ve Yaygınlaşma:** İnternet'in ticari kullanıma açılması ve World Wide Web'in (WWW) icat edilmesiyle İnternet, akademisyenlerin ve askeri birimlerin kullandığı dar bir ağdan, herkese açık kitlesel bir platforma dönüştü.
- ◊ **12 Nisan 1993 – Türkiye'nin İnternet'e Bağlanması:** Türkiye, Orta Doğu Teknik Üniversitesi (ODTÜ) ve TÜBİTAK iş birliğiyle yürütülen çalışmalar sonucunda, ODTÜ Bilgi İşlem Daire Başkanlığı sistemlerinden Washington D.C.'deki **NSFNET** (*National Science Foundation Network*) hattına bağlanan 64 Kbps kapasiteli kiralık bir hat ile İnternet'e ilk resmî bağlantısını gerçekleştirdi.

Bu kronolojiden çıkarılacak en önemli ders şudur: İnternet ile Web aynı şey değildir. İnternet, 1969'dan beri var olan fiziksel/mantıksal altyapıdır; Web ise bu altyapının üzerinde 1990'larda doğan, bizim en çok temas ettiğimiz servistir. Bu ayrımı netleştirmek için aşağıdaki analojiye bakalım.



Şekil 1.2: TCP/IP Katmanları

Pedagojik Not: İnternet ve Web Analojisi

Otoyol ve Araç Analojisi: İnternet'i bir ülkedeki **otoyol ağına** benzetebiliriz. Otoyollar, köprüler, tüneller ve kavşaklar fiziksel altyapıyı (kablolar, yönlendiriciler, sunucular) oluşturur. **Web (World Wide Web)** ise bu otoyol üzerinde hareket eden otobüs, kamyon veya binek araç türlerinden yalnızca **bir tanesidir**. E-posta (SMTP), dosya transferi (FTP) veya anlık mesajlaşma protokolleri ise aynı otoyolu kullanan diğer araç türleridir.

1.2 TCP/IP Protokol Kümesi ve Çalışma Mimarisi

Otoyol analojisinde gördüğümüz gibi, aynı yolu farklı "araç türleri" paylaşır; ama bu araçların birbirine çarpmadan, doğru adrese ve eksiksiz şekilde ulaşabilmesi için ortak bir trafik kuralları bütününe ihtiyaç vardır. Bölüm 1.1.1'de gördüğümüz basit istemci-sunucu ilişkisinin, dünyanın öbür ucundaki bambaşka bir donanım ve işletim sistemine sahip bir bilgisayarla dahi sorunsuz çalışabilmesini sağlayan da işte bu ortak kurallar bütünüdür. İnternet'in mimari omurgasını oluşturan **TCP/IP** (*Transmission Control Protocol/Internet Protocol*), farklı donanım, işletim sistemi ve ağ altyapılarına sahip bilgisayarların haberleşmesini sağlayan standart protokoller topluluğudur.

TCP/IP, veriyi tek bir devasa blok hâlinde göndermek yerine yönetilebilir küçük parçalara (**paketlere**) bölerek iletir. Her paket ağ üzerinde bağımsız bir yol izleyebilir; bu sayede hatlardan biri arızalansa dahi veri, alternatif rotalar üzerinden hedefe ulaşabilir.

Ş1.2.1 TCP/IP Katmanlı Mimari Yapısı ve Veri Kapsülleme

Bu esnekliği sağlamak için TCP/IP, karmaşık ağ iletişimini yönetilebilir kılan 4 katmanlı bir yapı kullanır (Şekil 1.2). Her katmanın kendine özgü bir görevi vardır ve bir katman, yalnızca kendisinin bir alt veya bir üst katmanıyla doğrudan konuşur. Bu "katmanlara bölme" fikri ilk bakışta karmaşık görünse de, aslında hepimizin bildiği bir mantığı izler: aşağıda vereceğimiz mektup örneğinde olduğu gibi, büyük bir işi birbirinden bağımsız çalışan küçük görevlere ayırmak.

Üst katmandaki bir veri aşağı doğru indikçe her katmanda kendi meta verisini (*header*) alarak zarflanır. Bu işleme **Veri Kapsülleme** (*encapsulation*) adı verilir. Hedef cihazda ise süreç tam tersine işler: veri yukarı doğru çıkarken her katman kendi başlığını sırayla çıkarır (*decapsulation*). Şimdi bu dört katmanı, en üstten en alta doğru inceleyelim.

- 1. Uygulama Katmanı (Application Layer):** Kullanıcının doğrudan etkileşimde bulunduğu yazılımlar ve protokoller bu katmanda yer alır. HTTP (Web), HTTPS, FTP (Dosya Aktarımı), SMTP (E-posta) ve DNS (Alan Adı Çözümleme) gibi servisler, veriyi bu katmanda üretir.
- 2. Taşıma Katmanı (Transport Layer):** Verinin uçtan uca güvenli veya hızlı bir şekilde iletilmesinden ve süreçlerin (process) port numaraları ile ayrıştırılmasından sorumludur. Bu katmanda, kullanım amacına göre birbirinden oldukça farklı iki protokol devreye girer:
 - ◊ **TCP (Transmission Control Protocol):** Bağlantı odaklıdır (*connection-oriented*); veriyi gönderen ile alan taraf arasında önceden kurulmuş bir bağlantı söz konusudur. Verinin eksiksiz, hatasız ve doğru sırada ulaştığını garanti eder (Örn: Web sayfaları, dosya indirme). Veri birimi **Segment**'tir.
 - ◊ **UDP (User Datagram Protocol):** Bağlantısızdır (*connectionless*); veri iletim hızını ön planda tu-

tar ve paket kaybı kontrolü yapmaz (Örn: Canlı video yayını, çevrim içi oyunlar). Veri birimi **Datagram**'dir.

3. **İnternet / Ağ Katmanı (Internet Layer)**: Paketlerin kaynak cihazdan hedef cihaza doğru yönlendirilmesinden (*routing*) ve mantıksal adreslemeden sorumludur.
 - ◊ **IP (Internet Protocol)**: Cihazlara mantıksal adresler (IPv4 için örnek adres: **192.0.2.1** / IPv6 için örnek adres: **2001:db8::8a2e:370:7334**) atayarak paketlerin ağ üzerindeki hedefini belirler. Veri birimi **Paket (Packet)**'tir.
 - ◊ **ICMP (Internet Control Message Protocol)**: Ağdaki cihazların veri iletim hatalarını raporlamak ve durum bilgisi paylaşmak için kullandığı bir ağ katmanı protokolüdür (Örn: **ping** komutu).
4. **Ağ Erişimi / Fiziksel Katman (Network Access Layer)**: Verinin fiziksel ortamda (bakır kablo, fiber optik, Wi-Fi dalgaları) sinyallere ve çerçevelere (*frames*) dönüştürülerek iletilmesini sağlar. MAC adresleri bu katmanda çalışır (Örn: Ethernet, Wi-Fi 802.11).

Bu dört katmanı somutlaştırmanın en kolay yolu, hepimizin aşına olduğu bir mektup gönderme sürecini hayal etmektir:

Mektup ve Zarf Analojisi

- ◊ **Uygulama Katmanı**: Mektubun içerisindeki yazıdır (HTML / HTTP içeriği).
- ◊ **Taşıma Katmanı (TCP)**: Mektubun sayfalarını numaralandıran ve alıcıya ulaşip ulaşmadığını teyit eden iadeli taahhütlü posta sistemidir.
- ◊ **İnternet Katmanı (IP)**: Zarfın üzerine yazılan Gönderen ve Alıcı adresleridir (IP Adresleri).
- ◊ **Ağ Erişimi Katmanı**: Mektubu taşıyan posta aracının veya uçağın katettiği fiziksel yoldur (Ethernet / Sinyal).

§1.2.2 TCP Üçlü El Sıkışması

Taşıma katmanında TCP'nin verdiği güvenilirlik garantisinin bir bedeli vardır: iki taraf, veri göndermeye başlamadan önce birbirini tanımalıdır. Bunu, telefonla arama yapmaya benzetebiliriz: karşı taraf açmadan ("Alo?" demeden) konuşmaya başlamazsınız; siz de karşı tarafın sizi duyduğundan emin olmadan asıl mesajınızı iletmezsiniz. Bu nedenle, uygulama katmanından gelen veriler (örneğin bir HTTP isteği) aktarılmadan önce istemci ile sunucu arasında mantıksal bir bağlantı kurulmalıdır. Bu süreç, 3 adımlı bir el sıkışma (*three-way handshake*) ile gerçekleşir:

1. **SYN (Synchronize)**: İstemci, sunucuya rastgele bir sıra numarası ($Seq = X$) içeren ve bağlantı kurmak istediğini belirten bir **SYN** paketi gönderir. ("Alo, beni duyuyor musun?")
2. **SYN-ACK (Synchronize-Acknowledge)**: Sunucu isteği onaylar ($Ack = X + 1$) ve kendi sıra numarasını ($Seq = Y$) ekleyerek istemciye bir **SYN-ACK** paketi ile yanıt verir. ("Evet, duyuyorum; sen beni duyuyor musun?")
3. **ACK (Acknowledge)**: İstemci, sunucunun yanıtını onaylar ($Ack = Y + 1$). Bağlantı artık kurulmuştur (**ESTABLISHED**) ve veri (*payload*) güvenle iletilir. ("Evet, seni de duyuyorum; konuşmaya başlayabiliriz.")

Peki, istemci bu el sıkışmayı hangi adrese göndereceğini nasıl bilir? İşte tam bu noktada devreye, IP adreslerini insan diline çeviren bir sistem girer: DNS.

1.3 Alan Adı Sistemi ve Alan Adı Çözümleme

TCP/IP mimarisinde (Bölüm 1.2) cihazlar birbirlerini yalnızca IP adresleri üzerinden bulabilir. Ancak **172.217.16.142** gibi sayısal bir diziyi ezberlemek insan hafızası için pratik değildir; biz isimleri hatırlamakta çok daha başarılıyızdır. Tıpkı telefonunuzdaki rehberde arkadaşınızın on bir haneli numarasını ezberlemek yerine ismine dokunmanız gibi. İşte bu insani ihtiyacı karşılamak için metinsel alan adı sistemi **DNS (Domain Name System)** geliştirilmiştir.

DNS, İnternet üzerindeki metinsel alan adlarını (*domain name*) (Örneğin **ornek.com**) bilgisayarların haberleşebildiği sayısal IP adreslerine (**192.0.2.1**) dönüştüren dağıtık ve hiyerarşik bir veri tabanı sistemidir. Kısacası DNS, iki dünya arasındaki iletişimi sağlayan İnternet'in telefon rehberi görevini üstlenir.

§1.3.1 DNS Ağacının Hiyerarşik Yapısı

DNS, tek bir merkezi sunucudan oluşmaz; aksine esneklik, performans ve güvenlik amacıyla ağaç yapısında hiyerarşik bir mimariyle yönetilir. Bu hiyerarşiyi en tepeden başlayarak inceleyelim:

1. **Kök Sunucular (Root DNS Servers)**: Hiyerarşinin en tepesinde yer alırlar (adres zincirindeki son görünmez noktalar: **ornek.com**). Dünya genelinde küresel olarak dağıtılmış 13 temel kök sunucu otoritesi (A'dan M'ye) bulunur ve bunlar, Üst Seviye Alan Adı (TLD) sunucularının adreslerini gös-

terirler.

2. **Üst Seviye Alan Adı Sunucuları (Top-Level Domain - TLD Servers):** Alan adı uzantılarını yönetirler.
 - ◊ **gTLD (Genel TLD):** .com, .org, .net, .edu
 - ◊ **ccTLD (Ülke Kodlu TLD):** .tr (Türkiye), .uk (İngiltere), .de (Almanya)
3. **Yetkili Ad Sunucuları (Authoritative Name Servers):** Bir alan adına ait gerçek DNS kayıtlarını (A, CNAME, MX vb.) barındıran ve nihai cevabı veren sunuculardır.
4. **Özyinelemeli Çözücü (Recursive Resolver / DNS Recursor):** İSS (İnternet Servis Sağlayıcı) veya kamuya açık (Örn: 8.8.8.8, 1.1.1.1) DNS sunucularıdır. Kullanıcının adına tüm DNS hiyerarşisini dolaşarak IP adresini bulur ve istemciye getirir.

Bu dört bileşenin gerçek bir sorguda nasıl bir araya geldiğini, adım adım takip edelim.

§1.3.2 DNS Adres Çözümleme Adımları

Bir web tarayıcısına <https://www.ornek.com> yazıldığında arka planda gerçekleşen adım adım DNS çözümleme (*Step-by-Step Resolution*) süreci şu şekildedir:

1. **Yerel Önbellek Denetimi:** Tarayıcı ve işletim sistemi, öncelikle kendi yerel önbelleğinde (*DNS Cache*) veya *hosts* dosyasında bu adrese ait bir IP kaydı olup olmadığını kontrol eder.
2. **Özyinelemeli Sorgu Başlatma:** Önbellekte kayıt yoksa istemci, tanımlı özyinelemeli çözücü (*Recursive Resolver*) sunucusuna bir sorgu gönderir.
3. **Kök Sunucuya Başvuru:** Resolver, Kök DNS sunucusuna giderek .com uzantısından kimin sorumlu olduğunu sorar.
4. **TLD Sunucusuna Başvuru:** Kök sunucunun yönlendirmesiyle Resolver, .com TLD sunucusuna gider ve *ornek.com*'un Yetkili DNS sunucusunu sorar.
5. **Yetkili Sunucudan IP Alma:** Resolver, Yetkili DNS sunucusuna ulaşarak *www.ornek.com*'un IP adresini talep eder. Sunucu, IP adresini (192.0.2.1) döndürür.
6. **Yanıt ve Önbellekleme:** Resolver, elde ettiği IP adresini istemciye sunar ve belirlenen **TTL (Time to Live)** süresince kendi önbelleğinde saklar. Artık TCP bağlantısı (Bölüm 1.2'de gördüğümüz üçlü el sıkışma) başlatılabilir.

Temel DNS Kayıt Türleri (DNS Record Types)

Bir DNS sunucusu üzerinde en sık kullanılan kayıt tipleri şunlardır:

- ◊ **A Kaydı (Address Record):** Domain adını bir IPv4 adresine yönlendirir.
- ◊ **AAAA Kaydı:** Domain adını bir IPv6 adresine yönlendirir.
- ◊ **CNAME (Canonical Name):** Bir domain adını başka bir domain adına bağlar (takma ad / alias).
- ◊ **MX (Mail Exchange):** Domain adına gelen e-postaların hangi e-posta sunucusuna teslim edileceğini belirtir.
- ◊ **TXT Kaydı:** Alan adı doğrulama (SPF, DKIM vb.) ve güvenlik amacıyla düz metin verileri barındırır.
- ◊ **NS (Name Server):** Alan adının yetkili DNS sunucularını tanımlar.

1.4 Web Nedir?

Artık İnternet'in nasıl bir altyapı sunduğunu ve isimlerin nasıl adrese çevrildiğini biliyoruz. Sıra, bu altyapının üzerinde çalışan ve bizim en sık kullandığımız servise geldi: Web. Çoğu zaman İnternet (Bölüm 1.1) ile Web kavramları birbirinin yerine kullanılsa da ikisi farklı katmanları temsil eder. **İnternet**, bilgisayarları ve ağları birbirine bağlayan temel altyapıdır; **www (world wide web)** ise bu altyapı üzerinde çalışan ve metin, görsel, ses, video gibi zengin içeriklerin birbirine **hiper-bağlantılar (hyperlink)** vasıtasıyla bağlandığı bir bilgi paylaşım servisi.

§1.4.1 Nasıl Ortaya Çıktı?

1989 yılında Avrupa Nükleer Araştırma Merkezi CERN'de çalışan İngiliz bilgisayar bilimci **Sir Tim Berners-Lee**, farklı ülkelerdeki bilim insanlarının araştırmalarını ve dokümanlarını kolayca paylaşabilmesi amacıyla "*Universal Linked Information System*" adıyla bilinen bir proje önerdi.

Berners-Lee; **HTML** (metin biçimlendirme dili), **HTTP** (iletişim protokolü) ve **URL** (adresleme sistemi) standartlarını geliştirerek 1991 yılında **World Wide Web**'i hayata geçirdi. Bu üç bileşen, ileride bu kitap boyunca sık sık karşımıza çıkacak: bu bölümün geri kalanında URL ve HTTP'yi, sonraki bölümlerde ise HTML'i ayrıntılı olarak ele alacağız. CERN'in 1993 yılında Web teknolojisini telifsiz ve herkesin kullanımına açık (*public domain*) hâle getirmesiyle birlikte Web, küresel bir bilgi devrimine dönüştü.

§1.4.2 Web Sunucusu Nedir ve Nasıl Davranır?

Bölüm 1.1.1’de tanıttığımız restoran analogisini hatırlayın: Web’in çalışabilmesi için bir tarafta içeriği sunan (mutfak), diğer tarafta ise bu içeriği talep eden (müşteri) bir bileşen bulunmalıdır. İlk olarak sunan tarafı, yani web sunucusunu inceleyelim. Web sunucusu (*web server*), hem donanımsal hem de yazılımsal bileşenleri olan bir sistemdir:

- ◊ **Donanımsal olarak:** Web sitesine ait dosyaları (HTML, CSS, JavaScript kodları, görseller, veri tabanları vb.) barındıran ve İnternet’e kesintisiz (7/24) yüksek bant genişliği ile bağlı olan fiziksel veya sanal bilgisayardır.
- ◊ **Yazılımsal olarak:** İstemcilerden (*client*) gelen HTTP isteklerini analiz eden, ilgili dosyaları veya dinamik çıktıları üreterek istemciye yanıt olarak gönderen programdır (Örn: Nginx, Apache HTTP Server, Microsoft IIS, Caddy).

Web Sunucusunun Davranış Döngüsü:

1. **Dinleme (*Listening*):** Sunucu, belirli bir ağ kapısı (*port*) üzerinden (şifresiz HTTP için varsayılan 80, şifreli HTTPS için 443) gelecek istekleri sürekli dinler.
2. **Kabul ve İşleme (*Parsing*):** İstemciden gelen HTTP paketini kabul eder, sözdizimini kontrol eder ve istenen kaynağı belirler.
3. **Yanıt Oluşturma (*Responding*):** İstenen kaynak mevcutsa uygun bir durum kodu (Örn: 200 OK) ile içeriği gövdeye ekler; aksi hâlde hata kodu (Örn: 404 Not Found) döner.
4. **Bağlantıyı Yönetme (*Connection Handling*):** İşlem tamamlandıktan sonra ağ kaynağını tasarruflu kullanmak adına bağlantıyı sonlandırır ya da takip eden istekler için açık tutar (*Keep-Alive*).

Peki istemci, sunucudan hangi kaynağı istediğini nasıl belirtir? Bunun cevabı, her web adresinin temelini oluşturan URL yapısında saklıdır; ama önce sunucunun bize gönderdiği dosyanın *içeriğine* bir göz atalım.

1.5 HTML Nedir?

Hiper Metin İşaretleme Dili **HTML** (*HyperText Markup Language*), web sayfalarının yapısını ve içeriğini oluşturmak için kullanılan standart işaretleme dilidir. Bir programlama dili değil, tarayıcılara metin, görsel, video ve bağlantıların nasıl görüntüleneceğini bildiren bir **etiket** (*tag*) sistemidir.

Web sunucudan istenen metin dosyaları, sunucudan sunucuya farklı standartlarda kaydedilmiş olabilir. Örneğin; Türkçe bir metin Türkçe Windows bilgisayarda ASCII kodlamasıyla olarak kaydedildiğinde, yerel karakterler (üğışçöğüşçö), Windows-1254 karakter kodlama standardına göre kaydedilir. Ancak bu dosya İngilizce Windows altında açtığımızda yerel karakterleri okuyamayız. Bir web tarayıcısında görüntülenmek üzere biçimlendirilmiş/anlamlandırılmış bu metinler için HTML standardı kullanılır.

Aşağıda basit bir HTML dosyasının içeriği verilmiştir;

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Sample page</title>
  </head>
  <body>
    <h1>Sample page</h1>
    <p>This is a <a href="demo.html">simple</a>sample.</p>
    <!-- this is a comment -->
  </body>
</html>
```

Görüldüğü üzere HTML belgeleri, öge (element) ve **metin ağacından** oluşur. Her bir öge `<body>` gibi bir **başlangıç etiketi** ile başlayıp `</body>` gibi bir **bitiş etiketiyle** biter. Ögenin görevine bağlı olarak aradaki metin değerlendirilir. Etiketler (tag), üst üste binmeden, tamamen birbirinin içinde olacak şekilde iç içe yerleştirilmelidir. Başlangıç ve bitiş etiketleri içine bir başka etiket yazılamaz.

Web sunucusunda bulunan bu anlamlandırılmış metin dosyaları `.html` veya `.htm` uzantısına sahip olurlar (örneğin `index.html`). Web tarayıcıları, HTML belgelerini bir web sunucusundan alır ve bu belgeleri görsel web sayfalarına dönüştürerek kullanıcıya gösterir.

- ◊ 1991 yılında "HTML Etiketleri" adlı belge ile HTML dilinin halka açıklaması ilk olarak Tim Berners-Lee tarafından yapıldı.
- ◊ 1995 yılında HTML 2 dil standardı yayımlandı.
- ◊ 1997 yılında HTML 3 ve 4 dil standardı World Wide Web Konsorsiyumu (kısaca W3C) tarafından yayımlandı
- ◊ 2014 yılında HTML 5 dil standardı yayımlandı

- ◇ HTML dilinin HTML 5.2 sürümü 2017 yılında yayımlanmıştır.
- ◇ 2019'dan beri HTML, W3C yerine WHATWG'nin "HTML Living Standard"ı olarak sürümsüz şekilde güncelleniyor; W3C de 2021'de bu standardı benimsedi.

İlerleyen bölümlerde, bir sunucunun bir HTML sayfasını tam olarak nasıl gönderdiğini ham hâliyle anlatılacaktır.

1.6 Tekbiçim Kaynak Bulucu ve Sunucuların Konumlandırılması

Tekbiçim Kaynak Bulucu **URL** (*Uniform Resource Locator*), internet üzerindeki bir kaynağın (web sayfası, görsel, dosya veya video) adresini belirten ve ona nasıl erişileceğini tanımlayan standart adrestir. Bu adresleri web tarayıcıya yazarak sunucu kaynaklarına erişiriz.

1994 yılında *Tim Berners-Lee* tarafından geliştirilen URL, web sitelerinin dijital "ev adresi" olarak işlev görür.

Bir URL genel olarak şu bileşenlerden oluşur:

<https://www.example.com:443/blog/index.html?id=10#section2>

- ◇ **Şema (Protocol/Scheme)**: Tarayıcının kaynakla nasıl iletişim kuracağını yani protokolü belirtir (<http://>, <https://>, <ftp://>).
- ◇ **Alan Adı (Domain Name)**: Kaynağın barındırıldığı sunucunun insan tarafından okunabilir adıdır (www.example.com).
- ◇ **Port (Port)**: Sunucu üzerindeki spesifik kapıyı gösterir (ör. HTTPS için varsayılan :443, HTTP için :80). Genelde gizlidir; yani şema [https](https://) seçildiğinde 443, şema [http](http://) seçildiğinde 80 varsayılan olarak kullanılır ve adres çubuğunda gösterilmez.
- ◇ **Yol (Path)**: Sunucu içindeki dosya veya kaynağın tam konumunu gösterir (</blog/index.html>).
- ◇ **Sorgu Parametreleri (Query String)**: Sunucuya iletilen ek anahtar-değer çiftleridir; ? işareti ile başlar ve parametreler birbirlerinden & karakteri ile ayrılır ([?id=10&page=2](http://www.example.com?id=10&page=2)).
- ◇ **Çapa / Çapa Bağlantısı (Fragment/Anchor)**: Sayfa içindeki belirli bir noktayı işaret eder; # işareti ile başlar (<#section2>).

1.7 Tekbiçim Kaynak Kimliği

URL'yi öğrendikten sonra karşınıza sıkça çıkacak, ondan biraz daha genel bir kavrama değinelim: Tekbiçim Kaynak Kimliği **URI** (*Uniform Resource Identifier*). URI, genel formülü şu şekilde olan daha geniş bir adresleme kalıbıdır:

`scheme:[//authority]path[?query][#fragment]`

- ◇ **Şema (Scheme)**: İletişim kuralı veya isim alanı (<https://>, <mailto:>, <urn:>)
- ◇ **Yetki (Authority)**: Kullanıcı bilgisi, sunucu ve port (user:pass@example.com:8080).
- ◇ **Yol (Path)**: Kaynağın dikey hiyerarşik konumu (</docs/file.pdf>).
- ◇ **Sorgu (Query)**: Kaynağa iletilen filtre/parametre ([?search=latex&lang=tr](http://www.example.com/?search=latex&lang=tr)).
- ◇ **Çapa (Fragment)**: Kaynak içindeki alt bölüm (<#section2>).

URL ile URI Arasındaki Fark:

- ◇ **URI (Uniform Resource Identifier)**: Sunucudaki bir kaynağı tanımlayan yani kimliklendiren adrestir. URL'nin üst kümesidir.
- ◇ **URL (Uniform Resource Locator)**: Kaynağın hem kimliğini hem de ona nasıl ulaşılabileceğini (konumunu) belirten özel bir URI türüdür.

Bileşen	Tanım	Örnek
Scheme (Şema)	İletişim kuralı veya isim alanı	https:// , mailto: , urn:
Authority (Yetki)	Kullanıcı bilgisi, sunucu ve port	user:pass@example.com:8080
Path (Yol)	Kaynağın dikey hiyerarşik konumu	/docs/file.pdf
Query (Sorgu)	Kaynağa iletilen filtre/parametre	?search=latex&lang=tr
Fragment (Çapa)	Kaynak içindeki alt bölüm/nokta	#section2

Tablo 1.1: URI Bileşenlerine Genel Bakış

Farklı URI Örnekleri:

- ◇ Web Sayfası (URL): <https://www.example.com/index.html>
- ◇ E-posta Adresi: <mailto:ornek@email.com>
- ◇ Kitap Standart Numarası (URN): <urn:isbn:9783161484100>
- ◇ Telefon Numarası: <tel:+90-555-123-4567>

◇ Dosya Yolu: `file:///C:/Users/Document.txt`

Özet olarak her URL bir URI'dır, ancak her URI bir URL değildir. Sunucu tarafını ve adresleme sistemini anladığımıza göre, artık madalyonun diğer yüzüne, yani isteği gönderen ve yanıtı bize gösteren tarayıcıya bakabiliriz.

1.8 Web Tarayıcısı Ne Yapar?

Web tarayıcısı (*browser*), kullanıcının Web kaynaklarına erişmesini, gelen kodları derleyip görsel arayüze dönüştürmesini sağlayan istemci tarafı (*client-side*) bir yazılımdır (Örn: Chrome, Firefox, Safari, Edge). Restoran analogimizdeki müşteri rolünü üstlenen taraf, teknik olarak işte bu yazılımdır.

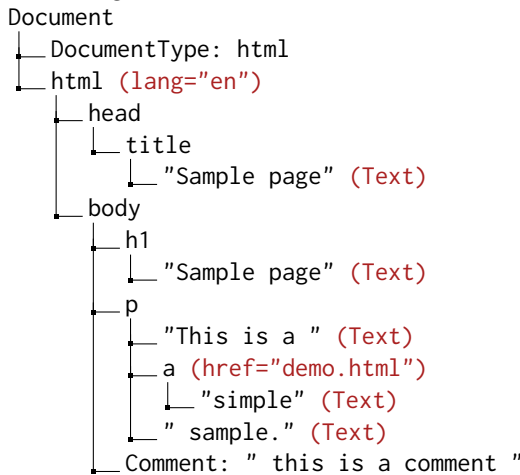
Web Tarayıcısının Çalışma Adımları (Critical Rendering Path):

1. **Adres Analizi ve Ağ İsteği:** URL ayrıştırılır, DNS çözünürlüğü yapılır (Bölüm 1.3) ve sunucuya HTTP isteği gönderilir.
2. **İçerik İşleme ve Görselleştirme:**
 - ◇ HTML öğeleri ayrıştırılarak **DOM** (*Document Object Model*) ağacı kurulur.
 - ◇ HTML öğelerini biçimlendirmede CSS (Cascading Sytle Sheet) standardı kullanılır. Bu CSS kodları ayrıştırılarak **CSSOM** (*CSS Object Model*) ağacı kurulur. İleride detaylı anlatılacaktır.
 - ◇ DOM ve CSSOM birleştirilerek **görselleştirme ağacı** (*render tree*) yapar.
 - ◇ Ekrandaki öğelerin boyut ve konumları hesaplanır (*layout/reflow*) ve HTML kodu pikseller halinde ekrana çizilir (*painting*).
3. **Kod Çalıştırma:** JavaScript kodları tarayıcı üzerinde çalıştırılarak HTML sayfaları dinamikleştirilebilmektedir.

Bu döngüdeki **sunucuya HTTP isteği gönderilir** adımının perde arkasında ne olduğunu şimdi ayrıntılı olarak inceleyeceğiz; çünkü HTTP, bir web programcısının en çok karşılaşacağı protokoldür.

§1.8.1 DOM Ağacı

Web tarayıcıları sunucudan alınan HTML öğelerini ayrıştırarak onu bir DOM ağacına dönüştürür. HTML ile işlem yapan tüm araçlar bu işlemi yapar! DOM ağaçları, özellikle `DocumentType` düğümü, `Element` düğümleri, `Text` düğümleri, `Comment` düğümleri ve bazı durumlarda `ProcessingInstruction` düğümleri olmak üzere çeşitli düğüm türleri içerir. Bölüm 1.5 altında verilen HTML dosyasının tarayıcı tarafından DOM ağacına aşağıdaki şekilde çevrilir;



Bu ağacın belge ögesi, HTML belgelerinde her zaman tepe konumunda bulunan öge (kök ögesi) olan `html` ögesidir. İki öge, `head` ve `body` ve bunların arasında bir metim (*text*) düğümü içerir.

§1.8.2 Java Script

1990'lı yıllardan itibaren web tarayıcıları JavaScript çalıştırmaya başlamıştır. Javascript ile HTML öğelerinden ve niteliklerinden herhangi birini ekleyebilir, değiştirebilir ve kaldırabilir. Ayrıca öğeleri biçimlendiren stillerden (*style*) herhangi birini değiştirebilir mevcut tüm olaylara (*event*) tepki verebilir veya yeni olaylar oluşturabilir. HTML sayfasının herhangi bir yerine `script` ögesi ile yerleştirilebilir ve tarayıcının bu kodu çalıştırması sağlanabilir.

```

<script>
document.getElementById("demo").innerHTML = "Hello JavaScript!";
const p = document.createElement("p");
p.innerText = "This is a paragraph";
document.body.appendChild(p);

```

</script>

§1.8.3 En Çok Kullanılan Web Tarayıcıları

Yıllara göre en çok kullanılan web tarayıcıları;

- ◊ Line Mode Browser, 1990, Tim Berners-Lee ve Nicola Pellow
- ◊ Netscape Navigator, 1994, Netscape
- ◊ Mosaic Web Browser, 1994, NCSA
- ◊ Internet Explorer, 1995, Microsoft
- ◊ Opera, 1995, Opera
- ◊ Safari, 2003, Apple
- ◊ Firefox, 2004, Mozilla Vakfı
- ◊ Chrome, 2008, Google (Open Source Chromium Project)

1.9 HTTP Nedir?

Hiper Metin Transfer Protokolü HTTP (*Hypertext Transfer Protocol*), Web üzerindeki istemci (tarayıcı) ile sunucu arasındaki veri aktarım standartlarını belirleyen, TCP/IP modelinin Uygulama Katmanında (*Application Layer*) çalışan bir iletişim protokolüdür. Bir başka deyişle HTTP, Bölüm 1.1.1’de tanımladığımız “müşteri isteğini nasıl ifade eder, mutfak cevabı nasıl paketler” sorusunun somut cevabıdır.

- ◊ HTTP, istemci-sunucu (*client-server*) mimarisine dayanır.
- ◊ **Durumsuzdur** (*stateless*): HTTP, varsayılan olarak her isteği birbirinden bağımsız değerlendirir. Sunucu, önceki isteklerin durumunu hatırla(ya)maz. Oturum takibi için *cookie* veya *token* gibi ek mekanizmalar kullanılır.

§1.9.1 HTTP Protokolünün Kronolojik Evrimi

HTTP, 1991’deki ilk hâinden bugüne, performans ihtiyaçları doğrultusunda dört büyük evrim geçirmiştir:

HTTP/0.9 (1991): Yalnızca *GET* komutunu destekleyen, başlık bilgisi veya durum kodu içermeyen ilkel sürümdür.

HTTP/1.0 (1996): RFC 1945 ile standartlaştı. HTTP Başlıkları (*Headers*), Durum Kodları (*Status Codes*) ve farklı içerikler için MIME-type desteği geldi. Ancak her istek için yeni bir TCP bağlantısı açılması performans darboğazı yarattı.

HTTP/1.1 (1997 / 1999): RFC 2616 ile yayımlandı. **Keep-Alive** (Kalıcı Bağlantı) ile tek bir TCP bağlantısından çoklu istek atılması sağlandı. *Host* başlığı zorunlu kılınarak tek sunucuda çoklu site barındırma (*Virtual Hosting*) imkânı doğdu.

HTTP/2 (2015): RFC 7540 ile yayımlandı. Metin yerine *ikili* (*binary*) format kullanıldı. **çoklama** (*multiplexing*) sayesinde tek bir TCP bağlantısı üzerinden eşzamanlı (*concurrent*) istekler birbirini engellemeden iletildi. Başlık (*Header*) sıkıştırma (HPACK) getirildi.

HTTP/3 (2022): RFC 9114 ile standartlaştı. Alt taşıma katmanında TCP yerine UDP tabanlı **QUIC** protokolü kullanıldı. Ağ değişimlerinde kopma yaşanması engellendi ve TCP seviyesindeki paket engellemeleri (*Transport HOL Blocking*) tamamen çözüldü.

HTTP’nin sürüm sürüm nasıl geliştiğini gördük; peki iki taraf arasında değiş tokuş edilen bu mesajlar tam olarak neye benziyor? Şimdi bir HTTP isteğinin ve yanıtının içeriğini, en somut örneklerle açalım.

§1.9.2 Web Tarayıcı ve Sunucu İletişimi

HTTP, istemci ile sunucu arasında mesaj paketleri transfer eden metin tabanlı (HTTP/1.x) veya ikili (HTTP/2-3) bir protokoldür. Her HTTP mesajı – ister istek ister yanıt olsun – aynı dört bölümlü iskeleti izler.

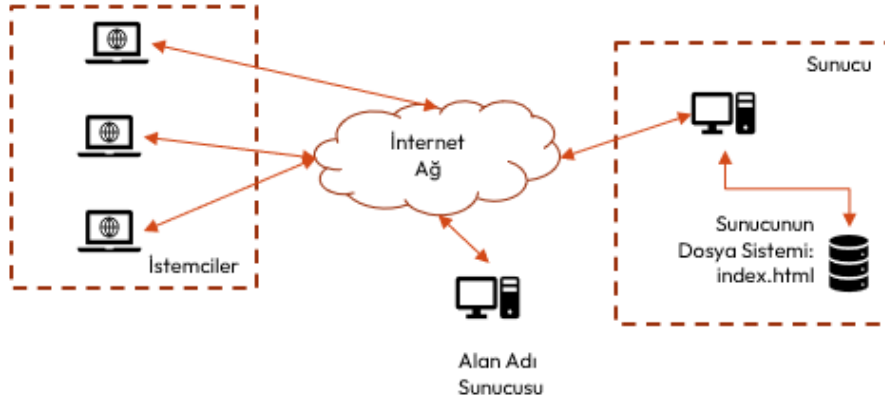
İstek Mesajı Yapısı

İstemci yani tarayıcı (*browser*) tarafında bir adres yazıldığında, aslında ekranda görmediğiniz bir metin dosyası hazırlanıp sunucuya gönderilir. Siz adres çubuğuna yalnızca kısa bir adres yazarsınız; ama tarayıcı bunu, sunucunun anlayacağı standart bir formata (*Request Message*) dönüştürür. İstek mesajı şu dört bölümden oluşur:

1. **İstek Satırı** (*Request Line*): Yöntem adı (*GET, PUT, POST, ...*), kaynak adresi (URI) ve HTTP sürümü.
2. **İstek Başlıkları** (*Request Headers*): İstemci ve medya bilgileri (anahtar-değer çiftleri).
3. **Boş Satır** (*Empty Line*): Başlıklar ile veri gövdesini ayıran zorunlu alan.
4. **Mesaj Gövdesi** (*Payload*): Sunucuya gönderilecek veri (POST/PUT detayları).

Bunu somut bir örnekle görelim. Diyelim ki tarayıcının adres çubuğuna bir görselin adresini yazdınız:

Web tarayıcıda yazılan görsel kaynak isteği (*logo.png*):



Şekil 1.3: Web Sunucusu-Tarayıcı İletişimi

www.example.com/images/logo.png

Tarayıcı, bu kısa adresi arka planda yukarıdaki dört bölümü içeren şu ham (*raw*) pakete dönüştürür:

görsel kaynak için tarayıcı tarafından oluşturulan ham (raw) HTTP istek paketi:

```
GET /images/logo.png HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: image/avif,image/webp,*/*
Connection: keep-alive
```

[Boş Satır]

Yanıt Mesajı Yapısı

Sunucu, gelen bu isteğe neredeyse simetrik bir olarak yanıt paketiyle (*response message*) karşılık verir:

1. **Durum Satırı (Status Line):** HTTP sürümü, Sayısal Durum Kodu ve Açıklama.
2. **Yanıt Başlıkları (Response Headers):** Sunucu ve içerik meta verileri.
3. **Boş Satır (Empty Line):** Gövde ayırıcı alan.
4. **Mesaj Gövdesi (Message Body):** İstenen içerik (HTML, JSON, Görsel vb.).

Az önceki görsel isteğine sunucunun vereceği yanıt şu şekildedir:

Görsel kaynağa ilişkin sunucu tarafından hazırlanan ham (raw) HTTP yanıt paketi:

```
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 12:00:00 GMT
Server: Nginx/1.24.0
Content-Type: image/png
Content-Length: 4512
Connection: keep-alive
```

[Boş Satır]

[Görsel İkili Verisi / Binary Data]

Bir HTML Sayfasına İlişkin İstek ve Yanıt Paketleri

Aynı yapı, bir görsel yerine bir HTML sayfası istendiğinde de birebir geçerlidir; yalnızca istenen kaynak yolu ve **Accept** başlığı değişir. Şimdi adres çubuğuna yalnızca site adını yazdığınızı düşünelim:

Bu durumda web tarayıcıda yazılan adres aşağıdaki gibidir:

www.example.com

Bu istek karşısında oluşan ham HTTP İstek Paketi:

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Accept-Language: tr-TR,tr;q=0.9
Connection: keep-alive
```

[Boş Satır]

Dikkat ederseniz siz herhangi bir dosya adı yazmadınız; ama istek paketinde `/index.html` yolu otomatik olarak belirdi. Bunun nedeni basittir: sunucuya bir dosya yolu belirtilmediğinde, sunucu **varsayılan olarak kök dizindeki `index.html`** dosyasını döndürecek şekilde yapılandırılmıştır. İleride kendi web sitenizi yayınladığınızda, ziyaretçilerin göreceği ilk sayfayı bu yüzden `index.html` olarak adlandırabilirsiniz.

Bu isteğe cevap olarak sunucu, gövdeye ikili veri yerine düz metin olarak HTML içeriğini yerleştirir; **Content-Type** başlığının değeri de buna göre değişir:

Örnek Raw HTTP Yanıt Paketi (`index.html` Sayfası):

```
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 12:00:00 GMT
Server: Nginx/1.24.0
Content-Type: text/html; charset=UTF-8
Content-Length: 187
Connection: keep-alive

<!DOCTYPE html>
<html lang="tr">
<head>
<meta charset="UTF-8">
<title>Ana Sayfa</title>
</head>
<body>
<h1>Hoş Geldiniz</h1>
<p>Bu, ornek.com sitesinin ana sayfasıdır.</p>
</body>
</html>
```

Tarayıcı, bu ham metni aldıktan sonra Bölüm 1.8’de anlatılan **Kritik Görselleştirme Yolu** (*Critical Rendering Path*) adımlarını işleterek DOM ağacını kurar ve sayfayı ekrana çizer.

Buraya kadar sunucunun size bir kaynağı nasıl ilettiğini gördük. Ama istemci sunucuya her zaman “bana bunu ver” demez; kimi zaman yeni bir kayıt oluşturmak, kimi zaman mevcut bir kaydı güncellemek ya da silmek ister. İşte bu farklı niyetleri ifade eden HTTP yöntemlerini şimdi tek tek inceleyelim.

En Popüler Web Sunucuları

Aşağıda yıllara sari çok kullanılan web sunucuları yer almaktadır;

- ◊ 1990, CERN httpd, *Tim Berners-Lee*, NextStepOS
- ◊ 1994, NCSA httpd, NCSA
- ◊ 1994, Netsite, Netscape
- ◊ 1995, Apache Http Server, Apache Vakfı
- ◊ 1995, IIS, Microsoft
- ◊ 2002, Hiawatha, *Hugo Leisink*
- ◊ 2003, lighttpd, *Jan Kneschke*
- ◊ 2004, Nginx, *Igor Sysoev* (reverse proxy)
- ◊ 2009 Node.js (cross-platform, open-source JavaScript runtime environment)

§1.9.3 HTTP Yöntemleri

Bir önceki başlıkta sunucudan veri okuma yöntemi olan **GET** yöntemine örnek verilmiştir. Başka yöntemler (*HTTP Methods*) veya eylemler (*HTTP verbs*) de vardır. Bu yöntemler, istemcinin sunucudaki hedef kaynak üzerinde gerçekleştirmek istediği eylemin amacını belirtir.

İşlem odaklı (RPC-*Remote Procedure Call* tarzı) mimarilerde HTTP yöntemleri standart amaçları dışında kullanılır. Sunucuda **Veri Oluşturma, Okuma, Güncelleme ve Silme** yani **CRUD** (*Create, Read, Update, Delete*) işlemleri için tek bir yöntem (**GET** veya **POST**) üzerinden yürütülür. Sunucuda yapılacak her bir işlem için ayrı bir URL kullanılır.

GET

RESTful olmayan yapıda **GET**, yalnızca veri okumak için değil, bazen silme veya güncelleme gibi eylemleri tetiklemek için de yanlış bir şekilde URL parametreleriyle kullanılır.

Örnek İstek (Eylem Odaklı URL ile Veri Getirme):

```
GET /services/urunGetir.php?urunId=42 HTTP/1.1
Host: www.ornek.com
Accept: application/json
```

[Boş Satır -- Gövde Yok]

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 72
```

```
{"status": "success", "data": {"id": 42, "ad": "Kablosuz Klavye", "fiyat": 349.90}}
```

HEAD

Sunucu yanıt olarak **yalnızca HTTP başlıklarını** döner, mesaj gövdesini (*body*) göndermez. Bu yöntem sunucudaki bir dosya veya servisin durumunu kontrol etmek için kullanılır.

Örnek İstek:

```
HEAD /dosyalar/getLogo.ashx?fileId=10 HTTP/1.1
Host: www.ornek.com
```

[Boş Satır -- Gövde Yok]

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: image/png
Content-Length: 24560
```

[Boş Satır -- Gövde Yine Yok]

POST

HTTP **POST** yöntemi, yeni kayıt oluşturmanın yanı sıra veri güncelleme, veri silme veya bir iş sürecini başlatmak için kullanılan genel bir "işlem yürütücü" haline gelir. Her işlem için ayrı bir URL fonksiyonu çağrılır.

Örnek İstek (İşlem Odaklı Kayıt Ekleme):

```
POST /api/v1/kullaniciEkle HTTP/1.1
Host: www.ornek.com
Content-Type: application/json
Content-Length: 58
```

```
{"action": "create_user", "ad": "Ayşe Yılmaz", "email": "ayse@ornek.com"}
```

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"resultCode": 0, "message": "Kullanici eklendi", "newId": 107}
```

Diğer Yöntemler

- ◊ **PUT**: Sunucudaki bir kaynağı tamamen değiştirmek için kullanılır. Bu yöntem ileride göreceğimiz RESTful mimaride kullanılır.
- ◊ **DELETE**: Sunucudaki bir kaynağı silmek için kullanılır. Bu yöntem de ileride göreceğimiz RESTful mimaride kullanılır. Silme işlemi bir **POST** yöntemi ile yapılır. Bazı durumlarda güvenlik açısından sakıncalı olmasına rağmen bir **GET** yöntemi üzerinden parametre gönderilerek tetiklenir.
- ◊ **PATCH**: Sunucudaki bir kaynağı kısmi olarak değiştirmek için kullanılır. Bu yöntem de ileride göreceğimiz RESTful sistemlerde kullanılır. Bütün güncelleme fonksiyonları gibi genel bir **POST** metoduna veya ilgili eylem URL'sine yönlendirilerek bu işlem yapılır.
- ◊ **OPTIONS**: Sunucunun desteklediği yöntemleri ve **Çapraz Kaynak Paylaşımı** olan CORS (*Cross-Origin Resource Sharing*) güvenlik kurallarını sorgular.
- ◊ **CONNECT**: Vekil sunucular üzerinden şifreli tünel (HTTPS) kurmak için kullanılır.
- ◊ **TRACE**: İstek paketinin izlediği yolu test eden teşhis yöntemidir.

Yukarıdaki örneklerde sürekli karşımıza çıkan **200 OK**, **201 Created** ve **204 No Content** gibi ifadeler, aslında sunucunun isteğe ne şekilde karşılık verdiğini özetleyen büyük bir sınıflandırma sisteminin yalnızca birer üyesidir. Bu sistemi şimdi bütünüyle ele alalım.

§1.9.4 HTTP Durum Kodları

HTTP durum kodları (*HTTP Status Codes*), sunucunun isteğe verdiği yanıtı sınıflandıran 3 haneli sayılardır. İlk hane, yanıtın genel kategorisini belirler:

- ♦ **1xx – Bilgilendirici (Informational)**: İsteğin alındığını ve sürecin devam ettiğini belirtir (Örn: **100 Continue**).
- ♦ **2xx – Başarılı (Successful)**: İşlemin başarıyla tamamlandığını gösterir (Örn: **200 OK**, **201 Created**, **204 No Content**).
- ♦ **3xx – Yönlendirme (Redirection)**: Ek işlem yapılması gerektiğini bildirir (Örn: **301 Moved Permanently**, **304 Not Modified**).
- ♦ **4xx – İstemci Hataları (Client Errors)**: İsteğin hatalı veya yetkisiz olduğunu gösterir (Örn: **400 Bad Request**, **401 Unauthorized**, **403 Forbidden**, **404 Not Found**).
- ♦ **5xx – Sunucu Hataları (Server Errors)**: Sunucunun isteği işlerken hataya düştüğünü gösterir (Örn: **500 Internal Server Error**, **502 Bad Gateway**, **503 Service Unavailable**).

1.10 Bölüm Özeti

Bu bölümde, bir tarayıcıya adres yazıp Enter'a bastığımız andan ekranda bir sayfa belirene kadar geçen yolculuğu en basit fikirden başlayarak adım adım izledik:

- ♦ Her şeyin temelinde basit bir **istemci-sunucu** ilişkisi vardır: biri ister, biri karşılar.
- ♦ **İnternet**, cihazları birbirine bağlayan fiziksel/mantıksal altyapıdır; **TCP/IP** ise bu altyapı üzerinde veriyi katman katman paketleyip taşıyan kurallar bütünüdür.
- ♦ **DNS**, insan dostu alan adlarını (*ornek.com* gibi) makinelerin anladığı IP adreslerine çevirir.
- ♦ **Web**, İnternet üzerinde çalışan; HTML, HTTP ve URL üçlüsüyle kurulmuş bir bilgi paylaşım servisi.
- ♦ **HTTP**, istemci ile sunucu arasındaki bu iletişimin dilini belirler: hangi yöntemle (**GET**, **POST**, ...) ne istendiğini ve sunucunun buna hangi durum koduyla yanıt verdiğini tanımlar.

Bu kavramlar, kitabın ilerleyen bölümlerinde yazacağınız sunucu tarafı kodların, tasarlayacağınız API'lerin ve çözeceğiniz performans problemlerinin ortak dilini oluşturacak. Şimdi öğrendiklerinizi pekiştirmek için aşağıdaki soruları çözmeyi deneyin.

1.11 Düşün ve Çöz

1. **İnternet ve Web Ayrımı**: Bir arkadaşınıza İnternet ve Web arasındaki farkı teknik terimlere boğulmadan, günlük hayattan metinde yer alan otoyol analogisi dışında özgün bir analogi kurgulayarak nasıl açıklarsınız?
2. **Protokol Evrimi ve Performans**: HTTP/1.1 kullanılan bir e-ticaret sitesinde 100 adet küçük ürün görseli aynı anda yüklenirken tarayıcının yaşadığı HOL (*Head-of-Line*) darboğazının nedeni nedir? HTTP/2 bu sorunu çoklama (*Multiplexing*) mekanizmasıyla tam olarak nasıl çözmektedir?
3. **Uçtan Uca İletişim Analizi**: Tarayıcıya <https://www.kitap.com/bolum1> yazılıp Enter'a basıldığı andan itibaren gerçekleşen DNS Çözümleme, TCP 3'lü El Sıkışması, TLS El Sıkışması, HTTP İstek/Yanıt ve DOM Rendering aşamalarının sırasını şematik olarak açıklayınız.

2. Bölüm

HTML

2.1 HTML Nedir?

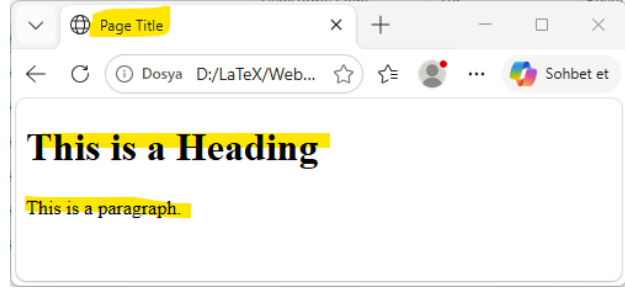
Hiper Metin İşaretleme Dili **HTML** (*HyperText Markup Language*), Bölüm 1.5’de bahsedildiği gibi web sayfalarının yapısını ve içeriğini oluşturmak için kullanılan standart işaretleme dilidir. Bir programlama dili değil, tarayıcılara metin, görsel, video ve bağlantıların nasıl görüntüleneceğini bildiren bir **etiket** (*tag*) sistemidir.

Web tarayıcısı, sunucudan talep edilen HTML belgelerini aldıktan sonra Bölüm 1.8’de bahsedildiği üzere belgeye ait DOM ve CSDOM ağacını oluşturur. Daha sonra bu ağaçlardan ekrandaki öğelerin boyut ve konumları hesaplanır ve tarayıcı penceresine HTML öğeleri çizilir. Bu işleme **görselleştirme** (*render*) adı verilir.

2.2 Etiketler

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Kod 2.1: Basit HTML Belgesi



Şekil 2.1: Basit HTML Belgesi Web Tarayıcı Görselleştirmesi

HTML belgeleri, birbirini kapsayan öge (*element*) ve metin ağacından oluşur. Bir HTML belgesi oluşturulurken şu temel kurallara dikkat edilmelidir:

- ♦ **Etiket Çiftleri:** Öğelerin büyük bir çoğunluğu `<body>` gibi bir **başlangıç etiketi** ve bölü işareti içeren `</body>` gibi bir **bitiş etiketi** arasında yazılır. İki etiket arasında kalan kısım öğenin içeriğini oluşturur.
- ♦ **İç İç Yerleşim (Nesting):** Etiketler birbiriyle çakışmayacak şekilde, tam anlamıyla birbirini kapsayacak biçimde yerleştirilmelidir. Bir etiket kapatılmadan önce, onun içinde açılan tüm alt etiketler kapatılmış olmalıdır.
- ♦ **Doctype Bildirimi:** Belgenin en başında yer alan `<!DOCTYPE html>` ifadesi bir HTML etiketi değil; tarayıcıya bu belgenin HTML5 standartlarında yazıldığını bildiren teknik bir deklarasyondur.
- ♦ **Kök Öge:** Her HTML belgesinin tüm içeriği, en dışta yer alan `html` başlangıç ve bitiş etiketlerinin arasında yer almalıdır.

Yukarıdaki temel örnekte geçen etiketlerin kullanım amaçları şu şekildedir:

- ♦ **head:** Web sayfasının karakter kodlaması, başlığı, harici stil dosyaları ve arama motoru optimizasyonu (SEO) bilgileri gibi kullanıcıya doğrudan görünmeyen **üst verilerini** (*metadata*) barındırır.
- ♦ **title:** Tarayıcı sekmesinde veya arama motoru sonuçlarında görünen sayfa başlığını belirler.
- ♦ **body:** Web sayfasında kullanıcıya doğrudan gösterilecek tüm metin, görsel, buton ve yapısal içeriklerin yer aldığı ana bölümdür.
- ♦ **h1:** Sayfadaki en önemli ana başlığı yani birinci seviye başlığı temsil eder (*Heading 1*). Arama motorları için sayfa içeriğinin özetini sunar.
- ♦ **p:** Metin paragrafları oluşturmak için kullanılır (*Paragraph*).

2.3 Etiketlerde Hiyerarşi

HTML’de metin içerisindeki vurguları doğru temsil etmek ve anlamsal (*semantic*) bir yapı kurmak son derece önemlidir. Etiketlerin sıralaması doğrudan tarayıcının metni doğru yorumlamasını sağlar.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
    <p>Bu <em>oldukca
      → <strong>yanlış!</strong></em></p>
    <p>Bu <em> çok
      → <strong>doğru</strong>.</em></p>
  </body>
</html>
```

Kod 2.2: Etiket Hiyerarşisi HTML Belgesi



Şekil 2.2: Etiket Hiyerarşisi Web Tarayıcı Görselleştirmesi

Yukarıdaki örnekte yer alan biçimlendirme etiketlerinin kullanım amaçları şu şekildedir:

- ♦ **em**: İçindeki metne vurgu (*emphasis*) kazandırır. Tarayıcılar bu metni varsayılan olarak *eğik (italik)* şekilde görselleştirir.
- ♦ **strong**: İçindeki metnin önemini ve ciddiyetini belirtir. Tarayıcılar bu metni varsayılan olarak **koyu (bold)** şekilde gösterir.

Sözdizimi İpucu: Etiketleri kapatırken "Son açılan etiket, ilk kapatılır" (LIFO - Last In, First Out) mantığı izlenmelidir. Örneğin; `<p>Bu paragrafta <em>burası vurguludur.</em></p>` doğru bir kullanımdır.

2.4 Etiket Nitelikleri

Öğeler (*element*), davranışlarını değiştiren, ek bilgi sağlayan veya görünüşlerini şekillendiren niteliklere (*attribute*) sahip olabilirler.

Nitelikler hakkında bilinmesi gereken temel kurallar şunlardır:

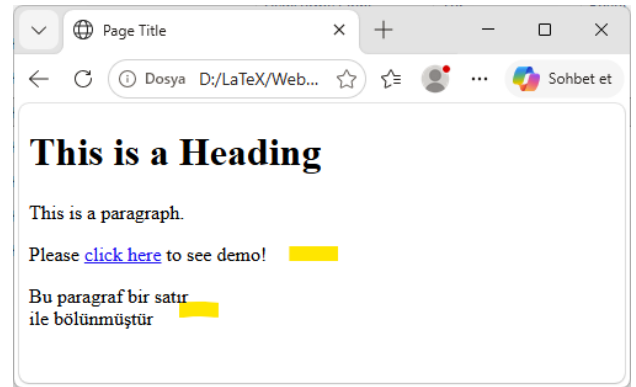
1. Nitelikler **her zaman başlangıç etiketinin içine** yazılır; bitiş etiketine asla yazılmaz.
2. Genellikle `anahtar="değer"` (`name="value"`) çifti şeklinde tanımlanırlar.
3. Nitelik isimleri boşluk veya " ' ' = < > gibi özel karakterler içeremez.

Aşağıdaki örnekte, **a** (*anchor*) etiketi ve bu etiketin hedef adresi belirtmesini sağlayan **href** niteliği kullanılarak bir köprü metni (*hyperlink*) oluşturulmuştur:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
    <p> Please <a href="demo.html">click
      → here</a> to see demo! </p>
    <p>Bu paragraf bir satır <br> ile
      → bölünmüştür</p>
  </body>
</html>
```

Kod 2.3: Etiket Nitelikleri HTML Belgesi

Örnekte geçen **a** etiketi ve **href** niteliği şu amaçla kullanılır:



Şekil 2.3: Etiket Nitelikleri Web Tarayıcı Görselleştirmesi

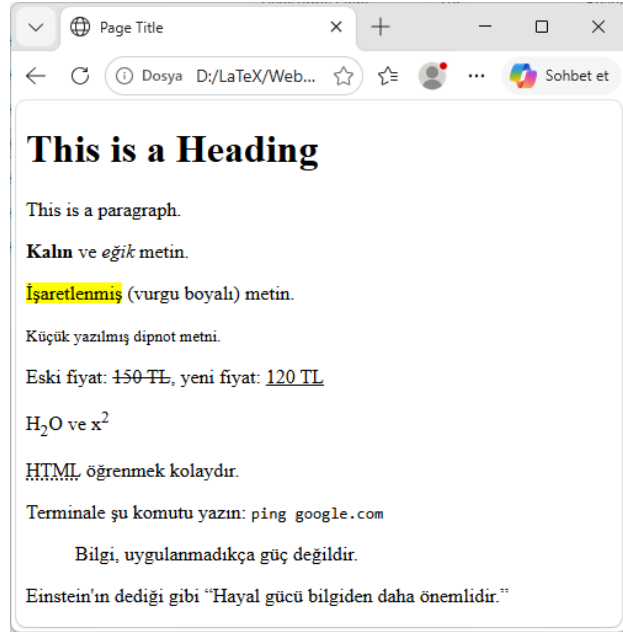
- ♦ **a**: Başka bir web sayfasına, aynı sayfa içerisindeki bir bölüme, bir dosyaya veya bir e-posta adresine köprü (bağlantı) oluşturmak için kullanılır.
- ♦ **href**: (*Hypertext Reference*) Kullanıcı bağlantıya tıkladığında yönlendirilecek hedef web adresini

(URL) veya dosya yolunu belirtir.

2.5 Diğer Metin Biçimlendirme Etiketleri

Bölüm 2.3’de `em` ve `strong` etiketlerini gördük. Bunların dışında, metnin anlamını veya sunumunu değiştiren birçok satır içi (*inline*) etiket daha vardır:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
    <p><b>Kalın</b> ve <i>eğik</i> metin.</p>
    <p><mark>İşaretlenmiş</mark> (vurgu boyalı)
    → metin.</p>
    <p><small>Küçük yazılmış dipnot
    → metni.</small></p>
    <p>Eski fiyat: <del>150 TL</del>, yeni
    → fiyat: <ins>120 TL</ins></p>
    <p>H<sub>2</sub>O ve x<sup>2</sup></p>
    <p><abbr title="Hiper Metin İşaretleme
    → Dili">HTML</abbr> öğrenmek
    → kolaydır.</p>
    <p>Terminale şu komutu yazın: <code>ping
    → google.com</code></p>
    <blockquote>Bilgi, uygulanmadıkça güç
    → değildir.</blockquote>
    <p>Einstein'ın dediği gibi <q>Hayal gücü
    → bilgiden daha önemlidir.</q></p>
  </body>
</html>
```



Şekil 2.4: Metin Biçimlendirme Web Tarayıcı Görselleştirmesi

Kod 2.4: Metin Biçimlendirme HTML Belgesi

- ◊ `b` / `i`: Metni sırasıyla kalın ve eğik gösterir; `strong`/`em`’den farkı, bir vurgu veya önem anlamı taşımadan yalnızca görsel bir stil uygulamalarıdır.
- ◊ `mark`: Metni bir vurgu kalemle işaretlenmiş gibi arka planı renkli gösterir.
- ◊ `small`: Yasal uyarı veya dipnot gibi ikincil önemdeki metinleri küçük punto ile gösterir.
- ◊ `del` / `ins`: Sırasıyla belgeden silinen (üstü çizili) ve belgeye sonradan eklenen (altı çizili) metinleri belirtir.
- ◊ `sub` / `sup`: Metni alt simge (*subscript*) veya üst simge (*superscript*) olarak gösterir; kimyasal formüller ve matematiksel ifadeler için kullanılır.
- ◊ `abbr`: Bir kısaltmayı temsil eder; `title` niteliği ile kısaltmanın açık hâli, fare üzerine getirildiğinde ipucu (*tooltip*) olarak gösterilir.
- ◊ `code`: Bilgisayar kodu parçalarını, tarayıcının varsayılan sabit genişlikli (*monospace*) yazı tipiyle gösterir.
- ◊ `blockquote`: Başka bir kaynaktan alınmış, kendi paragrafını oluşturan uzun alıntılarını belirtir; tarayıcı genellikle sola girinti uygular.
- ◊ `q`: Kısa, satır içi alıntılarını belirtir; tarayıcı içeriği otomatik olarak tırnak içine alır.

2.6 Listeler

Web sayfalarında birden fazla öğeyi (menü seçenekleri, tarif adımları, malzeme listeleri vb.) düzenli biçimde sunmak için üç tür liste etiketi kullanılır.

Ş2.6.1 Sırasız Liste

Öğeler arasında bir öncelik/sıra belirtilmeyen listelerdir (*unordered list*); tarayıcı varsayılan olarak her öğenin başına bir madde imi (*bullet*) koyar.

```
<ul>
  <li>Elma</li>
  <li>Armut</li>
  <li>Muz</li>
```

```
</ul>
```

ul (*Unordered List*) liste kapsayıcısıdır; **li** (*List Item*) ise listenin her bir öğesini temsil eder.

§2.6.2 Sıralı Liste

Sıralı liste (*ordered list*), öğelerin belirli bir sırayla (adım adım) sunulması gerektiğinde kullanılır; tarayıcı öğeleri otomatik olarak numaralandırır.

```
<ol>
  <li>Suyu kaynat.</li>
  <li>Poşeti suya at.</li>
  <li>5 dakika bekle.</li>
</ol>
```

ol (*Ordered List*); **type** niteliği ile numaralandırma biçimi (1, A, a, I, i) değiştirilebilir.

§2.6.3 Tanım Listesi

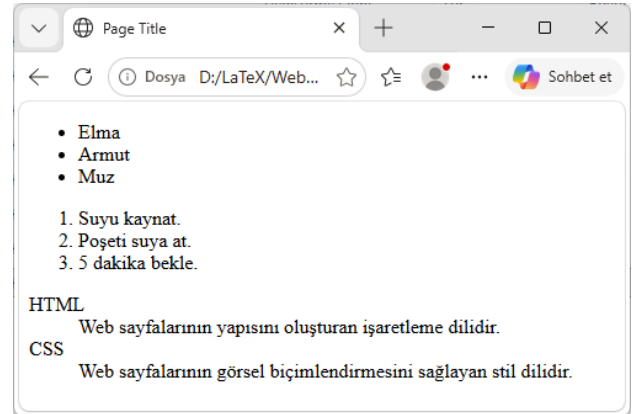
Tanım listesi (*description list*) ile o terimin açıklamasını eşleştirmek için kullanılır (sözlük, sıkça sorulan sorular gibi).

```
<dl>
  <dt>HTML</dt>
  <dd>Web sayfalarının yapısını oluşturan işaretleme dilidir.</dd>
  <dt>CSS</dt>
  <dd>Web sayfalarının görsel biçimlendirmesini sağlayan stil dilidir.</dd>
</dl>
```

dl (*Description List*) kapsayıcıdır; **dt** (*term*) terimi, **dd** (*description*) ise terimin açıklamasını belirtir.

Listeler iç içe de kullanılabilir: bir **li** öğesinin içine yeni bir **ul** veya **ol** yerleştirilerek alt maddeler oluşturulabilir.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <ul>
      <li>Elma</li>
      <li>Armut</li>
      <li>Muz</li>
    </ul>
    <ol>
      <li>Suyu kaynat.</li>
      <li>Poşeti suya at.</li>
      <li>5 dakika bekle.</li>
    </ol>
    <dl>
      <dt>HTML</dt>
      <dd>Web sayfalarının yapısını oluşturan
        → işaretleme dilidir.</dd>
      <dt>CSS</dt>
      <dd>Web sayfalarının görsel
        → biçimlendirmesini sağlayan stil
        → dilidir.</dd>
    </dl>
  </body>
</html>
```



Şekil 2.5: Listeler Web Tarayıcı Görselleştirmesi

Kod 2.5: Liseler HTML Belgesi

2.7 Görseller

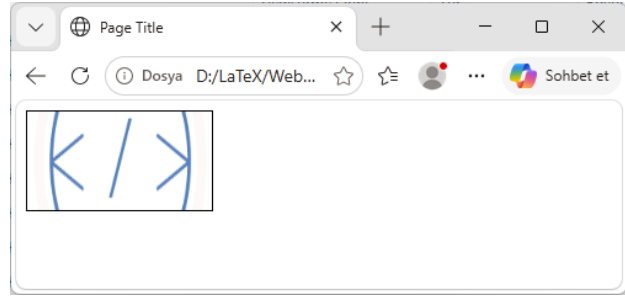
Bir web sayfasına görsel (*image*) eklemek için **img** etiketi kullanılır. Bu etiket, içerik veya bitiş etiketi barındırmayan boş bir öğedir.

```

```

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    
  </body>
</html>
```

Kod 2.6: Görsel İçeren HTML Belgesi



Şekil 2.6: Görsel Web Tarayıcı Görselleştirmesi

- ◊ **src:** (*Source*) Görsel dosyasının yolunu veya adresini (URL) belirtir. Sadece dosya adı varda HTML belgesi ile aynı dizinde yer aldığı varsayılır.
- ◊ **alt:** (*Alternative Text*) Görsel yüklenemediğinde gösterilecek alternatif metindir; ayrıca ekran okuyucu yazılımlar tarafından görme engelli kullanıcılara okunduğu için erişilebilirlik açısından zorunlu kabul edilir.
- ◊ **width / height:** Görselin piksel cinsinden genişlik ve yüksekliğini belirtir; tarayıcının sayfa yüklenirken görsel için doğru miktarda yer ayırmasını sağlar.

2.8 Tablolar

Satır ve sütunlardan oluşan verileri (ör. ders programı, fiyat listesi) düzenli biçimde göstermek için tablo (*table*) etiketleri kullanılır.

```
<table border="1">
  <caption>Haftalık Ders Programı</caption>
  <thead>
    <tr>
      <th>Gün</th>
      <th>Ders</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Pazartesi</td>
      <td>Web Programlama</td>
    </tr>
    <tr>
      <td>Salı</td>
      <td>Veri Tabanı Yönetim Sistemleri</td>
    </tr>
  </tbody>
</table>
```

Kod 2.7: Tablo İçeren HTML Belgesi

- ◊ **table:** Tablonun tamamını kapsayan ana etikettir.
- ◊ **caption:** Tablonun başlığını/açıklamasını belirtir.
- ◊ **thead / tbody / tfoot:** Tablonun sırasıyla başlık, gövde ve alt bilgi bölümlerini gruplar.
- ◊ **tr:** (*Table Row*) Bir tablo satırını temsil eder.
- ◊ **th:** (*Table Header*) Başlık hücrelerini belirtir; tarayıcı bu hücreleri varsayılan olarak kalın ve ortalı gösterir.
- ◊ **td:** (*Table Data*) Sıradan bir veri hücrelerini belirtir.

Gün	Ders
Pazartesi	Web Programlama
Salı	Veri Tabanı Yönetim Sistemleri

Şekil 2.7: Tablo Web Tarayıcı Görselleştirmesi

2.9 Blok ve Satır İçi Elemanlar

HTML öğeleri tarayıcı tarafından varsayılan olarak iki farklı biçimde görüntülenir:

- ◊ **Blok Düzey (*Block-level*)**: Kendinden önceki ve sonraki içerikten ayrı bir satırda, yatayda mevcut tüm genişliği kaplayarak görüntülenir (ör. `p`, `h1`, `ul`, `div`, `table`).
 - ◊ **Satır İçi (*Inline*)**: Yalnızca içeriği kadar yer kaplar ve aynı satırda diğer öğelerle yan yana durabilir (ör. `a`, `strong`, `em`, `span`, `img`).
- `div` (*division*) ve `span`, kendi başlarına herhangi bir anlamsal anlam taşımayan, salt gruplama amaçlı öğelerdir:
- ◊ `div`: Bir grup blok düzey içeriği (birkaç paragraf, başlık, resim vb.) tek bir kapsayıcı altında toplamak için kullanılır.
 - ◊ `span`: Bir metin akışı içindeki küçük bir bölümü (ör. yalnızca bir kelimeyi) diğerlerinden ayırmak için kullanılır.

```
<div>
  <h2>Ürün Bilgisi</h2>
  <p>Bu ürün <span>sınırlı sayıda</span> üretilmiştir.</p>
</div>
```

Önemli Not: `div` ve `span` öğeleri CSS olmadan kullanıldığında sayfanın görünümünde herhangi bir fark yaratmaz; asıl güçleri, ileriki bölümlerde öğreneceğiniz CSS ile birlikte kullanıldıklarında ortaya çıkar.

2.10 HTML5 Semantik Etiketleri

Erken dönem web sayfalarında, sayfanın her bölümü (üst menü, ana içerik, kenar çubuğu, alt bilgi) genellikle anlamsız `div` etiketleriyle oluşturuluyordu. HTML5 ile birlikte, sayfanın hangi bölümünün ne işe yaradığını hem tarayıcıya hem geliştiriciye hem de arama motorlarına açıkça belirten **semantik (anlamsal)** etiketler tanıtıldı.

```
<body>
  <header>
    <h1>Site Başlığı</h1>
    <nav>
      <a href="#">Ana Sayfa</a>
      <a href="#">Hakkımızda</a>
    </nav>
  </header>
  <main>
    <article>
      <h2>Blog Yazısı Başlığı</h2>
      <p>Yazının içeriği burada yer alır.</p>
    </article>
    <aside>
      <p>İlgili bağlantılar burada listelenir.</p>
    </aside>
  </main>
  <footer>
    <p>&copy; 2026 Tüm hakları saklıdır.</p>
  </footer>
</body>
```

Kod 2.8: HTML5 Etiketleri İçeren HTML Belgesi

- ◊ **header**: Sayfanın veya bir bölümün giriş kısmını (logo, başlık, gezinme menüsü) barındırır.
 - ◊ **nav**: (*Navigation*) Sayfa içi veya siteler arası bağlantı menülerini kapsar.
 - ◊ **main**: Sayfanın tekrar etmeyen, asıl/benzersiz içeriğini kapsar; bir sayfada yalnızca bir kez kullanılmalıdır.
 - ◊ **article**: Kendi başına anlamlı, bağımsız olarak dağıtılacak içerikleri (blog yazısı, haber, forum gönderisi) kapsar.
 - ◊ **aside**: Ana içerikle dolaylı olarak ilgili yan bilgileri (kenar çubuğu, reklam, ilgili bağlantılar) kapsar.
 - ◊ **footer**: Sayfanın veya bir bölümün alt bilgisini (telif hakkı, iletişim bilgisi) kapsar.
- Semantik etiketler, tarayıcı tarafından görsel olarak `div` ile aynı şekilde (blok öğe olarak) görüntülenir; asıl fark, kod okunabilirliği, erişilebilirlik ve arama motoru optimizasyonu (*SEO*) açısından ortaya çıkar.

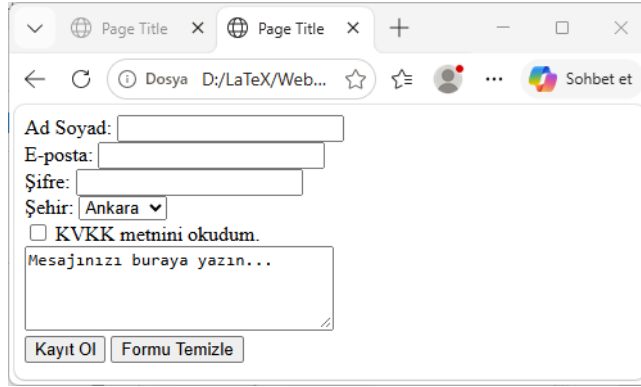
2.11 Formlar

Kullanıcıdan veri toplamak (giriş formu, arama kutusu, anket vb.) ve bu veriyi sunucuya göndermek için **form** etiketi ve alt elemanları kullanılır. Formların sunucuya veri gönderme sürecini (*GET* / *POST* yöntemleriyle) Bölüm 1.9'de incelemiştik; burada bu isteği tetikleyen arayüz elemanlarını ele alacağız.

```
<form action="/kayit" method="post">
  <label for="ad">Ad Soyad:</label>
  <input type="text" id="ad" name="ad"><br>
  <label for="eposta">E-posta:</label>
  <input type="email" id="eposta" name="eposta"><br>
  <label for="sifre">Şifre:</label>
  <input type="password" id="sifre" name="sifre"><br>
  <label for="sehir">Şehir:</label>
  <select id="sehir" name="sehir">
    <option value="ankara">Ankara</option>
    <option value="istanbul">İstanbul</option>
  </select>
  <label><br>
  <input type="checkbox" name="kvkk"> KVKK metnini okudum.
  </label><br>
  <textarea name="mesaj" rows="4" cols="30">Mesajınızı buraya yazın...</textarea>
  <br>
  <button type="submit">Kayıt Ol</button> <button type="reset">Formu Temizle</button>
</form>
```

Kod 2.9: Form İçeren HTML Belgesi

- ◇ **form**: Girilen tüm verileri gruplayan ve **action** niteliğinde belirtilen adrese **method** (GET/POST) ile gönderen ana kapsayıcıdır.
- ◇ **label**: Bir form elemanının ne işe yaradığını açıklayan etikettir; **for** niteliği, ilişkili olduğu elemanın **id** değeriyle eşleşmelidir.
- ◇ **input**: Kullanıcıdan tek satırlık veri almak için kullanılan çok amaçlı elemandır; **type** niteliğiyle **text**, **email**, **password**, **checkbox**, **radio**, **number**, **date** gibi birçok farklı veri türü olabilir.
- ◇ **select / option**: Kullanıcıya önceden tanımlı seçenekler arasından seçim yaptıran açılır liste oluşturur.
- ◇ **textarea**: Çok satırlı, serbest metin girişi için kullanılır.
- ◇ **button**: Formu göndermek (**submit**), sıfırlamak (**reset**) veya özel bir işlemi tetiklemek için kullanılan tıklanabilir elemandır.



Şekil 2.8: Form Web Tarayıcı Görselleştirmesi

2.12 Yorum Satırları

HTML kodu içine, tarayıcı tarafından görüntülenmeyen ama kaynak kodunda görünen açıklama notları eklemek için yorum satırı sözdizimini kullanılır:

```
<!-- Bu bir yorum satırıdır, tarayıcıda görüntülenmez -->
<p>Bu metin ise tarayıcıda görüntülenir.</p>
```

Yorumlar; kodun belirli bölümlerini geçici olarak devre dışı bırakmak, ekip arkadaşlarına not bırakmak veya kodun karmaşık kısımlarını açıklamak amacıyla sıkça kullanılır.

2.13 Kendi Kendini Kapanan Etiketler

HTML'de bazı öğeler içerisinde metin veya başka bir etiket barındırmaz (*Self-Closing Tags*). Bu tür öğelere **boş öğeler** (*void elements*) denir ve bunlarda bitiş etiketi bulunmaz. Bu bölümde gördüğünüz **br** etiketi buna bir örnektir.

En sık karşılaşılabilecek boş öğeler şunlardır:

- ◇ **
**: Bir metin akışı içinde satır sonu (*line break*) oluşturur.
- ◇ **<hr>**: (*Horizontal Rule*) İçerikler arasına yatay bir ayraç çizgisi ekler.
- ◇ ****: Sayfaya görsel ekler (bkz. Bölüm 2.7).

- ◇ `<input>`: Formlarda kullanıcıdan veri alan giriş alanını oluşturur.
- ◇ `<meta>`: `head` içinde karakter kodlaması, sayfa açıklaması gibi üst verileri tanımlar.
- ◇ `<link>`: Harici bir kaynağı (ör. bir CSS dosyasını) belgeye bağlar.

§2.13.1 Bölüm Özeti

Bu bölümde, bir HTML belgesinin iskeletini kurmak için ihtiyaç duyacağınız temel kelime dağarcığını uçtan uca inceledik:

- ◇ HTML, tarayıcıya içeriğin **yapısını** bildiren bir işaretleme dilidir; etiketler birbirini iç içe, çakışmadan kapsayacak şekilde yazılır.
- ◇ Metni biçimlendirmek için `em`, `strong`, `b`, `i`, `mark`, `sub`/`sup` gibi satır içi etiketler kullanılır.
- ◇ Verileri düzenli sunmak için `ul`/`ol`/`dl` listeleri ve `table` tabloları kullanılır.
- ◇ `div` ve `span`, kendi başlarına anlam taşımayan gruplama etiketleridir; **semantik etiketler** (`header`, `nav`, `main`, `article`, `aside`, `footer`) ise sayfanın bölümlerine anlam katar.
- ◇ `form` ve alt elemanları (`input`, `select`, `textarea`, `button`) kullanıcıdan veri toplayıp sunucuya göndermeyi sağlar.
- ◇ `img`, `br`, `input` gibi bazı öğeler **boş öğedir** (*void element*); bitiş etiketi bulunmaz.

Buraya kadar sayfanın **yapısını** nasıl kuracağınızı öğrendiniz; sıradaki bölümde bu yapıya CSS ile görsel kimlik kazandırmayı, yani **sunumu** öğreneceksiniz.

2.14 Düşün ve Çöz

1. **Anlamsal HTML:** Bir haber sitesinin ana sayfasını (üst menü, haber listesi, kenar çubuğu, alt bilgi) yalnızca semantik etiketleri kullanarak nasıl iskeletlersiniz? `header`, `nav`, `main`, `article`, `aside` ve `footer` etiketlerini nerede kullanacağınızı belirtin.
2. **Blok/Satır İçi Ayrımı:** `div` ve `span` etiketleri arasındaki temel fark nedir? Bir alışveriş sepeti sayfasında hangi durumda `div`, hangi durumda `span` kullanmayı tercih edersiniz?
3. **Form Tasarımı:** Bir üye kayıt formunda ad, e-posta, doğum tarihi ve cinsiyet bilgisi toplamak istiyorsunuz. Bu alanlar için en uygun `input type` değerlerini seçin ve seçiminizi gerekçelendirin.
4. **Liste Seçimi:** Bir yemek tarifi sayfasında “gerekli malzemeler” ve “yapılış adımları” bölümleri için hangi liste türünü (`ul` ya da `ol`) kullanırsınız? Neden?
5. **Boş Öğeler:** `img`, `br` ve `input` etiketlerinin ortak özelliği nedir? Bu öğelerin neden bir bitiş etiketi yoktur?

2.15 Uygulamalı Projeler

Bu bölümde öğrendiğiniz etiketleri pekiştirmek için, yalnızca HTML kullanarak (henüz CSS veya JavaScript olmadan) aşağıdaki beş projeden birini veya birkaçını tamamlamayı deneyin. Amaç görsel bir tasarım değil, doğru etiketi doğru yerde kullanma alışkanlığı kazanmaktır.

1. **Kişisel Özgeçmiş (CV) Sayfası:** Ad-soyad için `h1`, bölüm başlıkları (Eğitim, Deneyim, Yetenekler) için `h2`, yetenekler için `ul`/`li`, profil fotoğrafı için `img`, iletişim bağlantıları için `a` (biri `mailto:` ile) kullanın; sayfayı bir `header` ve `footer` ile çerçeveleyin.
2. **Yemek Tarifi Sayfası:** Malzemeler için `ul`, hazırlama adımları için sıralı liste `ol`, tarifi görseli için `img`, bir şef notu için `blockquote`, hazırlama/pişirme süresi gibi bilgiler için küçük bir `table` kullanın.
3. **Film veya Kitap Kataloğu:** `table`, `thead`, `tbody`, `tr`, `th` ve `td` ile ad/yıl/tür/puan sütunlarından oluşan bir liste oluşturun; her satırdaki esere bir detay sayfasına giden `a` bağlantısı ekleyin.
4. **Üyelik / İletişim Formu:** `form`, `label` ve çeşitli `input type` değerleri (`text`, `email`, `password`, `date`, `radio`, `checkbox`) ile birlikte `select`/`option`, `textarea` ve `button` kullanarak bir kayıt formu tasarlayın.
5. **Haber Ana Sayfası:** `header` içinde bir `nav` menüsü, `main` içinde birkaç `article` (her biri kendi `h2`, `p` ve tarih için `small` içersin), bir `aside` (popüler yazılar listesi) ve bir `footer` kullanarak semantik bir sayfa iskeleti kurunuz.

§2.15.1 Blog Ana Sayfa Örneği

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Teknoloji Günlüğü - Ana Sayfa</title>
  </head>
  <body>
    <header>
      <h1>Teknoloji Günlüğü</h1>
      <nav>
        <a href="#">Ana Sayfa</a>
        <a href="#">Yazılım</a>
```

```

    <a href="#">Donanım</a>
    <a href="#">Hakkımızda</a>
  </nav>
</header>
<hr>
<main>
  <article>
    <h2>Web Programlamaya Yeni Başlayanlar İçin 5 Tavsiye</h2>
    <p><small>Yayın Tarihi: 23 Eylül 2026 | Yazar: Ayşe Yılmaz</small></p>
    
    <p>
      Web programlamaya yeni başlayanlar sıklıkla nereden başlayacaklarını
      bilemiyor. Bu yazıda, öğrenme sürecinizi kolaylaştıracak
      <strong>beş temel tavsiyeyi</strong> sizler için derledik.
    </p>
    <ol>
      <li>Önce HTML ve CSS ile sağlam bir temel oluşturun.</li>
      <li>Küçük projeler yaparak öğrendiklerinizi pekiştirin.</li>
      <li>Tarayıcının geliştirici araçlarını kullanmayı öğrenin.</li>
    </ol>
    <p><a href="#">Devamını oku &rarr;</a></p>
  </article>
  <article>
    <h2>Yarı İletken Sektöründeki Son Gelişmeler</h2>
    <p><small>Yayın Tarihi: 20 Eylül 2026 | Yazar: Mehmet Kaya</small></p>
    <p>
      Donanım dünyasında bu hafta öne çıkan gelişmeleri bir araya
      getirdik. <em>Üretici firmaların açıklamalarına</em> göre önümüzdeki
      yıl büyük bir değişim bekleniyor.
    </p>
    <blockquote>
      Bu, sektörün son on yıldaki en önemli dönüm noktalarından biri olabilir.
    </blockquote>
    <p><a href="#">Devamını oku &rarr;</a></p>
  </article>
</main>
<aside>
  <h2>Popüler Yazılar</h2>
  <ul>
    <li><a href="#">HTML Etiketlerine Hızlı Bakış</a></li>
    <li><a href="#">CSS Kutu Modeli Nedir?</a></li>
    <li><a href="#">İlk JavaScript Programınız</a></li>
  </ul>
</aside>
<hr>
<footer>
  <p>&copy; 2026 Teknoloji Günlüğü. Tüm hakları saklıdır.</p>
  <p>İletişim: <a
    ↪ href="mailto:iletisim@teknolojigunlugu.com">iletisim@teknolojigunlugu.com</a></p>
</footer>
</body>
</html>

```

3. Bölüm

CSS

Bölüm ??’de bir HTML belgesiyle bir web sayfasının **iskeletini** nasıl kurduğunuzu gördünüz: başlıklar, paragraflar, listeler, tablolar, formlar. Ancak o iskelet, tarayıcının varsayılan (ve oldukça sade) görünümüyle (siyah yazı, beyaz arka plan, küçük boşluklar) karşınıza çıkıyordu. Bu bölümde, aynı iskelete renk, düzen ve kişilik kazandıran dili öğreneceksiniz: **CSS**. Eğer HTML bir binanın betonarme iskeletiyse, CSS o binanın boyası, mobilyası ve iç mimarisidir; iskelet olmadan dekorasyonun bir anlamı olmaz, ama dekorasyon olmadan da bina yaşanması olmaz.

3.1 CSS Nedir ve Neden Kullanılır?

HTML (*HyperText Markup Language*), web sayfalarının anlamsal iskeletini ve verinin yapısını oluşturan bir işaretleme dilidir. Ancak HTML, verinin kullanıcıya estetik olarak nasıl sunulacağı (renk, font, düzen, hizalama, boyut vb.) hususunda tek başına yetersiz kalır. Web sayfalarının görsel sunumunu yönlendirmek ve yapıyla tasarımı birbirinden ayırmak amacıyla **Kademeli Stil Şablonları** yani **CSS** (*Cascading Style Sheets*) geliştirilmiştir.

CSS yalnızca standart HTML belgelerini biçimlendirmekle kalmaz; XML (*Extensible Markup Language*) tabanlı olan SVG (*Scalable Vector Graphics*), MathML (*Mathematical Markup Language*) ve XHTML (*Extensible HTML*) gibi diğer işaretleme dillerindeki verilerin görselleştirilmesinde de aktif olarak kullanılır.

3.2 HTML Belgelerine CSS Uygulama Yöntemleri

CSS kurallarını bir HTML belgesine entegre etmenin üç temel yolu vardır. Her yöntemin kendine özgü avantajları, kullanım senaryoları ve öncelik sıraları mevcuttur.

§3.2.1 Satır İçi Stil Kullanımı

En temel stil verme yöntemi, doğrudan HTML ögesinin içerisine **style** niteliğinin (*attribute*) yazılmasıdır. Yani satır içi CSS (*Inline CSS*) kullanımıdır.

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Satır İçi Stil Örneği</title>
  </head>
  <body>
    <h1 style="color: red; font-size: 24px;">Bölüm 1</h1>
    <p style="color: blue; line-height: 1.5;">Paragraf 1</p>
  </body>
</html>
```

Değerlendirme

Satır içi stiller, o anki öğeye en yüksek önceliği kazandırır. Ancak tasarım ile içerik birbirine karıştığı, kod tekrarına yol açtığı ve bakım zorluğu ürettiği için **modern web geliştirmede zorunlu kalmadıkça tercih edilmemelidir**.

§3.2.2 Dahili Stil Kullanımı

Biçimlendirme kurallarını metinden kısmen uzaklaştırmak amacıyla stiller, HTML belgesinin **<head>** bölümü içerisine yerleştirilen **<style>** etiketi altında toplanabilir (*Internal CSS*):

```
<!DOCTYPE html>
<html lang="tr">
  <head>
```

```

<meta charset="UTF-8">
<title>Dahili Stil Örneği</title>
<style>
  h1 {
    color: red;
    font-size: 24px;
  }
  p {
    color: blue;
    line-height: 1.5;
  }
</style>
</head>
<body>
<h1>Bölüm 1</h1>
<p>Paragraf 1</p>
</body>
</html>

```

Dahili stiller tek bir HTML sayfasını özelleştirmek için uygundur. Ancak proje büyüdükçe ve sayfa sayısı arttıkça aynı stillerin her HTML dosyasına tekrar yazılması gerekir.

§3.2.3 Harici Stil Kullanımı

Büyük ölçekli web projelerinde tüm sayfaların tek elden çıkmış gibi tutarlı bir tasarıma sahip olması istenir. Bu durumda stiller, tamamen bağımsız `.css` uzantılı dosyalarda (*External CSS*) toplanır ve HTML belgesine `<head>` bölümünde yer alan `<link>` etiketi ile bağlanır:

```

<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Harici Stil Örneği</title>
    <link rel="stylesheet" type="text/css" href="styles.css">
  </head>
  <body>
    <h1>Bölüm 1</h1>
    <p>Paragraf 1</p>
  </body>
</html>

```

Bağlanan harici `styles.css` dosyasının içeriği ise yalnızca CSS sözdiziminden oluşur:

```

/* styles.css */
h1 {
  color: red;
  font-size: 24px;
}
p {
  color: blue;
  line-height: 1.5;
}

```

`<link>` etiketinde geçen niteliklerin anlamları şunlardır:

- ◇ `rel="stylesheet"`: Çağrılan dosyanın HTML belgesi ile olan ilişkisini (*relationship*) belirtir; bu dosyanın bir stil şablonu olduğunu gösterir.
- ◇ `type="text/css"`: Dosyanın MIME türünü ifade eder (HTML5 ile kullanımı opsiyonel hale gelmiştir).
- ◇ `href="styles.css"`: Yüklenecek CSS dosyasının ağ adresini veya dosya yolunu (*hypertext reference*) belirtir.

3.3 CSS Sözdizimi

Şimdiye kadar CSS kurallarını üç farklı yerde (satır içi, dahili, harici) nasıl *barındırdığımızı* gördük. Ama bu kuralların kendisi nasıl yazılır? Her CSS kuralı, birbirini tamamlayan üç temel parçadan oluşur:

```

selector {
  property: value;
}

```

Bunu somut bir örnekle açalım:

```

h1 {
  color: red;
  font-size: 24px;
}

```

- ◊ **Seçici (Selector):** Stilin **hangi** HTML ögesine/öğelerine uygulanacağını belirtir. Yukarıdaki örnekte **h1**, sayfadaki tüm **<h1>** etiketlerini hedefler.
- ◊ **Bildirim Bloğu (Declaration Block):** Süslü parantezler **{ }** arasında yer alan ve seçilen öğeye uygulanacak tüm kuralları barındıran bölümdür.
- ◊ **Bildirim (Declaration):** Bir **özellik (property)** ile bir **değeri (value)** iki nokta üst üste **(:)** ile birbirine bağlayan tek bir kuraldır (Örn: **color: red**).
- ◊ **Noktalı Virgül (;):** Her bildirimin sonuna konur ve bir sonraki bildirimden ayrılmasını sağlar. Bloktaki son bildirimden sonra teknik olarak zorunlu değildir, ancak yeni bir kural eklerken hata yapmamak için her zaman yazılması tavsiye edilir.

§3.3.1 Yorum Satırları

HTML'de olduğu gibi CSS'te de, tarayıcı tarafından yok sayılan ama kaynak kodda görünen açıklama notları eklenebilir. CSS yorumları **/*** ile başlar ve ***/** ile biter:

```
/* Bu bir CSS yorumudur, tarayıcı bu satırı işlemez */
h1 {
    color: red; /* Ana başlık rengini kırmızı yapar */
}
```

3.4 CSS Seçicileri

Stil kurallarının hangi öğelere uygulanacağını belirlemek, CSS öğrenmenin en kritik adımıdır. Seçiciler (*Selectors*), kabaca genelden öze doğru sıralanabilir.

§3.4.1 Öğe Seçicisi

Belgedeki belirli türdeki **tüm** etiketleri (*tag*) hedefler.

```
p {
    color: darkgray;
}
```

Bu kural, sayfadaki her **<p>** etiketini seçer.

§3.4.2 Sınıf Seçicisi

HTML etiketlerine **class** niteliğiyle verilen isimleri hedefler; başına nokta (.) konur. Aynı sınıf adı, sayfadaki birden fazla farklı etikete verilebilir; bu sayede birbirinden bağımsız öğeler aynı stili paylaşabilir.

```
<p class="uyari">Dikkat, bu alan boş bırakılamaz.</p>
<span class="uyari">Şifre en az 8 karakter olmalıdır.</span>
.uyari {
    color: red;
    font-weight: bold;
}
```

§3.4.3 Kimlik Seçicisi

HTML etiketlerine **id** niteliğiyle verilen, belge içinde **benzersiz** (yalnızca bir kez kullanılması gereken) ismi hedefler; başına diyez (#) konur.

```
<header id="site-basligi">Kitap Kütüphanesi</header>
#site-basligi {
    background-color: navy;
    color: white;
}
```

Bir sayfada aynı **id** değeri yalnızca bir öğeye verilmelidir; tekrarlanan sınıflar (**class**) için ise böyle bir kısıtlama yoktur.

§3.4.4 Evrensel Seçici

Yıldız (*) işareti, belgedeki **tüm** öğeleri hedefler. Genellikle tarayıcıların öğelere varsayılan olarak uyguladığı iç/dış boşlukları sıfırlamak gibi genel ayarlar için kullanılır:

```
* {
    margin: 0;
    padding: 0;
}
```

§3.4.5 Gruplama Seçicisi

Birden fazla farklı seçiciye aynı kuralları tek seferde uygulamak için seçiciler virgülle (,) ayrılarak gruplanabilir; bu, aynı satırların tekrar tekrar yazılmasını önler.

```
h1, h2, h3 {
```

```
font-family: Arial, sans-serif;
}
```

§3.4.6 Torun ve Çocuk Seçicileri

Bir öğenin yalnızca **belirli bir kapsayıcı içindeyken** stillendirilmesi istenebilir. Torun (*descendant*) ve çocuk (*child*) seçicileri buna örnek verilebilir.

- ◇ **Torun Seçicisi** (aralarında boşluk): Belirtilen atanın **herhangi bir derinlikteki** tüm alt öğelerini seçer.

```
nav a {
  text-decoration: none;
}
```

Bu kural, yalnızca `<nav>` etiketinin içindeki `<a>` bağlantılarını hedefler; `<nav>` dışındaki bağlantıları etkilemez.

- ◇ **Çocuk Seçicisi** (`>` işareti): Yalnızca belirtilen atanın **doğrudan** (bir alt seviye) çocuklarını seçer, torununu değil.

```
ul > li {
  list-style-type: square;
}
```

§3.4.7 Nitelik Seçicisi

Belirli bir HTML niteliğine (*attribute*) veya nitelik değerine sahip öğeleri hedefler; köşeli parantez (`[]`) içinde yazılır.

```
input[type="password"] {
  border: 1px solid gray;
}
```

Bu kural, yalnızca `type` niteliği `"password"` olan `<input>` öğelerini hedefler; diğer `input` türlerini (`text`, `email` vb.) etkilemez.

§3.4.8 Sözde Sınıflar ve Sözde Öğeler

Bazı stiller, öğenin kalıcı bir niteliğine değil, o anki **durumuna** veya içeriğin **belirli bir parçasına** bağlıdır.

- ◇ **Sözde Sınıf** (*Pseudo-class*, **tek iki nokta** `:`): Öğenin belirli bir durumunu hedefler (Örn: `:hover` fare üzerine geldiğinde, `:focus` klavye odağı aldığında, `:first-child` bir kapsayıcının ilk çocuğu olduğunda).

```
button:hover {
  background-color: darkorange;
}
```

- ◇ **Sözde Öğe** (*Pseudo-element*, **çift iki nokta** `::`): Öğenin içeriğinin belirli bir bölümünü hedefler (Örn: `::first-line` bir paragrafın ilk satırı, `::before`/`::after` öğeden önce/sonra içerik eklemek için).

```
p::first-line {
  font-weight: bold;
}
```

Pedagojik Not: Seçicileri Bir Arada Görmek

Aşağıdaki tek satır, öğrendiğimiz seçicilerin birçoğunu bir arada gösterir:

```
nav.menu > li:hover a[target="_blank"]
```

Bu, "sınıfı `menu` olan bir `nav` öğesinin doğrudan çocuğu olan, üzerine gelindiğinde (`:hover`) etkinleşen bir `li` içindeki, `target` niteliği `_blank` olan `a` etiketlerini seç" anlamına gelir. İlk bakışta karmaşık görünse de, her parçası tek başına öğrendiğiniz basit kurallardan oluşur.

Artık bir stil kuralını nasıl yazacağımızı ve hangi öğeleri hedefleyeceğimizi biliyoruz. Peki aynı öğeye birden fazla, hatta birbiriyle çelişen kural uygulandığında tarayıcı hangisine öncelik verir? Bu sorunun cevabı, CSS'in adındaki **kademeli** (*cascading*) kelimesinde saklıdır.

3.5 CSS'te Kademeli Yapı ve Kalıtım

CSS teriminin başındaki **kademeli** (*cascading*) kelimesi, stillerin bir şelale gibi yukarıdan aşağıya doğru aktarılması ve belgedeki öğelere uygulanma sırasını ifade eder.

§3.5.1 Kalıtım Mantiği

Aksi belirtilmedikçe, tarayıcının oluşturacağı DOM ağacında üst hiyerarşide (kök öğeye yakın) yer alan bir etikete uygulanan stil kuralları, o etiketin kapsadığı tüm alt etiketlere (*child elements*) **kademeli** olarak miras kalır.

Örneğin; `<body>` etiketine verilen bir yazı rengi, özel bir kural tanımlanmadığı sürece içerisindeki tüm `<h1>`, `<p>`, `<span>` öğeleri tarafından devralınır:

```
<body style="color: darkgreen;">
<h1>Bu başlık da yeşil görünür</h1>
<p>Bu paragraf da yeşil renk mirasını alır.</p>
</body>
```

§3.5.2 Seçici Özgüllüğü ve Öncelik Sırası

Aynı HTML ögesine birden fazla çelişen stil kuralı uygulandığında tarayıcı, **özellik** (*specificity*) hesabına göre hangi kuralın geçerli olacağına karar verir. Öncelik hiyerarşisi en yüksekte en düşüğe şu şekildedir:

1. **!important** kuralı (Varsayılan öncelik sırasını ezerek en üst sıraya geçer).
2. Satır İçi Stilleri (*Inline CSS*) — HTML etiketi içindeki `style="..."` niteliği.
3. ID Seçicileri (Örn: `#ana-baslik`).
4. Sınıf (*Class*), Nitelik (*Attribute*) ve Pseudo-class Seçicileri (Örn: `.baslik`, `[type="text"]`).
5. Öge (*Element/Tag*) Seçicileri (Örn: `h1`, `p`, `div`).

Eğer iki kural aynı özgüllük puanına sahipse, CSS dosyasında **en son yazılan** (en alttaki) kural geçerli olur.

Tasarın ve İçeriğin Birbirinden Ayrılması

Tasarım ve içeriğin birbirinden ayrılması (*Separation of Concerns*) ilkesi gereğince, üretim ortamındaki projelerde her zaman **Harici CSS** kullanımı tercih edilmelidir. Bu yöntem; belgenin önbelleğe alınmasını (*caching*) kolaylaştırır, sayfa yüklenme sürelerini azaltır ve tüm web sitesinin tek bir merkezden yönetilmesini sağlar.

Buraya kadar hangi kuralın ne zaman kazandığını öğrendik; şimdi bu kurallara yazabileceğimiz somut *değerler* – renklere ve ölçü birimlerine – bakalım.

3.6 Renkler ve Ölçü Birimleri

Bir CSS bildiriminin sağ tarafında (*value*) yazabileceğimiz değerlerin biçimini bilmek, tasarımı hayata geçirebilmek için gereklidir.

§3.6.1 Renk Belirtme Yöntemleri

CSS'te bir rengi ifade etmenin birden çok yolu vardır:

- ♦ **İsimli Renkler (Named Colors):** Tarayıcıların tanıdığı hazır renk isimleridir (Örn: `red`, `blue`, `navy`, `transparent`).
- ♦ **Onaltılık (Hexadecimal):** Kırmızı, yeşil ve mavi (*RGB*) bileşenlerinin her birini iki onaltılık basamakla ifade eder; `#` işaretiyle başlar (Örn: `#FF0000` kırmızı, `#000000` siyah, `#FFFFFF` beyaz).
- ♦ **RGB:** Kırmızı, yeşil, mavi bileşenlerini 0-255 arasında sayılarla ifade eder (Örn: `rgb(255, 0, 0)` kırmızı).
- ♦ **RGBA:** RGB'ye ek olarak bir de saydamlık (*alpha*) değeri (0 ile 1 arasında) alır (Örn: `rgba(255, 0, 0, 0.5)` yarı saydam kırmızı).
- ♦ **HSL:** Rengi ton (*hue*, 0-360 derece), doygunluk (*saturation*, %) ve parlaklık (*lightness*, %) olarak ifade eder; insan gözünün renk algısına daha yakın, sezgisel bir yöntemdir (Örn: `hsl(0, 100%, 50%)` kırmızı).

```
h1 {
  color: #2c3e50;           /* Onaltılık */
  background-color: rgb(236, 240, 241); /* RGB */
  border-color: rgba(0, 0, 0, 0.2);    /* Yarı saydam siyah */
}
```

§3.6.2 Ölçü Birimleri

Boyut, boşluk ve konum değerleri belirtilirken kullanılan başlıca birimler mutlak ve göreceli olmak üzere ikiye ayrılır:

- ♦ **px (piksel):** Ekrandaki sabit bir nokta sayısını ifade eden mutlak bir birimdir; ekran boyutundan veya kullanıcı ayarlarından etkilenmez.
- ♦ **% (yüzde):** Üst (ebeveyn) ögenin ilgili boyutuna göre göreceli bir orandır (Örn: genişliği `%50` olan bir kutu, kapsayıcısının yarısı kadar yer kaplar).
- ♦ **em:** O öğeye (veya bulunmuyorsa üst öğeye) uygulanan yazı tipi boyutuna göre göreceli bir birimdir (Örn: yazı boyutu `16px` olan bir öğede `2em`, `32px` anlamına gelir).
- ♦ **rem (root em):** `em`'e benzer, ancak her zaman kök ögenin (`<html>`) yazı tipi boyutuna göre hesaplanır; bu nedenle iç içe öğelerde `em`'in yol açabileceği katlanarak büyüme sorununu ortadan kaldırır ve modern projelerde tercih edilir.
- ♦ **vw / vh (viewport width/height):** Tarayıcı penceresinin görünür genişliğinin veya yüksekliğinin yüzde birine karşılık gelir (Örn: `50vw`, ekran genişliğinin yarısıdır).

Renkleri ve ölçü birimlerini artık biliyoruz; peki bir HTML ögesi sayfada tam olarak ne kadar yer kaplar ve bu

alan nasıl hesaplanır? Bu sorunun cevabı, CSS'in en temel modellerinden biri olan Kutu Modeli'nde saklıdır.

3.7 Kutu Modeli

Tarayıcı, sayfadaki her HTML öğesini – görünmez de olsa – dikdörtgen bir **kutu** olarak render eder. Bu kutunun toplam boyutunu belirleyen dört katman, içten dışa doğru şu şekildedir:

1. **İçerik (Content)**: Metnin, görselin veya diğer öğelerin bulunduğu asıl alandır; **width** ve **height** özellikleriyle boyutlandırılır.
2. **İç Boşluk (Padding)**: İçerik ile kutunun kenarlığı arasındaki boşluktur; kutunun arka plan rengini de kapsar.
3. **Kenarlık (Border)**: Kutuyu çevreleyen çizgidir; kalınlık, stil ve renk belirtilir (Örn: **1px solid black**).
4. **Dış Boşluk (Margin)**: Kutunun kendisi ile komşu öğeler arasındaki, kutunun **dışında** kalan ve şeffaf olan boşluktur.

```
.kutu {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

Kutu Modeli (*Box Model*) bir elemanın içten dışa doğru boyutlarını (**Content**, **Padding**, **Border**, **Margin**) belirlerken; Sayfa Düzeni (Layout) bu kutuların ekranda birbirine göre nasıl konumlanacağını yönetir. CSS'te kutuları yan yana, alt alta veya iç içe yerleştirmek için kullanılan temel yaklaşım ve yöntemler başlıklar halinde aşağıda verilmiştir.

§3.7.1 Kutu Boyutlandırma Hesabı

Sayfa düzeninde en sık yaşanan taşma ve kayma sorunlarının önüne geçmek için ilk kural **box-sizing** ayarıdır.

- ◊ **content-box** (Varsayılan): Genişlik (**width**) sadece içeriği kapsar. **padding** ve **border** genişliğe eklenir. (Örn: $\text{width: } 200\text{px} + \text{padding: } 20\text{px} = 240\text{px}$ toplam genişlik).
- ◊ **border-box** (Önerilen): **padding** ve **border** verilen genişliğin içine dahil edilir. Eleman tam olarak belirlendiğiniz boyutta kalır.

```
/* Tüm projede öngörülebilir kutu boyutları için varsayılan sıfırlama */
*, *::before, *::after {
  box-sizing: border-box;
}
```

Kutunun Gerçek Genişliği

Varsayılan kutu modelinde (**box-sizing: content-box**), **width** yalnızca **içerik** alanının genişliğini belirtir; **padding** ve **border** bu genişliğe **eklenir**. Yukarıdaki örnekte kutunun tarayıcıda kapladığı gerçek genişlik $200 + (20 \times 2) + (5 \times 2) = 250$ piksel olur – 200 değil! Bu şaşırtıcı davranışı önlemek için modern projelerde genellikle şu kural tüm sayfaya uygulanır:

```
* {
  box-sizing: border-box;
}
```

border-box modelinde ise **padding** ve **border**, belirtilen **width** değerinin **içine** dahil edilir; yani yukarıdaki kutu artık gerçekten de tam 200 piksel genişliğinde olur.

§3.7.2 Temel Konumlandırma Yöntemleri

Kutuların akış içindeki davranışını belirleyen 4 ana düzen yaklaşımı vardır:

Yöntem	Ne Zaman Kullanılır?	Öne Çıkan Özelliği
Normal Akış (Normal Flow)	Varsayılan metin ve blok yerleşimi	Blok elemanlar (div, p) alt alta; satır içi elemanlar (span, a) yan yana dizilir.
Flexbox (1D Düzen)	Tek yönlü (sadece satır veya sadece sütun) hizalama	Esnek genişlik, dikey/yatay ortalama ve eleman aralıklarını otomatik dağıtma.
CSS Grid (2D Düzen)	İki boyutlu (satır + sütun) karmaşık sayfa düzenleri	İzgara yapısı oluşturup kutuları istenen hücrelere tam hassasiyetle yerleştirme.
Positioning (Mutlak Konum)	Özel/Serbest konumlandırma (absolute, fixed)	Akıştan bağımsız olarak çakışan veya sabitlelen kutular oluşturma.

Tablo 3.1: Temel Konumlandırma Yöntemleri

§3.7.3 Uygulamalı Düzen Senaryoları

Kutuları İç İçe Yerleştirme

İç içe kutularda dış kutunun iç çeperinden boşluk bırakmak için `padding`, içteki kutunun dış çeperine mesafe koymak için `margin` kullanılır.

```
.dis-kutu {
  width: 300px;
  padding: 20px; /* İçerideki elemanlar kenara yapışmaz */
  background-color: #f0f0f0;
}

.ic-kutu {
  width: 100%; /* Dış kutunun padding hariç genişliğini kaplar */
  margin-bottom: 10px; /* Bir sonraki alt kutuyla arasına mesafe koyar */
  background-color: #ffffff;
}
```

Kutuları Yan Yana Dizme

Kutuları yan yana yerleştirmek için ebeveyn elemana `display: flex` verilmesi yeterlidir (*Flexbox*).

```
.container {
  display: flex;
  flex-direction: row; /* Kutuları yan yana dizer */
  justify-content: space-between; /* Kutuların arasına eşit boşluk koyar */
  align-items: center; /* Kutuları dikeyde ortalar */
  gap: 15px; /* Kutular arasına sabit boşluk ekler */
}

.kutu {
  flex: 1; /* Tüm kutular kalan alanı eşit paylaşır */
}
```

Karmaşık Izgara Düzeni

Birden fazla satır ve sütundan oluşan sayfa tasarımlarında kullanılır (*Grid*).

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 eşit sütun oluşturur */
  gap: 20px; /* Kutular arası dikey/yatay boşluk */
}
```

Özel Konumlandırma

Bir kutuyu diğer kutuların üzerine bindirmek veya ekranda sabitlemek için kullanılır (*position*).

- ◊ `static`: Varsayılan akış konumudur.
- ◊ `relative`: Elemanı kendi normal konumuna göre kaydırır, akıştaki yerini korur.
- ◊ `absolute`: Akıştan tamamen çıkar. `position: relative` verilmiş en yakın ata elemanına göre konumlanır.
- ◊ `fixed`: Tarayıcı penceresine göre konumlanır; sayfa kaysa da sabitlelen noktada kalır (Örn: Sabit menüler).

```
/* Dış kutuyu referans noktası yap */
.kart {
  position: relative;
  width: 250px;
  height: 150px;
}

/* Sağ üst köşeye Rozet/Etiket iliştir */
.rozet {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

§3.7.4 Bir kutuyu yatayda ve dikeyde ortalama

CSS'te bir kutuyu yatayda ve dikeyde (tam merkezde) ortalama geçmişte zorlu teknikler gerektirse de, modern CSS standartları sayesinde artık çok basittir. Günümüzde en sık kullanılan ve modern kabul edilen 3 temel ortalama yöntemi bulunmaktadır:

CSS Grid ile Tek Satırda Ortalama

En modern ve pratik yöntemdir. Ebeveyn (*parent*) kapsayıcıya sadece 2 satır CSS ekleyerek içerisindeki tüm çocuk (*child*) elemanları tam merkeze yerleştirebilirsiniz.

```
.kapsayici {
  display: grid;
  place-items: center; /* Hem yatay (justify) hem dikey (align) ortalar */
  min-height: 100vh; /* Kapsayıcının tüm ekran yüksekliğini kaplaması için */
}
.kutu {
  /* Ekstra konumlandırma kodu gerekmez */
  width: 200px;
  height: 200px;
  background-color: #4CAF50;
}
```

Esnek Düzen Yöntemi

En yaygın kullanılan yöntemdir. Esnek düzen (*Flexbox*) kullanarak yatay ve dikey eksenlerdeki hizalamayı ayrı ayrı kontrol edebilirsiniz.

```
.kapsayici {
  display: flex;
  justify-content: center; /* Yatayda ortalar */
  align-items: center; /* Dikeyde ortalar */
  min-height: 100vh;
}
.kutu {
  width: 200px;
  height: 200px;
  background-color: #2196F3;
}
```

Esnek düzen alternatifi olarak ebeveyne `display: flex` verip, ortalanan kutunun kendisine `margin: auto` yazarak da aynı sonucu elde edebilirsiniz;

```
.kapsayici {
  display: flex;
  min-height: 100vh;
}
.kutu {
  margin: auto; /* Tüm boşluğu eşit paylaştırarak merkeze oturur */
}
```

Absolute Position + Transform Yöntemi

Eğer kutuyu normal sayfa akışından çıkarıp (`position: absolute`), belirli bir ana elemanın üstünde bağımsız olarak ortalamak istiyorsanız bu yöntem kullanılır.

```
.kapsayici {
  position: relative; /* Kutunun referans alacağı kapsayıcı */
  min-height: 100vh;
}
.kutu {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%); /* Kendi genişlik/yüksekliğinin yarısı kadar geri kaydırır */
  width: 200px;
  height: 200px;
  background-color: #9C27B0;
}
```

Özetle Hangisini Seçmeliyiz?

- ◇ CSS Grid (`place-items: center`): Sayfa düzeninde hızlıca tek bir kutuyu veya kapsayıcıyı ortalamak istiyorsanız en modern ve en kısa yoldur.
- ◇ Flexbox (`justify-content` ve `align-items`): Yan yana veya alt alta dizilen birden fazla elemanın merkezlenmesini yönetiyorsanız en esnek yoldur.
- ◇ Absolute + Transform: Popup, modal veya kart üzeri etiket/rozet gibi diğer elemanların üzerine katman olarak binen öğeleri ortalamak için idealdir.

Bölüm Özeti

Bu bölümde, HTML ile kurduğumuz iskelete görsel kimlik kazandıran CSS dilini baştan sona inceledik:

- ◇ CSS, bir HTML belgesine **satır içi**, **dahili** veya **harici** olmak üzere üç farklı yolla uygulanabilir; büyük projelerde **harici CSS** tercih edilir.
- ◇ Her CSS kuralı bir **seçici** ve süslü parantez içindeki **bildirimlerden** (özellik: değer;) oluşur.
- ◇ Seçiciler; öğe, sınıf (.), kimlik (#), evrensel (*), torun/çocuk ve nitelik seçicileri gibi farklı düzeylerde hedefleme sağlar.
- ◇ Aynı öğeye çelişen kurallar uygulandığında tarayıcı, **özgüllük** (*specificity*) hesabına göre bir kazanan belirler; eşitlik durumunda en son yazılan kural geçerli olur.
- ◇ Üst öğelere uygulanan bazı stiller, **kalıtım** yoluyla alt öğelere otomatik olarak geçer.
- ◇ Renkler; isim, onaltılık, RGB(A) veya HSL ile, ölçüler ise `px`, `%`, `em`, `rem`, `vw`/`vh` gibi birimlerle ifade edilir.
- ◇ Her HTML öğesi tarayıcıda bir **kutu** olarak render edilir; bu kutunun toplam boyutu içerik, iç boşluk, kenarlık ve dış boşluk katmanlarının toplamıdır.

Buraya kadar sayfanın **yapısını** nasıl kuracağınızı ve biçimlendirmesini nasıl yapacağımızı kabaca öğrendik. CSS Biçimlendirmelerini W3Schools adresinde çevrimiçi deneme yaparak daha detaylı öğrenebilirsiniz¹. Bir sonraki bölümde bu kutuları sayfa ile kullanıcı arasındaki etkileşimi sağlayan JavaScript dilini öğreneceksiniz.

3.8 Düşün ve Çöz

1. **Uygulama Yöntemi Seçimi:** Beş sayfalık bir kurumsal web sitesinde tüm sayfaların aynı renk paletini ve yazı tipini kullanması isteniyor. Satır içi, dahili ve harici CSS yöntemlerinden hangisini tercih edersiniz? Bakım kolaylığı açısından gerekçelendiriniz.
2. **Özgüllük Hesabı:** Aşağıdaki üç kural aynı `<p id="giris" class="vurgu">` öğesini hedeflemektedir:

```
p { color: black; }
.vurgu { color: blue; }
#giris { color: red; }
```

Tarayıcı bu paragrafı hangi renkte gösterir? Özgüllük sıralamasına göre açıklayınız.

3. **Seçici Yazımı:** Yalnızca bir `<article>` etiketinin **doğrudan çocuğu** olan `<p>` etiketlerini (torunlarını değil) hedefleyen bir CSS seçicisi yazınız.
4. **Kutu Modeli Hesabı:** `width: 300px`, `padding: 15px` ve `border: 5px solid` değerlerine sahip bir kutunun, `box-sizing` özelliği (a) `content-box` ve (b) `border-box` olarak ayarlandığında tarayıcıda kaplayacağı toplam genişliği hesaplayınız.
5. **Birim Seçimi:** Bir web sitesinde kullanıcı, tarayıcı ayarlarından yazı tipi boyutunu büyütüyor. Sitedeki tüm yazı boyutlarının bu değişikliğe orantılı şekilde tepki vermesini istiyorsanız `px` yerine hangi birimi tercih etmelisiniz? Nedenini açıklayınız.

3.9 Uygulamalı Projeler

Bu bölümde öğrendiğiniz seçicileri, renkleri ve kutu modelini pekiştirmek için, HTML ile CSS'i (henüz JavaScript olmadan) bir arada kullanarak aşağıdaki beş projeden birini veya birkaçını tamamlamayı deneyin.

1. **Basit Bir Profil Kartı:** Bölüm 2'deki CV sayfanızı ele alın; kartı bir `div` içine alıp `padding`, `border` ve `margin` ile kutu modelini uygulayın, isim için farklı bir `color` ve `font-size` belirleyin.
2. **Stilize Edilmiş Bir Navigasyon Menüsü:** `ul/li` ile kurduğunuz bir menüde bağlantıların (a) alt çizgisini kaldırın, arka plan rengi ve iç boşluk (`padding`) verin, `:hover` sözde sınıfıyla üzerine gelindiğinde renk değiştiren bir etkileşim ekleyin.
3. **Blog Sayfasını Giydirme:** Bölüm 2'de HTML ile oluşturduğunuz blog ana sayfasını CSS ile stillendirin: `header` ve `footer`'a farklı arka plan renkleri, her `article` kutusuna kenarlık ve iç boşluk, `aside` kutusuna daha soluk bir arka plan rengi verin.
4. **Basit Bir Uyarı/Bilgi Kutusu Seti:** Sınıf seçicileriyle (`.basari`, `.hata`, `.uyari`) farklı renklerde üç ayrı bilgi kutusu tasarlayın; her kutu için `box-sizing: border-box` kullanarak genişliklerinin tutarlı kalmasını

¹<https://www.w3schools.com/css/default.asp>

sağlayın.

5. **Stilize Edilmiş Bir Form:** Bölüm 2'deki üyelik formunuza; `input` alanlarına kenarlık ve iç boşluk, odaklandığında (`:focus`) renk değiştiren bir kenarlık ve gönder butonuna `:hover` efekti ekleyin.

§3.9.1 Blog Ana Sayfa CSS Örneği

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Teknoloji Günlüğü - Ana Sayfa</title>
    <style>
      body {
        font-family: Arial, sans-serif;
        margin: 0;
        padding: 0;
        background-color: #f4f4f4;
        color: #2c3e50;
      }

      header {
        background-color: #2c3e50;
        color: white;
        padding: 20px;
      }

      header h1 {
        margin: 0;
      }

      nav a {
        color: white;
        text-decoration: none;
        margin-right: 15px;
      }

      nav a:hover {
        color: #f39c12;
      }

      main {
        padding: 20px;
      }

      article {
        background-color: white;
        border: 1px solid #ddd;
        padding: 15px;
        margin-bottom: 20px;
      }

      article h2 {
        color: #2c3e50;
      }

      blockquote {
        border-left: 4px solid #f39c12;
        padding-left: 15px;
        color: #555;
        font-style: italic;
      }

      aside {
        background-color: #ecf0f1;
        border: 1px solid #ddd;
        padding: 15px;
        margin: 0 20px 20px 20px;
      }
```

```

    }

    footer {
        background-color: #2c3e50;
        color: white;
        padding: 15px;
        text-align: center;
    }

    footer a {
        color: #f39c12;
    }
</style>
</head>
<body>

    <header>
        <h1>Teknoloji Günlüğü</h1>
        <nav>
            <a href="#">Ana Sayfa</a>
            <a href="#">Yazılım</a>
            <a href="#">Donanım</a>
            <a href="#">Hakkımızda</a>
        </nav>
    </header>

    <main>

        <article>
            <h2>Web Programlamaya Yeni Başlayanlar İçin 5 Tavsiye</h2>
            <p><small>Yayın Tarihi: 23 Eylül 2026 | Yazar: Ayşe Yılmaz</small></p>
            
            <p>
                Web programlamaya yeni başlayanlar sıklıkla nereden başlayacaklarını
                bilemiyor. Bu yazıda, öğrenme sürecinizi kolaylaştıracak
                <strong>beş temel tavsiyeyi</strong> sizler için derledik.
            </p>
            <ol>
                <li>Önce HTML ve CSS ile sağlam bir temel oluşturun.</li>
                <li>Küçük projeler yaparak öğrendiklerinizi pekiştirin.</li>
                <li>Tarayıcının geliştirici araçlarını kullanmayı öğrenin.</li>
            </ol>
            <p><a href="#">Devamını oku &rarr;</a></p>
        </article>

        <article>
            <h2>Yarı İletken Sektöründeki Son Gelişmeler</h2>
            <p><small>Yayın Tarihi: 20 Eylül 2026 | Yazar: Mehmet Kaya</small></p>
            <p>
                Donanım dünyasında bu hafta öne çıkan gelişmeleri bir araya
                getirdik. <em>Üretici firmaların açıklamalarına</em> göre önümüzdeki
                yıl büyük bir değişim bekleniyor.
            </p>
            <blockquote>
                Bu, sektörün son on yıldaki en önemli dönüm noktalarından biri olabilir.
            </blockquote>
            <p><a href="#">Devamını oku &rarr;</a></p>
        </article>

    </main>

    <aside>
        <h2>Popüler Yazılar</h2>
        <ul>
            <li><a href="#">HTML Etiketlerine Hızlı Bakış</a></li>
            <li><a href="#">CSS Kutu Modeli Nedir?</a></li>
        </ul>
    </aside>

```

```
        <li><a href="#">İlk JavaScript Programınız</a></li>
    </ul>
</aside>

<footer>
    <p>&copy; 2026 Teknoloji Günlüğü. Tüm hakları saklıdır.</p>
    <p>İletişim: <a
        ↪ href="mailto:iletisim@teknolojigunlugu.com">iletisim@teknolojigunlugu.com</a></p>
</footer>

</body>
</html>
```

4. Bölüm

Program ve Programlama

4.1 Programlama Dillerinin Tarihsel Gelişimi

§4.1.1 Düşük Düzey Programlama

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

İlk Programcı

Augusta Ada Byron, ilk programcı olarak kabul edilir.

1854 Yılında *George Boole*, doğru/yanlış (1/0) değerleri üzerinde çalışan mantıksal işlemleri matematiksel olarak formüleştirdiği **Boole cebirini** (*Boolean algebra*) geliştirmiştir. İkili sayı sisteminin kendisi çok daha eskiye (Leibniz, 17. yüzyıl) dayansa da bugün bilgisayarlarımızın tüm mantıksal devrelerinin temelini oluşturan VE, VEYA, DEĞİL işlemleri doğrudan Boole'un bu çalışmasına dayanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde **ENIAC** (*Electronic Numerical Integrator and Computer*) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. Neumann tarafından geliştirilmiş bu mimari (*Stored-program computer and Universal Turing machine § Stored-program computer*), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m² yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. MARK-I bilgisayarından bin kat daha hızlıydı.

Bu yıllardan sonra geliştirilen bilgisayarlarda; Sisteme ilk elektrik verildiği (sistemin başlatıldığı) andan itibaren işlemci (**CPU**), temel bir yaşam döngüsüne girer. İşlemci, elektrik kesilene veya sistem kapatılana kadar mikro saniyeler içinde milyarlarca kez tekrarlanan üç adımlık bellekten **Emir Alma** (*fetch*), alınan emri **Çözme** (*decode*) ve emri **İcra Etme** (*execute*) döngüsünü yürütür. Bilgisayar biliminde bu sonsuz döngüye **Emir Çevrimi** (*instruction cycle*) veya **Makine Çevrimi** (*machine cycle*) adı verilir.

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi emre (instruction), **bilgisayar programı** (*computer program*) denir. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlanması işlemine **programlama** (*programming*) denir. Programlama **emir kodları** (*instruction code*) ile yapıldığından bu programın dili, **makine dili** (*machine language*) olarak adlandırılır¹.

Düşük Düzey Programlama

Emir kodu ve hatırlatıcı isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama** (*low level programming*) olarak adlandırılır. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır. Bu programlama türünde hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

1943-1945 Yılları arasında *Konrad Zuse*, ilk yüksek düzey programlama dili tasarımlarından biri olan Plankalkül'ü geliştirmiştir; dil, döneminin **Z3** bilgisayarı düşünülerek tasarlanmış olsa da o yıllarda hiçbir makinede fiilen çalıştırılmamış, ilk gerçek uygulaması ancak 2000 yılında yapılabilmektedir.

¹ISO/IEC 2382:2015. ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system

Yüksek Düzey Programlama Dilleri

Emir kodlarını (*instruction code*) ve hatırlatıcı isimleri (*mnemonic*) içermeyen programlama dilleri, **yüksek düzey programlama dili** (*high level programming language*) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

§4.1.2 Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (*high level programming language*) olarak adlandırılır. Yani emir kodu (*instruction code*) ve hatırlatıcı isim (*mnemonic*) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (*statement*) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla emirden (*instruction*) oluşur.

Talimat

Yüksek seviyeli bir dilde talimatlar, yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (*logical sequence*). Talimatlar, anahtar kelimeler kullanılarak yazılır.

1954 Yılında **FORTRAN** (*FORmula TRANslation*), *John Backus* liderliğinde IBM 704 için geliştirilirdi. FORTRAN, **READ**, **WRITE**, **IF** gibi 32 farklı talimata (*statement*) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir. Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir².

```

1 C AREA OF A TRIANGLE WITH A STANDARD SQUARE ROOT FUNCTION
2 C INPUT - TAPE READER UNIT 5, INTEGER INPUT
3 C OUTPUT - LINE PRINTER UNIT 6, REAL OUTPUT
4 C INPUT ERROR DISPLAY ERROR OUTPUT CODE 1 IN JOB CONTROL LISTING
5 C READ INPUT TAPE 5, 501, IA, IB, IC
6 read(*,*) IA, IB, IC
7 C 501 FORMAT (3I5)
8 C IA, IB, AND IC MAY NOT BE NEGATIVE OR ZERO
9 C FURTHERMORE, THE SUM OF TWO SIDES OF A TRIANGLE
10 C MUST BE GREATER THAN THE THIRD SIDE, SO WE CHECK FOR THAT, TOO
11 IF (IA) 777, 777, 701
12 701 IF (IB) 777, 777, 702
13 702 IF (IC) 777, 777, 703
14 703 IF (IA+IB-IC) 777, 777, 704
15 704 IF (IA+IC-IB) 777, 777, 705
16 705 IF (IB+IC-IA) 777, 777, 799
17 777 STOP 1
18 C USING HERON'S FORMULA WE CALCULATE THE
19 C AREA OF THE TRIANGLE
20 799 S = FLOAT(IA + IB + IC) / 2.0
21 AREA = SQRT( S * (S - FLOAT(IA)) * (S - FLOAT(IB)) *
22 + (S - FLOAT(IC)))
23 C WRITE OUTPUT TAPE 6, 601, IA, IB, IC, AREA
24 write(*, 601) IA, IB, IC, AREA
25 601 FORMAT (4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
26 + 13H SQUARE UNITS)
27 END

```

Örnekte görüldüğü üzere FORTRAN II dilinde her talimat metnin altıncı sütunundan başlar ve 72. sütununda biter. Altıncı sütuna kadar olan yerlere numara ile satır etiketi konulabilir (12-17, 20 ve 25. satırlar). Eğer talimat bu aralığa sığmaz ise beşinci sütuna "+" karakteri konularak (22 ve 26. satırlar) talimata devam edilir. Kodun içine açıklamalar "C" karakteri ile başlayan satırlarla konulur. Kod içinde yer alan talimatlar kısa öz ve anlaşılırdır.

§4.1.3 Arapsaçı Kod

1955-1959 Yılları arasında, geliştirme sürecinde **B-0** (*Business Language, Version 0*) adıyla anılan ve daha sonra **FLOW-MATIC** adıyla piyasaya sürülen dil, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda bu dilde örnek bir program verilmiştir³.

²https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

³<https://gist.github.com/marcgg/f9f2da31a973ba319809>

```
(0) INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1) COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2) TRANSFER A TO D .
(3) WRITE-ITEM D .
(4) JUMP TO OPERATION 8 .
(5) TRANSFER A TO C .
(6) MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7) WRITE-ITEM C .
(8) READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9) JUMP TO OPERATION 1 .
(10) READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11) JUMP TO OPERATION 1 .
(12) SET OPERATION 9 TO GO TO OPERATION 2 .
(13) JUMP TO OPERATION 2 .
(14) TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
OTHERWISE GO TO OPERATION 15 .
(15) REWIND B .
(16) CLOSE-OUT FILES C ; D .
(17) STOP . (END)
```

Örnekte görüldüğü üzere talimatlar parantez içine yazılan satır numaraları ile başlar ve nokta karakteri ile biter. Böylece FORTRAN'dan daha kolay birden fazla metin satırından oluşan talimatlar yazılabilmektedir.

Arapsaçı Kod

Bu döneme kadar yayılan programlamada, bir sürü **GO TO** ya da **JUMP TO** talimatı kullanıldığından, talimatların hangi sırada çalıştığının yani **program akışının** (*control flow*) anlaşılması zordur. Bu nedenle programda **okunaklılık oldukça azdır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (*spaghetti code*) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

1958 Yılındaki FORTRAN diline çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, değer üreten fonksiyonlar için **FUNCTION**, **CALL** ve **RETURN** özellikleri eklenmiş ve **FORTRAN-II** adını almıştır. 1966 Yılında ise FORTRAN-IV diline **INTEGER**, **REAL**, **DOUBLE PRECISION**, **COMPLEX** ve **LOGICAL** veri tipleri (*data type*) ile Mantıksal **IF**, aritmetik (üç yollu) **IF** ve **DO** döngüsü talimatları (*statement*) eklenmiştir.

4.2 Temel Programlama Kavramları: Değişken ve Fonksiyon

§4.2.1 Değişken Nedir?

Değişkenler (*variable*), en temel tanımıyla cebirde kullandığımız soyut sembollerin bilgisayar bilimindeki doğrudan karşılığıdır. Matematikte değişken, en genel tanımıyla, belirli bir evren veya tanımlı bir küme içerisindeki herhangi bir öğeyi temsil eden ve süreç boyunca farklı öğeleri de temsil eden sembolik bir niceliktir. Matematiksel işlemlerde ve algoritmalarda doğrudan sabit sayıları kullanmak yerine, o an hangi değere sahip olacağı değişebilen, ancak formülün mantıksal yapısını koruyan sembolik ifadeler tercih edilir.

$$\begin{aligned} 3x^2 + 2x + 10 \\ c^2 = a^2 + b^2 \\ c = 2\pi r \end{aligned}$$

Yukarıda verilen cebirsel ifadelerin ilkinde matematiksel bir polinom, ikincisinde bir dik üçgenin kenar uzunlukları arasındaki geometrik ilişki (Pisagor Teoremi), üçüncüsünde ise bir çemberin çevre hesabı gösterilmiştir. Bu fonksiyonel ifadelerde yer alan x , a , b ve r değerleri dışarıdan bağımsız olarak değişebildiği için **bağımsız değişken**; bu girdilere bağlı olarak nihai sonucu üreten c ise **bağımlı değişkendir**. Formüllerdeki 3,2,10 sayıları ile matematiksel bir sabit olan π sayısı ise sürecin hiçbir aşamasında değişmeyen **sabit** (*constant*) değerlerdir.

Değişkenlerin Bellek Karşılığı

Değişkenler, bilgisayarlarda fiziksel bir miktar bellek bölgesini (adresini) işgal eder. Yüksek seviyeli dillerde (Python, C++, Java vb.) bir değişken tanımladığımızda, arka planda işletim sistemi ve derleyici, o değişkenin türüne göre ana bellekten (RAM) belirli sayıda bayt boyutunda bir alanı ayırır (rezerve eder). Değişkenin ismi, o bellek kutusunun üzerindeki etikettir; içindeki değer ise kutunun muhteviyatıdır.

Bilgisayar mimarisinde veriler, yapısal özelliklerine göre belleğe farklı matematiksel formatlarda kaydedilirler. Bu formatlar temel olarak ikiye ayrılır:

- ◊ **Tam Sayı Gösterimi (Fixed-point / Integer Representation):** Ondalık kısmı olmayan sayıların, ikili tabanda doğrudan bit dizileri hâlinde saklandığı formattır. Negatif sayılar genellikle **ikinin tümleyeni (two's complement)** yöntemiyle temsil edilir.
- ◊ **Kayan Noktalı Gösterim (Floating-point Representation):** Ondalık kesirli sayıların, işaret, üs (*exponent*) ve mantis (*mantissa*) bileşenlerinden oluşan bilimsel gösterime benzer bir formatla saklandığı yöntemdir; bugün neredeyse tüm diller ve işlemciler IEEE 754 standardını kullanır.

Bir değişkenin hangi formatta saklanacağını (dolayısıyla ne kadar bellek harcayacağını ve hangi işlemlere izin vereceğini) belirleyen şey, o değişkenin **veri tipidir (data type)**; bu kavrama Bölüm 4.3.1'da yeniden döneceğiz. Şimdi ise değişkenler kadar temel bir diğer yapı taşına, fonksiyonlara bakalım.

§4.2.2 Fonksiyon Nedir?

Matematikte **fonksiyon (function)**, bir kümenin bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir (relationship)**.

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 4.1: İki ile Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade (expression)** edilir.

$$f(x) = 2x \quad (4.1)$$

Burada f , fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x \quad (4.2)$$

Bu durumda bağımlı değişken y ile bağımsız değişken olan x arasındaki ilişkiyi formülüne eder. Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$f(x) = 2x + 33 \quad (4.3)$$

Bu durumda ise anlatılan değişkenler arasındaki ilişkiyi tanımlayan formüle f gibi bir isim verilir. İsmi belirlenen bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan $f(x)$ değerine eşleyen bir ifadedir. Yani $x = 1.0$ argümanını $f(1.0) = 5.0$ reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür (return)**.

$$g(a, b) = 2a + b \quad (4.4)$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile b elemanını, hesaplanan $g(a, b)$ değerine eşleyen bir ifadedir. Yani $a = 1.0$ ve $b = 1.0$ argümanlarını $g(1.0, 1.0)$ fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (*return*).

Parametre ve Argüman

Örneklerdeki x , a ve b bağımsız değişkenler olup **parametre (parameter)** olarak adlandırılır. $f(1.0)$ ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre (actual parameter)** ya da **argüman (argument)** olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0) \quad (4.5)$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler (call)**.

4.3 Programlama Paradigmaları

§4.3.1 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, birden fazla çağrı noktası olabilen **COROUTINE** ve **FUNCTION** terimleri kullanılıyor olsa da hala **GOTO** ve **JUMPTO** talimatları kullanılmakta ve kafa karışıklığına devam etmekteydi. 1968 Yılında **Edsger W. Dijkstra** önderliğinde **GOTO** talimatları **zararlı ilan edilmiştir**.

1970 Yılında **Niklaus Wirth** tarafından **Pascal** dili geliştirildi. Bu dil, yapısal programlama ilkelerini benimseyen ve yaygınlaştıran en bilinen erken örneklerden biridir (yapısal programlamanın kavramsal temelleri aslında 1960'ta ortaya çıkan **ALGOL 60** diline kadar uzanır). Ardından bunu **C** dili takip etmiştir.

Yapısal programlamada (*structural programming*) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (**if**, **switch**, **for**, **do**, **while**) birbirinden ayrılmıştır. Pascal, kendi kendini çağıran **özyinelemeli** (*recursive*) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (*pointer*) desteklemektedir⁴. Aşağıda iki sayıyı kullanıcıdan alan ve konsola toplamı yazdıran bir program örneği verilmiştir;

```
PROGRAM ToplamaIslemi;
VAR { Değişken tanımlama bölümü }
    sayi1, sayi2, toplam: integer;
BEGIN
    { Kullanıcıdan veri alma }
    WRITELN('Birinci sayiyi giriniz:');
    READLN(sayi1);
    WRITELN('İkinci sayiyi giriniz:');
    READLN(sayi2);
    { Hesaplama }
    toplam := sayi1 + sayi2;
    { Sonucu ekrana yazdırma }
    WRITELN('Girdiginiz sayıların toplamı: ', toplam);
    { Programın hemen kapanmaması için bir tuş beklemesi (isteğe bağlı) }
    READLN;
END.
```

Yapısal Programlamada Okunaklılık Yüksek

Yapısal programlamada, bir ana fonksiyon ve alt fonksiyonlara tanımlanarak fonksiyonların birbirini çağırmasıyla programlama yapılır. Her bir fonksiyonda verinin tutulacağı veri yapıları ile bu verileri işleyen kontrol yapıları vardır. Bu yapılar birbirinden ayrıldığından, arapsaçı programlamanın aksine, talimatların hangi sırayla icra edildiğinin anlaşılması yani kontrol akışı (*control flow*) kolay anlaşılır. Bu nedenle **yapısal programlamada okunaklılık çok yüksektir**.

Pascal dilinde önce bahsedilen dillerden farklı olarak satır numarası yoktur. Yani talimatlar FORTRAN veya FLOW-MATIC dilindeki gibi satır numaralarıyla yazılmaz. Pascal dilinde her talimat noktalı virgül karakteri ile biter. Talimatları boşluk, yeni satır ve sekme gibi karakterlerden istediğimiz kadarını kullanarak da yazabiliriz. Böylece gelişigüzel (*free format*) kod yazmanın önü de açılmıştır.

Yapısal programlama dillerinde aşağıdaki genel çerçeveye uyulur;

1. Programın başladığı yeri belirten bir **ana fonksiyon** (*main function*) tanımlanır. Yapılacak işi fonksiyonlar olarak yazar ve bu ana fonksiyonda birbirini çağıran fonksiyonlar olarak yazarız.
2. Ana fonksiyon dahil her fonksiyonda ilk önce **veri yapıları** (*data structure*) tanımlanır. Veri yapısı;
 - (a) En basit veri yapısı **char**, **integer**, **float** ve **double** gibi **ilkel veri tiplerinden** (*primitive data type*) olabilen **değişken** (*variable*),
 - (b) İlkel veri tiplerinden meydana gelen **dizi** (*array*),
 - (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (*structure*),
 - (d) Ya da dinamik olarak yani çalıştırma anında birbirine bağlanmış verilerden oluşan **bağlı liste** (*linked list*) olabilir.
3. Veri yapısı tanımlandığı her fonksiyonda bu yapıları işleyecek **kontrol yapıları** (*control structure*) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **if**, **if-else**, **switch**, **while**, **do-while**, **for**, **continue**, **break**, **goto** ve **return** **talimatlarından** (*statement*) oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten **BEGIN END**. bloğudur. C dilinde ana fonksiyon, **main()** fonksiyonudur. C programlama dili, en çok kullanılan, öğrenmesi kolay bir yapısal programlama dilidir, hızlı ve verimli programlar üretir, assembly dili desteği ile **düşük düzey programlamayı** (*low level programming*) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar

⁴Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda örneği verilen Pascal programının C dilindeki karşılığı aşağıda verilmiştir;

```
#include <stdio.h>
int main() {
    // Değişken tanımlama bölümü
    int sayi1, sayi2, toplam;
    // Kullanıcıdan veri alma
    printf("Birinci sayiyi giriniz:\n");
    scanf("%d", &sayi1);
    printf("İkinci sayiyi giriniz:\n");
    scanf("%d", &sayi2);
    // Hesaplama
    toplam = sayi1 + sayi2;
    // Sonucu ekrana yazdırma
    printf("Girdiginiz sayilarin toplami: %d\n", toplam);
    // Programın hemen kapanmaması için bir tuş beklemesi (isteğe bağlı)
    // Not: Pascal'daki boş 'readln' yerine C'de yaygın olarak getchar() kullanılır.
    getchar(); // scanf'ten kalan Enter karakterini temizlemek için
    getchar(); // Asıl tuş bekletme işlevi için
    return 0;
}
```

Programın icrası her zaman ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte **programın icrasını değiştirmeyen** bu tür talimatlara **ardışık işlem** (*sequential operation*) adı verilir.

Ardışık İşlemler

- Değişkenlerin kimliklendirildiği değişken tanımlamaları (identifier definition):
 - int yariCap=3;
 - const float PI=3.14;
 - float daireninAlani,daireninCevresi;
- İfadelerden (*expression*): (Sabit, değişken ve operatör içeren söz dizimlerine **ifade** adı verilir)
 - daireninAlani=PI*yariCap*yariCap;
 - float daireninCevresi=2.0*PI*yariCap;
- Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (*input-output operation*):
 - printf("Girdiginiz sayilarin toplami: %d\n", toplam);
 - scanf("%d", &sayi2);

Kontrol işlemleri (*control operation*) ise programın icra sırasını yani kontrol akışını (*control flow*) değiştiren kontrol yapılarıdır.

Kontrol İşlemleri

- Duruma göre seçimler** (*conditional choice*): if, if-else ve switch talimatları.
- İlişkisel döngü** (*relational loop*): while, do-while ve for talimatları.
- Dallanmalar** (*jump*): continue, break, goto ve return talimatları.

Fiziksel ve Mantıksal Satır

Kontrol işlemlerinde; **kodlanan mantıksal satırlar** (*logical sequence*) ile fiziksel olarak icra edilen satırlar (*physical sequence*) birbirinden farklıdır.

§4.3.2 Nesne Yönelimli Programlama

Yapısal programlama paradigmasında talimatlar (*statement*) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya emreden programlama paradigması (*imperative programming*) adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden programlama bir örneği olan yapısal programlamada talimatlar (*statements*) art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Emreden paradigmanın bir örneği olan yapısal programlamada;

◊ Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da

o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. Değişim yönetimi (*change management*) çok zordur ve uzun zaman alır.

- ◊ Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda hataların izini sürmek (*error handling*) işi zorlaştır.
- ◊ Yapısal programlamada, gösterici (*pointer*) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun güvenli (*safe code*) olarak çalıştığının incelenmesi çok zordur.
- ◊ Sürekli olarak benzer projelerde aynı kodları tekrar yazmak (*duplicate code*) gereklidir.
- ◊ Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi başarısız (*fail*) olmuştur. Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur.

Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillerindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi. Smalltalk, 1972 yılında Xerox Park şirketinde Alan Kay önderliğinde Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, Scott Wallace ve Peter Deutsch tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (*object*) temel yapıtaşlarıdır. Nesneler;

- ◊ Durum (*state*) ve davranışlara (*behavior*) sahiptir. Davranışlar yapısal programlamadaki fonksiyonlar gibi kodlanırlar ve yöntem (*method*) olarak adlandırılırlar.
- ◊ Programlama, nesnelerin birbirlerine ileti göndermesiyle (*message-passing*) yapılır.
- ◊ Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani yansıma (*reflection*) özelliği vardır.

Emreden paradigma (*imperative programming*) aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya nesne yönelimli programlama kısaca **OOP** (*object oriented programming*) paradigması adı verilir.

C++ programlama dili, C diline yapılan nesne yönelimli programlama eklentileri yapılarak geliştirilmiş bir programlama dilidir. Danimarkalı bilgisayar bilim adamı olan Bjarne Stroustrup, sonradan C++ olarak bilinecek olan "C with Classes" üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++, nesne yönelimli programlama kavramlarına sahip C programlama dilinin bir uzantısıdır. C++ gibi genel amaçlı (general purpose), Java ve C# gibi nesne yönelimli birçok programlama dili daha sonra geliştirilmiş ve günümüzde popüler olmuştur.

Şimdiye kadar anlatılan bu dillerin dolaylı da olsa bilgisayar mimarisi ile ilişkisi vardır. Gelişmiş yazılımlar yapabilmek için bir miktar mimariden anlamak gereklidir. Ayrıca Java, C# ve C++ nesnelerle ilgili birçok kütüphaneye sahiptir. Bu kütüphaneleri kullanabilmek için birçok kavramı bir anda özümsemek gerektiğinden öğrenme eğrisi diktir.

Bilgisayarın mimarisini bilmeden programlama yapmayı sağlayacak nesneler kullanılarak ve tasarlanarak programlama yapılır. Bu nedenle Python, şüphesiz harika bir dildir. Çünkü sağladığı işlevsellik miktarı başka hiçbir dilde mevcut değildir. Ancak Python dilindeki bellek tüketimi çok yüksek olduğundan bellek kısıtlamalarının olduğu bir yerde kötü bir seçim haline getirir.

4.4 Kod Nasıl Çalıştırılır? Derleme ve Yorumlama

§4.4.1 Derleyici ve Yorumlayıcı

Yüksek seviyeli programlama dilleriyle yazdığımız kaynak kodlar (*source code*), insanlar tarafından kolayca okunup anlaşılabilir semantik bir yapıya sahiptir. Ancak bilgisayar mimarisinin kalbi olan işlemci, daha önce gördüğümüz gibi sadece elektrik sinyallerine karşılık gelen 0 ve 1 dizilimlerinden, yani emir kodundan (*instruction code*) anlar. İnsan algısına uygun bu metinlerin, donanımın icra edebileceği deterministik emirlere dönüştürülmesini sağlayan iki temel yazılımsal dönüştürme stratejisi mevcuttur: **Derleyici** (*compiler*) ve **Yorumlayıcı** (*interpreter*).

§4.4.2 Derleyici

Derleyici(*compiler*), geliştirici tarafından yazılan kaynak kodun tamamını dil bilgisi (*syntax*) kuralları açısından **tek bir kerede** analiz eden ve **hedef işlemci mimarisinin emir setine uygun**, doğrudan çalıştırılabilir ikili (*binary executable*) bir dosya (*.exe*, *.out* vb.) üreten sistem yazılımıdır. Derleme sonrasında oluşan dosya defalarca çalıştırılabilir. Derleme felsefesi, yabancı dildeki bir kitabın tamamını bir kerede hedef dile çevirip matbaada yeni bir kitap olarak basan mütercime benzer. Bu mimarinin özellikleri şu şekildedir:

- ◊ **Performans Odaklılık:** Program çalıştırılmadan önce bir kez derlendiği için, yürütme esnasında tekrar bir çeviri işlemine gerek kalmaz; bu nedenle derlenen diller (C, C++, Rust vb.) **maksimum hızda ve verimli** çalışır.
- ◊ **Fikri Mülkiyet ve Dağıtım Kolaylığı:** Yazılım dağıtılırken ham kaynak kodların paylaşılmasına gerek yok-

tur. Kullanıcıya yalnızca derlenmiş ikili dosya (*executable binary file*) teslim edilir.

- ◊ **Platform Bağımlılığı:** Derleyici, kodu doğrudan hedef donanımın (Örn: x86-64 Intel işlemci ve Windows işletim sistemi) emir setine göre ürettiği için, o dosya başka bir mimaride (Örn: ARM tabanlı bir akıllı telefonda) doğrudan çalışmaz.
- ◊ **Katı Hata Denetimi:** Kaynak kodda tek bir virgül hatası dahi olsa derleme işlemi başarısız olur. Tüm hatalar raporlanır ve kodun düzeltilip **baştan aşağı yeniden derlenmesi** gerekir.

Derleme işlemi sırasında aşağıdaki adımlar izlenir;

- ◊ Koda ilişkin derleyiciye argüman olarak kod dosyası verilir.
- ◊ Derlenecek kodda hata yoksa işlemci özel makine kodlarından oluşan icra edilebilir (*executable*) dosya oluşur. Bu nedenle bu dosyaya doğal kod (native kode) adı da verilir.
- ◊ Kullanıcı arayüzünden verilen bir komutla icra edilebilir bu dosya işletim sistemi kontrolünde işlemci tarafından çalıştırılır. Çalıştırılan bu program ihtiyacımızı gördüğü sürece yeniden derlemeye gerek yoktur.

§4.4.3 Yorumlayıcı

Yorumlayıcı(*interpreter*), kaynak kodu baştan sona tek bir seferde çevirmek yerine, programın çalışması esnasında (*runtime*) **talimatlardan** (*statement*) **oluşan kod satırlarını** yukarıdan aşağıya doğru sırayla, satır satır okur; anlık olarak makine diline çevirir ve o anda işlemciye ileterek icra edilmesini sağlar. Yorumlama felsefesi, uluslararası bir konferansta konuşmacının sözlerini dinleyicilere eş zamanlı olarak aktaran simültane tercümana benzer. Ortada kalıcı, bağımsız çalışabilir ikili bir dosya oluşmaz. Programın her çalıştırılışında **yorumlayıcı yazılımın** arka planda aktif olması zorunludur.

Yorumlayıcı, bir programlama dilinde veya **betik** (*script*) dilde yazılmış talimatları, makine diline derleme olmadan doğrudan çalıştıran bir bilgisayar programıdır. Betik diller için derleme süreci yoktur. Betik diller her zaman bir ev sahibi uygulama tarafından her seferinde çalıştırılır. Örnek olarak;

- ◊ JavaScript için ev sahibi uygulama bir tarayıcıdır(*web browser*).
 - ◊ PHP için ev sahibi uygulama web sunucusudur (*web server*).
 - ◊ Windows'taki toplu işlem dosyaları (**.bat*) için ev sahibi uygulama Windows komut istemidir (*Windows Command Prompt*).
 - ◊ Linux için kabuk dosyaları (**.sh*) için ev sahibi uygulama */bin/bash* kabuğu (*shell*) uygulamasıdır.
- Kısaca yorumlayıcı, bir dilde yazılmış kaynak kodu talimatlar (*statement*) halinde satır satır (*logical sequence*) okuyup çalıştıran bir programdır. Yorumlayıcılarda hata oluşursa, hatanın olduğu satırda durulur ve sonrasını çalıştırılmaz. Yorumlayıcılar derleyicilere göre genellikle daha yavaştır çünkü her seferinde kodu çözümlemesi gerekir. Bir yorumlayıcı genellikle program çalıştırmak için aşağıdaki stratejilerden birini kullanır:
- ◊ Kaynak kodu ayrıştırır (*parse*) ve çalıştırılacak satırları belirleyerek sırayla icra eder.
 - ◊ Kaynak kodu daha kolay işleyecek bir biçime veya nesne koduna çevir ve ardından icra eder.
 - ◊ Kaynak kod derleyici tarafından yorumlayıcının sanal makinesiyle eşleşen bir bayt kodu üretir. Yorumlayıcı sanal makinesi bu kodu doğrudan çalıştırır. Python dilinde bu strateji kullanılır.

§4.4.4 Script Nedir?

Betik (*script*), başka bir program veya çalışma zamanı ortamı tarafından adım adım yorumlanarak çalıştırılan, derleme aşamasına ihtiyaç duymayan hafif kod parçalarından oluşan programlama dilleridir.

Betik diller tek başına çalışmazlar. Onların talimatlarını (*statements*) sırasıyla çalıştıracak bir ev sahibi programa ihtiyaç duyarlar. İşte JavaScript de web tarayıcısı tarafından çalıştırılan, endüstri standardı olmuş, bir betik dildir. Bölüm 1.8 başlığında web tarayıcısının HTML sayfalarını dinamik hale getirdiği belirtilmişti.

Betik dilleri, C veya C++ gibi dillerin aksine doğrudan makine koduna derlenmez. Bunun yerine bir yorumlayıcı (*interpreter*) veya JIT (*Just-In-Time*) derleyici tarafından satır satır (*logical sequences*) okunarak anlık olarak yürütülür.

Betik diller, hızlı geliştirme süreci, dinamik veri türleri (*dynamic typing*) ve belirli bir platform veya uygulama içerisinde otomasyon sağlama yeteneği verir.

Bölüm Özeti

Bu bölümde, bilgisayar programlarının doğuşundan günümüz betik dillerine uzanan geniş bir yolu izledik:

- ◊ Bir bilgisayar programı, işlemcinin tekrar tekrar yürüttüğü **emir çevrimi** (fetch-decode-execute) üzerine kuruludur; **düşük düzey** diller bu emirlere yakinken, **yüksek düzey** diller konuşma dilimize yakın talimatlar kullanır.
- ◊ Erken dönem dillerde (FORTRAN, FLOW-MATIC) yaygın **GOTO/JUMP** kullanımı, okunması güç **arapsaçı koda** yol açmış; bu sorun **yapısal programlama** (Pascal, C) ile veri yapıları ve kontrol yapılarının birbirinden ayrılmasıyla çözülmüştür.
- ◊ **Değişkenler** bellekte bir değeri, **fonksiyonlar** ise girdi ile çıktı arasındaki bir ilişkiyi temsil eder.
- ◊ Yapısal programlamanın büyüyen projelerde yarattığı bağımlılık ve bakım sorunları, nesneleri durum ve davranışlarıyla bir arada ele alan **nesne yönelimli programlama** (OOP) paradigmasının doğmasına yol

açmıştır.

- ◊ Kaynak kod, ya bir **derleyici** tarafından bir kerede makine diline çevrilip çalıştırılabilir bir dosya üretilir (C, C++, Pascal) ya da bir **yorumlayıcı** tarafından çalışma zamanında satır satır yürütülür.
 - ◊ **Betik diller** (JavaScript, PHP, Bash), tek başlarına çalışamayan, kendilerini çalıştıracak bir ev sahibi uygulamaya (tarayıcı, web sunucusu, kabuk) ihtiyaç duyan, derlemeye gerek duymayan yorumlanan dillerdir.
- Bu bölümde edindiğiniz bu kavramsal çerçeve – değişken, fonksiyon, kontrol akışı, paradigma, derleme/yorumlama – ileride öğreneceğiniz her programlama dilinde (JavaScript dahil) karşınıza aynı isimlerle çıkacaktır; bir sonraki bölümde bu kavramların JavaScript dilindeki somut karşılıklarını göreceksiniz.

4.5 Düşün ve Çöz

1. **Düşük mü, Yüksek mi?** Bir öğrenciye `MOV AX, 5` gibi bir Assembly komutu ile `toplam = a + b;` gibi bir C ifadesi gösteriliyor. Bu iki örnekten hangisinin düşük, hangisinin yüksek düzey programlamaya ait olduğunu, bölümde öğrendiğiniz "emir kodu" ve "hatırlatıcı isim" kavramlarına dayanarak açıklayınız.
2. **Arapsaçı Kod Analizi:** Bölümdeki FLOW-MATIC örneğinde talimatlar birbirine `JUMP TO OPERATION` ifadeleriyle bağlanmaktadır. Bu tür bir kod organizasyonunun, yapısal programlamadaki fonksiyon çağrılarına kıyasla okunabilirlik açısından ne gibi dezavantajları vardır?
3. **Parametre ve Argüman:** $f(x) = 3x + 7$ fonksiyonu için $f(4)$ ifadesindeki parametre ile argümanı ayırt ederek belirtiniz. Bu fonksiyonun bir programlama dilinde bir fonksiyon olarak nasıl tanımlanacağını (sözde kod ile) yazınız.
4. **Derleyici mi, Yorumlayıcı mı?** Bir masaüstü oyun motoru geliştiriyorsunuz ve en yüksek performansı elde etmek istiyorsunuz. Derlenen bir dil mi (C++) yoksa yorumlanan bir dil mi (JavaScript) tercih edersiniz? Kararınızı, bölümde öğrendiğiniz derleyici/yorumlayıcı farklarına dayanarak gerekçelendiriniz.
5. **Paradigma Seçimi:** Bir öğrenci bilgi sistemi geliştiriyorsunuz; her öğrencinin adı, notu ve devamsızlık durumu olacak ve bu öğrenciler üzerinde işlem yapılacak. Bu senaryoyu yapısal programlama ile mi yoksa nesne yönelimli programlama ile mi modellemek daha uygundur? Nedenini açıklayınız.

5. Bölüm

JavaScript

5.1 JavaScript Tarihçesi

JavaScript, web'e dinamik etkileşim kazandırmak amacıyla geliştirilmiş ve zamanla dünyanın en popüler dillerinden biri haline gelmiştir. Hem yapısal programlamayı hem de nesne yönelimli programlamayı destekler.

Yıl	Gelişme / Milat	Açıklama
1995	Netscape & Brendan Eich	<i>Brendan Eich</i> tarafından Netscape Navigator tarayıcısı için sadece 10 günde tasarlandı (İlk adı Mocha, sonra LiveScript).
1997	ECMAScript Standardı	Dilin standartlaşması için ECMA International'a sunuldu ve ES (<i>ECMAScript</i>) standartları doğdu.
2005	AJAX Devrimi	Sayfa yenilenmeden veri transferine izin veren AJAX teknolojisi ile modern web uygulamalarının önü açıldı.
2009	Node.js & ES5	<i>Ryan Dahl</i> tarafından Node.js tanıtıldı; JavaScript tarayıcı dışına çıkarak sunucu tarafında da çalışmaya başladı.
2015	ES6 (ECMAScript 2015)	<i>let/const</i> , ok fonksiyonları (<i>arrow functions</i>), <i>class</i> yapısı ve modüller gibi büyük modern özellikler eklendi.

Tablo 5.1: JavaScript Tarihçesi

Günümüzde hala popüler olan programlama dillerinin başında gelmektedir¹.

5.2 JavaScript Kullanım Alanları

Günümüzde JavaScript tam kapsamlı (*full-stack*) web geliştirme imkanı sunar:

- ❖ **İstemci Tarafı (Front-End Web):** React, Vue.js ve Angular ile kullanıcı arayüzleri ve dinamik web sayfaları oluşturma.
- ❖ **Sunucu Tarafı (Back-End):** Node.js ve Express.js ile yüksek performanslı, olay güdümlü (*event-driven*) API servisleri geliştirme.
- ❖ **Mobil Uygulama Geliştirme:** React Native veya Ionic kullanarak iOS ve Android için tek kod tabanından yerel görünümlü uygulamalar yazma.
- ❖ **Masaüstü Uygulamaları:** Electron.js altyapısı ile çapraz platform (*cross-platform*) masaüstü yazılımları (ör. VS Code, Slack) geliştirme.
- ❖ **Veri Tabanı İletişimi:** MongoDB gibi NoSQL veri tabanlarında sorgulama yapısı olarak JSON/JS formatlarını kullanma.

5.3 JavaScript Çalışma Mantığı

JavaScript, mantıksal olarak **tek iş parçacıklı** (*single-threaded*) bir çalışma yapısına sahiptir. Yani aynı anda tek bir komut satırını icra eder. Ancak ağ istekleri, dosya okuma veya zamanlayıcılar gibi uzun süren işlemlerde programın kilitlenmesini (*blocking*) önlemek için kesintisiz G/Ç (*non-blocking I/O*) mimarisini ve **Olay Döngüsü** (*Event Loop*) mekanizmasını kullanır.

Olay Döngüsü (Event Loop)

JavaScript ana iş parçacığında (*main thread*) yürütülen kodları **Çağrı Yığını** (*Call Stack*) üzerinde çalıştırır. Uzun süren asenkron görevler ise arka planda Web API'lerine devredilir. Tamamlanan asenkron görevlerin geri çağırma fonksiyonları (*callback*) bir **Görev Kuyruğu**'na (*Task Queue*) atılır. Olay Döngüsü, Çağrı Yığını boşaldığında kuyruktaki işleri sırayla yığına aktararak icra ettirir.

¹<https://spectrum.ieee.org/top-programming-languages-2025>

5.4 JavaScript'e Giriş

§5.4.1 Sözdizimi (Syntax) Kuralları

JavaScript sözdizimi C dili ailesinden türemiştir:

- ◊ **Talimatlar:** Her mantıksal satırın (talimatın) sonuna noktalı virgül (;) konulması iyi bir uygulamadır. (JavaScript otomatik noktalı virgül ekleme – *Automatic Semicolon Insertion (ASI)* özelliğine sahip olsa da açıkça yazılması önerilir).
- ◊ **Kod Blokları:** Mantıksal bloklar ve işlev etki alanları süslü parantezler { } ile tanımlanır.
- ◊ **Büyüklik/Küçüklük Harf Duyarlılığı (Case-Sensitivity):** JavaScript harf büyüklüğüne duyarlıdır. `sayi` ile `Sayi` iki farklı değişkendir.
- ◊ **Açıklama Satırları:** Tek satırlık açıklamalar için `//`, çok satırlı açıklamalar için `/* ... */` blokları kullanılır.

§5.4.2 Değişken Tanımlama

Bölüm 4.2.1 başlığında matematiksel kavram olarak gördüğümüz değişkenlerin burada somut karşılığını göreceksiniz. Her yapısal programlama dilinde olduğu gibi JavaScript'te de **veri yapıları** (*data structure*) ve **kontrol yapıları** (*control structure*) teorik olarak birbirinden ayrıdır. Ancak C gibi dillerden farklı olarak JavaScript, veriyi işleyecek kontrol yapısından önce verinin tipini ve bellek boyutunu katı bir şekilde tanımlamanızı zorunlu kılmaz.

C dilinde bir değişken tanımlandığında (örneğin `int x = 5;`), bellekte sabit boyutlu bir adres (kutu) ayrılır ve değer doğrudan buraya yazılır. JavaScript'te ise değişkenler bellek adresini temsil eden birer **etiket** (referans) gibidir. Atanan değerler belleğin başka bir bölgesinde dinamik olarak oluşturulur ve değişken yalnızca o değeri işaret eder.

JavaScript'te değişken tanımlamak için üç farklı anahtar kelime kullanılır:

1. **var:** Fonksiyon kapsamlıdır (*function-scoped*). Tanımlandığı fonksiyonun her yerinden erişilebilir. Yükseltme (*hoisting*) davranışına tabidir. Modern JavaScript (ES6+) standartlarında kafa karışıklığına ve hatalara yol açtığı için kullanımı önerilmez.
2. **let:** Blok kapsamlıdır (*block-scoped*). Yalnızca tanımlandığı { } parantezleri arasında geçerlidir. Değeri daha sonra değiştirilebilir.
3. **const:** Blok kapsamlıdır (*block-scoped*). Sabit (*constant*) değerler tanımlamak için kullanılır. İlk değer ataması (*initialization*) tanımlandığı anda yapılmak zorundadır ve referansı sonradan değiştirilemez.

§5.4.3 Değişken Kimliklendirme (Identifiers)

Program içinde değişkenlere, fonksiyonlara veya etiketlere verilen isimlere **tanımlayıcı** veya **kimlik** (*identifier*) adı verilir. JavaScript'te geçerli bir isim tanımlarken şu kurallara uyulmalıdır:

- ◊ İsimler yalnızca harf (`a-z`, `A-Z`), rakam (`0-9`), alt çizgi (`_`) veya dolar simgesi (`$`) içerebilir.
- ◊ Bir değişken ismi kesinlikle bir **rakam ile başlamaz** (örneğin `1sayi` geçersizdir, ancak `sayi1` geçerlidir).
- ◊ Dille bütünleşmiş **Ayrılmış Kelimeler** (*Reserved Keywords*) değişken ismi olarak kullanılamaz (`let`, `class`, `return`, `function` vb.).
- ◊ Naming Convention (İsimlendirme Standardı): JavaScript dünyasında değişken ve fonksiyon isimleri için **camelCase** kullanımı kabul görmüş standarttır (örneğin `kullaniciAdi`, `hesaplaToplam()`).

§5.4.4 Değişmezler (Literals)

Kod içinde doğrudan (sabit olarak) yazılan ham değerlere **değişmezler** (*literals*) denir. Bunlar bir değişkene atanan ilk verilerin kaynağıdır.

```
let tamsayi = 42;           // Numeric Literal (Sayısal Değişmez)
let metin = "Merhaba";     // String Literal (Metinsel Değişmez)
let durum = true;          // Boolean Literal (Mantıksal Değişmez)
let nesne = { ad: "Ali" }; // Object Literal (Nesne Değişmezi)
let dizi = [1, 2, 3];       // Array Literal (Dizi Değişmezi)
```

§5.4.5 İşleçler (Operators)

Değişkenler ve değerler üzerinde belirli matematiksel veya mantıksal işlemleri yürüten sembollere **işleç** (*operator*) denir:

- ◊ **Aritmetik İşleçler:** `+`, `-`, `*`, `/`, `%` (mod alma), `**` (üs alma).
- ◊ **Atama İşleçleri:** `=`, `+=`, `-=`, `*=`, `/=`.
- ◊ **Karşılaştırma İşleçleri:** `==` (değer eşitliği), `===` (kesin eşitlik: değer ve tip eşitliği), `!=`, `!==`, `>`, `<`, `>=`, `<=`.
- ◊ **Mantıksal İşleçler:** `&&` (VE), `||` (VEYA), `!` (DEĞİL).
- ◊ **Üçlü İşleç (Ternary Operator):** `koşul ? doğruysa : yanlışsa`.

Eşitlik Operatörü Kullanımı

JavaScript'te tip dönüşümü risklerinden kaçınmak için her zaman değer ile birlikte tip kontrolü de yapan **kesin eşitlik** (`===`) ve **kesin eşitsizlik** (`!==`) operatörleri tercih edilmelidir. `5 == "5"` ifadesi `true` dönerken, `5 === "5"` ifadesi `false` döner.

§5.4.6 İfadeler (Expressions)

En az bir değer, değişken ve operatörün bir araya gelerek yeni bir **değer ürettiği** kod parçacıklarına **ifade** (*expression*) denir.

```
5 + 3;           // 8 değerini üreten bir ifade
let x = 10;      // 10 değeri atanır
x * 2 > 15;      // true mantıksal değerini üreten bir ifade
```

Buna karşılık tek başına değer üretmeyen, bilgisayara bir eylem gerçekleştirmesini emreden yapılara ise daha önce incelediğimiz gibi **talimat** (*statement*) denir (Örn: `if` blokları, `for` döngüleri).

§5.4.7 Değer Atanamaz Değişken Kullanımı ve Referans Sabitliği

`const` anahtar kelimesi ile tanımlanan bir değişkene yeni bir değer atanamaz (*re-assignment yapılamaz*). Ancak bu durum, değişkenin tuttuğu içeriğin hiçbir şekilde değiştirilemeyeceği anlamına gelmez.

```
const sayi = 100;
sayi = 200; // HATA! TypeError: Assignment to constant variable.

const kullanıcı = { ad: "Ahmet" };
kullanıcı.ad = "Mehmet"; // GEÇERLİ! Nesne içeriği değiştirilebilir.
kullanıcı = { ad: "Ayşe" }; // HATA! Yeni bir nesne referansı atanamaz.
```

Özel Kural

Modern JavaScript geliştirmelerinde varsayılan olarak her zaman `const` tercih edilmelidir. Yalnızca değeri sonradan değiştirilecek (örneğin döngü sayaçları) değişkenler için `let` kullanılmalıdır. `var` kullanımı tamamen terk edilmelidir.

§5.4.8 İlkel Veri Tipleri

C dilinde `char`, `int`, `float` gibi doğrudan işlemci emir setine hitap eden ve bellekte kaplayacağı alan derleme aşamasında baştan belli olan **ilkel veri tipleri** (*primitive data type*) vardır.

JavaScript tarafında da temel seviyede değer tutan ilkel veri tipleri (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`) mevcuttur. Ancak C gibi dillerden farklı olarak JavaScript'te ilkel bir değer üzerinden bir yöntem veya özellik çağrıldığında (örneğin metin uzunluğunu almak için `"merhaba".length` yazıldığında), JavaScript arka planda bu ilkel değeri geçici olarak bir **nesneye** (*object wrapper*) dönüştürür ve işlem bittiğinde bu nesneyi bellekten siler. Bu durum, geliştiricilerde "JavaScript'te her şey bir nesnedir" şeklinde yaygın bir yanlışlığı oluşturur.

JavaScript'te Bulunan İlkel Veri Tipleri:

- ◊ **number**: Hem tamsayıları (*integer*) hem de ondalıklı sayıları (*floating-point*) temsil eder. C dilindeki gibi `int`, `float`, `double` ayrımı yoktur; tüm sayılar bellekte 64-bit çift duyarlıklılı (*double precision*) formatta saklanır. Özel sayısal değerler olan `Infinity`, `-Infinity` ve `NaN` (*Not-a-Number*) da bu tipe dahildir.


```
let tamSayi = 42;
let ondalikli = 3.14;
```
- ◊ **string**: Metinsel verileri temsil etmek için kullanılır. Karakter dizileri tek tırnak (`'`), çift tırnak (`"`) veya şablon dizimimleri (*template literals*) için ters tırnak (```) içinde tanımlanabilir.


```
let isim = "Ahmet";
let mesaj = `Merhaba ${isim}`; // Şablon dizilimi
```
- ◊ **boolean**: Mantıksal doğruluk durumunu ifade eder. Yalnızca iki değer alabilir: `true` (doğru) veya `false` (yanlış). Kontrol yapılarında (`if/else`) koşul değerlendirmeleri için kullanılır.


```
let aktifMi = true;
```
- ◊ **undefined**: Bir değişkenin tanımlandığını ancak henüz kendisine bir değer atanmadığını gösterir. JavaScript motoru tarafından otomatik olarak atanan varsayılan değerdir.


```
let x;
console.log(x); // undefined
```
- ◊ **null**: Bir değişkenin bilerek ve isteyerek "boş" veya "değersiz" bırakıldığını ifade eder. `undefined`'dan farkı, geliştirici tarafından kasıtlı olarak atanmasıdır.


```
let kullanıcı = null; // Henüz veri yüklenmedi/boş
```
- ◊ **symbol** (ES6+): Benzersiz (*unique*) ve değiştirilemez (*immutable*) tanımlayıcılar oluşturmak için kullanılır.

Özellikle nesnelerin gizli/özel mülk anahtarlarını oluştururken çakışmaları önlemek için tercih edilir.

```
let id1 = Symbol("id");
let id2 = Symbol("id"); // id1 !== id2 (her symbol benzersizdir)
```

- ◇ **bigint** (ES2020+): **number** tipinin güvenli temsil edebildiği sınırın ($2^{53} - 1$) üzerindeki çok büyük tamsayıları saklamak için kullanılır. Sayının sonuna **n** takısı eklenerek tanımlanır.

```
let buyukSayi = 9007199254740991n;
```

Bellek Mantiğı

İlkel veri tipleri belleğin yığın (*stack*) bölgesinde doğrudan tuttukları değerler ile saklanırlar. Bir ilkel değişken başka bir değişkene atandığında, değerin bir kopyası çıkartılır (*pass-by-value*). Bu durum, bellekte referans adresi tutan nesnelerden (*Objects/Arrays*) temel farklarıdır.

Dinamik Tiplendirme

C, C++ veya Java gibi **statik tipli** (*statically typed*) dillerde bir değişken tanımlanırken onun bellek alanı ve tipi baştan belirtilmek zorundadır:

```
int sayi = 42; // Bu degisken yalnızca tamsayı tutabilir.
sayi = "Ahmet"; // HATA! Derleme zamanında hata verir.
```

JavaScript ise **dinamik tipli** (*dynamically typed*) bir dildir. Bir değişken tanımlanırken tip belirtilmez. JavaScript yorumlayıcısı (*engine*), değişkene atanan değere bakarak veri tipini çalışma zamanında (*runtime*) otomatik olarak belirler. Aynı değişkene daha sonra farklı türde bir değer atanabilir:

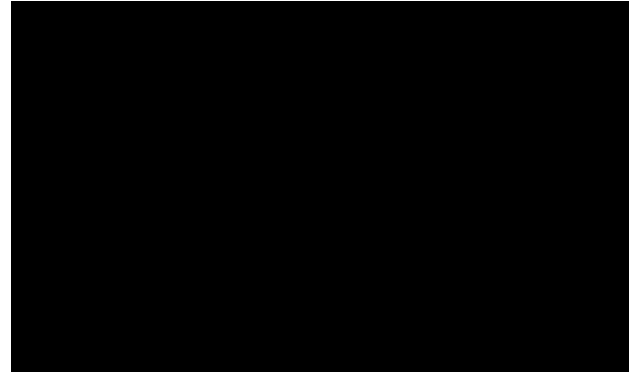
```
let veri = 42; // calisma zamanında 'number' tipini alır
veri = "Merhaba"; // şimdi 'string' tipine donustu
veri = true; // şimdi 'boolean' tipine donustu
```

```
let x = 10;
console.log(typeof x); // "number"

x = "Kodlama";
console.log(typeof x); // "string"

x = { ad: "Ali" };
console.log(typeof x); // "object"
```

Kod 5.1: `typeof` Operatörü ve Dinamik Tiplendirme



Şekil 5.1: Tarayıcı Konsol Çıktısı

Veri Tipi Kontrolü

Değişkenlerin çalışma zamanındaki güncel veri tipini öğrenmek için `typeof` operatörü kullanılır.

Yukarıdaki örnekte görüldüğü üzere `typeof` operatörü, `x` değişkeninin o an bellekte tuttuğu değerin türünü metin (*string*) olarak geri döndürür.

Sık Yapılan Hata

JavaScript'te `typeof null` ifadesi çalıştırıldığında çıktı olarak **"object"** döner. Bu durum JavaScript'in ilk versiyonlarından günümüze kadar gelen ve geriye dönük uyumluluğu bozmamak adına düzeltilmeyen bilinen bir dil hatasıdır (*bug*). `null` bir nesne değil, ilkel bir değerdir.

Otomatik Tip Dönüşümü

Dinamik tiplendirmenin bir sonucu olarak JavaScript, farklı veri tipleri birlikte işleme sokulduğunda tipleri otomatik olarak dönüştürmeye (*Type Coercion*) çalışır:

```
let sonuc = "5" + 2; // "52" (Sayiyi metne donusturup birlestirdi)
let fark = "5" - 2; // 3 (Metni sayiya donusturup cikarma yapti)
```

Bu esneklik küçük kodlarda kolaylık sağlasa da büyük projelerde beklenmeyen hatalara yol açabilir. Bu nedenle modern JavaScript geliştirmelerinde, kod derlenmeden önce tip kontrolleri yapan TypeScript dili yaygın olarak tercih edilmektedir.

5.5 JavaScript Kontrol Akışını Değiştiren Talimatlar

Programların sırayla yukarıdan aşağıya icra edilmesi sürecine **kontrol akışı** (*control flow*) denir. Belli şartlara göre bu doğrusal akışı değiştiren veya tekrarlayan yapılara ise kontrol yapıları denir.

§5.5.1 Duruma Göre Seçim Talimatları

Belirli mantıksal koşulların doğru (*true*) veya yanlış (*false*) olmasına göre farklı kod bloklarının çalıştırılmasını sağlar:

```
let not = 75;

if (not >= 85) {
  console.log("Pekiyi");
} else if (not >= 70) {
  console.log("İyi");
} else {
  console.log("Geçti");
}
```

Birden fazla sabit değer eşitlik kontrolü için *switch-case* yapısı tercih edilir:

```
let gun = 2;
switch (gun) {
  case 1: console.log("Pazartesi"); break;
  case 2: console.log("Salı"); break;
  default: console.log("Geçersiz Gün");
}
```

§5.5.2 İlişkisel Döngüler

Belirli bir kod bloğunu bir koşul sağlandığı sürece tekrar tekrar çalıştırmak için kullanılır:

- ◇ **for Döngüsü:** Sıra tabanlı mantıksal tekrarlar için kullanılır.
- ◇ **while Döngüsü:** Koşul doğru olduğu sürece çalışır.
- ◇ **for...in Döngüsü:** Bir nesnenin **anahtarları** (*keys*) üzerinde gezinir.
- ◇ **for...of Döngüsü:** Bir dizinin veya iterable yapının **değerleri** (*values*) üzerinde gezinir.

```
const dizi = ["Elma", "Armut", "Muz"];

// for...of ile eleman değerlerine doğrudan erişim
for (const meyve of dizi) {
  console.log(meyve);
}
```

§5.5.3 Fonksiyonlar

Bölüm 4.2.2 başlığında matematiksel kavram olarak gördüğümüz fonksiyonların burada somut karşılığını göreceksiniz. Fonksiyonlar, belirli bir görevi yerine getirmek için gruplanmış, tekrar kullanılabilir kod bloklarıdır. Alt program yapısını oluşturur.

```
// 1. Fonksiyon Bildirimi (Function Declaration)
function topla(a, b) {
  return a + b;
}

// 2. Ok Fonksiyonları (Arrow Functions - ES6)
const carp = (a, b) => a * b;
```

5.6 JavaScript Nesneler

§5.6.1 Nesne Nedir?

JavaScript'te **nesne** (*object*), ilişkili verilerin ve işlevlerin anahtar-değer (*key-value*) çiftleri halinde tutulduğu karmaşık (ilkel olmayan) bir veri yapısıdır. Nesneler belleğin **öbek** (*heap*) bölgesinde saklanır ve değişkenler bu nesnelere erişim sağlayan referans adreslerini tutar.

§5.6.2 Nesne Özellikleri

Bir nesnenin sahip olduğu verilere/durumlara **özellik** (*property*) denir.

```
const araba = {
  marka: "Toyota",
  model: 2022,
  renk: "Beyaz"
};
```

```
console.log(araba.marka); // Nokta notasyonu (Dot notation)
console.log(araba["renk"]); // Köşeli parantez notasyonu
```

§5.6.3 Nesne Yöntemleri

Bir nesne içerisine tanımlanan ve o nesneye ait eylemleri gerçekleştiren fonksiyonlara **yöntem** (*method*) denir.

```
const kisi = {
  ad: "Ahmet",
  soyad: "Yılmaz",
  tamAd: function() {
    return this.ad + " " + this.soyad;
  }
};
```

§5.6.4 this Kelimesi

this anahtar kelimesi, JavaScript'te o anki **yürütme bağlamını** (*execution context*) işaret eden dinamik bir referanstır. Nesne yöntemleri içinde kullanıldığında, yöntemin ait olduğu nesneyi temsil eder.

§5.6.5 Nesnenin Gösterilmesi

Nesneler doğrudan yazdırıldığında varsayılan olarak `[object Object]` çıktısını verebilir. Nesne içeriğini anlamlı biçimde metne dönüştürmek veya konsola basmak için `JSON.stringify()` kullanılır.

```
const veri = { id: 1, ad: "Test" };
console.log(JSON.stringify(veri)); // '{"id":1,"ad":"Test"}'
```

§5.6.6 Nesne Yapıcıları (Constructors)

Aynı yapıya sahip birden fazla nesne türetmek için **yapıcı fonksiyonlar** (*constructor functions*) veya ES6 ile gelen **class** yapısı kullanılır.

```
class Ogrenci {
  constructor(ad, numara) {
    this.ad = ad;
    this.numara = numara;
  }
}

const ogrenci1 = new Ogrenci("Ayşe", 101);
```

§5.6.7 Nesne Parçalama (Object Destructuring)

ES6 ile gelen nesne parçalama yöntemi, bir nesnenin özelliklerini doğrudan bağımsız değişkenlere çıkarmayı kolaylaştırır.

```
const kullanıcı = { id: 5, isim: "Can", rol: "Admin" };

// Nesne parçalama
const { isim, rol } = kullanıcı;
console.log(isim); // "Can"
```

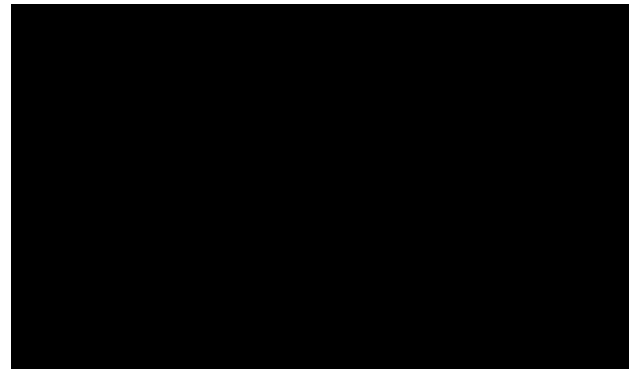
5.7 JavaScript Kapsam

§5.7.1 Değişkenlerin Kapsamı

Değişkenlerin kod içerisinde nerelerden erişilebilir olduğunu belirleyen kurala **kapsam** (*scope*) denir.

```
function testScope() {
  if (true) {
    var x = "var degiskeni";
    let y = "let degiskeni";
  }
  console.log(x); // "var degiskeni" (Erişildi!)
  console.log(y); // ReferenceError! (Erişilemez)
}
```

Kod 5.2: **var** ve **let** Kapsam Farkı



Şekil 5.2: Blok ve Fonksiyon Kapsamı

§5.7.2 Kod Blokları

Süslü parantezler `{ }` ile sınırlandırılmış alanlara kod bloğu denir. `let` ve `const` ile tanımlanan değişkenler bloğa özgü kapsam (*block-scope*) oluştururken, `var` bu blok sınırlarını tanımaz.

§5.7.3 Yükseltme Nedir?

JavaScript motoru kodları çalıştırmadan önce derleme aşamasında tüm değişken ve fonksiyon bildirimlerini bulundukları kapsamın en üstüne taşır. Bu olaya **yükseltme** (*hoisting*) denir.

```
console.log(a); // undefined döner (Hata vermez!)
var a = 10;

// Arka planda JavaScript kodu şu şekilde yorumlar:
// var a;
// console.log(a);
// a = 10;
```

`let` ve `const` ile tanımlanan değişkenler de yükseltilir; ancak tanımlandıkları satıra gelinceye kadar **Geçici Ölü Bölge** yani TDZ (*Temporal Dead Zone*) adı verilen bir alanda tutulurlar. Bu yüzden tanımlanmadan önce erişilmeye çalışıldığında `ReferenceError` hatası verirler.

§5.7.4 Katı Veri Tipi Kontrolü

JavaScript'e ES5 ile eklenen keti veri tipi kontrolü yapılması mümkündür. Katı Veri Tipi Kontrolü Modu yani **strict mode**, ("`use strict`") ile kodun daha katı kurallar çerçevesinde çalışmasını sağlar. Tanımlanmamış değişken kullanımını engeller, sessizce başarısız olan hataları görünür kılar ve daha güvenli kod yazılmasına olanak tanır.

```
"use strict";
x = 3.14; // HATA! ReferenceError: x is not defined (var/let/const zorunlu)
```

5.8 JavaScript Dahili Nesneleri

§5.8.1 Sayılar (Numbers)

JavaScript tüm sayıları IEEE 754 standardında 64-bit kayan noktalı sayı olarak saklar. Yöntemleri arasında `toFixed()` (ondalık basamak sınırlama) ve `parseInt()` (tamsayıya çevirme) sıklıkla kullanılır.

§5.8.2 Metinler (Strings)

Metinsel veriler üzerinde arama, dilimleme ve değiştirme gibi işlemler yapmayı sağlayan nesnedir. `slice()`, `substring()`, `replace()` ve `includes()` en yaygın yöntemlerindendir.

§5.8.3 Zamanlayıcılar (Timers)

Kodun belirli bir süre sonra veya belirli aralıklarla çalıştırılmasını sağlayan asenkron tarayıcı fonksiyonlarıdır:

```
// Belirli süre sonra 1 kez çalışır
setTimeout(() => console.log("Gecikmeli mesaj"), 1000);

// Belirli aralıklarla sürekli çalışır
setInterval(() => console.log("Periyodik mesaj"), 2000);
```

§5.8.4 Tarihler (Dates)

Zaman ve tarih verilerini yönetmek için kullanılan yerleşik `Date` nesnesidir. Zamanı milisaniye cinsinden izler.

§5.8.5 Diziler (Arrays)

Sıralı veri koleksiyonlarıdır. `map()`, `filter()`, `reduce()` gibi yüksek seviyeli fonksiyonel metodlar dizileri işlemek için tercih edilir.

§5.8.6 Kümeler (Sets)

Yalnızca **benzersiz** (*unique*) değerleri saklayan veri koleksiyonudur. Tekrarlayan verileri süzmek için idealdir.

§5.8.7 Haritalar (Maps)

Anahtar-değer çiftlerini saklayan koleksiyonlardır. Nesnelerden farklı olarak anahtar olarak ilkel tiplerin yanı sıra fonksiyonlar veya nesneler de kullanılabilir.

§5.8.8 Yineleyiciler (Iterators)

Bir koleksiyonun elemanları üzerinde sırayla gezinmeyi sağlayan bir arayüzdür. `Symbol.iterator` protokolünü uygulayan yapılar `for...of` döngüsü ile tüketilebilir.

§5.8.9 Matematik (Math)

Matematiksel sabitler ve fonksiyonlar sağlayan statik nesnedir (`Math.PI`, `Math.pow()`, `Math.random()`, `Math.floor()`).

§5.8.10 Düzenli İfadeler (RegExp)

Metinler içinde belirli karmaşık kalıpları aramak, doğrulamak ve değiştirmek için kullanılan güçlü ifade yapısıdır (örneğin e-posta adresi doğrulaması).

5.9 JavaScript Hata Yönetimi

§5.9.1 Hatalar ve Yakalama (try...catch)

Çalışma zamanında meydana gelen hataları yakalamak ve uygulamanın çökmesini engellemek için kullanılır.

```
try {
  // Hata oluşabilecek kod bloğu
  let sonuc = bilinmeyenFonksiyon();
} catch (error) {
  // Hata yakalandığında çalışacak blok
  console.error("Hata yakalandı:", error.message);
} finally {
  // Hata olsun ya da olmasın her durumda çalışacak blok
  console.log("İşlem tamamlandı.");
}
```

§5.9.2 Özel Hata Fırlatma (throw)

Geliştiriciler belirli mantıksal durumlarda kendi özel hatalarını `throw` ifadesi ile fırlatabilirler:

```
function yasKontrol(yas) {
  if (yas < 18) {
    throw new Error("18 yaşından küçükler kayıt olamaz.");
  }
  return true;
}
```

Bölüm Özeti

Bu bölümde, bilgisayar programlarının doğuşundan modern JavaScript'e uzanan geniş bir yolu izledik:

- ◊ Bir bilgisayar programı, işlemcinin tekrar tekrar yürüttüğü **emir çevrimi** (fetch-decode-execute) üzerine kuruludur; **düşük düzey** diller bu emirlere yakinken, **yüksek düzey** diller konuşma dilimize yakın talimatlar kullanır.
- ◊ **Değişkenler** bellekte bir değeri, **fonksiyonlar** ise girdi ile çıktı arasındaki bir ilişkiyi temsil eder.
- ◊ **Yapısal programlama**, veri yapılarıyla kontrol yapılarını birbirinden ayırarak *arapsaçı kod* sorununu çözerken; **nesne yönelimli programlama** (OOP), verileri ve davranışları tek bir nesne içinde bir araya getirir.
- ◊ Kaynak kod, ya bir **derleyici** tarafından bir kerede makine diline çevrilir (C, C++) ya da bir **yorumlayıcı** tarafından çalışma zamanında satır satır yürütülür (JavaScript, PHP).
- ◊ JavaScript; `let/const` ile blok kapsamlı değişkenler, dinamik tiplendirme, **nesneler**, **kapsam** (*scope*) kuralları ve `try...catch` ile hata yönetimi sunan, tarayıcı tarafından yorumlanan bir betik dilidir.

Bu bölümde JavaScript'in kendi başına nasıl çalıştığını öğrendiniz; bir sonraki bölümde bu dili, Bölüm 2'de kurduğunuz HTML iskeletiyle buluşturup sayfaları gerçek zamanlı olarak nasıl değiştireceğinizi (*DOM Manipülasyonu*) göreceksiniz.

5.10 Düşün ve Çöz

1. **Derleyici mi, Yorumlayıcı mı?** Bir masaüstü oyun motoru geliştiriyorsunuz ve en yüksek performansı elde etmek istiyorsunuz. Derlenen bir dil mi (C++) yoksa yorumlanan bir dil mi (JavaScript) tercih edersiniz? Kararınızı, bölümde öğrendiğiniz derleyici/yorumlayıcı farklarına dayanarak gerekçelendiriniz.
2. **Kapsam Analizi:** Aşağıdaki kod çalıştırıldığında hangi satırda hata alınır? Nedenini `var` ve `let` arasındaki **kapsam** (*scope*) farkına dayanarak açıklayınız.

```
function testKapsam() {
  if (true) {
    var a = "merhaba";
    let b = "dünya";
  }
  console.log(a);
  console.log(b);
}
```

3. **Tip Dönüşümü:** "10" + 5 ve "10" - 5 ifadelerinin JavaScript'te ürettiği sonuçları ve bu sonuçlara neden farklı tip dönüşümü kurallarının uygulandığını açıklayınız.
4. **Paradigma Seçimi:** Bir öğrenci bilgi sistemi geliştiriyorsunuz; her öğrencinin adı, notu ve devamsızlık durumu olacak ve bu öğrenciler üzerinde işlem yapılacak. Bu senaryoyu yapısal programlama ile mi yoksa nesne yönelimli programlama ile mi modellemek daha uygundur? Nedenini açıklayınız.
5. **Değişken Tanımlama Kuralları:** Aşağıdaki değişken isimlerinden hangileri JavaScript'te geçerlidir, hangileri geçersizdir? Geçersiz olanlar için nedenini belirtiniz: `1sonuc`, `_gizliDeger`, `kullanici-adi`, `let`, `toplamFiyat$`.

6. Bölüm

Tarayıcı Tarafında Programlama

6.1 Dinamik Web Sayfası Nedir?

Dinamik HTML kısaca DHTML, tek başına bir programlama dili veya yorumlama dili değildir. Tarayıcı tarafında dinamik ve kullanıcı etkileşimli web sayfaları yapmayı ifade eden kavramsal bir terimdir. HTML, JavaScript, CSS ve tarayıcı tarafından bellekte oluşturulan DOM (*Document Object Model*) ağacının birleşimidir. Dinamik web sayfalarında statik belgelerin aksine, kullanıcı bir düğmeye bastığında, bir formu doldurduğunda ya da fareyi bir öğenin üzerine getirdiğinde sayfa sunucudan yeniden yüklenmeksizin arayüz anlık olarak güncellenebilir.

6.2 JavaScript ve HTML DOM

Tarayıcı bir HTML belgesini yüklediğinde, belgenin hiyerarşik yapısını temsil eden bir nesne modeli oluşturur. Bu modele **HTML DOM** (*Document Object Model*) denir. DOM, belgedeki her bir etiketi, metni ve özniteliği (*attribute*) birer JavaScript nesnesi (düğüm – *node*) olarak belleğe yerleştirir.

§6.2.1 HTML DOM API

HTML DOM API, JavaScript dilinin tarayıcı belleğindeki bu düğüm ağacına erişmesini ve içeriği, yapıyı veya stilleri dinamik olarak değiştirmesini sağlayan standart arayüzdür.

DOM Ağaç Yapısı

HTML belgesindeki en tepe nesne *window* nesnesidir. Bunun altında bulunan *document* nesnesi, belgenin kök elemanı olan `<html>` etiketini, o da sırasıyla `<head>` ve `<body>` düğümlerini kapsar. Her düğüm bir ebeveyn (*parent*), çocuk (*child*) veya kardeş (*sibling*) ilişkisine sahiptir.

§6.2.2 HTML Öğelerini Seçme

JavaScript ile DOM üzerinde işlem yapabilmek için öncelikle ilgili belgenin/etiketin seçilmesi (elde edilmesi) gerekir. Bunun için temel yöntemler şunlardır:

- ◇ `document.getElementById(id)`: Belirtilen benzersiz *id* değerine sahip tek bir öğeyi seçer.
- ◇ `document.getElementsByClassName(className)`: Belirtilen CSS sınıfına sahip tüm öğeleri bir *HTMLCollection* dizisi olarak döner.
- ◇ `document.getElementsByTagName(tagName)`: Belirtilen etiket ismindeki tüm öğeleri seçer.
- ◇ `document.querySelector(cssSelector)`: Verilen CSS seçicisine uyan *ilk* öğeyi döner.
- ◇ `document.querySelectorAll(cssSelector)`: Verilen CSS seçicisine uyan *tüm* öğeleri bir *NodeList* koleksiyonu olarak döner.

```
// ID ile seçim
const baslik = document.getElementById("anaBaslik");

// CSS Seçicisi ile seçim
const butonlar = document.querySelectorAll(".btn-primary");
```

§6.2.3 HTML Değiştirme

HTML öğelerinin içerikleri veya öznitellikleri JavaScript ile dinamik olarak güncellenebilir:

```
const paragraf = document.getElementById("aciklama");

// Metin içeriğini değiştirme (Güvenli yöntem)
paragraf.textContent = "Yeni açıklama metni.";

// HTML içeriğini değiştirme
```

```
paragraf.innerHTML = "<strong>Koyu metin</strong> eklendi.";

// Öznitelik (Attribute) değiştirme
const resim = document.getElementById("profilResmi");
resim.setAttribute("src", "yeni_resim.png");
```

§6.2.4 CSS Değiştirme

Öğelerin stilleri doğrudan `style` nesnesi üzerinden ya da sınıf ekleyip çıkararak (`classList`) güncellenebilir:

```
const kutu = document.querySelector(".kutu");

// Doğrudan inline CSS değiştirme
kutu.style.backgroundColor = "blue";
kutu.style.fontSize = "20px";

// Sınıf ekleme/çıkarma (Önerilen yöntem)
kutu.classList.add("aktif");
kutu.classList.remove("pasif");
kutu.classList.toggle("gizli"); // Var ise siler, yok ise ekler
```

§6.2.5 Form Verilerini Doğrulama

Kullanıcının forma girdiği verilerin sunucuya gönderilmeden önce istemci tarafında denetlenmesi, sunucu yükünü azaltır ve kullanıcı deneyimini artırır.

```
const form = document.getElementById("girisFormu");

form.addEventListener("submit", function(event) {
  const eposta = document.getElementById("eposta").value;

  if (!eposta.includes("@")) {
    alert("Geçerli bir e-posta adresi giriniz!");
    event.preventDefault(); // Formun sunucuya gönderilmesini engeller
  }
});
```

§6.2.6 DOM Animasyonu

DOM öğelerini JavaScript ile hareket ettirmek için zamanlayıcılar (`setInterval`) veya yüksek performanslı grafik güncellemeleri sağlayan `requestAnimationFrame()` yöntemi kullanılır.

```
let pozisyon = 0;
const kutu = document.getElementById("animasyonKutusu");

function kareAkar() {
  if (pozisyon < 350) {
    pozisyon++;
    kutu.style.top = pozisyon + "px";
    kutu.style.left = pozisyon + "px";
    requestAnimationFrame(kareAkar); // Akıcı 60 FPS animasyon
  }
}

requestAnimationFrame(kareAkar);
```

§6.2.7 Document Properties and Methods

`document` nesnesi web belgesini temsil eder ve sayfa düzeyindeki erişimler için temel mülk (*property*) ve yöntemleri (*method*) sunar:

- ◇ `document.title`: Sayfa başlığını okur veya değiştirir.
- ◇ `document.body`: Belgenin `<body>` düğümüne doğrudan erişim sağlar.
- ◇ `document.createElement(tagName)`: Bellekte yeni bir HTML öğesi oluşturur.
- ◇ `document.createTextNode(text)`: Yeni bir metin düğümü oluşturur.

§6.2.8 Element Properties and Methods

Tekil HTML elemanları üzerindeki işlemleri yürütmek için kullanılan temeller şunlardır:

- ◇ `element.appendChild(childNode)`: Bir düğümü ilgili öğenin son çocuğu olarak ekler.
- ◇ `element.removeChild(childNode)`: Belirtilen çocuk düğümü kaldırır.
- ◇ `element.addEventListener(event, callback)`: Öğeye tıklama, odaklanma gibi olay dinleyicileri bağlar.
- ◇ `element.children`: Öğenin alt çocuk düğümlerini koleksiyon olarak verir.

6.3 XML

Extensible Markup Language (Kısaca XML) ise metinleri değil, verileri depolamak, iletmek ve yeniden oluşturmak için kullanılan bir işaretleme dili ve dosya biçimidir. XML Dili verilerin hem insanlar hem de makineler tarafından okunabilmesi için bir dizi kural tanımlar.

İlk standardı 1998 yılında W3C tarafından yayımlanan XML'in, günümüzde yaygın olarak kullanılan sürümü hâlâ XML 1.0'dır (en son gözden geçirilmiş beşinci baskısı 2008 yılında yayımlanmıştır); 2004'te tanıtılan XML 1.1 ise sınırlı sayıda özel karakter ihtiyacı dışında yaygın bir kabul görmemiştir.

Aşağıda bir CD kataloğu XML Belgesi içeriği verilmiştir:

```
<CATALOG>
  <CD>
    <TITLE>Empire Burlesque</TITLE>
    <ARTIST>Bob Dylan</ARTIST>
    <COUNTRY>USA</COUNTRY>
    <COMPANY>Columbia</COMPANY>
    <PRICE>10.90</PRICE>
    <YEAR>1985</YEAR>
  </CD>
  <CD>
    <TITLE>Hide your heart</TITLE>
    <ARTIST>Bonnie Tyler</ARTIST>
    <COUNTRY>UK</COUNTRY>
    <COMPANY>CBS Records</COMPANY>
    <PRICE>9.90</PRICE>
    <YEAR>1988</YEAR>
  </CD>
</CATALOG>
```

6.4 AJAX

§6.4.1 AJAX Nedir?

AJAX, İngilizce "*Asynchronous JavaScript and XML*" (Asenkron JavaScript ve XML) ifadesinin kısaltmasıdır ve istemci tarafında çeşitli web teknolojilerini kullanarak asenkron web uygulamaları oluşturan bir dizi web geliştirme tekniğini ifade eder. Sayfanın tamamı yeniden yüklenmeden, yalnızca ihtiyaç duyulan verilerin arka planda sunucudan alınarak güncellenmesine imkan tanır.

§6.4.2 AJAX XMLHttpRequest

Tarayıcının dahili XMLHttpRequest nesnesi, web sayfalarında Ajax kodunu icra eder ve web sitelerinin sayfayı yenilemeden ekrana içerik yüklemesine olanak tanır.

Ajax'ın kendisi yeni bir teknoloji veya yeni bir dil değil, var olan tarayıcı özelliklerini bir araya getiren bir tekniktir; bu tekniğin temelini oluşturan XMLHttpRequest nesnesi ise W3C tarafından 2006 yılında taslak olarak yayımlanmış ve 2014 yılında resmî bir öneri (*Recommendation*) hâline gelmiştir.

§6.4.3 AJAX Request

Aşağıda XMLHttpRequest nesnesi kullanılarak yapılan örnek bir istek mantığı verilmiştir:

```
<!DOCTYPE html>
<html>
  <body>
    <h2>Using the XMLHttpRequest Object</h2>
    <div id="demo">
      <button type="button" onclick="loadXMLDoc()">Content Değiştir</button>
    </div>
    <script>
      function loadXMLDoc() {
        var xhttp = new XMLHttpRequest();
        xhttp.onreadystatechange = function() {
          if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").innerHTML = this.responseText;
          }
        };
        xhttp.open("GET", "xmlhttp_info.txt", true);
        xhttp.send();
      }
    </script>
```

```
</body>
</html>
```

§6.4.4 AJAX Response

Sunucudan gelen yanıtlar `XMLHttpRequest` nesnesinin durum özelliklerine göre işlenir:

- ◇ **readyState**: İstek durumunu temsil eder (0: Tanımlanmadı, 1: Sunucu bağlantısı kuruldu, 2: İstek alındı, 3: İstek işleniyor, 4: İstek tamamlandı ve yanıt hazır).
- ◇ **status**: HTTP Durum Kodu (200: "OK" / Başarılı, 404: "Not Found" / Bulunamadı).
- ◇ **responseText**: Sunucudan gelen ham metinsel yanıtı saklar.
- ◇ **responseXML**: Sunucudan gelen yanıt XML biçimindeyse bir XML DOM belgesi olarak saklar.

§6.4.5 AJAX ile XML Dosyası Uygulaması

Sunucuda bulunan bir XML belgesini okuyarak içeriğini dinamik bir HTML tablosuna dönüştüren AJAX uygulaması aşağıda verilmiştir:

```
function loadCDCatalog() {
    const xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            const xmlDoc = this.responseXML;
            const cdler = xmlDoc.getElementsByTagName("CD");
            let tablo = "<tr><th>Başlık</th><th>Sanatçı</th></tr>";

            for (let i = 0; i < cdler.length; i++) {
                tablo += "<tr><td>" +
                    cdler[i].getElementsByTagName("TITLE")[0].childNodes[0].nodeValue +
                    "</td><td>" +
                    cdler[i].getElementsByTagName("ARTIST")[0].childNodes[0].nodeValue +
                    "</td></tr>";
            }
            document.getElementById("cdTablosu").innerHTML = tablo;
        }
    };
    xhttp.open("GET", "cd_catalog.xml", true);
    xhttp.send();
}
```

6.5 JQUERY

jQuery, "Less Write, Do More" (Daha Az Yaz, Daha Çok İş Yap) felsefesiyle geliştirilmiş, HTML DOM gezintisini, olay yönetimini, animasyonları ve AJAX etkileşimlerini basitleştiren hafif bir JavaScript kütüphanesidir.

§6.5.1 jQuery Selectors

jQuery seçicileri, HTML elemanlarını bulmak ve üzerinde işlem yapmak için CSS seçici sözdizimini kullanır. Seçim işlemleri varsayılan olarak dolar işareti (\$) ile başlar:

```
$("#p").hide(); // Tüm <p> etiketlerini gizler
$("#test").show(); // id="test" olan elemanı gösterir
$(".ornek").css("color", "red"); // class="ornek" olanların rengini kırmızı yapar
```

§6.5.2 jQuery HTML

jQuery, DOM içeriklerini ve özniteliklerini kolayca manipüle etmeye yönelik yöntemler sunar:

- ◇ **text()**: Seçilen elemanların metin içeriğini ayarlar veya okur.
- ◇ **html()**: HTML içeriğini (etiketleriyle birlikte) ayarlar veya okur.
- ◇ **val()**: Form alanlarının değerini okur veya ayarlar.
- ◇ **attr()**: Öznitelik (*attribute*) değerlerini değiştirir.

§6.5.3 jQuery CSS

jQuery ile CSS özelliklerini okumak veya doğrudan elemana uygulamak oldukça pratiktir:

```
// Tekil CSS özelliği değiştirme
$("#h1").css("background-color", "yellow");

// Çoklu CSS özelliği değiştirme
$("#p").css({
    "background-color": "gray",
    "font-size": "18px",
```

```
"padding": "10px"
});
```

§6.5.4 jQuery DOM

jQuery DOM hiyerarşisinde gezinmeyi (*traversing*) ve yeni düğümler eklemeyi kolaylaştırır:

```
// Yeni eleman ekleme
$("body").append("<p>Yeni Paragraf</p>"); // Sona ekler
$("body").prepend("<p>İlk Paragraf</p>"); // Başa ekler

// Düğüm ağacında gezinme
$("#cocukDugum").parent(); // Bir üst ebeveyne erişir
$(".kutu").find("span"); // Kutu içindeki tüm span'ları arar
```

6.6 Tek Sayfa Uygulaması

Tek Sayfa Uygulaması yani kısaca **SPA** (*Single Page Application*), sayfa geçişlerinde tarayıcının her seferinde sunucudan tüm sayfayı yeniden istediği geleneksel yöntemin aksine, tek bir HTML belgesini yükleyip geri kalan tüm etkileşimi JavaScript ile istemci tarafında yöneten bir web uygulaması mimarisidir. Amaç, web sitesinin bir masaüstü/mobil uygulaması gibi hissettirmesini sağlayan, anlık ve akıcı geçişlerdir.

§6.6.1 Geleneksel Çok Sayfalı Uygulama ile SPA Karşılaştırması

Bu farkı somutlaştırmak için, bugüne kadar öğrendiğimiz geleneksel modeli (Bölüm 1.9'de gördüğümüz her tıklamada yeni bir HTTP isteği ve tam sayfa yeniden yüklemesi) SPA modeliyle karşılaştıralım:

Özellik	Çok Sayfalı Uygulama (MPA)	Tek Sayfa Uygulaması (SPA)
Sayfa geçişi	Her bağlantıda sunucudan yeni bir HTML dosyası indirilir ve sayfa baştan render edilir.	Tek bir HTML dosyası bir kez yüklenir; geçişlerde yalnızca gereken veri indirilip DOM güncellenir.
Hız (geçişlerde)	Her geçişte tam bir HTTP isteği/render döngüsü yaşanır, gözle görülür bir "beyaz an" (flaş) oluşabilir.	Geçişler anlıktır; sayfa hiç "yanıp sönmez".
Sunucu yükü	Her istek, sunucudan tam bir HTML sayfası üretilmesini gerektirir.	Sunucu genellikle yalnızca ham veri (ör. JSON) döndürür; HTML üretimi tarayıcıda yapılır.
SEO ve ilk yükleme	Arama motorları içeriği doğrudan HTML içinde bulur; ilk sayfa hızlı görünür.	İçerik JavaScript ile sonradan oluşturulduğu için ek önlemler (<i>Server-Side Rendering</i> vb.) gerekmedikçe arama motorları ve ilk yükleme süresi dezavantajlı olabilir.

Tablo 6.1: Tek Sayfa ve Çok Sayfalı Uygulamaların Karşılaştırılması

§6.6.2 SPA Nasıl Çalışır?

Bir SPA'nın perde arkasında üç temel mekanizma birlikte çalışır. Dikkat ederseniz üçü de bu bölümde zaten öğrendiğiniz kavramlardır:

- 1. Tek Bir HTML İskeleti:** Sunucu yalnızca boş denecek kadar sade, genellikle içinde tek bir `<div id="app"></div>` barındıran bir `index.html` dosyası gönderir.
- 2. İstemci Tarafı Yönlendirme (*Client-Side Routing*):** Kullanıcı bir bağlantıya tıkladığında tarayıcı sunucuya **yeni bir sayfa istemez**; bunun yerine JavaScript, adres çubuğundaki URL'yi (`location.hash` ile ya da Tarayıcı Geçmişi API'sindeki `history.pushState()` yöntemiyle) kendisi günceller ve hangi içeriğin gösterileceğine kendisi karar verir.
- 3. Dinamik İçerik Güncelleme:** Yeni "sayfanın" içeriği, Bölüm 1.9'de öğrendiğimiz AJAX teknikleriyle (bu bölümdeki `XMLHttpRequest` veya modern tarayıcılarda onun yerini alan `fetch()` ile) sunucudan alınır ve önceki alt bölümlerde gördüğümüz `innerHTML` ya da DOM API'siyle sayfaya –sayfa hiç yenilenmeden– basılır.

Minik Bir SPA Örneği

Herhangi bir çatı (*framework*) kullanmadan, yalnızca bu bölümde öğrendiğiniz `querySelector`, `innerHTML` ve olay dinleyicileriyle iki "sayfalı" basit bir SPA kuralım. Önce HTML iskeletimiz:

```
<!DOCTYPE html>
<html lang="tr">
  <body>
    <nav>
```

```

    <a href="#">Ana Sayfa</a>
    <a href="#/hakkimizda">Hakkımızda</a>
  </nav>

  <!-- Tüm "sayfalar" bundan sonra bu tek kapsayıcının içine basılacak -->
  <div id="app"></div>

  <script src="spa.js"></script>
</body>
</html>

```

Şimdi de yönlendirme mantığını kuran `spa.js` dosyasını yazalım:

```

// 1. Adım: Her "sayfanın" HTML içeriğini bir nesnede saklıyoruz
const rotalar = {
  "/": "<h1>Ana Sayfa</h1><p>SPA örneğimize hoş geldiniz!</p>",
  "/hakkimizda": "<h1>Hakkımızda</h1><p>Bu sayfa hiç yeniden yüklenmedi.</p>"
};

// 2. Adım: Adres çubuğundaki '#' işaretinden sonrasını okuyup ilgili içeriği basan fonksiyon
function sayfayiRenderEt() {
  const mevcutYol = location.hash.slice(1) || "/";
  const icerik = rotalar[mevcutYol] || "<h1>404</h1><p>Sayfa bulunamadı.</p>";

  document.getElementById("app").innerHTML = icerik;
}

// 3. Adım: Adres çubuğu her değiştiğinde (bağlantıya tıklanınca) yeniden render et
window.addEventListener("hashchange", sayfayiRenderEt);

```

// 4. Adım: Sayfa ilk açıldığında da bir kez çalıştır

```

window.addEventListener("DOMContentLoaded", sayfayiRenderEt);

```

Kullanıcı "Hakkımızda" bağlantısına tıkladığında tarayıcı sunucudan **hiçbir yeni HTML dosyası istemez**; yalnızca `hashchange` olayı tetiklenir, JavaScript `rotalar` nesnesinden ilgili içeriği bulur ve `#app` kapsayıcısının içeriğini değiştirir. Adres çubuğunda `index.html#/hakkimizda` görünmesine rağmen sunucuya ikinci bir istek gitmemiştir.

Neden `location.hash`?

Gerçek SPA çatıları (React Router, Vue Router vb.) genellikle daha temiz görünen adresler (`/hakkimizda` gibi, `#` işaretsiz) üretmek için `history.pushState()` yöntemini kullanır. Ancak bu yöntem, sunucunun **her yolu** `index.html`'e yönlendirecek şekilde ayrıca yapılandırılmasını gerektirir. `location.hash` kullanımı ise herhangi bir sunucu yapılandırması gerektirmediği için öğrenmeye başlarken daha basit bir başlangıç noktasıdır.

AJAX ile Veri Çeken Bir SPA Rotası Örneği

Şimdi bu iskeleti, bölümün AJAX kısmında öğrendiğimiz tekniklerle birleştirelim. Aşağıdaki örnekte `/urunler` rotası, sayfa açıldığında sunucudan gerçek veri çeker ve listeler:

```

async function urunlerSayfasi() {
  const kapsayici = document.getElementById("app");
  kapsayici.innerHTML = "<h1>Ürünler</h1><p>Yükleniyor...</p>";

  // fetch(), modern tarayıcılarda XMLHttpRequest'in yerini alan,
  // Bölüm 5.3'te gördüğümüz AJAX isteklerini daha kısa yazmamızı sağlayan bir API'dir.
  const yanit = await fetch("/api/urunler");
  const urunler = await yanit.json();

  let liste = "<h1>Ürünler</h1><ul>";
  urunler.forEach(urun => {
    liste += `<li>${urun.ad} - ${urun.fiyat} TL</li>`;
  });
  liste += "</ul>";

  kapsayici.innerHTML = liste;
}

```

```
// rotalar nesnesini fonksiyonlarla eşleştirip genişletiyoruz
const rotaFonksiyonlari = {
  "/": () => { document.getElementById("app").innerHTML = "<h1>Ana Sayfa</h1>"; },
  "/urunler": urunlerSayfasi
};

function sayfayiRenderEt() {
  const mevcutYol = location.hash.slice(1) || "/";
  (rotaFonksiyonlari[mevcutYol] || (() => {
    document.getElementById("app").innerHTML = "<h1>404</h1>";
  })))();
}
```

Bu örnek, bölümün başından beri öğrendiğimiz üç ayrı konunun – **DOM seçme/değiştirme**, **AJAX ile veri çekme** ve **istemci tarafı yönlendirme** – pratikte nasıl bir araya gelerek gerçek bir SPA'yı oluşturduğunu gösterir. Üretimdeki büyük SPA projelerinde bu mantığın tamamı (rota eşleştirme, önbellekleme, geçiş animasyonları, hata yönetimi vb.) elle yazmak yerine, aşağıda tanıtılan hazır çatılar aracılığıyla yönetilir.

§6.6.3 SPA Kütüphaneleri ve Çatıları

SPA'ların arkasında JavaScript ile oluşturulmuş kütüphaneler ve çatılar (*frameworks*) bulunur:

- ◇ **Angular:** Google tarafından geliştirilen, TypeScript tabanlı, yönlendirme ve durum yönetimi gibi araçları kutudan çıktığı gibi (*out-of-the-box*) sunan, büyük ölçekli kurumsal SPA'lar için tercih edilen kapsamlı bir çatıdır.
- ◇ **React.js:** Meta (Facebook) tarafından geliştirilen, yalnızca kullanıcı arayüzü katmanına odaklanan, Sanal DOM (*Virtual DOM*) tekniğiyle performansı artıran hafif bir kütüphanedir; yönlendirme veya durum yönetimi gibi araçlar ayrı kütüphanelerle (*React Router* vb.) eklenir.
- ◇ **Vue.js:** Küçük ve orta ölçekli projeler için ve kademeli benimsemenin gerektiği durumlarda idealdir.
- ◇ **Svelte:** Performansa duyarlı uygulamalar ve daha küçük projeler için en iyisidir.
- ◇ **Ember.js:** Yapılandırma yerine gelenekselliğe ihtiyaç duyan iddialı web uygulamaları için mükemmeldir.
- ◇ **Backbone.js:** Esnekliğin gerekli olduğu hafif projeler için idealdir.
- ◇ **Aurelia:** Modülerliğe sahip modern bir çerçeveye ihtiyaç duyan büyük uygulamalar için uygundur.
- ◇ **Mithril.js:** Performansın kritik olduğu küçük projeler için idealdir.
- ◇ **Polymer:** Tekrar kullanılabilir bileşenlere sahip web bileşenleri ve uygulamalar geliştirmek için idealdir.
- ◇ **Alpine.js:** Ağır çerçevelere ihtiyaç duymayan, etkileşim gerektiren küçük projeler için en iyisidir.

6.7 Uygulamalı Projeler

Bu bölümde öğrendiğiniz DOM manipülasyonu, olay yönetimi, AJAX ve SPA kavramlarını pekiştirmek için aşağıdaki beş projeden birini veya birkaçını tamamlamayı deneyin. Hepsi yalnızca HTML ve JavaScript ile (istediğiniz kadar CSS ekleyerek) tamamlanabilir.

1. **Basit Sayaç Uygulaması:** + ve - butonlarıyla artan/azalan bir sayaç yapın. `document.getElementById` ile sayının yazılı olduğu öğeyi seçin, düğmelere `addEventListener("click", ...)` ile olay dinleyicisi ekleyin ve değeri `innerHTML` veya `textContent` ile güncelleyin.
2. **Yapılacaklar Listesi (To-Do List):** Bir `input` ve bir "Ekle" butonuyla kullanıcının yazdığı görevi bir `ul` listesine yeni bir `li` olarak ekleyin; her `li`'nin yanına bir "Sil" butonu koyup tıklandığında o öğeyi DOM'dan kaldırın.
3. **Basit Form Doğrulama (Validation):** Bir kayıt formunda, kullanıcı "Gönder" butonuna bastığında JavaScript ile alanların boş olup olmadığını kontrol edin; boş alanların yanında (sayfa hiç yenilenmeden) bir hata mesajı gösterin.
4. **Karanlık/Aydınlık Mod Değiştirici:** Sayfanın bir köşesine bir düğme koyun; tıklandığında `document.body.classList.toggle("koyu-tema")` ile `<body>` etiketine bir sınıf ekleyip çıkararak (CSS'te bu sınıf için farklı arka plan/yazı renkleri tanımlayarak) tema değişimini simüle edin.
5. **Üç Sayfalı Mini SPA:** Bölümde öğrendiğiniz `location.hash` tabanlı yönlendirmeyi kullanarak "Ana Sayfa", "Hakkımızda" ve "İletişim" rotalarından oluşan, sayfa hiç yeniden yüklenmeden gezinilen küçük bir tek sayfa uygulaması (SPA) kurun.

6.8 Yapılacaklar Listesi Örneği

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Yapılacaklar Listesi</title>
    <style>
```

```

body {
  font-family: Arial, sans-serif;
  max-width: 400px;
  margin: 40px auto;
  padding: 20px;
}

h1 {
  text-align: center;
}

#yeniGorev {
  width: 70%;
  padding: 8px;
}

#ekleButonu {
  padding: 8px 12px;
}

ul {
  list-style: none;
  padding: 0;
}

li {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 1px solid #ddd;
  padding: 8px 0;
}

.silButonu {
  background-color: #e74c3c;
  color: white;
  border: none;
  padding: 4px 10px;
  cursor: pointer;
}
</style>
</head>
<body>

<h1>Yapılacaklar Listem</h1>

<div>
  <input type="text" id="yeniGorev" placeholder="Yeni görev yazın...">
  <button id="ekleButonu">Ekle</button>
</div>

<ul id="gorevListesi"></ul>

<script>
// 1. Adım: Sayfadaki ilgili öğeleri seçiyoruz
const girisAlani = document.getElementById("yeniGorev");
const ekleButonu = document.getElementById("ekleButonu");
const liste = document.getElementById("gorevListesi");

// 2. Adım: Yeni bir görev ekleyen fonksiyonu tanımlıyoruz
function gorevEkle() {
  const gorevMetni = girisAlani.value.trim();

  if (gorevMetni === "") {
    alert("Lütfen bir görev yazın!");
    return;
  }

```

```
}

// Yeni bir <li> ögesi oluşturuyoruz
const yeniOge = document.createElement("li");

// Görev metnini tutacak bir <span> ekliyoruz
const metinSpani = document.createElement("span");
metinSpani.textContent = gorevMetni;

// Sil butonunu oluşturuyoruz
const silButonu = document.createElement("button");
silButonu.textContent = "Sil";
silButonu.className = "silButonu";

// Sil butonuna tıklanınca bu <li> ögesini listeden kaldır
silButonu.addEventListener("click", function () {
    liste.removeChild(yeniOge);
});

// Parçaları birleştirip listeye ekliyoruz
yeniOge.appendChild(metinSpani);
yeniOge.appendChild(silButonu);
liste.appendChild(yeniOge);

// Giriş kutusunu temizliyoruz
girisAlani.value = "";
girisAlani.focus();
}

// 3. Adım: "Ekle" butonuna tıklanınca gorevEkle() çalışsın
ekleButonu.addEventListener("click", gorevEkle);

// Bonus: Enter tuşuna basınca da görev eklensin
girisAlani.addEventListener("keydown", function (olay) {
    if (olay.key === "Enter") {
        gorevEkle();
    }
});
</script>

</body>
</html>
```

7. Bölüm

Web Sunucusu Tarafında Programlama

7.1 Common Gateway Interface

Şu ana kadar istemci olarak web tarayıcılarının akıllı hale getirilmesi için HTML-JS-CSS ile XML teknolojilerini öğrendik. Tarayıcılara benzer şekilde modern web sunucuları sadece istekleri karşılayan ve ona cevap veren sunucular değildir.

İstemci tarafında nasıl JS varsa sunucu tarafında da istekleri işleyip cevaplayan teknolojiler bulunur. Bu teknolojiler; HTML dosyasını cevap olarak göndermek yerine, içerisine veriler ekleyerek gönderebilir. Eklenenecek veriler bir başka sunucudan alınabileceği gibi veri tabanından da alabilir. Gelen isteklerden dosyaları değiştirebilir, silebilir veya veri tabanlarına veri yazabilir.

Ortak Ağ geçidi Arayüzü yani kısaca **CGI** (*Common Gateway Interface*), web sunucularının HTTP kullanıcı isteklerini işlemek için harici bir programı çalıştırmasını sağlayan bir ara yüzdür. 1993 yılında NCSA tarafından Httpd için geliştirilmiştir. Bu sayede Web sunucusunun kendisi veya uzantı (*extension*) modülleri aracılığıyla sunucu tarafı betik (*script*) çalıştırma devri 1996 yılında başlamıştır.

7.2 Sunucu Tarafı Betik

Sunucu tarafında betik (*server-side script*) çalıştırma; her kullanıcının web sitesine olan isteğine göre özelleştirilmiş bir yanıt üreten bir web sunucusunda betik (*script*) kullanmayı içeren bir tekniktir.

Sunucu tarafı betikler, PHP (*Personal Home Page*), ASP (*Active Server Pages*) ve JSP (*JavaServer Pages*) gibi birçok programlama dilinden herhangi birinde yazılabilir. Bu betikler, JavaScript gibi HTML belgesine gömülü betiklerin bir web tarayıcısında çalıştırıldığı gibi çalışmaz.

Şu ana kadar öğrendiğimiz ve içerisinde sadece HTML öğeleri bulunan dosyalar, statik web sayfaları (*static web pages*) olarak adlandırılır. Sunucu tarafında çalıştırılacak betik içeren sayfalar ise *.html veya *.htm olarak değil farklı uzantılara sahip olurlar. Sunucuda PHP betiği içeren dosyalar *.php uzantısıyla, ASP betiği içeren dosyalar *.asp ve JSP betiği içeren dosyalar ise *.jsp uzantısıyla saklanırlar.

Bölüm ??'de gösterilen tarayıcı-sunucu iletişimi, sunucu tarafı betik olduğunda aşağıdaki şekilde çalışır;

- ◊ Web tarayıcıda betik içeren URL (örneğin www.ornek.com/demo.php) yazılır.
- ◊ Sunucu talep edilen betik dosyasını (demo.php) tarayıcıya göndermeden önce betik kurallarına göre yorumlar. Betik çalıştırdıktan sonra oluşan nihai HTML belgesi tarayıcıya gönderilir.
- ◊ Tarayıcı gelen HTML belgesinden Belge nesne modeli yani DOM (*Document Object Model*), Kademeli Stil Sayfası Nesne Modeli olan CSSOM (*Cascading Style Sheet Object Model*) ağaçlarını oluşturur ve oluşan bu ağaçlardan görselleştirme (*render*) yaparak kullanıcıya sunar.

Sunucu tarafında en çok kullanılan betikler¹;

- ◊ **ASP** (*Active Server Pages*), Microsoft'un dinamik web sayfaları için ilk sunucu tarafı betik dili ve motorudur. 1996 ile 2000 yılları arasında popüler olmuştur.
- ◊ **JSP** (*JavaServer Pages*; 2017'den itibaren Eclipse Vakfı bünyesinde *Jakarta Server Pages* olarak da anılır), Yazılım geliştiricilerin HTML, XML, SOAP veya diğer belge türlerine dayalı dinamik olarak oluşturulmuş web sayfaları için sunucu tarafında çalışan bir teknolojidir. MVC (*Model-View-Controller*) desenini (*pattern*) kullanır.
- ◊ **PHP** (*Personal Home Page*), web geliştirmeye yönelik genel amaçlı bir betik dilidir. 1995 yılından sonra birçok projede kullanılmaktadır.
- ◊ **ASP.NET Web Forms**, Microsoft ASP.NET teknolojisini kullanan bir web uygulama çerçevesidir (*application framework*). 2002 ile 2010 yılları arasında çok kullanılmıştır.
- ◊ **Razor**, C# veya VB.NET programlama dilleriyle dinamik web sayfaları oluşturmak için kullanılan bir ASP.NET teknolojisidir. 2010 yılından beri kullanılmaktadır.
- ◊ **ASP.NET MVC**, Microsoft tarafından geliştirilen, MVC desenini uygulayan bir web uygulama teknolojisidir. 2012 yılından beri Apache License 2.0 ile yayımlanmaktadır.

¹https://w3techs.com/technologies/overview/programming_language

- ◇ **Node.js**, JavaScript geleneksel olarak istemci tarafı bir dil olsa da, Node.js JavaScript'i sunucu tarafında betik olarak çalışır.
- ◇ Bunların yanında **Google Apps Script**, **Go**, **Ruby**, **Python** ve **Perl** dilleri de web sunucularında betik olarak kullanılmaktadır.

Bu bölümde dünyada oldukça popüler olan Apache-PHP-MySQL üçlüsü üzerinde durulacaktır. İzleyen bölümlerde ise daha modern yaklaşımları olan ASP.NET teknolojisine örnek verilecektir.

Kişisel Ana Sayfa yani kısaca **PHP** (*Personal Home Page*) başlangıçta web üzerinde kişisel başlangıç sayfaları oluşturmak için geliştirilmiştir. Günümüzde **Hiper Metin Ön İşlemcisi** (*Hypertext Preprocessor*) olarak oldukça yaygın olarak kullanılmaktadır.

PHP nin C++ diline benzeyen bir yazım biçimi vardır. HTML belgesine PHP kodları derç edilerek sunucunun dosyayı göndermeden önce PHP dili ile işlenmesini sağlar. Özetle PHP dili sunucuda çalışır ve yorumlanır ve sadece çıktısı kullanıcı bilgisayarındaki tarayıcıya gönderilir. Böylece veritabanı sunucusunda bulunan verilere sadece web sunucusu üzerinden erişim sağlanır.

Sunucu üzerinde çalışan PHP dilini kullanabilmek için bir web sunucusuna ihtiyaç duyarız. PHP dili ile en çok kullanılan Apache web sunucusudur. Kullanıcılar kendi kişisel bilgisayarlarından veri tabanına direk müdahale edemezler.

Eğer veritabanı kullanılacak ise web sunucusuna bir veritabanı kurulması gereklidir. PHP ile en çok kullanılan veri tabanı sunucusu MySQL veri tabanıdır.

Şimdi bu üçlüyü başlıklar halinde açıklayalım.

7.3 Apache Web Sunucusu

Apache HTTP Sunucusu (*Apache HTTP Server*), dünyada en yaygın kullanılan açık kaynaklı, modüler ve platformlar arası web sunucu yazılımlarından biridir. Tarayıcılardan gelen HTTP/HTTPS isteklerini karşılar, statik dosyaları (HTML, CSS, JS, resim) doğrudan sunar veya PHP gibi yorumlayıcı modülleri tetikleyerek dinamik içerik üretip istemciye geri yanıt döner.

Apache Mantiğı ve Çalışma Prensibi

Apache, Varsayılan olarak **Port 80** (HTTP) ve **Port 443** (HTTPS) ağ kapılarını dinler. Tarayıcıdan bir web sayfasına erişmek istediğinde, Apache arka planda bir süreç (*process*) veya iş parçacığı (*thread*) oluşturarak isteği işler ve **DocumentRoot** dizinindeki dosyaları sunar.

§7.3.1 Apache2 Paketinin Kurulması

Linux ortamında Apache kurulumu paket yöneticisi (**apt**) aracılığıyla kolayca gerçekleştirilir. Kurulumu başlamanın önce sistem paket depolarının güncelliğinden emin olunmalıdır. Apache sunucu yazılımı Debian tabanlı sistemlerde **apache2** adıyla bulunur:

```
sudo apt update && sudo apt upgrade -y
sudo apt install apache2 -y
```

§7.3.2 Servis Durumunun ve Portların Kontrolü

Kurulum tamamlandıktan sonra Apache servisinin çalışıp çalışmadığı denetlenir:

```
# Apache servis durumunu kontrol etme
sudo systemctl status apache2

# Servisi başlatma / durdurma / yeniden başlatma komutları
sudo systemctl start apache2
sudo systemctl stop apache2
sudo systemctl restart apache2

# Sistemin her açılışında otomatik başlamasını sağlama
sudo systemctl enable apache2
```

Güvenlik Duvarı (UFW) İzinleri

Eğer sisteminizde **ufw** (*Uncomplicated Firewall*) aktif ise, dış dünyadan gelen web trafiğine izin verilmelidir:

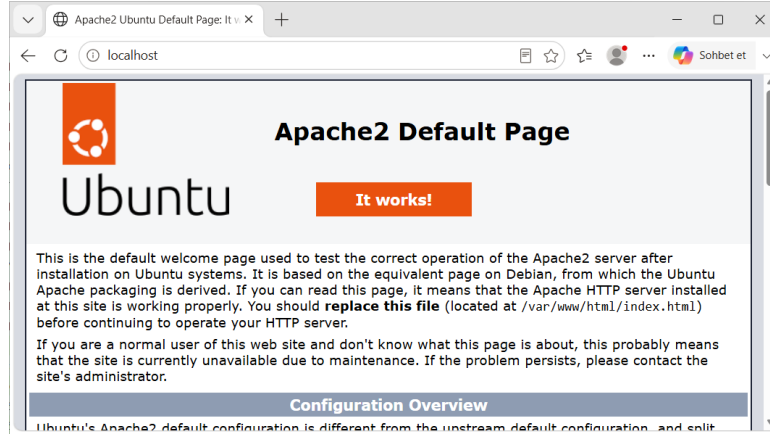
```
sudo ufw allow 'Apache Full' # Port 80 ve 443'e izin verir
sudo ufw reload
```

§7.3.3 Apache Dizin Yapısı ve Önemli Dosyalar

Ubuntu/Debian dağıtımlarında Apache konfigürasyon dosyaları düzenli bir dizin hiyerarşisine sahiptir:

- ◇ **/var/www/html**: Web sitesinin kaynak dosyalarının (HTML, PHP vb.) tutulduğu varsayılan **DocumentRoot** dizinidir. Sanal konak kullanmayacaksa PHP dosyalarımızı buraya koymalıyız.
- ◇ **/etc/apache2/apache2.conf**: Apache sunucusunun ana konfigürasyon dosyasıdır.
- ◇ **/etc/apache2/sites-available/**: Sunucuda tanımlanmış tüm sanal konakların (*Virtual Host*) yapılandırma dosyalarının yer aldığı dizindir.
- ◇ **/etc/apache2/sites-enabled/**: Aktif olarak çalışan sanal konakların sembolik bağlantılarının (*symlink*) bulunduğu dizindir.
- ◇ **/var/log/apache2/**: Erişim (*access.log*) ve hata (*error.log*) kayıtlarının tutulduğu dizindir.

Sistem düzgün kurulmuş ise tarayıcıda aşağıdaki sayfayı görürüz;



Şekil 7.1: `/var/www/html` dizinindeki varsayılan `index.html` belgesinin tarayıcı görselleştirmesi

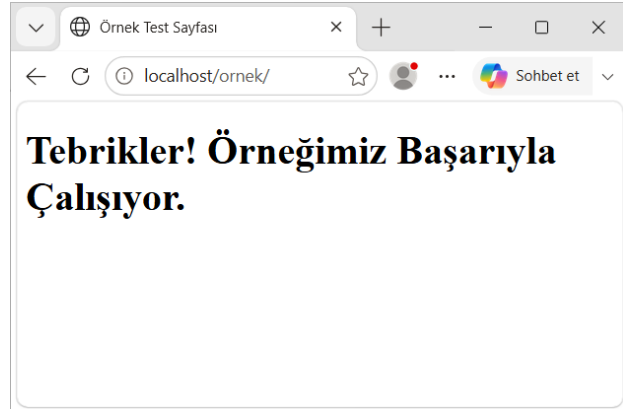
Daha sonra `/var/www/html` klasörüne geçip, *ornek* adında bir klasör oluşturulur;

```
sudo mkdir /var/www/html/ornek
```

Sonrasında `index.html` adlı dosyayı `/var/www/html/ornek` klasörü altında aşağıdaki `index.html` dosyasını oluşturuyoruz. Daha sonra bu dizini, PHP örnekleri ile dolduracağız.

```
<!-- /var/www/ornekproje/index.html -->
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Örnek Test Sayfası</title>
  </head>
  <body>
    <h1>Tebrikler! Örneğimiz Başarıyla
    ↪ Çalışıyor.</h1>
  </body>
</html>
```

Kod 7.1: *ornek* klasöründeki HTML Belgesi



Şekil 7.2: *ornek* klasöründeki HTML Belgesinin Tarayıcı Görselleştirmesi

Daha detaylı kurulum için sanal konak (*virtual host*) kurulumunu araştırabilirsiniz. Sanal konak, tek bir fiziki sunucu ve IP adresi üzerinde birden fazla farklı alan adının (*domain*) veya web sitesinin barındırılmasına olanak tanır.

§7.3.4 Apache Modülleri ve PHP Entegrasyonu

Apache, esnek modüler mimarisi sayesinde fonksiyonel kütüphaneler eklenerek genişletilebilir. Örneğin, yönlendirme kuralları için `mod_rewrite` modülü ve PHP dosyalarını işleyebilmek için `libapache2-mod-php` kullanılır.

```
# URL Yönlendirme (Rewrite) modülünü aktifleştirme
sudo a2enmod rewrite
```

```
# PHP modülünün aktif olduğunu doğrulama ve Apache'yi yenileme
sudo systemctl restart apache2
```

7.4 PHP

PHP (*Hypertext Preprocessor*), web sunucularında dinamik web sayfaları ve web uygulamaları geliştirmek için kullanılan, sunucu tarafında yorumlanan açık kaynaklı bir betik dilidir. C dili ailesinden türemiş bir sözdizimine sahiptir ve HTML kodları içerisinde doğrudan gömülebilir.

PHP Çalışma Prensibi

PHP kodları doğrudan kullanıcının tarayıcısında çalışmaz. Kullanıcı bir PHP sayfasına istek gönderdiğinde, web sunucusu (örneğin Apache) PHP yorumlayıcısını tetikler. PHP kodu sunucuda çalıştırılır, gerekli veritabanı veya dosya işlemleri yapılır ve sonuç olarak istemciye yalnızca **saf HTML/CSS/JS** çıktısı gönderilir.

§7.4.1 Linux PHP Kurulumu

Linux (Ubuntu/Debian) sistemlerde PHP yorumlayıcısını ve Apache sunucusu ile iletişimini sağlayan modülleri kurmak için uçbirim (*terminal*) üzerinden aşağıdaki komutlar yürütülür:

```
# Paket listelerini güncelleme
sudo apt update

# PHP ve Apache entegrasyon modülünün kurulumu
sudo apt install php libapache2-mod-php php-mysql -y

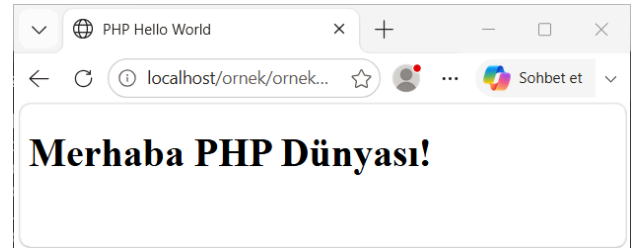
# Kurulumun ve PHP sürümünün kontrolü
php -v
```

§7.4.2 PHP Hello Uygulaması

PHP kod bloğu `<?php` etiketi ile başlar ve `?>` etiketi ile biter. Kod bloğu içerisindeki her talimat noktalı virgül (;) ile sonlandırılmalıdır. Metinsel çıktı üretmek için en sık kullanılan talimat `echo`'dur.

Aşağıda HTML belgesine gömülmüş temel bir PHP uygulaması yer almaktadır:

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>PHP Hello World</title>
  </head>
  <body>
    <h1>
      <?php
        echo "Merhaba PHP Dünyası!";
      ?>
    </h1>
  </body>
</html>
```



Şekil 7.3: PHP Merhaba Belgesi Tarayıcı Görselleştirmesi

Kod 7.2: PHP Merhaba Belgesi

§7.4.3 PHP Değişkenleri ve Veri Tipleri

Bölüm 4.2.1 başlığında matematiksel kavram olarak gördüğümüz değişkenlerin burada somut karşılığını göreceksiniz. PHP dilinde değişkenler, programın çalışması sırasında verilerin bellekte saklandığı ve bu verilere erişimi sağlayan isimlendirilmiş bellek alanlarıdır. C tabanlı bir sözdizimine sahip olan PHP'de tüm değişken isimleri mutlaka dolar simgesi (\$) ile başlar. Değişken adı dolar işaretinden sonra bir harf veya alt çizgi (_) karakteri ile başlamalıdır; rakam ile başlayamaz.

Büyüklik/Küçüklük Harf Duyarlılığı (Case-Sensitivity)

PHP'de değişken isimleri **büyük/küçük harfe duyarlıdır** (*case-sensitive*). Yani `$sayi` ile `$SAYI` bellek üzerinde iki farklı değişken olarak ele alınır. Ancak PHP fonksiyon isimleri, sınıf isimleri ve anahtar kelimeler (`echo`, `if`, `while` vb.) harf duyarlı değildir.

```
<?php
// Geçerli değişken tanımlamaları
```

```
$ogrenciAdi = "Ahmet";
$_yas = 20;
$sayi_1 = 100;

// Geçersiz değişken tanımlaması (Sözdizimi Hatası)
// $1sayi = 50;
?>
```

Dinamik Tiplendirme Mantığı

JavaScript dilinde olduğu gibi PHP de **zayıf ve dinamik tipli** (*weakly & dynamically typed*) bir dildir. Bir değişken tanımlanırken veri tipi (*integer, string, boolean* vb.) açıkça bildirilmez. PHP yorumlayıcısı, değişkene atanan değer türünü çalışma zamanında (*runtime*) otomatik olarak tespit eder. Değişkenin türü, programın ilerleyen adımlarında farklı türde bir değere atanarak değiştirilebilir.

```
<?php
$veri = "Merhaba"; // Şu an 'string' türünde
$veri = 42;        // Artık 'integer' türünde
$veri = true;      // Şimdi ise 'boolean' türünde
?>
```

Temel Veri Tipleri

PHP varsayılan olarak sekiz temel yani ilkel veri tipini destekler:

- ♦ **Integer (Tamsayı):** Ondalık kısmı olmayan pozitif veya negatif tam sayılardır (... - 2, -1, 0, 1, 2...).
- ♦ **Float / Double (Ondalık Sayı):** Kesirli ve ondalıklı sayıları temsil eder (Örn: 3.14, -0.001).
- ♦ **String (Metin):** Tırnak işaretleri arasına yazılan karakter dizileridir.
- ♦ **Boolean (Mantıksal):** Sadece **true** (doğru) veya **false** (yanlış) değerini alabilir.
- ♦ **Array (Dizi):** Birden fazla değeri tek bir değişkende saklayan bileşik veri tipidir.
- ♦ **Object (Nesne):** Sınıflardan türetilen nesne örnekleridir.
- ♦ **Resource (Kaynak):** Dosya, veritabanı bağlantısı gibi PHP dışındaki kaynaklara tutulan özel bir referans türüdür.
- ♦ **NULL:** Değişkenin herhangi bir değere sahip olmadığını veya henüz başlatılmadığını (ilklenirmediğini) belirtir.

```
<?php
$metin = "Bilgisayar Programlama"; // String
$tamSayi = 1024;                    // Integer
$ondalikSayi = 99.99;               // Float
$aktifMi = true;                    // Boolean
$bosDeger = null;                   // NULL

// Değişkenin tipini ve değerini ayrıntılı inceleme (Hata ayıklama için)
var_dump($metin);                  // string(22) "Bilgisayar Programlama"
var_dump($ondalikSayi);            // float(99.99)
?>
```

Tek Tırnak ve Çift Tırnak Arasındaki Fark

PHP’de metinsel ifadeler tanımlanırken tırnak kullanımı kritik öneme sahiptir. Çift tırnak içerisinde yazılan değişkenler yorumlanarak değerleri çıktıya basılır (*Variable Interpolation*). Tek tırnak içerisinde yazılan ifadeler ise tamamen ham metin olarak kabul edilir ve değişken isimleri yorumlanmaz.

```
<?php
$ad = "Zeynep";
// Çift Tırnak: Değişken yorumlanır
echo "Hoş geldin $ad"; // Çıktı: Hoş geldin Zeynep

// Tek Tırnak: Değişken olduğu gibi yazdırılır
echo 'Hoş geldin $ad'; // Çıktı: Hoş geldin $ad
?>
```

PHP İşleçleri ve Metin Birleştirme

PHP’de matematiksel ve mantıksal işlemler için standart işleçler (*operators*) kullanılır. Metin birleştirme işleminde JavaScript’teki + operatöründen farklı olarak **nokta (.)** operatörü tercih edilir.

```
<?php
$isim = "Ahmet";
$soyisim = "Yılmaz";

// Metin Birleştirme (String Concatenation)
```

```
$tamIsim = $isim . " " . $soyisim; // Ahmet Yılmaz

// Aritmetik İşleçler (Operators)
$a = 15;
$b = 4;
$toplam = $a + $b; // 19
$kalani = $a % $b; // 3 (Mod alma)

// Birleştirerek Atama Operatörü (.=)
$mesaj = "Merhaba ";
$mesaj .= $tamIsim; // Merhaba Ahmet Yılmaz
?>
```

Tip Dönüşümü ve Tip Zorlaması

PHP, işlem sırasında farklı tipteki verileri otomatik olarak birbirine dönüştürebilir (*Implicit Casting*). Ancak yazılımcı isterse parantez içerisinde hedef veri tipini belirterek açık tip dönüşümü de (*Explicit Casting*) yapabilir.

```
<?php
// Otomatik Tip Dönüşümü (Type Coercion)
$sayiMetin = "10";
$toplam = $sayiMetin + 5; // PHP metni otomatik sayıya çevirir -> 15

// Açık Tip Dönüşümü (Explicit Casting)
$ondalik = 8.75;
$tamSayi = (int) $ondalik; // 8 (Ondalık kısım atılır)

$sayi = 100;
$metinYap = (string) $sayi; // "100" metnine dönüştürülür
?>
```

Değişken Kapsamı

PHP’de değişkenlerin erişilebilirliği tanımlandıkları yere göre belirlenir. Değişkenlerin yaşam alanı olan kapsam (*scope*), JavaScript’teki `var`, `let`, `const` ayırımından farklı olarak PHP’de ana kapsam mantığı fonksiyon sınırlarına dayanır.

- ◊ **Global Scope (Küresel Kapsam):** Fonksiyonların dışında tanımlanan değişkenlerdir. Sadece fonksiyonların dışındaki alanda doğrudan erişilebilirler. Fonksiyon içerisinde kullanılabilmeleri için `global` anahtar kelimesi ile çağrılmaları gerekir.
- ◊ **Local Scope (Yerel Kapsam):** Bir fonksiyonun içerisinde tanımlanan değişkenlerdir. Sadece tanımlandıkları fonksiyon bloğunun içinden erişilebilirler; fonksiyon bittiğinde bellekten silinirler.
- ◊ **Static Scope (Statik Kapsam):** Fonksiyon içerisinde `static` anahtar kelimesi ile tanımlanan değişkenlerdir. Fonksiyonun çalışması tamamlansa dahi değerini bellekte korur.

```
<?php
$globalSayi = 50; // Global Kapsam
function test() {
    // Fonksiyon içinde global değişkene erişmek için:
    global $globalSayi;
    $yerelSayi = 10; // Local Kapsam
    echo $globalSayi + $yerelSayi;
}
test(); // Çıktı: 60
// Statik Değişken Örneği
function sayac() {
    static $sayi = 0; // İlk çağrıda oluşturulur, değerini korur
    $sayi++;
    echo $sayi . " ";
}
sayac(); // Çıktı: 1
sayac(); // Çıktı: 2
sayac(); // Çıktı: 3
?>
```

Sabitler ile define()/const

PHP’de değeri programın çalışması boyunca değiştirilemeyen yapılara sabit (*Constants*) denir. Sabitlerin başında dolar (\$) simgesi kullanılmaz ve varsayılan olarak küresel kapsama sahiptirler. Sabitler `define()` fonksiyonu veya `const` anahtar kelimesi ile tanımlanır.

```
<?php
define("PI_SAYISI", 3.14159);
const SITE_URL = "https://orneksite.com";
echo PI_SAYISI; // 3.14159
?>
```

§7.4.4 PHP Dizileri

PHP’de dizi (*Array*), tek bir değişken içerisinde birden fazla değeri depolamanızı sağlayan özel bir veri yapısıdır. PHP dizileri aslında sıralı haritalardır (*ordered maps*); yani anahtar (*key*) ve değer (*value*) çiftlerinden oluşur. PHP’de temel olarak 3 tür dizi bulunur:

İndeksli Diziler

Elemanların otomatik olarak 0’dan başlayan tam sayı indislerle (*index*) erişildiği dizilerdir (*Indexed Arrays*).

```
<?php
// Dizi tanımlama
$meyveler = ["Elma", "Muz", "Portakal"];

// Elemana erişim
echo $meyveler[0]; // Çıktı: Elma

// Yeni eleman ekleme
$meyveler[] = "Çilek";
?>
```

İlişkisel Diziler

İndeksler yerine kendi tanımladığınız özel metinsel veya sayısal anahtarları (*keys*) kullanan dizilerdir (*Associative Arrays*).

```
<?php
$kullanici = [
    "ad" => "Ahmet",
    "eposta" => "ahmet@example.com",
    "rol" => "Yazılımcı"
];

// Elemana erişim
echo $kullanici["ad"]; // Çıktı: Ahmet
?>
```

Çok Boyutlu Diziler

İçinde bir veya daha fazla dizi barındıran karmaşık dizilerdir (*Multidimensional Arrays*).

```
<?php
$urunler = [
    ["ad" => "Laptop", "fiyat" => 25000, "stok" => 10],
    ["ad" => "Telefon", "fiyat" => 15000, "stok" => 25]
];

// İkinci ürünün adına erişim
echo $urunler[1]["ad"]; // Çıktı: Telefon
?>
```

En Çok Kullanılana Dizi Fonksiyonları

§7.4.5 PHP Kontrol Akışı ve Karar Yapıları

Programlama dillerinde kod blokları varsayılan olarak yukarıdan aşağıya ve satır satır (*sequential execution*) yürütülür. Ancak gerçek dünya senaryolarında, belirli koşulların sağlanıp sağlanmamasına göre kodun farklı dallara ayrılması veya tekrarlanması gerekir. Programın bu yürütme sırasını yönlendiren yapılara **kontrol akışı mekanizmaları** denir.

C dili ailesinden türeyen PHP, kontrol akışı konusunda JavaScript ile son derece benzer bir sözdizimine (*syntax*)

Fonksiyon	Açıklama	Örnek
<code>count()</code>	Dizideki eleman sayısını döndürür.	<code>count(\$meyveler)</code>
<code>in_array()</code>	Bir değerin dizide olup olmadığını arar (<code>true/false</code>).	<code>in_array("Muz", \$meyveler)</code>
<code>array_push()</code>	Dizinin sonuna bir veya daha fazla eleman ekler.	<code>array_push(\$meyveler, "Kivi")</code>
<code>array_merge()</code>	İki veya daha fazla diziyi birleştirir.	<code>array_merge(\$dizi1, \$dizi2)</code>
<code>array_filter()</code>	Belirli bir kurala göre dizi elemanlarını filtreler.	<code>array_filter(\$sayilar, fn(\$n) => \$n > 10)</code>
<code>array_map()</code>	Tüm elemanlara bir fonksiyon uygular.	<code>array_map(fn(\$n) => \$n * 2, \$sayilar)</code>

Tablo 7.1: En Çok Kullanılan Dizi Fonksiyonları

sahiptir.

Koşullu İfadeler (If - ElseIf - Else)

Bir veya birden fazla mantıksal ifadenin doğruluk değerine (`true/false`) göre farklı kod bloklarının çalıştırılmasını sağlar.

Karşılaştırma ve Mantıksal Operatörler

PHP'de koşul ifadeleri yazılırken mantıksal operatörler kullanılır:

- ◊ **Eşitlik:** `==` (Sadece değer eşitliği) ve `===` (Hem değer hem **veri tipi** eşitliği).
- ◊ **Mantıksal VE / VEYA:** `&&` (VE), `||` (VEYA), `!` (DEĞİL).

```
<?php
$not = 75;
if ($not >= 85) {
    echo "Harf Notu: AA";
} elseif ($not >= 70) {
    echo "Harf Notu: BB"; // Bu blok çalışacaktır
} elseif ($not >= 50) {
    echo "Harf Notu: CC";
} else {
    echo "Harf Notu: FF (Kaldı)";
}
?>
```

Ternary (Üçlü) İşleç ve Null Coalescing İşleci: Kısa koşul ifadeleri için `if-else` yerine üçlü işleç (`? :`) kullanılır. PHP 7.0 ile birlikte gelen Null Birleştirme İşleci (**null coalescing operator**) (`??`) ise bir değişkenin tanımlı ve `null` olup olmadığını kontrol etmek için büyük kolaylık sağlar.

```
<?php
$yas = 20;
// Ternary Operatör
$durum = ($yas >= 18) ? "Yetişkin" : "Reşit Değil";

// Null Coalescing Operatörü (??)
// $_GET['kullanici'] tanımlı ise onu al, tanımlı değilse "Misafir" ata.
$kullaniciAdi = $_GET['kullanici'] ?? "Misafir";
?>
```

Çoklu Seçim Yapısı (Switch - Case)

Bir değişkenin alabileceği birden fazla somut değere göre farklı kod bloklarının çalıştırılması gereken durumlarda, ardışık `if-elseif` blokları yerine `switch-case` yapısı tercih edilir.

```
<?php
$gun = "Salı";
switch ($gun) {
    case "Pazartesi":
        echo "Haftanın ilk iş günü.";
        break;
    case "Salı":
        echo "İkinci iş günü."; // Çalışacak blok
        break;
    case "Cumartesi":
        // ...

```

```

case "Pazar":
    echo "Hafta sonu!";
    break;
default:
    echo "Geçersiz gün bilgisi.";
    break;
}
?>

```

break Talimatının Kritik Önemi

`case` bloklarının sonuna `break` konulmazsa, PHP eşleşen bloktan sonraki tüm `case` bloklarını koşula bakmaksızın çalıştırmaya devam eder (*Fall-through davranışı*).

Döngü Yapıları (Loops)

Belirli bir kod bloğunu belirli bir koşul sağlandığı sürece tekrar tekrar çalıştırmak için kullanılır.

- ◇ **while:** Koşul doğrulandığı sürece döngü çalışır. Koşul en başta kontrol edilir.
- ◇ **do-while:** Kod bloğu en az bir kez çalıştırılır, koşul kontrolü döngünün sonunda yapılır.
- ◇ **for:** Sayaç değişkeni, döngü koşulu ve artış miktarının tek satırda tanımlandığı yapılandırılmış döngüdür.
- ◇ **foreach:** PHP'ye özgü olup dizilerin (*array*) ve nesnelerin tüm elemanları üzerinde kolayca gezinmeyi yani yineleme (*iteration*) sağlar.

```

<?php
// Standard For Döngüsü
for ($i = 1; $i <= 3; $i++) {
    echo "Adım: $i <br>";
}
// Foreach Döngüsü (Diziler için)
$ogrenciler = ["Ahmet", "Mehmet", "Zeynep"];
// Sadece Değer İle Gezinme
foreach ($ogrenciler as $ogrenci) {
    echo "Öğrenci: $ogrenci <br>";
}
// Anahtar (Key) ve Değer (Value) İle Gezinme
$personel = ["Ad" => "Can", "Unvan" => "Geliştirici"];
foreach ($personel as $anahtar => $deger) {
    echo "$anahtar: $deger <br>";
}
?>

```

Diziler Üzerinde Döngü Kullanımı

Dizi elemanlarını gezmek için en pratik yöntem `foreach` döngüsüdür:

```

<?php
$personel = [
    "CEO" => "Zeynep",
    "CTO" => "Mehmet",
    "Dev" => "Ali"
];

foreach ($personel as $unvan => $isim) {
    echo "$unvan: $isim\n";
}
?>

```

Kontrol Akışını Kesen Talimatlar (Break ve Continue)

Döngünün doğal akışını erkenden sonlandırmak veya mevcut adımı atlamak için kullanılır.

- ◇ **break:** İçinde bulunduğu döngüyü tamamen sonlandırır ve kontrolü döngü dışındaki ilk satıra devreder.
- ◇ **continue:** Döngünün o anki adımını (iterasyonunu) anında sonlandırır ve sayaç güncelleme adımına geçerek bir sonraki adımdan devam eder.

```

<?php
// Break ve Continue Örneği
for ($sayi = 1; $sayi <= 10; $sayi++) {
    if ($sayi == 3) {
        continue; // 3 sayısını yazatlamadan sonraki adıma geç
    }
}

```

```

if ($sayi == 7) {
    break; // Sayı 7 olduğunda döngüyü tamamen bitirir
}
echo "$sayi ";
}
// Çıktı: 1 2 4 5 6
?>

```

PHP'de İç İçe Döngülerden Çıkış (Seviyeli Break/Continue)

JavaScript'te iç içe döngülerden çıkmak için *label* (etiket) kullanılırken, PHP'de **break** ve **continue** komutlarına sayısal parametre verilebilir. Örneğin **break 2**; talimatı iç içe geçmiş **2 döngüyü birden** sonlandırır.

```

<?php
// İç İçe Döngülerde Seviyeli Break
for ($x = 1; $x <= 3; $x++) {
    for ($y = 1; $y <= 3; $y++) {
        if ($x == 2 && $y == 2) {
            break 2; // Her iki döngüyü de anında bitirir
        }
        echo "x:$x, y:$y | ";
    }
}
// Çıktı: x:1, y:1 | x:1, y:2 | x:1, y:3 | x:2, y:1 |
?>

```

Alternatif Kontrol Akışı Sözdizimi (HTML İçinde PHP Kullanımı)

PHP dosyalarında HTML ve PHP kodları iç içe yazılırken küme parantezleri ({}) kafa karışıklığına yol açabilir. Bu nedenle PHP, şablon görünümünde okunabilirliği artırmak için alternatif bir sözdizimi sunar.

```

<!-- HTML İçerisinde Alternatif Koşul Kullanımı -->
<?php if ($kullaniciGirisYapti == true): ?>
<h2>Hoş geldiniz, Panelinize Erişim Sağlayabilirsiniz.</h2>
<?php else: ?>
<h2>Lütfen Önce Giriş Yapınız.</h2>
<?php endif; ?>
<!-- HTML İçerisinde Alternatif Döngü Kullanımı -->
<ul>
<?php foreach ($urunler as $urun): ?>
<li><?php echo $urun; ?></li>
<?php endforeach; ?>
</ul>

```

§7.4.6 PHP Fonksiyonları

Bölüm 4.2.2 başlığında matematiksel kavram olarak gördüğümüz fonksiyonların burada somut karşılığını göreceksiniz. Fonksiyonlar, belirli bir görevi yerine getirmek üzere tasarlanmış, bir ismi olan ve program içerisinde ihtiyaç duyulduğunda tekrar tekrar çağrılabilen bağımsız kod bloklarıdır. Fonksiyon kullanımı; kod tekrarını önler (*DRY - Don't Repeat Yourself*), modülerliği artırır, bakım ve hata ayıklama süreçlerini kolaylaştırır.

Fonksiyon Tanımlama ve Çağırma

PHP'de kullanıcı tanımlı bir fonksiyon **function** anahtar kelimesi ile başlar. Fonksiyon isimleri harf veya alt çizgi (_) ile başlamalı, rakamla başlamamalıdır.

PHP'de Fonksiyon İsmi Harf Duyarsızlığı

Değişkenlerin aksine, PHP'de fonksiyon isimleri **büyük/küçük harfe duyarsızdır** (*case-insensitive*). Örneğin **selamVer()** şeklinde tanımlanan bir fonksiyon **SELAMVER()** veya **SelamVer()** olarak çağrılabilir. Ancak kod okunabilirliği açısından tanımlandığı biçimde çağırılması tavsiye edilir.

```

<?php
// Fonksiyon Tanımlama
function mesajYazdir() {

```

```

    echo "PHP Fonksiyon Dünyasına Hoş Geldiniz!";
}

// Fonksiyon Çağırma (Invocation)
mesajYazdir();
?>

```

Parametre Alan ve Değer Döndüren Fonksiyonlar

Fonksiyonlar dış dünyadan girdi almak için parametre kullanırlar. Hesaplanan veya üretilen veriyi ana programa aktarmak için ise `return` ifadesi kullanılır. `return` çalıştırıldığı an fonksiyonun yürütülmesi durdurulur ve kontrol çağrının yapıldığı satıra geri döner.

```

<?php
function topla($sayi1, $sayi2) {
    $sonuc = $sayi1 + $sayi2;
    return $sonuc; // Değeri çağrıldığı yere döndürür
}

$toplamSonuc = topla(15, 25); // 40
?>

```

Varsayılan Argümanlar (Default Arguments)

Fonksiyon çağrılırken bir argüman gönderilmezse, parametreye önceden atanan varsayılan değer devreye girer. Bu argümanlar daima zorunlu parametrelerin **sağına** yazılmalıdır.

```

<?php
function karsilama($isim, $unvan = "Ziyaretçi") {
    return "Merhaba " . $unvan . " " . $isim;
}

echo karsilama("Ahmet", "Dr."); // Merhaba Dr. Ahmet
echo karsilama("Mehmet");      // Merhaba Ziyaretçi Mehmet
?>

```

Katı Tipleme ve Tür Bildirimi (Type Hints & Strict Types)

PHP 7.0 ile birlikte fonksiyon parametrelerine ve dönüş değerlerine belirgin tür atama desteği gelmiştir. Bu sayede hatalı veri tiplerinin fonksiyona girmesi engellenir.

```

<?php
declare(strict_types=1); // Katı veri tipi kontrolünü aktifleştirir

// Parametrelerin int olmasını ve dönüşün float olmasını zorunlu kılar
function bolme(int $a, int $b): float {
    return $a / $b;
}

echo bolme(10, 2); // 5.0 (float)
// bolme("10", 2); // Strict types aktifken TypeError hatası fırlatır
?>

```

Değer ile Referans İle Aktarım (Pass by Value vs. Pass by Reference)

Varsayılan olarak PHP'de argümanlar fonksiyona **değer ile** (*pass by value*) aktarılır; yani verinin bir kopyası oluşturulur. Ancak parametrenin başına ampersand (&) simgesi konulursa, değişkenin kendisi/bellek adresi (*pass by reference*) aktarılır ve fonksiyon içindeki değişiklikler orijinal değişkeni etkiler.

```

<?php
function artirReferans(&$sayi) {
    $sayi += 10;
}

$originalSayi = 5;
artirReferans($originalSayi);
echo $originalSayi; // Çıktı: 15 (Orijinal değişken değişti)
?>

```

Sınırsız Sayıda Parametre (Variadic Functions)

Kaç tane argüman alacağı önceden bilinmeyen fonksiyonlar için üç nokta (...) işleci (*Splat Operator*) kullanılır. Bu işeç gelen tüm argümanları bir diziye dönüştürür.

```
<?php
function ortalamaHesapla(...$sayilar): float {
    if (count($sayilar) === 0) return 0.0;
    return array_sum($sayilar) / count($sayilar);
}

echo ortalamaHesapla(10, 20, 30, 40); // 25
?>
```

Anonim Fonksiyonlar ve Ok Fonksiyonları (Arrow Functions)

İsmi olmayan ve genellikle bir değişkene atanarak veya başka bir fonksiyona parametre (*callback*) olarak aktarılan fonksiyonlardır. PHP 7.4 ile kısa yazım sunan **Ok Fonksiyonları** (*Arrow Functions*) eklenmiştir.

Anonim vs Ok Fonksiyonlarında Kapsam (use vs Otomatik Kapsam)

Geleneksel anonim fonksiyonlar dışarıdaki bir değişkene erişmek için **use** (*\$degisken*) sözdizimini kullanmak zorundadır. Ok fonksiyonları (**fn()** => ...) ise dış kapsamdaki değişkenlere otomatik olarak erişebilir.

```
<?php
// Geleneksel Anonim Fonksiyon
$carpan = 2;
$katiniAl = function($sayi) use ($carpan) {
    return $sayi * $carpan;
};

// PHP 7.4+ Ok Fonksiyonu (Arrow Function)
// Dışarıdaki $carpan değişkenine 'use' gerekmeden otomatik erişir
$katiniAlKisa = fn($sayi) => $sayi * $carpan;

echo $katiniAlKisa(5); // 10
?>
```

Özyinelemeli (Recursive) Fonksiyonlar

Kendi kendisini çağıran fonksiyonlardır. Karmaşık veri yapılarını (örneğin dizin ağaçları veya kategoriler) taramak için kullanılır. Sonsuz döngüye girmemesi için mutlaka bir **taban koşulu** (*base case*) bulunmalıdır.

```
<?php
function faktoriyel(int $n): int {
    // Taban Koşul (Base Case)
    if ($n <= 1) {
        return 1;
    }
    // Özyinelemeli Çağrı (Recursive Call)
    return $n * faktoriyel($n - 1);
}

echo faktoriyel(5); // 120
?>
```

§7.4.7 PHP Nesne Yönelimli Programlama (OOP) ve Nesneler

Bölüm 4.3.2 başlığında teorisi verilen Nesne Yönelimli Programlama yani **OOP** (*Object-Oriented Programming*), yazılımı birbirleriyle etkileşim halinde olan nesneler (*objects*) etrafında kurgulayan bir programlama yaklaşımıdır. PHP 5 ile birlikte güçlü bir OOP altyapısına kavuşmuş, PHP 7 ve 8 sürümleriyle de katı tiplendirme ve modern mimari kalıplarını destekleyen gelişmiş bir yapıya bürünmüştür.

Sınıf (Class) ve Nesne (Object) Mantığı

Sınıf (Class): Bir nesnenin durumunu/özelliklerini (*states/attributes*) ve davranışlarını/ yöntemlerini (*behavior/method*) tanımlayan bir soyut taslak veya şablondur.

Nesne (Object): Bu sınıftan **new** anahtar kelimesi ile öbek bellekte (*Heap*) somut olarak üretilen (örneklenen) dinamik bir varlıktır.

Temel Sınıf Tanımlama ve Nesne Örneklemesi

Bir sınıf tanımlanırken **class** anahtar kelimesi kullanılır. Sınıf içerisindeki değişkenlere **özellik** (*property*), fonksiyonlara ise **yöntem** (*method*) denir. Sınıf içindeki özellik ve yöntemlere erişmek için **->** (*object operator*) kullanılır.

```
<?php
class Otomobil {
    // Özellik Tanımlamaları
    public string $marka;
    public string $renk;

    // Yöntem (method) Tanımlaması
    public function bilgiVer(): string {
        // $this anahtar kelimesi mevcut nesne örneğini temsil eder
        return "Aracın Markası: " . $this->marka . ", Rengi: " . $this->renk;
    }
}

// Nesne Oluşturma (Instantiation)
$arac1 = new Otomobil();
$arac1->marka = "Toyota";
$arac1->renk = "Kırmızı";

echo $arac1->bilgiVer();
// Çıktı: Aracın Markası: Toyota, Rengi: Kırmızı
?>
```

Erişim Belirleyiciler ve Kapsülleme

Kapsülleme (*Encapsulation*), bir nesnenin iç durumunu dış müdahalelerden korumak ve veriye erişimi denetim altına almak için kullanılır. PHP'de üç temel erişim belirleyici (*Access Modifiers*) bulunur:

- ◇ **public**: Sınıf içinden, türetilen sınıflardan ve sınıf dışından (her yerden) erişilebilir.
- ◇ **protected**: Sadece tanımlandığı sınıf içinden ve o sınıftan türetilen (miras alan) alt sınıflardan erişilebilir.
- ◇ **private**: Sadece tanımlandığı sınıfın içinden erişilebilir; dışarıdan veya alt sınıflardan erişilemez.

```
<?php
class BankaHesabi {
    private float $bakiye = 0.0; // Dışarıdan doğrudan değiştirilemez

    // Getter Metodu
    public function getBakiye(): float {
        return $this->bakiye;
    }

    // Setter Metodu
    public function paraYatir(float $miktar): void {
        if ($miktar > 0) {
            $this->bakiye += $miktar;
        }
    }
}

$hesap = new BankaHesabi();
$hesap->paraYatir(500);
echo "Güncel Bakiye: " . $hesap->getBakiye(); // 500
// $hesap->bakiye = 1000; // Fatal Error: Private özelliğe dışarıdan erişilemez!
?>
```

Sihirli Yöntemler

PHP'de çift alt çizgi (__) ile başlayan özel fonksiyonlara **sihirli yöntemler** (*magic methods*) denir. Bunlar; kurucu (**__construct**) ve yıkıcı (**__destruct**) yöntemlerdir.

- ◇ **__construct()**: Nesne **new** ile bellekte oluşturulduğu an otomatik olarak çalışan kurucu yöntemdir. Nesneyle ilişkin verilerin ilk değerlerini atamak için kullanılır.
- ◇ **__destruct()**: Nesnenin kullanımı bittiğinde veya bellekten silindiğinde (*Garbage Collector*) otomatik olarak çalışan yıkıcı yöntemdir.

```
<?php
class Kullanici {
```

```

    public string $ad;

    // Kurucu Yöntem
    public function __construct(string $ad) {
        $this->ad = $ad;
        echo "Kullanıcı '$this->ad' oluşturuldu.<br>";
    }

    // Yıkıcı Yöntem
    public function __destruct() {
        echo "Kullanıcı '$this->ad' bellekten silindi.<br>";
    }
}

$kullanici1 = new Kullanici("Zeynep");
?>

```

Kalıtım

Kalıtım (*Inheritance*), mevcut bir sınıfın tüm **public** ve **protected** durum ile yöntemlerini yeni bir sınıfa aktarmasını sağlar. PHP’de kalıtım **extends** anahtar kelimesi ile gerçekleştirilir. Bir alt sınıf, üst sınıfın yöntemlerini ezebilir (*Method Overriding*).

```

<?php
// Üst Sınıf (Base/Parent Class)
class Personel {
    protected string $ad;
    protected float $maas;

    public function __construct(string $ad, float $maas) {
        $this->ad = $ad;
        $this->maas = $maas;
    }

    public function detayGoster(): string {
        return "Ad: " . $this->ad . " - Maaş: " . $this->maas;
    }
}

// Alt Sınıf (Child Class)
class Yazilimci extends Personel {
    public string $uzmanlikDili;

    public function __construct(string $ad, float $maas, string $uzmanlikDili) {
        // Üst sınıfın kurucu metodunu çağırma
        parent::__construct($ad, $maas);
        $this->uzmanlikDili = $uzmanlikDili;
    }

    // Yöntem Ezme (Method Overriding)
    public function detayGoster(): string {
        return parent::detayGoster() . " - Dil: " . $this->uzmanlikDili;
    }
}

$yazilimci = new Yazilimci("Ahmet", 45000, "PHP");
echo $yazilimci->detayGoster();
?>

```

Statik Özellikler ve Yöntemler

Statik üyelere erişmek için nesne örneği (**new**) oluşturmaya gerek yoktur. Sınıf ismi üzerinden doğrudan **Kapsam Çözümleme İşleci (::)** kullanılarak erişilir. Statik yöntemler içerisinde **\$this** kullanılamaz; bunun yerine **self::** ifadesi tercih edilir.

```

<?php
class Matematik {
    public static float $pi = 3.14;
}

```

```

    public static function kareAl(int $sayi): int {
        return $sayi * $sayi;
    }
}

// Nesne türetmeden erişim
echo Matematik::$pi; // 3.14
echo Matematik::kareAl(4); // 16
?>

```

Soyut Sınıflar ve Arayüzler

Gelişmiş yazılım mimarilerinde alt sınıflara standart kurallar koymak ve çok biçimliliği (*Polymorphism*) sağlamak için soyutlama kullanılır.

Soyut sınıftan (*abstract class*) nesne imal edilemez. Soyut olamayan sınıflar somut sınıf (*concrete class*) olarak adlandırılır. Ara yüzler (*interface*) ise nesnenin sahip olacağı rollerdir.

Abstract Class vs Interface Farkı

Abstract Class: Gövdeli (normal) yöntemler barındırabilir. Bir sınıf yalnızca tek bir soyut sınıftan türetilir (*extends*).

Interface: Yalnızca gövdesiz yöntem imzaları (*method signature*) barındırır. Bir sınıf birden fazla arayüzü yani rolü uygulayabilir (*implements*).

```

<?php
// Arayüz Tanımlama
interface OdemeYontemi {
    public function odemeYap(float $tutari): bool;
}

class KrediKarti implements OdemeYontemi {
    public function odemeYap(float $tutari): bool {
        echo "$tutari TL Kredi Kartı ile çekildi.";
        return true;
    }
}

class Havale implements OdemeYontemi {
    public function odemeYap(float $tutari): bool {
        echo "$tutari TL Havale/EFT ile gönderildi.";
        return true;
    }
}
?>

```

§7.4.8 PHP Dahili Nesneleri

PHP, nesne yönelimli programlama süreçlerini kolaylaştırmak ve sık karşılaşılan karmaşık operasyonları standart bir yapıya bağlamak için bünyesinde birçok yerleşik (dahili) sınıf ve nesne barındırır (*Built-in Objects*). Bu sınıflar herhangi bir harici kütüphane veya modül yüklemeye gerek kalmaksızın doğrudan kullanılabilir.

Tarih ve Zaman Yönetimi (DateTime Nesnesi)

Geleneksel metinsel veya Unix zaman damgası (*timestamp*) tabanlı tarih işlemlerinin aksine, *DateTime* sınıfı tarih ve saat verilerini nesne yönelimli, güvenli ve zaman dilimlerine (*timezone*) duyarlı bir şekilde yönetmeyi sağlar.

```

<?php
// Mevcut tarih ve saat ile bir DateTime nesnesi oluşturma
$simdi = new DateTime();
echo "Şu anki Zaman: " . $simdi->format('Y-m-d H:i:s') . "<br>";

// Özel bir zaman dilimi (Timezone) belirleme
$zamanDilimi = new DateTimeZone('Europe/Istanbul');
$tarih = new DateTime('now', $zamanDilimi);

// Tarih Üzerinde Aritmetik İşlemler (DateInterval)
// P2M3D = 2 Ay (Month) ve 3 Gün (Day) ekle

```

```
$tarih->add(new DateInterval('P2M3D'));
echo "Gelecek Tarih: " . $tarih->format('d.m.Y') . "<br>";

// İki Tarih Arasındaki Farkı Hesaplama (diff)
$dogumGunu = new DateTime('1995-05-15');
$bugun = new DateTime('today');
$fark = $dogumGunu->diff($bugun);

echo "Yaşınız: " . $fark->y . " yıl, " . $fark->m . " ay, " . $fark->d . " gün.";
?>
```

Anonim ve Jenerik Nesneler (stdClass)

stdClass, PHP'nin varsayılan jenerik boş sınıfıdır. Önceden özel bir **class** şablonu tanımlamaya gerek kalmadan, dinamik olarak özellik (*property*) eklenebilen geçici nesneler oluşturmak için kullanılır. Ayrıca dizi tür dönüştürmelerinde (*type casting*) varsayılan dönüşüm hedefidir.

```
<?php
// Boş bir stdClass nesnesi oluşturma
$urun = new stdClass();
$urun->id = 101;
$urun->baslik = "Kablosuz Klavye";
$urun->fiyat = 450.00;

// İlişkisel bir Diziyi (Associative Array) Nesneye Dönüştürme
$dizi = ["ad" => "Can", "rol" => "Yönetici"];
$kullaniciNesne = (object) $dizi;

echo $kullaniciNesne->ad; // Çıktı: Can
?>
```

Hata ve İstisna Yönetimi

PHP 7 ile birlikte tüm sistem hataları ve istisnalar (**Exception**) nesne tabanlı bir hiyerarşiye bağlanmıştır. **Throwable** arayüzü, hem sistem hatalarının (**Error**) hem de kullanıcı tanımlı istisnaların (**Exception**) türediği en üst çatıdır.

Try-Catch Bloğu ve Exception Mantığı

Hata potansiyeli taşıyan kodlar **try** bloğuna alınır. Bir aksaklık durumunda **throw** ile bir **Exception** nesnesi fırlatılır ve **catch** bloğu tarafından yakalanarak programın çökmesi engellenir.

```
<?php
function bolmeIslemi(float $bolunen, float $bolen): float {
    if ($bolen === 0.0) {
        // Dahili Exception nesnesini fırlatma
        throw new Exception("Sıfıra bölme hatası gerçekleşti!", 400);
    }
    return $bolunen / $bolen;
}

try {
    echo bolmeIslemi(10, 0);
} catch (Exception $e) {
    // Exception nesnesinin dahili yöntemlerini kullanma
    echo "Hata Yakalandı: " . $e->getMessage() . "<br>";
    echo "Hata Kodu: " . $e->getCode() . "<br>";
    echo "Satır: " . $e->getLine();
} finally {
    echo "<br>İşlem tamamlandı."; // Her durumda çalışacak blok
}
?>
```

Anonim Fonksiyon Nesnesi

PHP'de tanımlanan tüm anonim fonksiyonlar (ve ok fonksiyonları) arka planda otomatik olarak dahili **Closure** sınıfının birer nesne örneğidir. **Closure** nesneleri sayesinde fonksiyonlar değişkenlere atanabilir, başka fonksiyonlara argüman olarak aktarılabilir veya çalışma zamanında bir sınıfa dinamik yöntem olarak bağlan-

abilir (*bindTo*).

```
<?php
// Anonim fonksiyon aslında bir Closure nesnesidir
$selamlama = function(string $isim): string {
    return "Merhaba $isim";
};

// Closure nesnesi olduğunu doğrulama
var_dump($selamlama instanceof Closure); // bool(true)

// Closure Nesnesinin Yöntemleri (bindTo ile bağlama)
class Sunucu {
    private string $durum = "Aktif";
}

$durumOku = function() {
    return $this->durum;
};

// Private özelliğe erişebilmek için closure nesnesini sınıfa bağlama
$bagliFonksiyon = $durumOku->bindTo(new Sunucu(), 'Sunucu');
echo $bagliFonksiyon(); // Çıktı: Aktif
?>
```

Öne Çıkan Diğer Dahili PHP Nesneleri

- ◊ **ArrayIterator / ArrayObject:** Dizilerin nesne gibi davranmasını ve özelleştirilmiş iterasyon mantıklarının uygulanmasını sağlar.
- ◊ **ReflectionClass:** Çalışma zamanında sınıfların, yöntemlerin ve parametrelerin iç yapısını incelemeye olanak tanır (*Reverse Engineering / Metaprogramming*).
- ◊ **PDO / PDOStatement:** Güvenli veritabanı bağlantısı ve sorgu yürütme süreçlerini yöneten dahili nesnelerdir.

§7.4.9 PHP Modülleri ve Eklentileri

PHP, çekirdek (*core*) mimarisi oldukça hafif tutulmuş, ihtiyaca göre fonksiyonel modüller ve eklentiler (*extensions*) yüklenerek genişletilebilen modüler bir yapıya sahiptir. Veritabanı sürücüler, görsel işleme kütüphaneleri, şifreleme araçları ve ağ protokolleri PHP'ye birer modül olarak dahil edilir.

PHP Modül Mimarisi ve Çalışma Mantığı

PHP modülleri genellikle C/C++ dili ile yazılmış ve derlenmiş paylaşımlı kütüphanelerdir (*Linux ortamında .so, Windows ortamında .dll*). PHP çalışma zamanı (*runtime*), bu modülleri *php.ini* konfigürasyon dosyası aracılığıyla belleğe yükler ve içerisindeki fonksiyonları PHP kodlarının kullanımına sunar.

En Yaygın Kullanılan PHP Modülleri

Modern bir web uygulamasının ihtiyaç duyduğu temel PHP modülleri şunlardır:

- ◊ **php-mysql / php-pdo:** MySQL ve diğer ilişkisel veritabanlarına (PostgreSQL, SQLite vb.) güvenli erişim sağlayan veritabanı sürücüleridir.
- ◊ **php-gd / php-imagick:** Resim boyutlandırma, filigran (*watermark*) ekleme ve format dönüştürme gibi görsel işleme operasyonlarını yürütür.
- ◊ **php-mbstring:** Çok baytlı karakter dizilerini (*Multi-Byte String*) ve UTF-8 (Türkçe karakterler vb.) metin işlemlerini hatasız işlemek için kullanılır.
- ◊ **php-curl:** HTTP, FTP ve HTTPS protokolleri üzerinden uzak sunuculara istek atmayı ve REST API'ler ile haberleşmeyi sağlar.
- ◊ **php-json:** JSON (*JavaScript Object Notation*) formatındaki verileri çözümlemek (*decode*) ve oluşturmak (*encode*) için kullanılan çekirdek modüldür.
- ◊ **php-xml / php-zip:** XML belgelerini ayrıştırma ve ZIP arşivleri oluşturma/açma işlemlerini üstlenir.
- ◊ **php-opcache:** PHP kodlarını derleyerek bellekte ön belleğe alır; her istekte kodun tekrar tekrar yorumlanmasını engelleyerek sunucu performansını ciddi oranda artırır.

Sunucu Ortamında Modül Yönetimi

Linux/Ubuntu sistemlerde PHP eklentileri paket yöneticisi (**apt**) aracılığıyla kolayca yüklenebilir ve yönetilebilir.

```
# Yüklü olan tüm PHP modüllerini listeleme
php -m

# Belirli bir PHP modülünü arama (Örn: GD kütüphanesi)
sudo apt search php-gd

# Yeni bir PHP modülü kurma (Örn: cURL ve PDO MySQL)
sudo apt install php-curl php-mysql -y

# Kurulan modülü aktiveleştirme / devre dışı bırakma
sudo phpenmod curl
sudo phpdismod curl

# Değişikliklerin geçerli olması için web sunucusunu/SAPI'yi yeniden başlatma
sudo systemctl restart apache2
# veya Nginx + PHP-FPM kullanılıyorsa:
# sudo systemctl restart php8.2-fpm
```

Konfigürasyon Dosyası (**php.ini**) ve Modül Etkinleştirme

PHP modüllerinin davranışı ve limitleri **php.ini** dosyası üzerinden ayarlanır. Modüller bu dosyada **extension** direktifi ile tanımlanır.

```
; php.ini İçerisinde Modül Etkinleştirme Örneği

; Linux sistemlerde .so uzantılı kütüphane
extension=curl.so
extension=pdo_mysql.so

; Windows sistemlerde .dll uzantılı kütüphane
; extension=php_gd.dll

; Modül Yapılandırma Parametreleri Örneği (OPcache)
opcache.enable=1
opcache.memory_consumption=128
```

Kod İçerisinde Modül Varlığını Kontrol Etme

Yazdığınız bir PHP uygulamasının ihtiyaç duyduğu bir eklentinin sunucuda yüklü olup olmadığını çalışma zamanında denetlemek, olası sistem hatalarının (**Fatal Error**) önüne geçer.

```
<?php
// extension_loaded() fonksiyonu ile modül kontrolü
if (extension_loaded('gd')) {
    echo "GD Kütüphanesi aktif, görsel işleme yapılabilir.";
} else {
    echo "Hata: GD Kütüphanesi sunucuda kurulu değil!";
}

// Belirli bir fonksiyonun tanımlı olup olmadığını kontrol etme
if (function_exists('curl_init')) {
    echo "cURL fonksiyonları kullanılabilir.";
}
?>
```

Apache Modülü (libapache2-mod-php) ve PHP-FPM Ayrımı

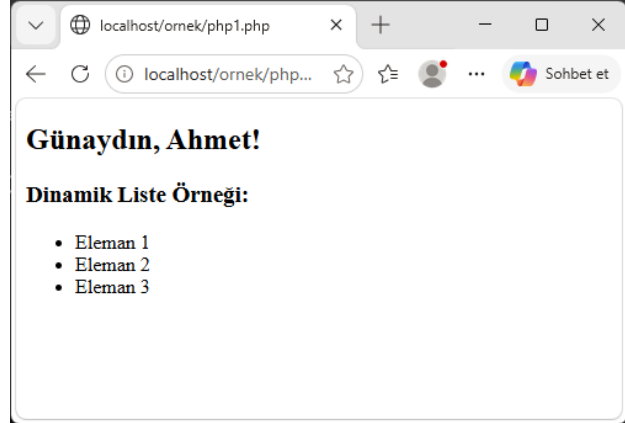
PHP'nin web sunucusu ile etkileşime girme biçimi iki temel modülle sağlanır:

- ♦ **SAPI (Apache Modülü):** PHP doğrudan Apache'nin bir iç modülü (**mod_php**) olarak çalışır.
- ♦ **PHP-FPM (FastCGI Process Manager):** PHP'yi bağımsız bir servis olarak çalıştırır. Nginx ve yüksek trafikli Apache mimarilerinde performans ve kaynak yönetimi açısından tercih edilen modern standarttır.

§7.4.10 PHP ile Çıktı HTML Dokümanını Değiştirme

PHP, sunucu tarafında mantıksal kararlar vererek veya döngüler çalıştırarak dinamik HTML içerikleri üretebilir.

```
<?php
$kullaniciAd = "Ahmet";
$saat = date("H"); // Sunucunun güncel saatini alır
↪ (00-23)
if ($saat < 12) {
    $mesaj = "Günaydın";
} else {
    $mesaj = "Tünaydın";
}
?>
<!DOCTYPE html>
<html lang="tr">
<body>
    <h2><?php echo $mesaj . ", " . $kullaniciAd
    ↪ . "!"; ?></h2>
    <h3>Dinamik Liste Örneği:</h3>
    <ul>
        <?php
        for ($i = 1; $i <= 3; $i++) {
            echo "<li>Eleman " . $i . "</li>";
        }
        ?>
    </ul>
</body>
</html>
?>
```



Şekil 7.4: PHP tarafından Değiştirilmiş HTML'in Web Tarayıcı Görselleştirmesi

Kod 7.3: Çıktı HTML Belgesini Değiştirecek PHP

§7.4.11 HTML Form Verilerini HTTP GET Yöntemi ile Tarayıcıdan Alma

İstemci (tarayıcı), bir HTML formu veya URL parametreleri aracılığıyla sunucuya veri gönderebilir. HTTP GET yöntemi kullanıldığında veriler URL adres çubuğunda (ornek.php?ad=Ahmet&yasi=25) görünür biçimde taşınır. PHP, bu verilere global \$_GET süper dizisi aracılığıyla erişir.

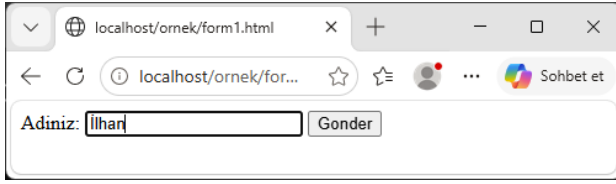
```
<!-- form.html (İstemci Tarafı Formu) -->
<html lang="tr">
<body>
    <form action="islem.php" method="GET">
        <label for="kullanici">Adınız:</label>
        <input type="text" id="kullanici" name="kullanici_adi">
        <button type="submit">Gönder</button>
    </form>
</body>
</html>
```

Sunucudaki veriyi işleyen islem.php dosyası:

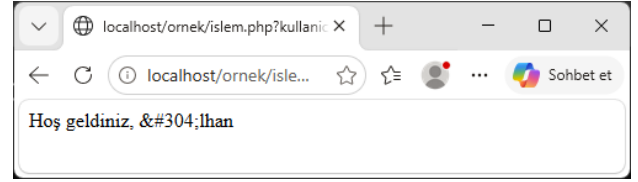
```
<?php
// islem.php (Sunucu Tarafı)
if (isset($_GET['kullanici_adi'])) {
    $gelenAd = htmlspecialchars($_GET['kullanici_adi']);
    echo "Hoş geldiniz, " . $gelenAd;
} else {
    echo "Form verisi alınamadı!";
}
?>
```

Burada dikkat edilmesi gereken formda **Gönder** butonuna tıklandığında verilerin URL üzerinden **islem.php** dosyasına aktarılmasıdır. Yani sağdaki görseldeki adres aşağıdaki şekilde olacaktır;

http://localhost/ornek/islem.php?kullanici_adi=%26%23304%3B1han



Şekil 7.5: Form HTML Sayfasının Görselleştirilmesi



Şekil 7.6: Form Tarafından Gönderilen Verileri İşlemiş İşlem Sayfasının Görselleştirilmesi

Ş7.4.12 HTML Form Verilerini HTTP POST Yöntemi ile Tarayıcıdan Alma

Bölüm 1.9’de öğrendiğimiz gibi **GET** ve **POST** yöntemleri farklı amaçlara hizmet eder. Şifre, kredi kartı bilgisi gibi hassas verilerde veya büyük hacimli formlarda veriler adres çubuğunda görünmemeli; bu nedenle HTTP **POST** yöntemi tercih edilir. PHP, bu şekilde gönderilen verilere global **\$_POST** süper dizisi aracılığıyla erişir.

\$_GET ile \$_POST Arasındaki Fark

- ◇ **\$_GET**: Veriler URL üzerinden taşındığı için adres çubuğunda görünür, tarayıcı geçmişinde (*history*) saklanır ve boyut sınırı vardır. Arama, filtreleme gibi tekrar bağlantı verilebilmesi istenen işlemler için uygundur.
- ◇ **\$_POST**: Veriler HTTP isteğinin gövdesinde (*body*) taşındığı için adres çubuğunda görünmez ve pratikte boyut sınırı yoktur. Giriş formları, kayıt formları gibi hassas veya büyük veriler için tercih edilir.

```
<!-- kayıt_formu.html (İstemci Tarafı Formu) -->
<form action="kayit_islem.php" method="POST">
  <label for="eposta">E-posta:</label>
  <input type="email" id="eposta" name="eposta">

  <label for="sifre">Şifre:</label>
  <input type="password" id="sifre" name="sifre">

  <button type="submit">Kayıt Ol</button>
</form>
```

Sunucudaki veriyi işleyen **kayit_islem.php** dosyası:

```
<?php
// kayıt_islem.php (Sunucu Tarafı)
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $eposta = htmlspecialchars($_POST['eposta'] ?? '');
  $sifre = $_POST['sifre'] ?? '';

  if (!empty($eposta) && !empty($sifre)) {
    echo "Kayıt isteği alındı: " . $eposta;
    // Bu noktadan sonra veri, ilerleyen bölümde göreceğimiz
    // gibi bir veritabanına kaydedilebilir.
  } else {
    echo "Lütfen tüm alanları doldurunuz!";
  }
}
?>
```

7.5 MySQL

MySQL, ilişkisel veri tabanıdır yönetim sistemidir yani **RDBMS** (Relational Database Management System). PHP ile tam uyumlu çalışarak verilerin kalıcı olarak saklanması, sorgulanmasını ve güncellenmesini sağlar.

Ş7.5.1 Linux MySQL Kurulumu

```
# MySQL Server kurulumu
sudo apt install mysql-server -y

# Servisi başlatma ve kontrol etme
sudo systemctl start mysql
sudo systemctl status mysql
```

Veritabanı sunucusu kuruluşuna göre **okul** veritabanını oluşturabiliriz. Parola sorduğunda Linux **root** parolasını girmelisiniz. Böylece örneklerimizde veritabanına bağlanmak için **devuser** adlı bir kullanıcı oluşturmuş

olacağız.

```
hoydabre@OZKAN:~$ sudo mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.4.11-0ubuntu0.26.04.1 (Ubuntu)
```

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql>CREATE USER 'devuser'@'localhost' IDENTIFIED BY 'YourStrongPassword123!';
ALL PRIVILEGES ON *.* TO 'devuser'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> GRANT ALL PRIVILEGES ON *.* TO 'devuser'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0.00 sec)
```

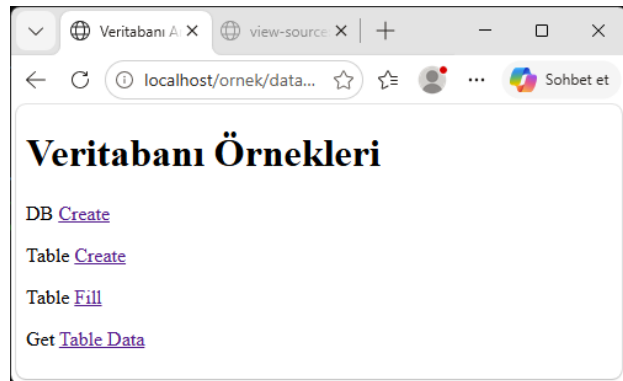
```
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> EXIT;
hoydabre@OZKAN:~$
```

Aşağıda sırasıyla veritabanı, tablo oluşturup tabloyu dolduran ve verileri getiren PHP sayfaları verilmiştir.

§7.5.2 Veritabanı Ana Sayfa

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="utf-8">
  <title>Veritabanı Ana Sayfa</title>
</head>
<body>
  <div id="content">
    <h1>Veritabanı Örnekleri</h1>
    <p> DB <a
      ↪ href="CreateDB.php">Create</a></p>
    <p> Table <a
      ↪ href="CreateTable.php">Create</a></p>
    <p> Table <a
      ↪ href="FillTable.php">Fill</a></p>
    <p> Get <a href="GetTable.php">Table
      ↪ Data</a></p>
  </div><!--content-->
</body>
</html>
```



Şekil 7.7: Veritabanı Ana Sayfasının Görselleştirilmesi

Kod 7.4: Veritabanı Ana Sayfa HTML

PHP'nin PDO (*PHP Data Objects*) kütüphanesi kullanılarak güvenli bir şekilde veritabanı bağlantısı kurulur ve temel **CRUD** (*Create, Read, Update, Delete*) işlemleri gerçekleştirilir.

§7.5.3 Veritabanı Oluşturan PHP

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>MySQL Veritabanı Oluşturma</title>
</head>
<body>
  <h1>Veri Tabanı Oluşturma</h1>
```

```

<p>
<?php
$host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!'; $charset = 'utf8mb4';
$dsn = "mysql:host=$host;charset=$charset";
try {
    // 1. Connect to MySQL server without a dbname parameter
    $pdo = new PDO("mysql:host=$host;charset=utf8mb4", $user,$pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);

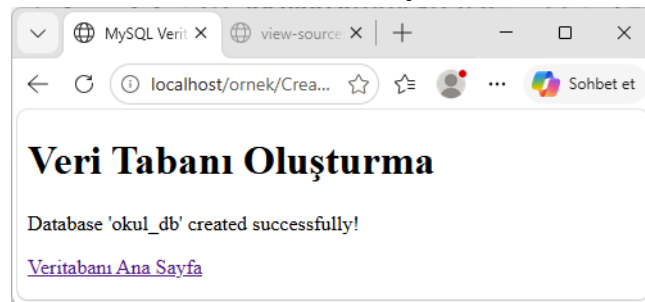
    // 2. SQL query to create database
    $dbname = "okul_db";
    $sql = "CREATE DATABASE IF NOT EXISTS `dbname` CHARACTER SET utf8mb4 COLLATE
        ↪ utf8mb4_unicode_ci";

    // 3. Execute query
    $pdo->exec($sql);
    echo "Database '$dbname' created successfully!";

    // 4. (Optional) Switch to the newly created database
    $pdo->exec("USE `dbname`");
} catch (PDOException $e) {
    echo "Veritabanı Hatası: " . $e->getMessage();
}
?>
</p>
<p><a href="database.html">Veritabanı Ana Sayfa</a></p>
</body>
</html>

```

Kod 7.5: Veritabanı Oluşturan PHP



Şekil 7.8: CreateDB.PHP Sayfasının Görselleştirilmesi

§7.5.4 Veritabanında Tablo Oluşturan PHP

```

<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>MySQL Veritabanı Tablo Oluşturma</title>
</head>
<body>
    <h1>Veri Tabanında Tablo Oluşturma</h1>
    <p>
    <?php
    $host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
    $charset = 'utf8mb4'; $dbname = "okul_db";
    $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
    try {
        // 1. Connect to MySQL server without a dbname parameter
        $pdo = new PDO($dsn, $user,$pass, [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);

        // 2. SQL query to create table
        $tablename = "Ogrenciler";

```

```

$sql = "DROP TABLE IF EXISTS $tablename";
$pdo->exec($sql);

$sql = "CREATE TABLE $tablename ("
    "id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,"
    "adi VARCHAR(30) NOT NULL,"
    "soyadi VARCHAR(30) NOT NULL,"
    "eposta VARCHAR(50),"
    "kayittarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP"
    ")";

// 3. Execute query
$pdo->exec($sql);
echo "Database '$tablename' created successfully!";
} catch (PDOException $e) {
    echo "Veritabanı Hatası: " . $e->getMessage();
}
?>
</p>
<p><a href="database.html">Veritabanı Ana Sayfa</a></p>
</body>
</html>

```

Kod 7.6: Veritabanında Tablo Oluşturan PHP



Şekil 7.9: CreateTable.PHP Sayfasının Görselleştirilmesi

§7.5.5 Veritabanında Tabloyu Veri İle Dolduran PHP

```

<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>MySQL Veritabanı Tablo Doldurma</title>
</head>
<body>
    <h1>Veri Tabanında Tablo Doldurma</h1>
    <p>
        <?php
            $host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
            $charset = 'utf8mb4'; $dbname = "okul_db"; $tablename = "Ogrenciler";
            $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
            try {
                // 1. Connect to MySQL server without a dbname parameter
                $pdo = new PDO($dsn, $user, $pass, [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);
                // 2. SQL query to create table
                $sql = "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Ahmet', 'Yıldırım',
                    ↳ 'ahmet@example.com')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('İlhan', 'Özkan',
                    ↳ 'ilhan@example.com.tr')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Ayşen', 'AĞLI', 'aysen@example.com')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Kağan', 'TÜRK',
                    ↳ 'kagan@example.com.tr')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Ajda', 'TAŞKIN', 'aö@example.com')";
                $pdo->exec($sql);
            }
        </?php>
    </p>

```

```

        echo "Table '$tablename' filled successfully!";
    } catch (PDOException $e) {
        echo "Veritabanı Hatası: " . $e->getMessage();
    }
    ?>
</p>
<p><a href="database.html">Veritabanı Ana Sayfa</a></p>
</body>
</html>

```

Kod 7.7: Veritabanında Tabloyu Veri İle Dolduran PHP



Şekil 7.10: FillTable.PHP Sayfasının Görselleştirilmesi

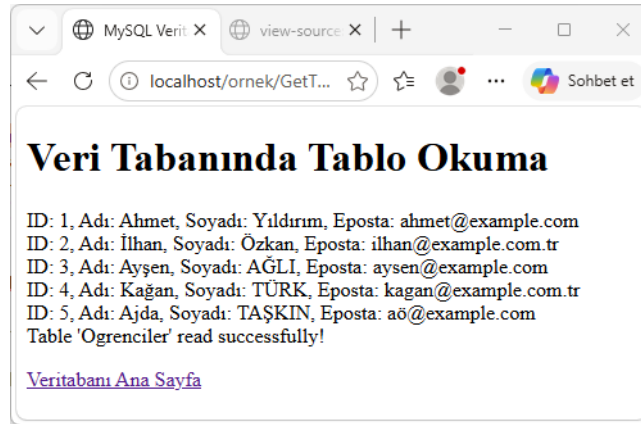
§7.5.6 Veritabanından Veri Okuyan PHP

```

<!DOCTYPE html>
<html lang="tr">
    <head>
        <meta charset="UTF-8">
        <title>MySQL Veritabanı Tablo Okuma</title>
    </head>
    <body>
        <h1>Veri Tabanında Tablo Okuma</h1>
        <p>
            <?php
                $host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
                $charset = 'utf8mb4'; $dbname = "okul_db"; $tablename = "Ogrenciler";
                $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
                try {
                    // 1. Connect to MySQL server without a dbname parameter
                    $pdo = new PDO($dsn, $user,$pass, [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
                    // 2. SQL query to create table
                    $sql = "SELECT * FROM $tablename;";
                    $result=$pdo->query($sql); // Okuma İçin QUERY() Yöntemi Kullanılıyor.
                    foreach ($result as $row) {
                        echo "ID: " . $row['id'] . ", Adı: " . $row['adi'] . ", Soyadı: " . $row['soyadi'] .
                            ", Eposta: " . $row['eposta'] . "<br>";
                    };
                    echo "Table '$tablename' read successfully!";
                } catch (PDOException $e) {
                    echo "Veritabanı Hatası: " . $e->getMessage();
                }
            ?>
        </p>
        <p><a href="database.html">Veritabanı Ana Sayfa</a></p>
    </body>
</html>

```

Kod 7.8: Veritabanından Veri Okuyan PHP



Şekil 7.11: GetTable.PHP Sayfasının Görselleştirilmesi

§7.5.7 PHP ile Veritabanında CRUD İşlemleri

```
<?php
$host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
$charset = 'utf8mb4'; $dbname = "okul_db"; $tablename = "Ogrenciler";
$dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";

try {
    // 1. Veritabanı Bağlantısı
    $pdo = new PDO($dsn,$user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // 2. Tablo Oluşturma (Create Table)
    $sqlTable = "CREATE TABLE IF NOT EXISTS $tablename (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    numara INT NOT NULL
    )";
    $pdo->exec($sqlTable);

    // 3. Kayıt Oluşturma (Create) - Hazırlıklı İfade (Prepared Statement)
    $sqlInsert = "INSERT INTO $tablename (ad, numara) VALUES (:ad, :numara)";
    $stmt = $pdo->prepare($sqlInsert);
    $stmt->execute(['ad' => 'Ahmet Özkan', 'numara' => 101]);

    // 4. Bütün Kayıtları Okuma (ReadAll)
    $sqlSelect = "SELECT * FROM $tablename";
    $stmt = $pdo->query($sqlSelect); //QUERY() metodu SELECT sorgularında kullanılır, EXEC() metodu ise
    // → INSERT, UPDATE, DELETE gibi sorgularda kullanılır.
    $ogrenciler = $stmt->fetchAll(PDO::FETCH_ASSOC);

    foreach ($ogrenciler as $ogrenci) {
        echo "ID: " . $ogrenci['id'] . " - Ad: " . $ogrenci['ad'] . "<br>";
    }

    // 5. Kayıt Güncelleme (Update)
    $sqlUpdate = "UPDATE $tablename SET ad = :ad WHERE id = :id";
    $pdo->prepare($sqlUpdate)->execute(['ad' => 'Ahmet Can', 'id' => 1]);

    // 6. Kayıt Silme (Delete)
    $sqlDelete = "DELETE FROM $tablename WHERE id = :id";
    $pdo->prepare($sqlDelete)->execute(['id' => 1]);
} catch (PDOException $e) {
    echo "Veritabanı Hatası: " . $e->getMessage();
}
?>
```

§7.5.8 PHP ile Form Verilerini Veritabanına Kaydetme

Hassas verilerin gönderiminde HTTP POST yöntemi tercih edilmelidir. Güvenlik açığı olan *SQL Injection* saldırılarını önlemek için kullanıcı girdileri doğrudan SQL ifadesine eklenmemeli, **Hazırlıklı İfadeler** (*Prepared Statements*) kullanılmalıdır. Çünkü POST ile gelen değerler doğrudan veritabanına uygulanmaktadır.

```
<?php
// POST metodu ile gelen form verisini güvenli şekilde kaydetme
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $ogrenciAd = $_POST['ogrenci_ad'];
    $ogrenciNo = $_POST['ogrenci_no'];

    $host = 'localhost'; $db = 'okul'; $user = 'devuser';
    $pass = 'YourStrongPassword123!'; $charset = 'utf8mb4';

    $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
    $pdo = new PDO($dsn,$user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "INSERT INTO ogrenciler (ad, numara) VALUES (?, ?)";
    $stmt = $pdo->prepare($sql); $stmt->execute([$ogrenciAd, $ogrenciNo]);

    echo "Öğrenci veritabanına başarıyla kaydedildi.";
}
?>
```

§7.5.9 Hazırlıklı İfadeler

Hazırlıklı İfadeler (*Prepared Statements*), SQL Enjeksiyonu (*SQL Injection*) saldırılarını engellemenin en etkili ve standart yoludur. Temel çalışma prensibi, sorgunun mantıksal yapısı (SQL kodu) ile kullanıcının gönderdiği veri (parametreler) birbirinden tamamen ayrılmasına dayanır.

Nasıl Çalışır? Normal (güvenli olmayan) bir sorguda veritabanı, gelen tüm metni tek bir SQL komutu olarak algılar ve derler. Hazırlıklı ifadelerde ise süreç iki aşamada işler;

- ◇ Hazırlık (*Prepare*): SQL sorgu şablonu veritabanına gönderilir ve derlenir. Veritabanı sorgunun ne yapacağını (örneğin hangi tablodan veri çekileceğini) önceden anlar.
- ◇ Çalıştırma (*Execute*): Kullanıcıdan gelen parametreler veritabanına gönderilir. Veritabanı bu değerleri çalıştırma kodunun bir parçası olarak değil, sadece düz veri (metin/sayı) olarak işler.

◇

Bu sayede kullanıcının girdiği veri içerisinde ' OR '1'='1 gibi SQL komutları yer alsa bile veritabanı bunları kod olarak yürütmez, yalnızca aranan metnin bir parçası kabul eder.

§7.5.10 Veritabanının Güvenli Olan ve Olmayan Kullanımı

Giriş formu üzerinden gönderilen bir kullanıcı adı (*\$kullanici_adi*) ve şifre (*\$sifre*) senaryosunu inceleyelim.

Güvenlik Açığı Bulunan Kod

```
// DİKKAT: SQL Injection'a açıktır!
$sql = "SELECT * FROM kullanicilar WHERE kullanici_adi = '$kullanici_adi' AND sifre = '$sifre'";
$sonuc = $db->query($sql);
```

Saldırı Senaryosu: Kullanıcı, kullanıcı adı alanına *admin* girdiğinde sorgu şu hale gelir:

```
SELECT * FROM kullanicilar WHERE kullanici_adi = 'admin' --' AND sifre = '...'
-- karakterleri SQL'de yorum satırı anlamına geldiği için şifre kontrolü tamamen atlanır ve saldırgan
↳ şifresiz giriş yapar.
```

Güvenli Kod

```
// 1. Şablon hazırlanır (Kullanıcı girdisi yerine '?' yer tutucuları konur)
$stmt = $db->prepare("SELECT * FROM kullanicilar WHERE kullanici_adi = ? AND sifre = ?");
```

```
// 2. Parametreler güvenli bir şekilde bağlanır ve sorgu çalıştırılır
$stmt->execute([$kullanici_adi, $sifre]);
$kullanici = $stmt->fetch();
```

Saldırı Engelleme: Saldırgan kullanıcı adı alanına *admin* girse dahi, veritabanı bunu derlenmiş sorgunun içine mantıksal bir SQL komutu olarak eklemeyiz. Veritabanı, tam olarak adı *admin* olan bir kullanıcı arar ve bulamayacağı için erişimi engeller.

İsmlendirilmiş Parametreler Örneği

Soru işaretleri (?) yerine, okunabilirliği artırmak için ismlendirilmiş parametreler (*Named Parameters*) de kullanılabilir:

```
$stmt = $db->prepare("SELECT * FROM urunler WHERE kategori = :kategori AND fiyat < :fiyat");

$stmt->execute([
    ':kategori' => $gelen_kategori,
    ':fiyat'    => $gelen_fiyat
]);
```

Özet Olarak

Güvenlik: Kullanıcı girdisini veriden koda dönüştürme ihtimalini ortadan kaldırarak SQL Injection'ı %100'e yakın oranla engeller.

Performans: Aynı sorgu farklı parametrelerle defalarca çalıştırılacaksa (örneğin toplu veri ekleme), sorgu mantığı veritabanında bir kez derlendiği için işlem daha hızlı gerçekleşir.

7.6 Oturum ve Çerez Yönetimi

Bölüm 1.9'de HTTP'nin **durumsuz** (*stateless*) bir protokol olduğunu, yani sunucunun önceki isteklerin durumunu hatırla(ya)madığını öğrenmiştik. Ancak gerçek dünyadaki bir web uygulamasında, bir kullanıcının giriş yaptıktan sonraki sayfalarda hâlâ "giriş yapmış" olarak tanınması gerekir. PHP bu sorunu iki tamamlayıcı mekanizmayla çözer: **çerezler** (*cookies*) ve **oturumlar** (*sessions*).

§7.6.1 Çerezler (Cookies)

Çerez, sunucunun tarayıcıya gönderip tarayıcının kendi bilgisayarında sakladığı, sonraki her istekte tarayıcının otomatik olarak sunucuya geri gönderdiği küçük bir metin verisidir. PHP'de bir çerez oluşturmak için `setcookie()` fonksiyonu, okumak için ise `$_COOKIE` süper dizisi kullanılır.

```
<?php
// Çerez Oluşturma: isim, değer, geçerlilik süresi (saniye)
// Aşağıdaki örnek, çerezi 1 gün (86400 saniye) geçerli kılar.
setcookie("kullaniciAdi", "Ahmet", time() + 86400);
?>

<?php
// Bir sonraki istekte çerezi okuma
if (isset($_COOKIE["kullaniciAdi"])) {
    echo "Merhaba tekrar, " . $_COOKIE["kullaniciAdi"];
} else {
    echo "Çerez bulunamadı.";
}
?>
```

Çerezlerin Sınırlılıkları

Çerezler kullanıcının kendi tarayıcısında saklandığı için tarayıcı geliştirici araçlarından görülebilir ve değiştirilebilir. Bu nedenle şifre gibi hassas verilerin doğrudan çerezde saklanması güvenli değildir; bunun yerine bir sonraki alt bölümde göreceğimiz oturum (*session*) mekanizması tercih edilmelidir.

§7.6.2 Oturumlar

Oturum, çerezlerin aksine asıl veriyi **sunucu tarafında** saklar; tarayıcıya yalnızca o oturumu tanımlayan benzersiz bir kimlik (genellikle `PHPSESSID` adlı bir çerez içinde) gönderilir. Böylece hassas veriler hiçbir zaman kullanıcının tarayıcısına gitmez.

Bir sayfada oturum değişkenlerini kullanabilmek için, sayfanın en başında (herhangi bir HTML çıktısından önce) `session_start()` fonksiyonu çağrılmalıdır. Oturum verileri, global `$_SESSION` dizisinde saklanır.

```
<?php
session_start(); // Oturumu başlatır veya var olan oturuma bağlanır

$_SESSION['kullaniciAdi'] = "Ahmet"; // Oturuma veri yazma
$_SESSION['girisYapildi'] = true;
?>
```

Örnek: Basit Bir Giriş Sistemi

Aşağıdaki üç dosya, oturum yönetiminin en klasik kullanım senaryosunu – bir giriş (*Login*) sistemini – birlikte oluşturur.

1) **giris.php** – Giriş Formu ve Doğrulama:

```
<?php
session_start();

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $kullanici = $_POST['kullanici'] ?? '';
    $sifre = $_POST['sifre'] ?? '';

    // Gerçek bir projede bu bilgi veritabanından ve
    // şifrelenmiş (hashlenmiş) olarak kontrol edilir.
    if ($kullanici === "admin" && $sifre === "1234") {
        $_SESSION['girisYapildi'] = true;
        $_SESSION['kullaniciAdi'] = $kullanici;

        header("Location: panel.php"); // Başarılı girişte panele yönlendir
        exit();
    } else {
        $hataMesaji = "Kullanıcı adı veya şifre hatalı!";
    }
}

?>
<!DOCTYPE html>
<html lang="tr">
<body>
    <?php if (isset($hataMesaji)): ?>
    <p style="color:red;"><?php echo $hataMesaji; ?></p>
    <?php endif; ?>
    <form action="giris.php" method="POST">
    <input type="text" name="kullanici" placeholder="Kullanıcı Adı">
    <input type="password" name="sifre" placeholder="Şifre">
    <button type="submit">Giriş Yap</button>
    </form>
</body>
</html>
```

2) **panel.php** – Yalnızca Giriş Yapmış Kullanıcıların Görebileceği Sayfa:

```
<?php
session_start();

// Oturum kontrolü: giriş yapılmamışsa giriş sayfasına geri gönder
if (!isset($_SESSION['girisYapildi']) || $_SESSION['girisYapildi'] !== true) {
    header("Location: giris.php");
    exit();
}

?>
<!DOCTYPE html>
<html lang="tr">
<body>
    <h1>Hoş geldin, <?php echo htmlspecialchars($_SESSION['kullaniciAdi']); ?></h1>
    <a href="cikis.php">Çıkış Yap</a>
</body>
</html>
```

3) **cikis.php** – Oturumu Sonlandırma:

```
<?php
session_start();
session_unset(); // Oturumdaki tüm değişkenleri temizler
session_destroy(); // Oturumun kendisini sunucudan siler

header("Location: giris.php");
exit();
?>
```

header() ve Yönlendirme-Sonrası-Gönderme (PRG) Deseni

`header("Location: ...")` fonksiyonu, tarayıcıya farklı bir sayfaya yönlendiğini söyleyen bir HTTP yönlendirme başlığı gönderir; bu fonksiyonun çalışabilmesi için sayfada **hiçbir HTML çıktısı üretilmemiş olmalıdır**. Bir form işlendikten hemen sonra başka bir sayfaya yönlendirme yapılması (yukarıdaki `giris.php` örneğinde olduğu gibi), kullanıcının sayfayı yenilemesi (F5) durumunda formun yanlışlıkla tekrar gönderilmesini engelleyen **Post/Redirect/Get (PRG)** deseninin temelini oluşturur.

§7.6.3 PHP ve AJAX Örnekleri

AJAX (*Asynchronous JavaScript and XML*), web sayfasını tamamen yeniden yüklemeyen sunucudan arka planda veri çekmeyi ve arayüzü güncellemeyi sağlar.

Aşağıdaki örnekte kullanıcı açılır menüden (`select/combo`) bir albüm kategorisi seçtiğinde, JavaScript arka planda PHP sunucusuna istek atar; PHP ise veritabanından filtrelenen CD kataloğunu getirerek sayfayı anlık olarak günceller.

PHP Sunucu Tarafı (`get_cd.php`):

```
<?php
// get_cd.php
header('Content-Type: application/json');

$kategori = $_GET['kategori'] ?? '';

// Veritabanından kategoriye göre CD sorgusu yapıldığı varsayılmaktadır
$cdKatalog = [
    "pop" => ["Tarkan - Karma", "Sezen Aksu - Gülümse"],
    "rock" => ["Duman - Seni Kendime Sakladım", "Manga - Şehr-i Hüzn"]
];

$sonuc = $cdKatalog[$kategori] ?? [];
echo json_encode($sonuc); // JSON formatında yanıt döndürülür
?>
```

İstemci Tarafı HTML + JavaScript AJAX Mantığı:

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>AJAX CD Kataloğu</title>
  </head>
  <body>
    <label for="kategoriSelect">Kategori Seçin:</label>
    <select id="kategoriSelect" onchange="cdGetir(this.value)">
      <option value="">Seçiniz...</option>
      <option value="pop">Pop</option>
      <option value="rock">Rock</option>
    </select>

    <h4>Katalog Sonuçları:</h4>
    <ul id="cdListesi"></ul>

    <script>
      function cdGetir(kategori) {
        if (!kategori) return;

        // Asenkron HTTP isteği (Fetch API)
        fetch('get_cd.php?kategori=' + kategori)
          .then(response => response.json())
          .then(data => {
            const liste = document.getElementById('cdListesi');
            liste.innerHTML = ''; // Liste içeriğini temizle

            data.forEach(album => {
              const li = document.createElement('li');
              li.textContent = album;
              liste.appendChild(li);
            });
          });
      }
    </script>
  </body>
</html>
```

```

    });
  })
  .catch(error => console.error('AJAX Hatası:', error));
}
</script>
</body>
</html>

```

Bölüm Özeti

Bu bölümde, web sunucu tarafı programlamanın temel çalışma prensiplerini ve dinamik web uygulaması geliştirmede kullanılan teknolojileri inceledik:

- ♦ **CGI ve Sunucu Betikleri:** Web sunucularının dış programlarla iletişim kurmasını sağlayan CGI protokolünden başlayarak, sunucu tarafında yorumlanan PHP, ASP ve JSP gibi betik dillerinin çalışma mantığını gördük.
- ♦ **PHP Mimarisi:** HTML kodları arasına gömülebilen PHP dilinin sözdizimi, dinamik tiplendirmeli değişkenleri ve sunucu tarafında çıktı üretme mekanizmasını öğrendik.
- ♦ **Form Handling:** HTTP GET ve POST yöntemleriyle istemciden alınan verilerin PHP tarafından işlenmesini ve URL/Form parametrelerinin kullanımını inceledik.
- ♦ **Oturum ve Çerez Yönetimi:** HTTP'nin durumsuz yapısına rağmen `$_SESSION` ve `$_COOKIE` mekanizmalarıyla kullanıcı durumunun (ör. giriş yapılmış olma bilgisi) sayfalar arasında nasıl korunacağını, basit bir giriş sistemi üzerinden uyguladık.
- ♦ **Veri Tabanı İletişimi (MySQL):** RDBMS kavramını ve PHP PDO kütüphanesi kullanarak veritabanında tablo oluşturma, verileri okuma, ekleme, güncelleme ve silme (CRUD) adımlarını uyguladık.
- ♦ **AJAX ile Asenkron İletişim:** Sayfa yenilenmeden sunucuya arka planda veri alışverişi sağlayan AJAX mimarisini PHP-JavaScript entegrasyonu ile pratikleştirdik.

7.7 Düşün ve Çöz

1. **Sunucu ve İstemci Betikleri:** Bir web uygulamasında JavaScript kodu ile PHP kodunun çalıştırıldığı yer ve zaman bakımından temel farkları nelerdir? Neden kullanıcı şifresi doğrulama işlemi sadece JavaScript ile yapılamaz?
2. **HTTP Yöntemleri Seçimi:** Bir e-ticaret sitesinde ürün arama formu için HTTP GET metodu kullanılırken, kullanıcı giriş formu için neden HTTP POST metodu tercih edilmelidir? Güvenlik ve URL yapısı açısından açıklayınız.
3. **SQL Injection Güvenliği:** Aşağıdaki PHP kod satırı neden bir güvenlik açığı oluşturur? Bu sorunun önüne geçmek için yapılması gereken düzeltmeyi `PDO Prepared Statement` mantığı ile açıklayınız.

```
$sql = "SELECT * FROM kullanicilar WHERE ad = '" . $_POST['kullanici'] . "'";
```
4. **AJAX Kullanım Senaryosu:** Bir web sayfasında kullanıcı il seçtiğinde ilçe listesinin sayfa yeniden yüklenmeden güncellenmesini istiyorsunuz. AJAX yapısı kullanıldığında tarayıcı, PHP sunucusu ve veritabanı arasındaki veri akış sırasını adım adım açıklayınız.
5. **Oturum mu, Çerez mi?** Bir alışveriş sitesinde (a) kullanıcının "beni hatırla" tercihini ve (b) kullanıcının o an sepetinde bulunan ürünlerin toplam tutarını saklamak istiyorsunuz. Bu iki bilgidenden hangisi çerezde, hangisi oturumda saklanmalıdır? Güvenlik ve kalıcılık açısından gerekçelendiriniz.

7.8 Uygulamalı Projeler

Bu bölümde öğrendiğiniz form işleme, oturum yönetimi, PDO ile veritabanı işlemleri ve AJAX kavramlarını pekiştirmek için aşağıdaki beş projeden birini veya birkaçını tamamlamayı deneyin. Yerel bir Apache/PHP/MySQL ortamına ihtiyacınız olacaktır.

1. **Basit Hesap Makinesi:** İki sayı ve bir işlem (toplama, çıkarma, çarpma, bölme) seçimi alan bir `GET` formu oluşturun; PHP tarafında `switch` yapısıyla seçilen işlemi uygulayan bir fonksiyon yazıp sonucu ekrana yazdırın. Sıfıra bölme durumunu ayrıca kontrol edin.
2. **Ziyaretçi Defteri:** Ad ve mesaj alanları olan bir `POST` formuyla gönderilen verileri PDO kullanarak bir MySQL tablosuna kaydedin; sayfanın altında, tablodaki tüm mesajları en yeniden en eskiye doğru listelein.
3. **Veritabanı Destekli Giriş Sistemi:** Bölümdeki `giris.php` örneğini genişletin: kullanıcı adı ve şifreyi sabit kodlamak yerine, PDO ile bir `kullanicilar` tablosunda arayın (şifreleri `password_hash()` ile saklayıp `password_verify()` ile doğrulayın) ve başarılı girişte oturumu (`$_SESSION`) başlatın.
4. **Ürün Kataloğu Yönetim Paneli:** Bir MySQL tablosundaki ürünleri (ad, fiyat) listeleyen, yeni ürün eklemeye (`POST`) ve mevcut bir ürünü silmeye (bağlantı üzerinden `GET` ile ürün `id`'sini göndererek) imkan tanıyan basit bir yönetim sayfası oluşturun.
5. **AJAX ile Anlık Arama:** Bir arama kutusuna her tuş vuruşunda (`keyup` olayı), `fetch()` ile bir PHP betiğine

istek gönderin; bu betik, gelen anahtar kelimeyi PDO ile veritabanında arayıp sonuçları JSON olarak döndürsün ve tarayıcı bu sonuçları sayfa yenilenmeden listelesin.

§7.8.1 Ziyaretçi Defteri Örneği

```
<?php
// ziyaretci-defteri.php
// Not: Çalıştırmadan önce aşağıdaki veritabanı bilgilerini kendi
// XAMPP/Apache/MySQL ortamınıza göre düzenleyin ve 'web_programlama' veritabanında 'ziyaretler'
//   tablosunu oluşturun:
//
// CREATE TABLE ziyaretler (
//   id INT AUTO_INCREMENT PRIMARY KEY,
//   ad VARCHAR(100) NOT NULL,
//   mesaj TEXT NOT NULL,
//   tarih DATETIME DEFAULT CURRENT_TIMESTAMP
// );

$sunucu = 'localhost';
$kullanici = 'devuser';
$sifre = 'YourStrongPassword123!';
$veritabani = 'web_programlama';

try {
    $pdo = new PDO("mysql:host=$sunucu;dbname=$veritabani;charset=utf8", $kullanici, $sifre);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $hata) {
    die("Veritabanı bağlantı hatası: " . $hata->getMessage());
}

$hataMesaji = "";

// Form POST ile gönderildiyse yeni mesajı kaydet
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $ad = trim($_POST['ad'] ?? '');
    $mesaj = trim($_POST['mesaj'] ?? '');

    if ($ad != "" && $mesaj != "") {
        // Prepared Statement kullanarak SQL Injection riskini ortadan kaldırıyoruz
        $sorgu = $pdo->prepare("INSERT INTO ziyaretler (ad, mesaj) VALUES (:ad, :mesaj)");
        $sorgu->execute([
            ':ad' => $ad,
            ':mesaj' => $mesaj
        ]);

        // Post/Redirect/Get deseni: kaydettikten sonra aynı sayfaya yönlendir
        header("Location: " . $_SERVER['PHP_SELF']);
        exit();
    } else {
        $hataMesaji = "Lütfen adınızı ve mesajınızı giriniz.";
    }
}

// Tüm mesajları en yenden en eskiye doğru çekiyoruz
$sonuc = $pdo->query("SELECT ad, mesaj, tarih FROM ziyaretler ORDER BY tarih DESC");
$mesajlar = $sonuc->fetchAll(PDO::FETCH_ASSOC);
?>
<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>Ziyaretçi Defteri</title>
    <style>
    body { font-family: Arial, sans-serif; max-width: 500px; margin: 40px auto; }
    form { margin-bottom: 30px; }
    input, textarea { width: 100%; padding: 8px; margin-bottom: 10px; box-sizing: border-box; }
    .mesaj-kutusu { border-bottom: 1px solid #ddd; padding: 10px 0; }
```

```
.hata { color: red; }
</style>
</head>
<body>

<h1>Ziyaretçi Defteri</h1>

<?php if ($hataMesaji): ?>
<p class="hata"><?php echo htmlspecialchars($hataMesaji); ?></p>
<?php endif; ?>

<form action="ziyaretci-defteri.php" method="POST">
<input type="text" name="ad" placeholder="Adınız">
<textarea name="mesaj" rows="3" placeholder="Mesajınız"></textarea>
<button type="submit">Gönder</button>
</form>

<h2>Gelen Mesajlar</h2>

<?php if (count($mesajlar) === 0): ?>
<p>Henüz mesaj yok. İlk mesajı siz bırakın!</p>
<?php endif; ?>

<?php foreach ($mesajlar as $satir): ?>
<div class="mesaj-kutusu">
<strong><?php echo htmlspecialchars($satir['ad']); ?></strong>
<small> - <?php echo $satir['tarih']; ?></small>
<p><?php echo htmlspecialchars($satir['mesaj']); ?></p>
</div>
<?php endforeach; ?>

</body>
</html>
```

8. Bölüm

Web Sunucusu Tarafında Programlama

8.1 Common Gateway Interface

Şu ana kadar istemci olarak web tarayıcılarının akıllı hale getirilmesi için HTML-JS-CSS ile XML teknolojilerini öğrendik. Tarayıcılara benzer şekilde modern web sunucuları sadece istekleri karşılayan ve ona cevap veren sunucular değildir.

İstemci tarafında nasıl JS varsa sunucu tarafında da istekleri işleyip cevaplayan teknolojiler bulunur. Bu teknolojiler; HTML dosyasını cevap olarak göndermek yerine, içerisine veriler ekleyerek gönderebilir. Eklenenecek veriler bir başka sunucudan alınabileceği gibi veri tabanından da alabilir. Gelen isteklerden dosyaları değiştirebilir, silebilir veya veri tabanlarına veri yazabilir.

Ortak Ağ geçidi Arayüzü yani kısaca **CGI** (*Common Gateway Interface*), web sunucularının HTTP kullanıcı isteklerini işlemek için harici bir programı çalıştırmasını sağlayan bir ara yüzdür. 1993 yılında NCSA tarafından Httpd için geliştirilmiştir. Bu sayede Web sunucusunun kendisi veya uzantı (*extension*) modülleri aracılığıyla sunucu tarafı betik (*script*) çalıştırma devri 1996 yılında başlamıştır.

8.2 Sunucu Tarafı Betik

Sunucu tarafında betik (*server-side script*) çalıştırma; her kullanıcının web sitesine olan isteğine göre özelleştirilmiş bir yanıt üreten bir web sunucusunda betik (*script*) kullanmayı içeren bir tekniktir.

Sunucu tarafı betikler, PHP (*Personal Home Page*), ASP (*Active Server Pages*) ve JSP (*JavaServer Pages*) gibi birçok programlama dilinden herhangi birinde yazılabilir. Bu betikler, JavaScript gibi HTML belgesine gömülü betiklerin bir web tarayıcısında çalıştırıldığı gibi çalışmaz.

Şu ana kadar öğrendiğimiz ve içerisinde sadece HTML öğeleri bulunan dosyalar, statik web sayfaları (*static web pages*) olarak adlandırılır. Sunucu tarafında çalıştırılacak betik içeren sayfalar ise *.html veya *.htm olarak değil farklı uzantılara sahip olurlar. Sunucuda PHP betiği içeren dosyalar *.php uzantısıyla, ASP betiği içeren dosyalar *.asp ve JSP betiği içeren dosyalar ise *.jsp uzantısıyla saklanırlar.

Bölüm ??'de gösterilen tarayıcı-sunucu iletişimi, sunucu tarafı betik olduğunda aşağıdaki şekilde çalışır;

- ♦ Web tarayıcıda betik içeren URL (örneğin www.ornek.com/demo.php) yazılır.
- ♦ Sunucu talep edilen betik dosyasını (demo.php) tarayıcıya göndermeden önce betik kurallarına göre yorumlar. Betik çalıştırdıktan sonra oluşan nihai HTML belgesi tarayıcıya gönderilir.
- ♦ Tarayıcı gelen HTML belgesinden Belge nesne modeli yani DOM (*Document Object Model*), Kademeli Stil Sayfası Nesne Modeli olan CSSOM (*Cascading Style Sheet Object Model*) ağaçlarını oluşturur ve oluşan bu ağaçlardan görselleştirme (*render*) yaparak kullanıcıya sunar.

Sunucu tarafında en çok kullanılan betikler¹;

- ♦ **ASP** (*Active Server Pages*), Microsoft'un dinamik web sayfaları için ilk sunucu tarafı betik dili ve motorudur. 1996 ile 2000 yılları arasında popüler olmuştur.
- ♦ **JSP** (*JavaServer Pages*; 2017'den itibaren Eclipse Vakfı bünyesinde *Jakarta Server Pages* olarak da anılır), Yazılım geliştiricilerin HTML, XML, SOAP veya diğer belge türlerine dayalı dinamik olarak oluşturulmuş web sayfaları için sunucu tarafında çalışan bir teknolojidir. MVC (*Model-View-Controller*) desenini (*pattern*) kullanır.
- ♦ **PHP** (*Personal Home Page*), web geliştirmeye yönelik genel amaçlı bir betik dilidir. 1995 yılından sonra birçok projede kullanılmaktadır.
- ♦ **ASP.NET Web Forms**, Microsoft ASP.NET teknolojisini kullanan bir web uygulama çerçevesidir (*application framework*). 2002 ile 2010 yılları arasında çok kullanılmıştır.
- ♦ **Razor**, C# veya VB.NET programlama dilleriyle dinamik web sayfaları oluşturmak için kullanılan bir ASP.NET teknolojisidir. 2010 yılından beri kullanılmaktadır.
- ♦ **ASP.NET MVC**, Microsoft tarafından geliştirilen, MVC desenini uygulayan bir web uygulama teknolojisidir. 2012 yılından beri Apache License 2.0 ile yayımlanmaktadır.

¹https://w3techs.com/technologies/overview/programming_language

- ◇ **Node.js**, JavaScript geleneksel olarak istemci tarafı bir dil olsa da, Node.js JavaScript'i sunucu tarafında betik olarak çalışır.
- ◇ Bunların yanında **Google Apps Script**, **Go**, **Ruby**, **Python** ve **Perl** dilleri de web sunucularında betik olarak kullanılmaktadır.

Bu bölümde dünyada oldukça popüler olan Apache-PHP-MySQL üçlüsü üzerinde durulacaktır. İzleyen bölümlerde ise daha modern yaklaşımları olan ASP.NET teknolojisine örnek verilecektir.

Kişisel Ana Sayfa yani kısaca **PHP** (*Personal Home Page*) başlangıçta web üzerinde kişisel başlangıç sayfaları oluşturmak için geliştirilmiştir. Günümüzde **Hiper Metin Ön İşlemcisi** (*Hypertext Preprocessor*) olarak oldukça yaygın olarak kullanılmaktadır.

PHP nin C++ diline benzeyen bir yazım biçimi vardır. HTML belgesine PHP kodları derç edilerek sunucunun dosyayı göndermeden önce PHP dili ile işlenmesini sağlar. Özetle PHP dili sunucuda çalışır ve yorumlanır ve sadece çıktısı kullanıcı bilgisayarındaki tarayıcıya gönderilir. Böylece veritabanı sunucusunda bulunan verilere sadece web sunucusu üzerinden erişim sağlanır.

Sunucu üzerinde çalışan PHP dilini kullanabilmek için bir web sunucusuna ihtiyaç duyarız. PHP dili ile en çok kullanılan Apache web sunucusudur. Kullanıcılar kendi kişisel bilgisayarlarından veri tabanına direk müdahale edemezler.

Eğer veritabanı kullanılacak ise web sunucusuna bir veritabanı kurulması gereklidir. PHP ile en çok kullanılan veri tabanı sunucusu MySQL veri tabanıdır.

Şimdi bu üçlüyü başlıklar halinde açıklayalım.

8.3 Apache Web Sunucusu

Apache HTTP Sunucusu (*Apache HTTP Server*), dünyada en yaygın kullanılan açık kaynaklı, modüler ve platformlar arası web sunucu yazılımlarından biridir. Tarayıcılardan gelen HTTP/HTTPS isteklerini karşılar, statik dosyaları (HTML, CSS, JS, resim) doğrudan sunar veya PHP gibi yorumlayıcı modülleri tetikleyerek dinamik içerik üretip istemciye geri yanıt döner.

Apache Mantiğı ve Çalışma Prensibi

Apache, Varsayılan olarak **Port 80** (HTTP) ve **Port 443** (HTTPS) ağ kapılarını dinler. Tarayıcıdan bir web sayfasına erişmek istediğinde, Apache arka planda bir süreç (*process*) veya iş parçacığı (*thread*) oluşturarak isteği işler ve **DocumentRoot** dizinindeki dosyaları sunar.

§8.3.1 Apache2 Paketinin Kurulması

Linux ortamında Apache kurulumu paket yöneticisi (**apt**) aracılığıyla kolayca gerçekleştirilir. Kurulumu başlamanın önce sistem paket depolarının güncelliğinden emin olunmalıdır. Apache sunucu yazılımı Debian tabanlı sistemlerde **apache2** adıyla bulunur:

```
sudo apt update && sudo apt upgrade -y
sudo apt install apache2 -y
```

§8.3.2 Servis Durumunun ve Portların Kontrolü

Kurulum tamamlandıktan sonra Apache servisinin çalışıp çalışmadığı denetlenir:

```
# Apache servis durumunu kontrol etme
sudo systemctl status apache2

# Servisi başlatma / durdurma / yeniden başlatma komutları
sudo systemctl start apache2
sudo systemctl stop apache2
sudo systemctl restart apache2

# Sistemin her açılışında otomatik başlamasını sağlama
sudo systemctl enable apache2
```

Güvenlik Duvarı (UFW) İzinleri

Eğer sisteminizde **ufw** (*Uncomplicated Firewall*) aktif ise, dış dünyadan gelen web trafiğine izin verilmelidir:

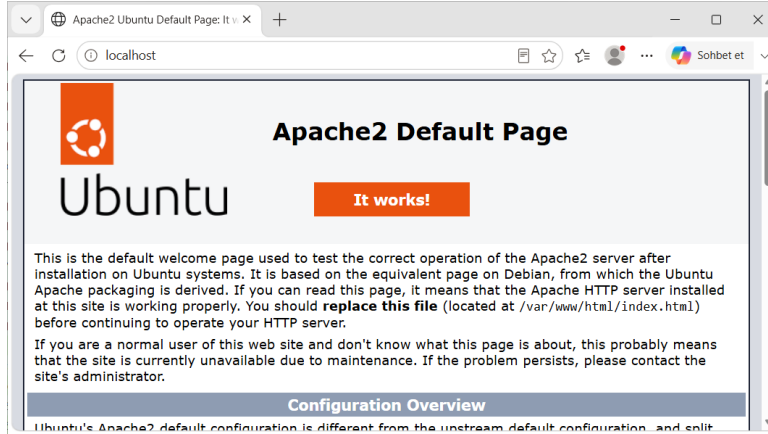
```
sudo ufw allow 'Apache Full' # Port 80 ve 443'e izin verir
sudo ufw reload
```

§8.3.3 Apache Dizin Yapısı ve Önemli Dosyalar

Ubuntu/Debian dağıtımlarında Apache konfigürasyon dosyaları düzenli bir dizin hiyerarşisine sahiptir:

- ◇ **/var/www/html**: Web sitesinin kaynak dosyalarının (HTML, PHP vb.) tutulduğu varsayılan **DocumentRoot** dizinidir. Sanal konak kullanmayacaksa PHP dosyalarımızı buraya koymalıyız.
- ◇ **/etc/apache2/apache2.conf**: Apache sunucusunun ana konfigürasyon dosyasıdır.
- ◇ **/etc/apache2/sites-available/**: Sunucuda tanımlanmış tüm sanal konakların (*Virtual Host*) yapılandırma dosyalarının yer aldığı dizindir.
- ◇ **/etc/apache2/sites-enabled/**: Aktif olarak çalışan sanal konakların sembolik bağlantılarının (*symlink*) bulunduğu dizindir.
- ◇ **/var/log/apache2/**: Erişim (*access.log*) ve hata (*error.log*) kayıtlarının tutulduğu dizindir.

Sistem düzgün kurulmuş ise tarayıcıda aşağıdaki sayfayı görürüz;



Şekil 8.1: **/var/www/html** dizinindeki varsayılan **index.html** belgesinin tarayıcı görselleştirmesi

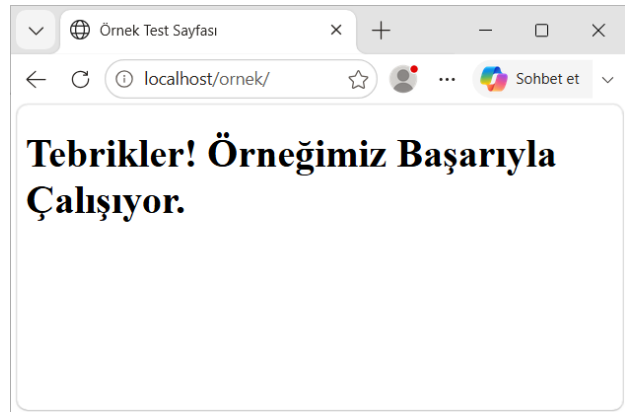
Daha sonra **/var/www/html** klasörüne geçip, **ornek** adında bir klasör oluşturulur;

```
sudo mkdir /var/www/html/ornek
```

Sonrasında **index.html** adlı dosyayı **/var/www/html/ornek** klasörü altında aşağıdaki **index.html** dosyasını oluşturuyoruz. Daha sonra bu dizini, PHP örnekleri ile dolduracağız.

```
<!-- /var/www/ornekproje/index.html -->
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>Örnek Test Sayfası</title>
  </head>
  <body>
    <h1>Tebrikler! Örneğimiz Başarıyla
    ↪ Çalışıyor.</h1>
  </body>
</html>
```

Kod 8.1: **ornek** klasöründeki HTML Belgesi



Şekil 8.2: **ornek** klasöründeki HTML Belgesinin Tarayıcı Görselleştirmesi

Daha detaylı kurulum için sanal konak (*virtual host*) kurulumunu araştırabilirsiniz. Sanal konak, tek bir fiziki sunucu ve IP adresi üzerinde birden fazla farklı alan adının (*domain*) veya web sitesinin barındırılmasına olanak tanır.

§8.3.4 Apache Modülleri ve PHP Entegrasyonu

Apache, esnek modüler mimarisi sayesinde fonksiyonel kütüphaneler eklenerek genişletilebilir. Örneğin, yönlendirme kuralları için **mod_rewrite** modülü ve PHP dosyalarını işleyebilmek için **libapache2-mod-php** kullanılır.

```
# URL Yönlendirme (Rewrite) modülünü aktifleştirme
sudo a2enmod rewrite
```

```
# PHP modülünün aktif olduğunu doğrulama ve Apache'yi yenileme
sudo systemctl restart apache2
```

8.4 PHP

PHP (*Hypertext Preprocessor*), web sunucularında dinamik web sayfaları ve web uygulamaları geliştirmek için kullanılan, sunucu tarafında yorumlanan açık kaynaklı bir betik dilidir. C dili ailesinden türemiş bir sözdizimine sahiptir ve HTML kodları içerisinde doğrudan gömülebilir.

PHP Çalışma Prensibi

PHP kodları doğrudan kullanıcının tarayıcısında çalışmaz. Kullanıcı bir PHP sayfasına istek gönderdiğinde, web sunucusu (örneğin Apache) PHP yorumlayıcısını tetikler. PHP kodu sunucuda çalıştırılır, gerekli veritabanı veya dosya işlemleri yapılır ve sonuç olarak istemciye yalnızca **saf HTML/CSS/JS** çıktısı gönderilir.

§8.4.1 Linux PHP Kurulumu

Linux (Ubuntu/Debian) sistemlerde PHP yorumlayıcısını ve Apache sunucusu ile iletişimini sağlayan modülleri kurmak için uçbirim (*terminal*) üzerinden aşağıdaki komutlar yürütülür:

```
# Paket listelerini güncelleme
sudo apt update

# PHP ve Apache entegrasyon modülünün kurulumu
sudo apt install php libapache2-mod-php php-mysql -y

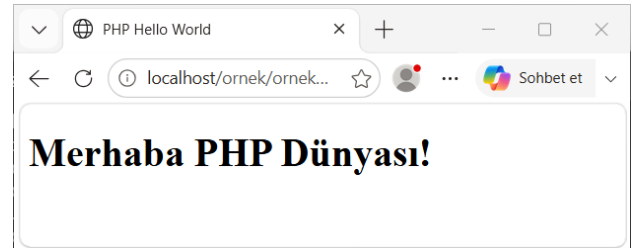
# Kurulumun ve PHP sürümünün kontrolü
php -v
```

§8.4.2 PHP Hello Uygulaması

PHP kod bloğu `<?php` etiketi ile başlar ve `?>` etiketi ile biter. Kod bloğu içerisindeki her talimat noktalı virgül (;) ile sonlandırılmalıdır. Metinsel çıktı üretmek için en sık kullanılan talimat `echo`'dur.

Aşağıda HTML belgesine gömülmüş temel bir PHP uygulaması yer almaktadır:

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>PHP Hello World</title>
  </head>
  <body>
    <h1>
      <?php
        echo "Merhaba PHP Dünyası!";
      ?>
    </h1>
  </body>
</html>
```



Şekil 8.3: PHP Merhaba Belgesi Tarayıcı Görselleştirmesi

Kod 8.2: PHP Merhaba Belgesi

§8.4.3 PHP Değişkenleri ve Veri Tipleri

Bölüm 4.2.1 başlığında matematiksel kavram olarak gördüğümüz değişkenlerin burada somut karşılığını göreceksiniz. PHP dilinde değişkenler, programın çalışması sırasında verilerin bellekte saklandığı ve bu verilere erişimi sağlayan isimlendirilmiş bellek alanlarıdır. C tabanlı bir sözdizimine sahip olan PHP'de tüm değişken isimleri mutlaka dolar simgesi (\$) ile başlar. Değişken adı dolar işaretinden sonra bir harf veya alt çizgi (_) karakteri ile başlamalıdır; rakam ile başlayamaz.

Büyüklik/Küçüklük Harf Duyarlılığı (Case-Sensitivity)

PHP'de değişken isimleri **büyük/küçük harfe duyarlıdır** (*case-sensitive*). Yani `$sayi` ile `$SAYI` bellek üzerinde iki farklı değişken olarak ele alınır. Ancak PHP fonksiyon isimleri, sınıf isimleri ve anahtar kelimeler (`echo`, `if`, `while` vb.) harf duyarlı değildir.

```
<?php
// Geçerli değişken tanımlamaları
```

```
$ogrenciAdi = "Ahmet";
$_yas = 20;
$sayi_1 = 100;

// Geçersiz değişken tanımlaması (Sözdizimi Hatası)
// $1sayi = 50;
?>
```

Dinamik Tiplendirme Mantığı

JavaScript dilinde olduğu gibi PHP de **zayıf ve dinamik tipli** (*weakly & dynamically typed*) bir dildir. Bir değişken tanımlanırken veri tipi (*integer, string, boolean* vb.) açıkça bildirilmez. PHP yorumlayıcısı, değişkene atanan değer türünü çalışma zamanında (*runtime*) otomatik olarak tespit eder. Değişkenin türü, programın ilerleyen adımlarında farklı türde bir değere atanarak değiştirilebilir.

```
<?php
$veri = "Merhaba"; // Şu an 'string' türünde
$veri = 42;        // Artık 'integer' türünde
$veri = true;      // Şimdi ise 'boolean' türünde
?>
```

Temel Veri Tipleri

PHP varsayılan olarak sekiz temel yani ilkel veri tipini destekler:

- ◊ **Integer (Tamsayı):** Ondalık kısmı olmayan pozitif veya negatif tam sayılardır (... - 2, -1, 0, 1, 2...).
- ◊ **Float / Double (Ondalık Sayı):** Kesirli ve ondalıklı sayıları temsil eder (Örn: 3.14, -0.001).
- ◊ **String (Metin):** Tırnak işaretleri arasına yazılan karakter dizileridir.
- ◊ **Boolean (Mantıksal):** Sadece **true** (doğru) veya **false** (yanlış) değerini alabilir.
- ◊ **Array (Dizi):** Birden fazla değeri tek bir değişkende saklayan bileşik veri tipidir.
- ◊ **Object (Nesne):** Sınıflardan türetilen nesne örnekleridir.
- ◊ **Resource (Kaynak):** Dosya, veritabanı bağlantısı gibi PHP dışındaki kaynaklara tutulan özel bir referans türüdür.
- ◊ **NULL:** Değişkenin herhangi bir değere sahip olmadığını veya henüz başlatılmadığını (ilklenendirilmediğini) belirtir.

```
<?php
$metin = "Bilgisayar Programlama"; // String
$tamSayi = 1024;                    // Integer
$ondalikSayi = 99.99;               // Float
$aktifMi = true;                    // Boolean
$bosDeger = null;                   // NULL

// Değişkenin tipini ve değerini ayrıntılı inceleme (Hata ayıklama için)
var_dump($metin);                  // string(22) "Bilgisayar Programlama"
var_dump($ondalikSayi);            // float(99.99)
?>
```

Tek Tırnak ve Çift Tırnak Arasındaki Fark

PHP’de metinsel ifadeler tanımlanırken tırnak kullanımı kritik öneme sahiptir. Çift tırnak içerisinde yazılan değişkenler yorumlanarak değerleri çıktıya basılır (*Variable Interpolation*). Tek tırnak içerisinde yazılan ifadeler ise tamamen ham metin olarak kabul edilir ve değişken isimleri yorumlanmaz.

```
<?php
$ad = "Zeynep";
// Çift Tırnak: Değişken yorumlanır
echo "Hoş geldin $ad"; // Çıktı: Hoş geldin Zeynep

// Tek Tırnak: Değişken olduğu gibi yazdırılır
echo 'Hoş geldin $ad'; // Çıktı: Hoş geldin $ad
?>
```

PHP İşleçleri ve Metin Birleştirme

PHP’de matematiksel ve mantıksal işlemler için standart işleçler (*operators*) kullanılır. Metin birleştirme işleminde JavaScript’teki + operatöründen farklı olarak **nokta (.)** operatörü tercih edilir.

```
<?php
$isim = "Ahmet";
$soyisim = "Yılmaz";

// Metin Birleştirme (String Concatenation)
```

```
$tamIsim = $isim . " " . $soyisim; // Ahmet Yılmaz

// Aritmetik İşleçler (Operators)
$a = 15;
$b = 4;
$toplam = $a + $b; // 19
$kalani = $a % $b; // 3 (Mod alma)

// Birleştirerek Atama Operatörü (.=)
$mesaj = "Merhaba ";
$mesaj .= $tamIsim; // Merhaba Ahmet Yılmaz
?>
```

Tip Dönüşümü ve Tip Zorlaması

PHP, işlem sırasında farklı tipteki verileri otomatik olarak birbirine dönüştürebilir (*Implicit Casting*). Ancak yazılımcı isterse parantez içerisinde hedef veri tipini belirterek açık tip dönüşümü de (*Explicit Casting*) yapabilir.

```
<?php
// Otomatik Tip Dönüşümü (Type Coercion)
$sayiMetin = "10";
$toplam = $sayiMetin + 5; // PHP metni otomatik sayıya çevirir -> 15

// Açık Tip Dönüşümü (Explicit Casting)
$ondalik = 8.75;
$tamSayi = (int) $ondalik; // 8 (Ondalık kısım atılır)

$sayi = 100;
$metinYap = (string) $sayi; // "100" metnine dönüştürülür
?>
```

Değişken Kapsamı

PHP’de değişkenlerin erişilebilirliği tanımlandıkları yere göre belirlenir. Değişkenlerin yaşam alanı olan kapsam (*scope*), JavaScript’teki **var**, **let**, **const** ayırımından farklı olarak PHP’de ana kapsam mantığı fonksiyon sınırlarına dayanır.

- ◊ **Global Scope (Küresel Kapsam):** Fonksiyonların dışında tanımlanan değişkenlerdir. Sadece fonksiyonların dışındaki alanda doğrudan erişilebilirler. Fonksiyon içerisinde kullanılabilmeleri için **global** anahtar kelimesi ile çağrılmaları gerekir.
- ◊ **Local Scope (Yerel Kapsam):** Bir fonksiyonun içerisinde tanımlanan değişkenlerdir. Sadece tanımlandıkları fonksiyon bloğunun içinden erişilebilirler; fonksiyon bittiğinde bellekten silinirler.
- ◊ **Static Scope (Statik Kapsam):** Fonksiyon içerisinde **static** anahtar kelimesi ile tanımlanan değişkenlerdir. Fonksiyonun çalışması tamamlansa dahi değerini bellekte korur.

```
<?php
$globalSayi = 50; // Global Kapsam
function test() {
    // Fonksiyon içinde global değişkene erişmek için:
    global $globalSayi;
    $yerelSayi = 10; // Local Kapsam
    echo $globalSayi + $yerelSayi;
}
test(); // Çıktı: 60
// Statik Değişken Örneği
function sayac() {
    static $sayi = 0; // İlk çağrıda oluşturulur, değerini korur
    $sayi++;
    echo $sayi . " ";
}
sayac(); // Çıktı: 1
sayac(); // Çıktı: 2
sayac(); // Çıktı: 3
?>
```

Sabitler ile define()/const

PHP’de değeri programın çalışması boyunca değiştirilemeyen yapılara sabit (*Constants*) denir. Sabitlerin başında dolar (\$) simgesi kullanılmaz ve varsayılan olarak küresel kapsama sahiptirler. Sabitler `define()` fonksiyonu veya `const` anahtar kelimesi ile tanımlanır.

```
<?php
define("PI_SAYISI", 3.14159);
const SITE_URL = "https://orneksite.com";
echo PI_SAYISI; // 3.14159
?>
```

§8.4.4 PHP Dizileri

PHP’de dizi (*Array*), tek bir değişken içerisinde birden fazla değeri depolamanızı sağlayan özel bir veri yapısıdır. PHP dizileri aslında sıralı haritalardır (*ordered maps*); yani anahtar (*key*) ve değer (*value*) çiftlerinden oluşur. PHP’de temel olarak 3 tür dizi bulunur:

İndeksli Diziler

Elemanların otomatik olarak 0’dan başlayan tam sayı indislerle (*index*) erişildiği dizilerdir (*Indexed Arrays*).

```
<?php
// Dizi tanımlama
$meyveler = ["Elma", "Muz", "Portakal"];

// Elemana erişim
echo $meyveler[0]; // Çıktı: Elma

// Yeni eleman ekleme
$meyveler[] = "Çilek";
?>
```

İlişkisel Diziler

İndeksler yerine kendi tanımladığınız özel metinsel veya sayısal anahtarları (*keys*) kullanan dizilerdir (*Associative Arrays*).

```
<?php
$kullanici = [
    "ad" => "Ahmet",
    "eposta" => "ahmet@example.com",
    "rol" => "Yazılımcı"
];

// Elemana erişim
echo $kullanici["ad"]; // Çıktı: Ahmet
?>
```

Çok Boyutlu Diziler

İçinde bir veya daha fazla dizi barındıran karmaşık dizilerdir (*Multidimensional Arrays*).

```
<?php
$urunler = [
    ["ad" => "Laptop", "fiyat" => 25000, "stok" => 10],
    ["ad" => "Telefon", "fiyat" => 15000, "stok" => 25]
];

// İkinci ürünün adına erişim
echo $urunler[1]["ad"]; // Çıktı: Telefon
?>
```

En Çok Kullanılana Dizi Fonksiyonları

§8.4.5 PHP Kontrol Akışı ve Karar Yapıları

Programlama dillerinde kod blokları varsayılan olarak yukarıdan aşağıya ve satır satır (*sequential execution*) yürütülür. Ancak gerçek dünya senaryolarında, belirli koşulların sağlanıp sağlanmamasına göre kodun farklı dallara ayrılması veya tekrarlanması gerekir. Programın bu yürütme sırasını yönlendiren yapılara **kontrol akışı mekanizmaları** denir.

C dili ailesinden türeyen PHP, kontrol akışı hususunda JavaScript ile son derece benzer bir sözdizimine (*syntax*)

Fonksiyon	Açıklama	Örnek
<code>count()</code>	Dizideki eleman sayısını döndürür.	<code>count(\$meyveler)</code>
<code>in_array()</code>	Bir değerin dizide olup olmadığını arar (<code>true/false</code>).	<code>in_array("Muz", \$meyveler)</code>
<code>array_push()</code>	Dizinin sonuna bir veya daha fazla eleman ekler.	<code>array_push(\$meyveler, "Kivi")</code>
<code>array_merge()</code>	İki veya daha fazla diziyi birleştirir.	<code>array_merge(\$dizi1, \$dizi2)</code>
<code>array_filter()</code>	Belirli bir kurala göre dizi elemanlarını filtreler.	<code>array_filter(\$sayilar, fn(\$n) => \$n > 10)</code>
<code>array_map()</code>	Tüm elemanlara bir fonksiyon uygular.	<code>array_map(fn(\$n) => \$n * 2, \$sayilar)</code>

Tablo 8.1: En Çok Kullanılan Dizi Fonksiyonları

sahiptir.

Koşullu İfadeler (If - ElseIf - Else)

Bir veya birden fazla mantıksal ifadenin doğruluk değerine (`true/false`) göre farklı kod bloklarının çalıştırılmasını sağlar.

Karşılaştırma ve Mantıksal Operatörler

PHP'de koşul ifadeleri yazılırken mantıksal operatörler kullanılır:

- ◊ **Eşitlik:** `==` (Sadece değer eşitliği) ve `===` (Hem değer hem **veri tipi** eşitliği).
- ◊ **Mantıksal VE / VEYA:** `&&` (VE), `||` (VEYA), `!` (DEĞİL).

```
<?php
$not = 75;
if ($not >= 85) {
    echo "Harf Notu: AA";
} elseif ($not >= 70) {
    echo "Harf Notu: BB"; // Bu blok çalışacaktır
} elseif ($not >= 50) {
    echo "Harf Notu: CC";
} else {
    echo "Harf Notu: FF (Kaldı)";
}
?>
```

Ternary (Üçlü) İşleç ve Null Coalescing İşleci: Kısa koşul ifadeleri için `if-else` yerine üçlü işleç (`? :`) kullanılır. PHP 7.0 ile birlikte gelen Null Birleştirme İşleci (**null coalescing operator**) (`??`) ise bir değişkenin tanımlı ve `null` olup olmadığını kontrol etmek için büyük kolaylık sağlar.

```
<?php
$yas = 20;
// Ternary Operatör
$durum = ($yas >= 18) ? "Yetişkin" : "Reşit Değil";

// Null Coalescing Operatörü (??)
// $_GET['kullanici'] tanımlı ise onu al, tanımlı değilse "Misafir" ata.
$kullaniciAdi = $_GET['kullanici'] ?? "Misafir";
?>
```

Çoklu Seçim Yapısı (Switch - Case)

Bir değişkenin alabileceği birden fazla somut değere göre farklı kod bloklarının çalıştırılması gereken durumlarda, ardışık `if-elseif` blokları yerine `switch-case` yapısı tercih edilir.

```
<?php
$gun = "Salı";
switch ($gun) {
    case "Pazartesi":
        echo "Haftanın ilk iş günü.";
        break;
    case "Salı":
        echo "İkinci iş günü."; // Çalışacak blok
        break;
    case "Cumartesi":
        // ...

```

```

case "Pazar":
    echo "Hafta sonu!";
    break;
default:
    echo "Geçersiz gün bilgisi.";
    break;
}
?>

```

break Talimatının Kritik Önemi

`case` bloklarının sonuna `break` konulmazsa, PHP eşleşen bloktan sonraki tüm `case` bloklarını koşula bakmaksızın çalıştırmaya devam eder (*Fall-through davranışı*).

Döngü Yapıları (Loops)

Belirli bir kod bloğunu belirli bir koşul sağlandığı sürece tekrar tekrar çalıştırmak için kullanılır.

- ◇ **while:** Koşul doğrulandığı sürece döngü çalışır. Koşul en başta kontrol edilir.
- ◇ **do-while:** Kod bloğu en az bir kez çalıştırılır, koşul kontrolü döngünün sonunda yapılır.
- ◇ **for:** Sayaç değişkeni, döngü koşulu ve artış miktarının tek satırda tanımlandığı yapılandırılmış döngüdür.
- ◇ **foreach:** PHP'ye özgü olup dizilerin (*array*) ve nesnelerin tüm elemanları üzerinde kolayca gezinmeyi yani yineleme (*iteration*) sağlar.

```

<?php
// Standard For Döngüsü
for ($i = 1; $i <= 3; $i++) {
    echo "Adım: $i <br>";
}
// Foreach Döngüsü (Diziler için)
$ogrenciler = ["Ahmet", "Mehmet", "Zeynep"];
// Sadece Değer İle Gezinme
foreach ($ogrenciler as $ogrenci) {
    echo "Öğrenci: $ogrenci <br>";
}
// Anahtar (Key) ve Değer (Value) İle Gezinme
$personel = ["Ad" => "Can", "Unvan" => "Geliştirici"];
foreach ($personel as $anahtar => $deger) {
    echo "$anahtar: $deger <br>";
}
?>

```

Diziler Üzerinde Döngü Kullanımı

Dizi elemanlarını gezmek için en pratik yöntem `foreach` döngüsüdür:

```

<?php
$personel = [
    "CEO" => "Zeynep",
    "CTO" => "Mehmet",
    "Dev" => "Ali"
];

foreach ($personel as $unvan => $isim) {
    echo "$unvan: $isim\n";
}
?>

```

Kontrol Akışını Kesen Talimatlar (Break ve Continue)

Döngünün doğal akışını erkenden sonlandırmak veya mevcut adımı atlamak için kullanılır.

- ◇ **break:** İçinde bulunduğu döngüyü tamamen sonlandırır ve kontrolü döngü dışındaki ilk satıra devreder.
- ◇ **continue:** Döngünün o anki adımını (iterasyonunu) anında sonlandırır ve sayaç güncelleme adımına geçerek bir sonraki adımdan devam eder.

```

<?php
// Break ve Continue Örneği
for ($sayi = 1; $sayi <= 10; $sayi++) {
    if ($sayi == 3) {
        continue; // 3 sayısını yazatlamadan sonraki adıma geç
    }
}

```

```

if ($sayi == 7) {
    break; // Sayı 7 olduğunda döngüyü tamamen bitirir
}
echo "$sayi ";
}
// Çıktı: 1 2 4 5 6
?>

```

PHP'de İç İçe Döngülerden Çıkış (Seviyeli Break/Continue)

JavaScript'te iç içe döngülerden çıkmak için *label* (etiket) kullanılırken, PHP'de **break** ve **continue** komutlarına sayısal parametre verilebilir. Örneğin **break 2**; talimatı iç içe geçmiş **2 döngüyü birden** sonlandırır.

```

<?php
// İç İçe Döngülerde Seviyeli Break
for ($x = 1; $x <= 3; $x++) {
    for ($y = 1; $y <= 3; $y++) {
        if ($x == 2 && $y == 2) {
            break 2; // Her iki döngüyü de anında bitirir
        }
        echo "x:$x, y:$y | ";
    }
}
// Çıktı: x:1, y:1 | x:1, y:2 | x:1, y:3 | x:2, y:1 |
?>

```

Alternatif Kontrol Akışı Sözdizimi (HTML İçinde PHP Kullanımı)

PHP dosyalarında HTML ve PHP kodları iç içe yazılırken küme parantezleri ({}) kafa karışıklığına yol açabilir. Bu nedenle PHP, şablon görünümünde okunabilirliği artırmak için alternatif bir sözdizimi sunar.

```

<!-- HTML İçerisinde Alternatif Koşul Kullanımı -->
<?php if ($kullaniciGirisYapti == true): ?>
<h2>Hoş geldiniz, Panelinize Erişim Sağlayabilirsiniz.</h2>
<?php else: ?>
<h2>Lütfen Önce Giriş Yapınız.</h2>
<?php endif; ?>
<!-- HTML İçerisinde Alternatif Döngü Kullanımı -->
<ul>
<?php foreach ($urunler as $urun): ?>
<li><?php echo $urun; ?></li>
<?php endforeach; ?>
</ul>

```

§8.4.6 PHP Fonksiyonları

Bölüm 4.2.2 başlığında matematiksel kavram olarak gördüğümüz fonksiyonların burada somut karşılığını göreceksiniz. Fonksiyonlar, belirli bir görevi yerine getirmek üzere tasarlanmış, bir ismi olan ve program içerisinde ihtiyaç duyulduğunda tekrar tekrar çağrılabilen bağımsız kod bloklarıdır. Fonksiyon kullanımı; kod tekrarını önler (*DRY - Don't Repeat Yourself*), modülerliği artırır, bakım ve hata ayıklama süreçlerini kolaylaştırır.

Fonksiyon Tanımlama ve Çağırma

PHP'de kullanıcı tanımlı bir fonksiyon **function** anahtar kelimesi ile başlar. Fonksiyon isimleri harf veya alt çizgi (_) ile başlamalı, rakamla başlamamalıdır.

PHP'de Fonksiyon İsmi Harf Duyarsızlığı

Değişkenlerin aksine, PHP'de fonksiyon isimleri **büyük/küçük harfe duyarsızdır** (*case-insensitive*). Örneğin **selamVer()** şeklinde tanımlanan bir fonksiyon **SELAMVER()** veya **SelamVer()** olarak çağrılabilir. Ancak kod okunabilirliği açısından tanımlandığı biçimde çağırılması tavsiye edilir.

```

<?php
// Fonksiyon Tanımlama
function mesajYazdir() {

```

```

    echo "PHP Fonksiyon Dünyasına Hoş Geldiniz!";
}

// Fonksiyon Çağırma (Invocation)
mesajYazdir();
?>

```

Parametre Alan ve Değer Döndüren Fonksiyonlar

Fonksiyonlar dış dünyadan girdi almak için parametre kullanırlar. Hesaplanan veya üretilen veriyi ana programa aktarmak için ise `return` ifadesi kullanılır. `return` çalıştırıldığı an fonksiyonun yürütülmesi durdurulur ve kontrol çağrının yapıldığı satıra geri döner.

```

<?php
function topla($sayi1, $sayi2) {
    $sonuc = $sayi1 + $sayi2;
    return $sonuc; // Değeri çağrıldığı yere döndürür
}

$toplamSonuc = topla(15, 25); // 40
?>

```

Varsayılan Argümanlar (Default Arguments)

Fonksiyon çağrılırken bir argüman gönderilmezse, parametreye önceden atanan varsayılan değer devreye girer. Bu argümanlar daima zorunlu parametrelerin **sağına** yazılmalıdır.

```

<?php
function karsilama($isim, $unvan = "Ziyaretçi") {
    return "Merhaba " . $unvan . " " . $isim;
}

echo karsilama("Ahmet", "Dr."); // Merhaba Dr. Ahmet
echo karsilama("Mehmet");      // Merhaba Ziyaretçi Mehmet
?>

```

Katı Tipleme ve Tür Bildirimi (Type Hints & Strict Types)

PHP 7.0 ile birlikte fonksiyon parametrelerine ve dönüş değerlerine belirgin tür atama desteği gelmiştir. Bu sayede hatalı veri tiplerinin fonksiyona girmesi engellenir.

```

<?php
declare(strict_types=1); // Katı veri tipi kontrolünü aktifleştirir

// Parametrelerin int olmasını ve dönüşün float olmasını zorunlu kılar
function bolme(int $a, int $b): float {
    return $a / $b;
}

echo bolme(10, 2); // 5.0 (float)
// bolme("10", 2); // Strict types aktifken TypeError hatası fırlatır
?>

```

Değer ile Referans İle Aktarım (Pass by Value vs. Pass by Reference)

Varsayılan olarak PHP'de argümanlar fonksiyona **değer ile** (*pass by value*) aktarılır; yani verinin bir kopyası oluşturulur. Ancak parametrenin başına ampersand (&) simgesi konulursa, değişkenin kendisi/bellek adresi (*pass by reference*) aktarılır ve fonksiyon içindeki değişiklikler orijinal değişkeni etkiler.

```

<?php
function artirReferans(&$sayi) {
    $sayi += 10;
}

$originalSayi = 5;
artirReferans($originalSayi);
echo $originalSayi; // Çıktı: 15 (Orijinal değişken değişti)
?>

```

Sınırsız Sayıda Parametre (Variadic Functions)

Kaç tane argüman alacağı önceden bilinmeyen fonksiyonlar için üç nokta (...) işleci (*Splat Operator*) kullanılır. Bu işeç gelen tüm argümanları bir diziye dönüştürür.

```
<?php
function ortalamaHesapla(...$sayilar): float {
    if (count($sayilar) === 0) return 0.0;
    return array_sum($sayilar) / count($sayilar);
}

echo ortalamaHesapla(10, 20, 30, 40); // 25
?>
```

Anonim Fonksiyonlar ve Ok Fonksiyonları (Arrow Functions)

İsmi olmayan ve genellikle bir değişkene atanarak veya başka bir fonksiyona parametre (*callback*) olarak aktarılan fonksiyonlardır. PHP 7.4 ile kısa yazım sunan **Ok Fonksiyonları** (*Arrow Functions*) eklenmiştir.

Anonim vs Ok Fonksiyonlarında Kapsam (use vs Otomatik Kapsam)

Geleneksel anonim fonksiyonlar dışarıdaki bir değişkene erişmek için *use* (*\$degisken*) sözdizimini kullanmak zorundadır. Ok fonksiyonları (*fn() => ...*) ise dış kapsamdaki değişkenlere otomatik olarak erişebilir.

```
<?php
// Geleneksel Anonim Fonksiyon
$carpan = 2;
$katiniAl = function($sayi) use ($carpan) {
    return $sayi * $carpan;
};

// PHP 7.4+ Ok Fonksiyonu (Arrow Function)
// Dışarıdaki $carpan değişkenine 'use' gerekmeden otomatik erişir
$katiniAlKisa = fn($sayi) => $sayi * $carpan;

echo $katiniAlKisa(5); // 10
?>
```

Özyinelemeli (Recursive) Fonksiyonlar

Kendi kendisini çağıran fonksiyonlardır. Karmaşık veri yapılarını (örneğin dizin ağaçları veya kategoriler) taramak için kullanılır. Sonsuz döngüye girmemesi için mutlaka bir **taban koşulu** (*base case*) bulunmalıdır.

```
<?php
function faktoriyel(int $n): int {
    // Taban Koşul (Base Case)
    if ($n <= 1) {
        return 1;
    }
    // Özyinelemeli Çağrı (Recursive Call)
    return $n * faktoriyel($n - 1);
}

echo faktoriyel(5); // 120
?>
```

§8.4.7 PHP Nesne Yönelimli Programlama (OOP) ve Nesneler

Bölüm 4.3.2 başlığında teorisi verilen Nesne Yönelimli Programlama yani **OOP** (*Object-Oriented Programming*), yazılımı birbirleriyle etkileşim halinde olan nesneler (*objects*) etrafında kurgulayan bir programlama yaklaşımıdır. PHP 5 ile birlikte güçlü bir OOP altyapısına kavuşmuş, PHP 7 ve 8 sürümleriyle de katı tiplere ve modern mimari kalıplarını destekleyen gelişmiş bir yapıya bürünmüştür.

Sınıf (Class) ve Nesne (Object) Mantığı

Sınıf (Class): Bir nesnenin durumunu/özelliklerini (*states/attributes*) ve davranışlarını/ yöntemlerini (*behavior/method*) tanımlayan bir soyut taslak veya şablondur.

Nesne (Object): Bu sınıftan *new* anahtar kelimesi ile öbek bellekte (*Heap*) somut olarak üretilen (örneklenen) dinamik bir varlıktır.

Temel Sınıf Tanımlama ve Nesne Örneklemesi

Bir sınıf tanımlanırken **class** anahtar kelimesi kullanılır. Sınıf içerisindeki değişkenlere **özellik** (*property*), fonksiyonlara ise **yöntem** (*method*) denir. Sınıf içindeki özellik ve yöntemlere erişmek için **->** (*object operator*) kullanılır.

```
<?php
class Otomobil {
    // Özellik Tanımlamaları
    public string $marka;
    public string $renk;

    // Yöntem (method) Tanımlaması
    public function bilgiVer(): string {
        // $this anahtar kelimesi mevcut nesne örneğini temsil eder
        return "Aracın Markası: " . $this->marka . ", Rengi: " . $this->renk;
    }
}

// Nesne Oluşturma (Instantiation)
$arac1 = new Otomobil();
$arac1->marka = "Toyota";
$arac1->renk = "Kırmızı";

echo $arac1->bilgiVer();
// Çıktı: Aracın Markası: Toyota, Rengi: Kırmızı
?>
```

Erişim Belirleyiciler ve Kapsülleme

Kapsülleme (*Encapsulation*), bir nesnenin iç durumunu dış müdahalelerden korumak ve veriye erişimi denetim altına almak için kullanılır. PHP'de üç temel erişim belirleyici (*Access Modifiers*) bulunur:

- ◇ **public**: Sınıf içinden, türetilen sınıflardan ve sınıf dışından (her yerden) erişilebilir.
- ◇ **protected**: Sadece tanımlandığı sınıf içinden ve o sınıftan türetilen (miras alan) alt sınıflardan erişilebilir.
- ◇ **private**: Sadece tanımlandığı sınıfın içinden erişilebilir; dışarıdan veya alt sınıflardan erişilemez.

```
<?php
class BankaHesabi {
    private float $bakiye = 0.0; // Dışarıdan doğrudan değiştirilemez

    // Getter Metodu
    public function getBakiye(): float {
        return $this->bakiye;
    }

    // Setter Metodu
    public function paraYatir(float $miktar): void {
        if ($miktar > 0) {
            $this->bakiye += $miktar;
        }
    }
}

$hesap = new BankaHesabi();
$hesap->paraYatir(500);
echo "Güncel Bakiye: " . $hesap->getBakiye(); // 500
// $hesap->bakiye = 1000; // Fatal Error: Private özelliğe dışarıdan erişilemez!
?>
```

Sihirli Yöntemler

PHP'de çift alt çizgi (__) ile başlayan özel fonksiyonlara **sihirli yöntemler** (*magic methods*) denir. Bunlar; kurucu (**__construct**) ve yıkıcı (**__destruct**) yöntemlerdir.

- ◇ **__construct()**: Nesne **new** ile bellekte oluşturulduğu an otomatik olarak çalışan kurucu yöntemdir. Nesneyle ilişkin verilerin ilk değerlerini atamak için kullanılır.
- ◇ **__destruct()**: Nesnenin kullanımı bittiğinde veya bellekten silindiğinde (*Garbage Collector*) otomatik olarak çalışan yıkıcı yöntemdir.

```
<?php
class Kullanici {
```

```

    public string $ad;

    // Kurucu Yöntem
    public function __construct(string $ad) {
        $this->ad = $ad;
        echo "Kullanıcı '$this->ad' oluşturuldu.<br>";
    }

    // Yıkıcı Yöntem
    public function __destruct() {
        echo "Kullanıcı '$this->ad' bellekten silindi.<br>";
    }
}

$kullanici1 = new Kullanici("Zeynep");
?>

```

Kalıtım

Kalıtım (*Inheritance*), mevcut bir sınıfın tüm **public** ve **protected** durum ile yöntemlerini yeni bir sınıfa aktarmasını sağlar. PHP’de kalıtım **extends** anahtar kelimesi ile gerçekleştirilir. Bir alt sınıf, üst sınıfın yöntemlerini ezebilir (*Method Overriding*).

```

<?php
// Üst Sınıf (Base/Parent Class)
class Personel {
    protected string $ad;
    protected float $maas;

    public function __construct(string $ad, float $maas) {
        $this->ad = $ad;
        $this->maas = $maas;
    }

    public function detayGoster(): string {
        return "Ad: " . $this->ad . " - Maaş: " . $this->maas;
    }
}

// Alt Sınıf (Child Class)
class Yazilimci extends Personel {
    public string $uzmanlikDili;

    public function __construct(string $ad, float $maas, string $uzmanlikDili) {
        // Üst sınıfın kurucu metodunu çağırma
        parent::__construct($ad, $maas);
        $this->uzmanlikDili = $uzmanlikDili;
    }

    // Yöntem Ezme (Method Overriding)
    public function detayGoster(): string {
        return parent::detayGoster() . " - Dil: " . $this->uzmanlikDili;
    }
}

$yazilimci = new Yazilimci("Ahmet", 45000, "PHP");
echo $yazilimci->detayGoster();
?>

```

Statik Özellikler ve Yöntemler

Statik üyelere erişmek için nesne örneği (**new**) oluşturmaya gerek yoktur. Sınıf ismi üzerinden doğrudan **Kapsam Çözümleme İşleci (::)** kullanılarak erişilir. Statik yöntemler içerisinde **\$this** kullanılamaz; bunun yerine **self::** ifadesi tercih edilir.

```

<?php
class Matematik {
    public static float $pi = 3.14;
}

```

```

    public static function kareAl(int $sayi): int {
        return $sayi * $sayi;
    }
}

// Nesne türetmeden erişim
echo Matematik::$pi; // 3.14
echo Matematik::kareAl(4); // 16
?>

```

Soyut Sınıflar ve Arayüzler

Gelişmiş yazılım mimarilerinde alt sınıflara standart kurallar koymak ve çok biçimliliği (*Polymorphism*) sağlamak için soyutlama kullanılır.

Soyut sınıftan (*abstract class*) nesne imal edilemez. Soyut olamayan sınıflar somut sınıf (*concrete class*) olarak adlandırılır. Ara yüzler (*interface*) ise nesnenin sahip olacağı rollerdir.

Abstract Class vs Interface Farkı

Abstract Class: Gövdeli (normal) yöntemler barındırabilir. Bir sınıf yalnızca tek bir soyut sınıftan türetilir (*extends*).

Interface: Yalnızca gövdesiz yöntem imzaları (*method signature*) barındırır. Bir sınıf birden fazla arayüzü yani rolü uygulayabilir (*implements*).

```

<?php
// Arayüz Tanımlama
interface OdemeYontemi {
    public function odemeYap(float $tutari): bool;
}

class KrediKarti implements OdemeYontemi {
    public function odemeYap(float $tutari): bool {
        echo "$tutari TL Kredi Kartı ile çekildi.";
        return true;
    }
}

class Havale implements OdemeYontemi {
    public function odemeYap(float $tutari): bool {
        echo "$tutari TL Havale/EFT ile gönderildi.";
        return true;
    }
}
?>

```

§8.4.8 PHP Dahili Nesneleri

PHP, nesne yönelimli programlama süreçlerini kolaylaştırmak ve sık karşılaşılan karmaşık operasyonları standart bir yapıya bağlamak için bünyesinde birçok yerleşik (dahili) sınıf ve nesne barındırır (*Built-in Objects*). Bu sınıflar herhangi bir harici kütüphane veya modül yüklemeye gerek kalmaksızın doğrudan kullanılabilir.

Tarih ve Zaman Yönetimi (DateTime Nesnesi)

Geleneksel metinsel veya Unix zaman damgası (*timestamp*) tabanlı tarih işlemlerinin aksine, *DateTime* sınıfı tarih ve saat verilerini nesne yönelimli, güvenli ve zaman dilimlerine (*timezone*) duyarlı bir şekilde yönetmeyi sağlar.

```

<?php
// Mevcut tarih ve saat ile bir DateTime nesnesi oluşturma
$simdi = new DateTime();
echo "Şu anki Zaman: " . $simdi->format('Y-m-d H:i:s') . "<br>";

// Özel bir zaman dilimi (Timezone) belirleme
$zamanDilimi = new DateTimeZone('Europe/Istanbul');
$tarih = new DateTime('now', $zamanDilimi);

// Tarih Üzerinde Aritmetik İşlemler (DateInterval)
// P2M3D = 2 Ay (Month) ve 3 Gün (Day) ekle

```

```
$tarih->add(new DateInterval('P2M3D'));
echo "Gelecek Tarih: " . $tarih->format('d.m.Y') . "<br>";

// İki Tarih Arasındaki Farkı Hesaplama (diff)
$dogumGunu = new DateTime('1995-05-15');
$bugun = new DateTime('today');
$fark = $dogumGunu->diff($bugun);

echo "Yaşınız: " . $fark->y . " yıl, " . $fark->m . " ay, " . $fark->d . " gün.";
?>
```

Anonim ve Jenerik Nesneler (stdClass)

stdClass, PHP'nin varsayılan jenerik boş sınıfıdır. Önceden özel bir **class** şablonu tanımlamaya gerek kalmadan, dinamik olarak özellik (*property*) eklenebilen geçici nesneler oluşturmak için kullanılır. Ayrıca dizi tür dönüştürmelerinde (*type casting*) varsayılan dönüşüm hedefidir.

```
<?php
// Boş bir stdClass nesnesi oluşturma
$urun = new stdClass();
$urun->id = 101;
$urun->baslik = "Kablosuz Klavye";
$urun->fiyat = 450.00;

// İlişkisel bir Diziyi (Associative Array) Nesneye Dönüştürme
$dizi = ["ad" => "Can", "rol" => "Yönetici"];
$kullaniciNesne = (object) $dizi;

echo $kullaniciNesne->ad; // Çıktı: Can
?>
```

Hata ve İstisna Yönetimi

PHP 7 ile birlikte tüm sistem hataları ve istisnalar (**Exception**) nesne tabanlı bir hiyerarşiye bağlanmıştır. **Throwable** arayüzü, hem sistem hatalarının (**Error**) hem de kullanıcı tanımlı istisnaların (**Exception**) türediği en üst çatıdır.

Try-Catch Bloğu ve Exception Mantığı

Hata potansiyeli taşıyan kodlar **try** bloğuna alınır. Bir aksaklık durumunda **throw** ile bir **Exception** nesnesi fırlatılır ve **catch** bloğu tarafından yakalanarak programın çökmesi engellenir.

```
<?php
function bolmeIslemi(float $bolunen, float $bolen): float {
    if ($bolen === 0.0) {
        // Dahili Exception nesnesini fırlatma
        throw new Exception("Sıfıra bölme hatası gerçekleşti!", 400);
    }
    return $bolunen / $bolen;
}

try {
    echo bolmeIslemi(10, 0);
} catch (Exception $e) {
    // Exception nesnesinin dahili yöntemlerini kullanma
    echo "Hata Yakalandı: " . $e->getMessage() . "<br>";
    echo "Hata Kodu: " . $e->getCode() . "<br>";
    echo "Satır: " . $e->getLine();
} finally {
    echo "<br>İşlem tamamlandı."; // Her durumda çalışacak blok
}
?>
```

Anonim Fonksiyon Nesnesi

PHP'de tanımlanan tüm anonim fonksiyonlar (ve ok fonksiyonları) arka planda otomatik olarak dahili **Closure** sınıfının birer nesne örneğidir. **Closure** nesneleri sayesinde fonksiyonlar değişkenlere atanabilir, başka fonksiyonlara argüman olarak aktarılabilir veya çalışma zamanında bir sınıfa dinamik yöntem olarak bağlan-

abilir (*bindTo*).

```
<?php
// Anonim fonksiyon aslında bir Closure nesnesidir
$selamlama = function(string $isim): string {
    return "Merhaba $isim";
};

// Closure nesnesi olduğunu doğrulama
var_dump($selamlama instanceof Closure); // bool(true)

// Closure Nesnesinin Yöntemleri (bindTo ile bağlama)
class Sunucu {
    private string $durum = "Aktif";
}

$durumOku = function() {
    return $this->durum;
};

// Private özelliğe erişebilmek için closure nesnesini sınıfa bağlama
$bagliFonksiyon = $durumOku->bindTo(new Sunucu(), 'Sunucu');
echo $bagliFonksiyon(); // Çıktı: Aktif
?>
```

Öne Çıkan Diğer Dahili PHP Nesneleri

- ◊ **ArrayIterator / ArrayObject:** Dizilerin nesne gibi davranmasını ve özelleştirilmiş iterasyon mantıklarının uygulanmasını sağlar.
- ◊ **ReflectionClass:** Çalışma zamanında sınıfların, yöntemlerin ve parametrelerin iç yapısını incelemeye olanak tanır (*Reverse Engineering / Metaprogramming*).
- ◊ **PDO / PDOStatement:** Güvenli veritabanı bağlantısı ve sorgu yürütme süreçlerini yöneten dahili nesnelerdir.

§8.4.9 PHP Modülleri ve Eklentileri

PHP, çekirdek (*core*) mimarisi oldukça hafif tutulmuş, ihtiyaca göre fonksiyonel modüller ve eklentiler (*extensions*) yüklenerek genişletilebilen modüler bir yapıya sahiptir. Veritabanı sürücüler, görsel işleme kütüphaneleri, şifreleme araçları ve ağ protokolleri PHP'ye birer modül olarak dahil edilir.

PHP Modül Mimarisi ve Çalışma Mantığı

PHP modülleri genellikle C/C++ dili ile yazılmış ve derlenmiş paylaşımlı kütüphanelerdir (*Linux ortamında .so, Windows ortamında .dll*). PHP çalışma zamanı (*runtime*), bu modülleri *php.ini* konfigürasyon dosyası aracılığıyla belleğe yükler ve içerisindeki fonksiyonları PHP kodlarının kullanımına sunar.

En Yaygın Kullanılan PHP Modülleri

Modern bir web uygulamasının ihtiyaç duyduğu temel PHP modülleri şunlardır:

- ◊ **php-mysql / php-pdo:** MySQL ve diğer ilişkisel veritabanlarına (PostgreSQL, SQLite vb.) güvenli erişim sağlayan veritabanı sürücüleridir.
- ◊ **php-gd / php-imagick:** Resim boyutlandırma, filigran (*watermark*) ekleme ve format dönüştürme gibi görsel işleme operasyonlarını yürütür.
- ◊ **php-mbstring:** Çok baytlı karakter dizilerini (*Multi-Byte String*) ve UTF-8 (Türkçe karakterler vb.) metin işlemlerini hatasız işlemek için kullanılır.
- ◊ **php-curl:** HTTP, FTP ve HTTPS protokolleri üzerinden uzak sunuculara istek atmayı ve REST API'ler ile haberleşmeyi sağlar.
- ◊ **php-json:** JSON (*JavaScript Object Notation*) formatındaki verileri çözümlemek (*decode*) ve oluşturmak (*encode*) için kullanılan çekirdek modüldür.
- ◊ **php-xml / php-zip:** XML belgelerini ayrıştırma ve ZIP arşivleri oluşturma/açma işlemlerini üstlenir.
- ◊ **php-opcache:** PHP kodlarını derleyerek bellekte ön belleğe alır; her istekte kodun tekrar tekrar yorumlanmasını engelleyerek sunucu performansını ciddi oranda artırır.

Sunucu Ortamında Modül Yönetimi

Linux/Ubuntu sistemlerde PHP eklentileri paket yöneticisi (**apt**) aracılığıyla kolayca yüklenebilir ve yönetilebilir.

```
# Yüklü olan tüm PHP modüllerini listeleme
php -m

# Belirli bir PHP modülünü arama (Örn: GD kütüphanesi)
sudo apt search php-gd

# Yeni bir PHP modülü kurma (Örn: cURL ve PDO MySQL)
sudo apt install php-curl php-mysql -y

# Kurulan modülü aktiveleştirme / devre dışı bırakma
sudo phpenmod curl
sudo phpdismod curl

# Değişikliklerin geçerli olması için web sunucusunu/SAPI'yi yeniden başlatma
sudo systemctl restart apache2
# veya Nginx + PHP-FPM kullanılıyorsa:
# sudo systemctl restart php8.2-fpm
```

Konfigürasyon Dosyası (**php.ini**) ve Modül Etkinleştirme

PHP modüllerinin davranışı ve limitleri **php.ini** dosyası üzerinden ayarlanır. Modüller bu dosyada **extension** direktifi ile tanımlanır.

```
; php.ini İçerisinde Modül Etkinleştirme Örneği

; Linux sistemlerde .so uzantılı kütüphane
extension=curl.so
extension=pdo_mysql.so

; Windows sistemlerde .dll uzantılı kütüphane
; extension=php_gd.dll

; Modül Yapılandırma Parametreleri Örneği (OPcache)
opcache.enable=1
opcache.memory_consumption=128
```

Kod İçerisinde Modül Varlığını Kontrol Etme

Yazdığınız bir PHP uygulamasının ihtiyaç duyduğu bir eklentinin sunucuda yüklü olup olmadığını çalışma zamanında denetlemek, olası sistem hatalarının (**Fatal Error**) önüne geçer.

```
<?php
// extension_loaded() fonksiyonu ile modül kontrolü
if (extension_loaded('gd')) {
    echo "GD Kütüphanesi aktif, görsel işleme yapılabilir.";
} else {
    echo "Hata: GD Kütüphanesi sunucuda kurulu değil!";
}

// Belirli bir fonksiyonun tanımlı olup olmadığını kontrol etme
if (function_exists('curl_init')) {
    echo "cURL fonksiyonları kullanılabilir.";
}
?>
```

Apache Modülü (libapache2-mod-php) ve PHP-FPM Ayrımı

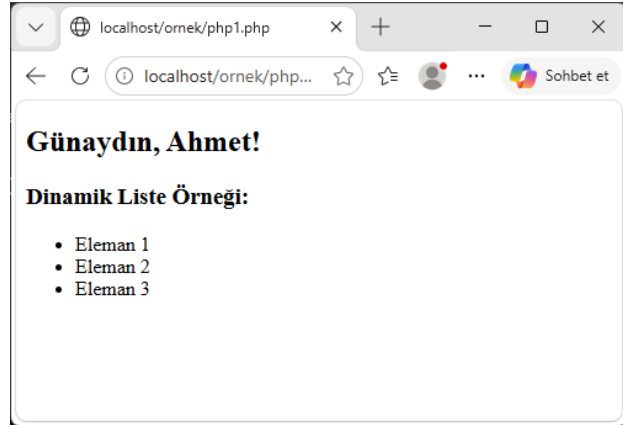
PHP'nin web sunucusu ile etkileşime girme biçimi iki temel modülle sağlanır:

- ♦ **SAPI (Apache Modülü):** PHP doğrudan Apache'nin bir iç modülü (**mod_php**) olarak çalışır.
- ♦ **PHP-FPM (FastCGI Process Manager):** PHP'yi bağımsız bir servis olarak çalıştırır. Nginx ve yüksek trafikli Apache mimarilerinde performans ve kaynak yönetimi açısından tercih edilen modern standarttır.

§8.4.10 PHP ile Çıktı HTML Dokümanını Değiştirme

PHP, sunucu tarafında mantıksal kararlar vererek veya döngüler çalıştırarak dinamik HTML içerikleri üretebilir.

```
<?php
$kullaniciAd = "Ahmet";
$saat = date("H"); // Sunucunun güncel saatini alır
↪ (00-23)
if ($saat < 12) {
    $mesaj = "Günaydın";
} else {
    $mesaj = "Tünaydın";
}
?>
<!DOCTYPE html>
<html lang="tr">
<body>
    <h2><?php echo $mesaj . ", " . $kullaniciAd
    ↪ . "!"; ?></h2>
    <h3>Dinamik Liste Örneği:</h3>
    <ul>
        <?php
        for ($i = 1; $i <= 3; $i++) {
            echo "<li>Eleman " . $i . "</li>";
        }
        ?>
    </ul>
</body>
</html>
?>
```



Şekil 8.4: PHP tarafından Değiştirilmiş HTML'in Web Tarayıcı Görselleştirmesi

Kod 8.3: Çıktı HTML Belgesini Değiştirecek PHP

§8.4.11 HTML Form Verilerini HTTP GET Yöntemi ile Tarayıcıdan Alma

İstemci (tarayıcı), bir HTML formu veya URL parametreleri aracılığıyla sunucuya veri gönderebilir. HTTP GET yöntemi kullanıldığında veriler URL adres çubuğunda (ornek.php?ad=Ahmet&yasi=25) görünür biçimde taşınır. PHP, bu verilere global \$_GET süper dizisi aracılığıyla erişir.

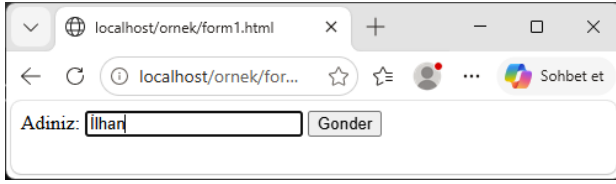
```
<!-- form.html (İstemci Tarafı Formu) -->
<html lang="tr">
<body>
    <form action="islem.php" method="GET">
        <label for="kullanici">Adınız:</label>
        <input type="text" id="kullanici" name="kullanici_adi">
        <button type="submit">Gönder</button>
    </form>
</body>
</html>
```

Sunucudaki veriyi işleyen islem.php dosyası:

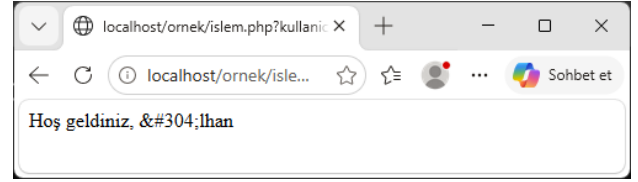
```
<?php
// islem.php (Sunucu Tarafı)
if (isset($_GET['kullanici_adi'])) {
    $gelenAd = htmlspecialchars($_GET['kullanici_adi']);
    echo "Hoş geldiniz, " . $gelenAd;
} else {
    echo "Form verisi alınamadı!";
}
?>
```

Burada dikkat edilmesi gereken formda **Gönder** butonuna tıklandığında verilerin URL üzerinden **islem.php** dosyasına aktarılmasıdır. Yani sağdaki görseldeki adres aşağıdaki şekilde olacaktır;

http://localhost/ornek/islem.php?kullanici_adi=%26%23304%3B1han



Şekil 8.5: Form HTML Sayfasının Görselleştirilmesi



Şekil 8.6: Form Tarafından Gönderilen Verileri İşlemiş İşlem Sayfasının Görselleştirilmesi

Ş8.4.12 HTML Form Verilerini HTTP POST Yöntemi ile Tarayıcıdan Alma

Bölüm 1.9’de öğrendiğimiz gibi **GET** ve **POST** yöntemleri farklı amaçlara hizmet eder. Şifre, kredi kartı bilgisi gibi hassas verilerde veya büyük hacimli formlarda veriler adres çubuğunda görünmemeli; bu nedenle HTTP **POST** yöntemi tercih edilir. PHP, bu şekilde gönderilen verilere global **\$_POST** süper dizisi aracılığıyla erişir.

\$_GET ile \$_POST Arasındaki Fark

- ◇ **\$_GET**: Veriler URL üzerinden taşındığı için adres çubuğunda görünür, tarayıcı geçmişinde (*history*) saklanır ve boyut sınırı vardır. Arama, filtreleme gibi tekrar bağlantı verilebilmesi istenen işlemler için uygundur.
- ◇ **\$_POST**: Veriler HTTP isteğinin gövdesinde (*body*) taşındığı için adres çubuğunda görünmez ve pratikte boyut sınırı yoktur. Giriş formları, kayıt formları gibi hassas veya büyük veriler için tercih edilir.

```
<!-- kayıt_formu.html (İstemci Tarafı Formu) -->
<form action="kayit_islem.php" method="POST">
  <label for="eposta">E-posta:</label>
  <input type="email" id="eposta" name="eposta">

  <label for="sifre">Şifre:</label>
  <input type="password" id="sifre" name="sifre">

  <button type="submit">Kayıt Ol</button>
</form>
```

Sunucudaki veriyi işleyen **kayit_islem.php** dosyası:

```
<?php
// kayıt_islem.php (Sunucu Tarafı)
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $eposta = htmlspecialchars($_POST['eposta'] ?? '');
  $sifre = $_POST['sifre'] ?? '';

  if (!empty($eposta) && !empty($sifre)) {
    echo "Kayıt isteği alındı: " . $eposta;
    // Bu noktadan sonra veri, ilerleyen bölümde göreceğimiz
    // gibi bir veritabanına kaydedilebilir.
  } else {
    echo "Lütfen tüm alanları doldurunuz!";
  }
}
?>
```

8.5 MySQL

MySQL, ilişkisel veri tabanıdır yönetim sistemidir yani **RDBMS** (Relational Database Management System). PHP ile tam uyumlu çalışarak verilerin kalıcı olarak saklanması, sorgulanmasını ve güncellenmesini sağlar.

Ş8.5.1 Linux MySQL Kurulumu

```
# MySQL Server kurulumu
sudo apt install mysql-server -y

# Servisi başlatma ve kontrol etme
sudo systemctl start mysql
sudo systemctl status mysql
```

Veritabanı sunucusu kuruluşuna göre **okul** veritabanını oluşturabiliriz. Parola sorduğunda Linux **root** parolasını girmelisiniz. Böylece örneklerimizde veritabanına bağlanmak için **devuser** adlı bir kullanıcı oluşturmuş

olacağız.

```
hoydabre@OZKAN:~$ sudo mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.4.11-0ubuntu0.26.04.1 (Ubuntu)
```

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql>CREATE USER 'devuser'@'localhost' IDENTIFIED BY 'YourStrongPassword123!';
ALL PRIVILEGES ON *.* TO 'devuser'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> GRANT ALL PRIVILEGES ON *.* TO 'devuser'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0.00 sec)
```

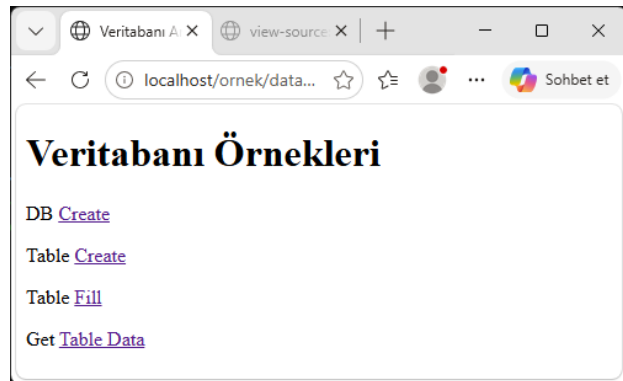
```
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> EXIT;
hoydabre@OZKAN:~$
```

Aşağıda sırasıyla veritabanı, tablo oluşturup tabloyu dolduran ve verileri getiren PHP sayfaları verilmiştir.

§8.5.2 Veritabanı Ana Sayfa

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="utf-8">
    <title>Veritabanı Ana Sayfa</title>
  </head>
  <body>
    <div id="content">
      <h1>Veritabanı Örnekleri</h1>
      <p> DB <a
        ↪ href="CreateDB.php">Create</a></p>
      <p> Table <a
        ↪ href="CreateTable.php">Create</a></p>
      <p> Table <a
        ↪ href="FillTable.php">Fill</a></p>
      <p> Get <a href="GetTable.php">Table
        ↪ Data</a></p>
    </div><!--content-->
  </body>
</html>
```



Şekil 8.7: Veritabanı Ana Sayfasının Görselleştirilmesi

Kod 8.4: Veritabanı Ana Sayfa HTML

PHP'nin PDO (*PHP Data Objects*) kütüphanesi kullanılarak güvenli bir şekilde veritabanı bağlantısı kurulur ve temel **CRUD** (*Create, Read, Update, Delete*) işlemleri gerçekleştirilir.

§8.5.3 Veritabanı Oluşturan PHP

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>MySQL Veritabanı Oluşturma</title>
  </head>
  <body>
    <h1>Veri Tabanı Oluşturma</h1>
```

```

<p>
<?php
$host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!'; $charset = 'utf8mb4';
$dsn = "mysql:host=$host;charset=$charset";
try {
    // 1. Connect to MySQL server without a dbname parameter
    $pdo = new PDO("mysql:host=$host;charset=utf8mb4", $user,$pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);

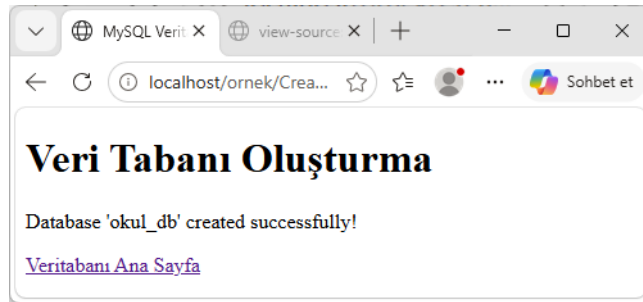
    // 2. SQL query to create database
    $dbname = "okul_db";
    $sql = "CREATE DATABASE IF NOT EXISTS `dbname` CHARACTER SET utf8mb4 COLLATE
        ↪ utf8mb4_unicode_ci";

    // 3. Execute query
    $pdo->exec($sql);
    echo "Database '$dbname' created successfully!";

    // 4. (Optional) Switch to the newly created database
    $pdo->exec("USE `dbname`");
} catch (PDOException $e) {
    echo "Veritabanı Hatası: " . $e->getMessage();
}
?>
</p>
<p><a href="database.html">Veritabanı Ana Sayfa</a></p>
</body>
</html>

```

Kod 8.5: Veritabanı Oluşturan PHP



Şekil 8.8: CreateDB.PHP Sayfasının Görselleştirilmesi

§8.5.4 Veritabanında Tablo Oluşturan PHP

```

<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>MySQL Veritabanı Tablo Oluşturma</title>
</head>
<body>
    <h1>Veri Tabanında Tablo Oluşturma</h1>
    <p>
    <?php
    $host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
    $charset = 'utf8mb4'; $dbname = "okul_db";
    $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
    try {
        // 1. Connect to MySQL server without a dbname parameter
        $pdo = new PDO($dsn, $user,$pass, [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);

        // 2. SQL query to create table
        $tablename = "Ogrenciler";

```

```

$sql = "DROP TABLE IF EXISTS $tablename";
$pdo->exec($sql);

$sql = "CREATE TABLE $tablename ("
    "id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,"
    "adi VARCHAR(30) NOT NULL,"
    "soyadi VARCHAR(30) NOT NULL,"
    "eposta VARCHAR(50),"
    "kayittarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP"
    ")";

// 3. Execute query
$pdo->exec($sql);
echo "Database '$tablename' created successfully!";
} catch (PDOException $e) {
    echo "Veritabanı Hatası: " . $e->getMessage();
}
?>
</p>
<p><a href="database.html">Veritabanı Ana Sayfa</a></p>
</body>
</html>

```

Kod 8.6: Veritabanında Tablo Oluşturan PHP



Şekil 8.9: CreateTable.PHP Sayfasının Görselleştirilmesi

§8.5.5 Veritabanında Tabloyu Veri İle Dolduran PHP

```

<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>MySQL Veritabanı Tablo Doldurma</title>
</head>
<body>
    <h1>Veri Tabanında Tablo Doldurma</h1>
    <p>
        <?php
            $host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
            $charset = 'utf8mb4'; $dbname = "okul_db"; $tablename = "Ogrenciler";
            $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
            try {
                // 1. Connect to MySQL server without a dbname parameter
                $pdo = new PDO($dsn, $user, $pass, [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);
                // 2. SQL query to create table
                $sql = "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Ahmet', 'Yıldırım',
                    ↳ 'ahmet@example.com')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('İlhan', 'Özkan',
                    ↳ 'ilhan@example.com.tr')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Ayşen', 'AĞLI', 'aysen@example.com')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Kağan', 'TÜRK',
                    ↳ 'kagan@example.com.tr')";
                "INSERT INTO $tablename (adi, soyadi, eposta) VALUES ('Ajda', 'TAŞKIN', 'aö@example.com')";
                $pdo->exec($sql);
            }
        </?php>
    </p>

```

```

        echo "Table '$tablename' filled successfully!";
    } catch (PDOException $e) {
        echo "Veritabanı Hatası: " . $e->getMessage();
    }
    ?>
</p>
<p><a href="database.html">Veritabanı Ana Sayfa</a></p>
</body>
</html>

```

Kod 8.7: Veritabanında Tabloyu Veri İle Dolduran PHP



Şekil 8.10: FillTable.PHP Sayfasının Görselleştirilmesi

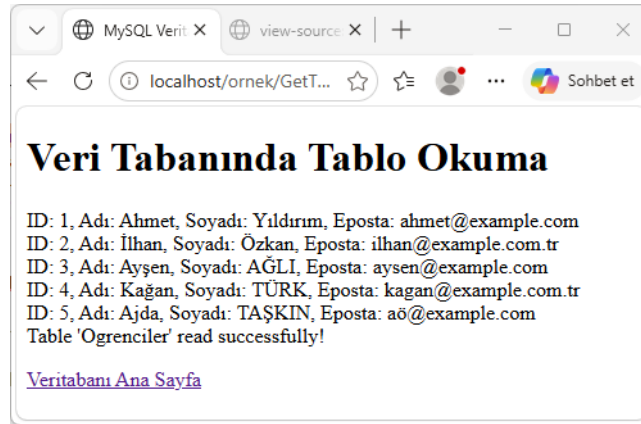
§8.5.6 Veritabanından Veri Okuyan PHP

```

<!DOCTYPE html>
<html lang="tr">
    <head>
        <meta charset="UTF-8">
        <title>MySQL Veritabanı Tablo Okuma</title>
    </head>
    <body>
        <h1>Veri Tabanında Tablo Okuma</h1>
        <p>
            <?php
                $host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
                $charset = 'utf8mb4'; $dbname = "okul_db"; $tablename = "Ogrenciler";
                $dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";
                try {
                    // 1. Connect to MySQL server without a dbname parameter
                    $pdo = new PDO($dsn, $user,$pass, [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
                    // 2. SQL query to create table
                    $sql = "SELECT * FROM $tablename;";
                    $result=$pdo->query($sql); // Okuma İçin QUERY() Yöntemi Kullanılıyor.
                    foreach ($result as $row) {
                        echo "ID: " . $row['id'] . ", Adı: " . $row['adi'] . ", Soyadı: " . $row['soyadi'] .
                            ", Eposta: " . $row['eposta'] . "<br>";
                    };
                    echo "Table '$tablename' read successfully!";
                } catch (PDOException $e) {
                    echo "Veritabanı Hatası: " . $e->getMessage();
                }
            ?>
        </p>
        <p><a href="database.html">Veritabanı Ana Sayfa</a></p>
    </body>
</html>

```

Kod 8.8: Veritabanından Veri Okuyan PHP



Şekil 8.11: GetTable.PHP Sayfasının Görselleştirilmesi

§8.5.7 PHP ile Veritabanında CRUD İşlemleri

```
<?php
$host = 'localhost'; $user = 'devuser'; $pass = 'YourStrongPassword123!';
$charset = 'utf8mb4'; $dbname = "okul_db"; $tablename = "Ogrenciler";
$dsn = "mysql:host=$host;dbname=$dbname;charset=$charset";

try {
    // 1. Veritabanı Bağlantısı
    $pdo = new PDO($dsn,$user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // 2. Tablo Oluşturma (Create Table)
    $sqlTable = "CREATE TABLE IF NOT EXISTS $tablename (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    numara INT NOT NULL
    )";
    $pdo->exec($sqlTable);

    // 3. Kayıt Oluşturma (Create) - Hazırlıklı İfade (Prepared Statement)
    $sqlInsert = "INSERT INTO $tablename (ad, numara) VALUES (:ad, :numara)";
    $stmt = $pdo->prepare($sqlInsert);
    $stmt->execute(['ad' => 'Ahmet Özkan', 'numara' => 101]);

    // 4. Bütün Kayıtları Okuma (ReadAll)
    $sqlSelect = "SELECT * FROM $tablename";
    $stmt = $pdo->query($sqlSelect); //QUERY() metodu SELECT sorgularında kullanılır, EXEC() metodu ise
    ↳ INSERT, UPDATE, DELETE gibi sorgularda kullanılır.
    $ogrenciler = $stmt->fetchAll(PDO::FETCH_ASSOC);

    foreach ($ogrenciler as $ogrenci) {
        echo "ID: " . $ogrenci['id'] . " - Ad: " . $ogrenci['ad'] . "<br>";
    }

    // 5. Kayıt Güncelleme (Update)
    $sqlUpdate = "UPDATE $tablename SET ad = :ad WHERE id = :id";
    $pdo->prepare($sqlUpdate)->execute(['ad' => 'Ahmet Can', 'id' => 1]);

    // 6. Kayıt Silme (Delete)
    $sqlDelete = "DELETE FROM $tablename WHERE id = :id";
    $pdo->prepare($sqlDelete)->execute(['id' => 1]);
} catch (PDOException $e) {
    echo "Veritabanı Hatası: " . $e->getMessage();
}
?>
```

§8.5.8 PHP ile Form Verilerini Veritabanına Kaydetme

Hassas verilerin gönderiminde HTTP POST yöntemi tercih edilmelidir. Güvenlik açığı olan *SQL Injection* saldırılarını önlemek için kullanıcı girdileri doğrudan SQL ifadesine eklenmemeli, **Hazırlıklı İfadeler** (*Prepared Statements*) kullanılmalıdır. Çünkü POST ile gelen değerler doğrudan veritabanına uygulanmaktadır.

```
<?php
// POST metodu ile gelen form verisini güvenli şekilde kaydetme
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $ogrenciAd = $_POST['ogrenci_ad'];
    $ogrenciNo = $_POST['ogrenci_no'];

    $host = 'localhost'; $db = 'okul'; $user = 'devuser';
    $pass = 'YourStrongPassword123!'; $charset = 'utf8mb4';

    $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
    $pdo = new PDO($dsn,$user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "INSERT INTO ogrenciler (ad, numara) VALUES (?, ?)";
    $stmt = $pdo->prepare($sql); $stmt->execute([$ogrenciAd, $ogrenciNo]);

    echo "Öğrenci veritabanına başarıyla kaydedildi.";
}
?>
```

§8.5.9 Hazırlıklı İfadeler

Hazırlıklı İfadeler (*Prepared Statements*), SQL Enjeksiyonu (*SQL Injection*) saldırılarını engellemenin en etkili ve standart yoludur. Temel çalışma prensibi, sorgunun mantıksal yapısı (SQL kodu) ile kullanıcının gönderdiği veri (parametreler) birbirinden tamamen ayrılmasına dayanır.

Nasıl Çalışır? Normal (güvenli olmayan) bir sorguda veritabanı, gelen tüm metni tek bir SQL komutu olarak algılar ve derler. Hazırlıklı ifadelerde ise süreç iki aşamada işler;

- ◇ Hazırlık (*Prepare*): SQL sorgu şablonu veritabanına gönderilir ve derlenir. Veritabanı sorgunun ne yapacağını (örneğin hangi tablodan veri çekileceğini) önceden anlar.
- ◇ Çalıştırma (*Execute*): Kullanıcıdan gelen parametreler veritabanına gönderilir. Veritabanı bu değerleri çalıştırma kodunun bir parçası olarak değil, sadece düz veri (metin/sayı) olarak işler.

◇

Bu sayede kullanıcının girdiği veri içerisinde ' OR '1'='1 gibi SQL komutları yer alsa bile veritabanı bunları kod olarak yürütmez, yalnızca aranan metnin bir parçası kabul eder.

§8.5.10 Veritabanının Güvenli Olan ve Olmayan Kullanımı

Giriş formu üzerinden gönderilen bir kullanıcı adı (*\$kullanici_adi*) ve şifre (*\$sifre*) senaryosunu inceleyelim.

Güvenlik Açığı Bulunan Kod

```
// DİKKAT: SQL Injection'a açıktır!
$sql = "SELECT * FROM kullanicilar WHERE kullanici_adi = '$kullanici_adi' AND sifre = '$sifre'";
$sonuc = $db->query($sql);
```

Saldırı Senaryosu: Kullanıcı, kullanıcı adı alanına *admin* girdiğinde sorgu şu hale gelir:

```
SELECT * FROM kullanicilar WHERE kullanici_adi = 'admin' --' AND sifre = '...'
-- karakterleri SQL'de yorum satırı anlamına geldiği için şifre kontrolü tamamen atlanır ve saldırgan
↳ şifresiz giriş yapar.
```

Güvenli Kod

```
// 1. Şablon hazırlanır (Kullanıcı girdisi yerine '?' yer tutucuları konur)
$stmt = $db->prepare("SELECT * FROM kullanicilar WHERE kullanici_adi = ? AND sifre = ?");
```

```
// 2. Parametreler güvenli bir şekilde bağlanır ve sorgu çalıştırılır
$stmt->execute([$kullanici_adi, $sifre]);
$kullanici = $stmt->fetch();
```

Saldırı Engelleme: Saldırgan kullanıcı adı alanına *admin* girse dahi, veritabanı bunu derlenmiş sorgunun içine mantıksal bir SQL komutu olarak eklemeyiz. Veritabanı, tam olarak adı *admin* olan bir kullanıcı arar ve bulamayacağı için erişimi engeller.

İsmlendirilmiş Parametreler Örneği

Soru işaretleri (?) yerine, okunabilirliği artırmak için ismlendirilmiş parametreler (*Named Parameters*) de kullanılabilir:

```
$stmt = $db->prepare("SELECT * FROM urunler WHERE kategori = :kategori AND fiyat < :fiyat");

$stmt->execute([
    ':kategori' => $gelen_kategori,
    ':fiyat'    => $gelen_fiyat
]);
```

Özet Olarak

Güvenlik: Kullanıcı girdisini veriden koda dönüştürme ihtimalini ortadan kaldırarak SQL Injection'ı %100'e yakın oranla engeller.

Performans: Aynı sorgu farklı parametrelerle defalarca çalıştırılacaksa (örneğin toplu veri ekleme), sorgu mantığı veritabanında bir kez derlendiği için işlem daha hızlı gerçekleşir.

8.6 Oturum ve Çerez Yönetimi

Bölüm 1.9'de HTTP'nin **durumsuz** (*stateless*) bir protokol olduğunu, yani sunucunun önceki isteklerin durumunu hatırla(ya)madığını öğrenmiştik. Ancak gerçek dünyadaki bir web uygulamasında, bir kullanıcının giriş yaptıktan sonraki sayfalarda hâlâ "giriş yapmış" olarak tanınması gerekir. PHP bu sorunu iki tamamlayıcı mekanizmayla çözer: **çerezler** (*cookies*) ve **oturumlar** (*sessions*).

§8.6.1 Çerezler (Cookies)

Çerez, sunucunun tarayıcıya gönderip tarayıcının kendi bilgisayarında sakladığı, sonraki her istekte tarayıcının otomatik olarak sunucuya geri gönderdiği küçük bir metin verisidir. PHP'de bir çerez oluşturmak için `setcookie()` fonksiyonu, okumak için ise `$_COOKIE` süper dizisi kullanılır.

```
<?php
// Çerez Oluşturma: isim, değer, geçerlilik süresi (saniye)
// Aşağıdaki örnek, çerezi 1 gün (86400 saniye) geçerli kılar.
setcookie("kullaniciAdi", "Ahmet", time() + 86400);
?>

<?php
// Bir sonraki istekte çerezi okuma
if (isset($_COOKIE["kullaniciAdi"])) {
    echo "Merhaba tekrar, " . $_COOKIE["kullaniciAdi"];
} else {
    echo "Çerez bulunamadı.";
}
?>
```

Çerezlerin Sınırlılıkları

Çerezler kullanıcının kendi tarayıcısında saklandığı için tarayıcı geliştirici araçlarından görülebilir ve değiştirilebilir. Bu nedenle şifre gibi hassas verilerin doğrudan çerezde saklanması güvenli değildir; bunun yerine bir sonraki alt bölümde göreceğimiz oturum (*session*) mekanizması tercih edilmelidir.

§8.6.2 Oturumlar

Oturum, çerezlerin aksine asıl veriyi **sunucu tarafında** saklar; tarayıcıya yalnızca o oturumu tanımlayan benzersiz bir kimlik (genellikle `PHPSESSID` adlı bir çerez içinde) gönderilir. Böylece hassas veriler hiçbir zaman kullanıcının tarayıcısına gitmez.

Bir sayfada oturum değişkenlerini kullanabilmek için, sayfanın en başında (herhangi bir HTML çıktısından önce) `session_start()` fonksiyonu çağrılmalıdır. Oturum verileri, global `$_SESSION` dizisinde saklanır.

```
<?php
session_start(); // Oturumu başlatır veya var olan oturuma bağlanır

$_SESSION['kullaniciAdi'] = "Ahmet"; // Oturuma veri yazma
$_SESSION['girisYapildi'] = true;
?>
```

Örnek: Basit Bir Giriş Sistemi

Aşağıdaki üç dosya, oturum yönetiminin en klasik kullanım senaryosunu – bir giriş (*Login*) sistemini – birlikte oluşturur.

1) **giris.php** – Giriş Formu ve Doğrulama:

```
<?php
session_start();

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $kullanici = $_POST['kullanici'] ?? '';
    $sifre = $_POST['sifre'] ?? '';

    // Gerçek bir projede bu bilgi veritabanından ve
    // şifrelenmiş (hashlenmiş) olarak kontrol edilir.
    if ($kullanici === "admin" && $sifre === "1234") {
        $_SESSION['girisYapildi'] = true;
        $_SESSION['kullaniciAdi'] = $kullanici;

        header("Location: panel.php"); // Başarılı girişte panele yönlendir
        exit();
    } else {
        $hataMesaji = "Kullanıcı adı veya şifre hatalı!";
    }
}

?>
<!DOCTYPE html>
<html lang="tr">
<body>
    <?php if (isset($hataMesaji)): ?>
    <p style="color:red;"><?php echo $hataMesaji; ?></p>
    <?php endif; ?>
    <form action="giris.php" method="POST">
    <input type="text" name="kullanici" placeholder="Kullanıcı Adı">
    <input type="password" name="sifre" placeholder="Şifre">
    <button type="submit">Giriş Yap</button>
    </form>
</body>
</html>
```

2) **panel.php** – Yalnızca Giriş Yapmış Kullanıcıların Görebileceği Sayfa:

```
<?php
session_start();

// Oturum kontrolü: giriş yapılmamışsa giriş sayfasına geri gönder
if (!isset($_SESSION['girisYapildi']) || $_SESSION['girisYapildi'] !== true) {
    header("Location: giris.php");
    exit();
}

?>
<!DOCTYPE html>
<html lang="tr">
<body>
    <h1>Hoş geldin, <?php echo htmlspecialchars($_SESSION['kullaniciAdi']); ?></h1>
    <a href="cikis.php">Çıkış Yap</a>
</body>
</html>
```

3) **cikis.php** – Oturumu Sonlandırma:

```
<?php
session_start();
session_unset(); // Oturumdaki tüm değişkenleri temizler
session_destroy(); // Oturumun kendisini sunucudan siler

header("Location: giris.php");
exit();
?>
```

header() ve Yönlendirme-Sonrası-Gönderme (PRG) Deseni

`header("Location: ...")` fonksiyonu, tarayıcıya farklı bir sayfaya yönlendiğini söyleyen bir HTTP yönlendirme başlığı gönderir; bu fonksiyonun çalışabilmesi için sayfada **hiçbir HTML çıktısı üretilmemiş olmalıdır**. Bir form işlendikten hemen sonra başka bir sayfaya yönlendirme yapılması (yukarıdaki `giris.php` örneğinde olduğu gibi), kullanıcının sayfayı yenilemesi (F5) durumunda formun yanlışlıkla tekrar gönderilmesini engelleyen **Post/Redirect/Get (PRG)** deseninin temelini oluşturur.

§8.6.3 PHP ve AJAX Örnekleri

AJAX (*Asynchronous JavaScript and XML*), web sayfasını tamamen yeniden yüklemeyen sunucudan arka planda veri çekmeyi ve arayüzü güncellemeyi sağlar.

Aşağıdaki örnekte kullanıcı açılır menüden (`select/combo`) bir albüm kategorisi seçtiğinde, JavaScript arka planda PHP sunucusuna istek atar; PHP ise veritabanından filtrelenen CD kataloğunu getirerek sayfayı anlık olarak günceller.

PHP Sunucu Tarafı (`get_cd.php`):

```
<?php
// get_cd.php
header('Content-Type: application/json');

$kategori = $_GET['kategori'] ?? '';

// Veritabanından kategoriye göre CD sorgusu yapıldığı varsayılmaktadır
$cdKatalog = [
    "pop" => ["Tarkan - Karma", "Sezen Aksu - Gülümse"],
    "rock" => ["Duman - Seni Kendime Sakladım", "Manga - Şehr-i Hüzn"]
];

$sonuc = $cdKatalog[$kategori] ?? [];
echo json_encode($sonuc); // JSON formatında yanıt döndürülür
?>
```

İstemci Tarafı HTML + JavaScript AJAX Mantığı:

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8">
    <title>AJAX CD Kataloğu</title>
  </head>
  <body>
    <label for="kategoriSelect">Kategori Seçin:</label>
    <select id="kategoriSelect" onchange="cdGetir(this.value)">
      <option value="">Seçiniz...</option>
      <option value="pop">Pop</option>
      <option value="rock">Rock</option>
    </select>

    <h4>Katalog Sonuçları:</h4>
    <ul id="cdListesi"></ul>

    <script>
      function cdGetir(kategori) {
        if (!kategori) return;

        // Asenkron HTTP isteği (Fetch API)
        fetch('get_cd.php?kategori=' + kategori)
          .then(response => response.json())
          .then(data => {
            const liste = document.getElementById('cdListesi');
            liste.innerHTML = ''; // Liste içeriğini temizle

            data.forEach(album => {
              const li = document.createElement('li');
              li.textContent = album;
              liste.appendChild(li);
            });
          });
      }
    </script>
  </body>
</html>
```

```

    });
  })
  .catch(error => console.error('AJAX Hatası:', error));
}
</script>
</body>
</html>

```

Bölüm Özeti

Bu bölümde, web sunucu tarafı programlamanın temel çalışma prensiplerini ve dinamik web uygulaması geliştirmede kullanılan teknolojileri inceledik:

- ♦ **CGI ve Sunucu Betikleri:** Web sunucularının dış programlarla iletişim kurmasını sağlayan CGI protokolünden başlayarak, sunucu tarafında yorumlanan PHP, ASP ve JSP gibi betik dillerinin çalışma mantığını gördük.
- ♦ **PHP Mimarisi:** HTML kodları arasına gömülebilen PHP dilinin sözdizimi, dinamik tiplendirmeli değişkenleri ve sunucu tarafında çıktı üretme mekanizmasını öğrendik.
- ♦ **Form Handling:** HTTP GET ve POST yöntemleriyle istemciden alınan verilerin PHP tarafından işlenmesini ve URL/Form parametrelerinin kullanımını inceledik.
- ♦ **Oturum ve Çerez Yönetimi:** HTTP'nin durumsuz yapısına rağmen `$_SESSION` ve `$_COOKIE` mekanizmalarıyla kullanıcı durumunun (ör. giriş yapılmış olma bilgisi) sayfalar arasında nasıl korunacağını, basit bir giriş sistemi üzerinden uyguladık.
- ♦ **Veri Tabanı İletişimi (MySQL):** RDBMS kavramını ve PHP PDO kütüphanesi kullanarak veritabanında tablo oluşturma, verileri okuma, ekleme, güncelleme ve silme (CRUD) adımlarını uyguladık.
- ♦ **AJAX ile Asenkron İletişim:** Sayfa yenilenmeden sunucuya arka planda veri alışverişi sağlayan AJAX mimarisini PHP-JavaScript entegrasyonu ile pratikleştirdik.

8.7 Düşün ve Çöz

1. **Sunucu ve İstemci Betikleri:** Bir web uygulamasında JavaScript kodu ile PHP kodunun çalıştırıldığı yer ve zaman bakımından temel farkları nelerdir? Neden kullanıcı şifresi doğrulama işlemi sadece JavaScript ile yapılamaz?
2. **HTTP Yöntemleri Seçimi:** Bir e-ticaret sitesinde ürün arama formu için HTTP GET metodu kullanılırken, kullanıcı giriş formu için neden HTTP POST metodu tercih edilmelidir? Güvenlik ve URL yapısı açısından açıklayınız.
3. **SQL Injection Güvenliği:** Aşağıdaki PHP kod satırı neden bir güvenlik açığı oluşturur? Bu sorunun önüne geçmek için yapılması gereken düzeltmeyi `PDO Prepared Statement` mantığı ile açıklayınız.

```
$sql = "SELECT * FROM kullanicilar WHERE ad = '" . $_POST['kullanici'] . "'";
```
4. **AJAX Kullanım Senaryosu:** Bir web sayfasında kullanıcı il seçtiğinde ilçe listesinin sayfa yeniden yüklenmeden güncellenmesini istiyorsunuz. AJAX yapısı kullanıldığında tarayıcı, PHP sunucusu ve veritabanı arasındaki veri akış sırasını adım adım açıklayınız.
5. **Oturum mu, Çerez mi?** Bir alışveriş sitesinde (a) kullanıcının "beni hatırla" tercihini ve (b) kullanıcının o an sepetinde bulunan ürünlerin toplam tutarını saklamak istiyorsunuz. Bu iki bilgidен hangisi çerezde, hangisi oturumda saklanmalıdır? Güvenlik ve kalıcılık açısından gerekçelendiriniz.

8.8 Uygulamalı Projeler

Bu bölümde öğrendiğiniz form işleme, oturum yönetimi, PDO ile veritabanı işlemleri ve AJAX kavramlarını pekiştirmek için aşağıdaki beş projeden birini veya birkaçını tamamlamayı deneyin. Yerel bir Apache/PHP/MySQL ortamına ihtiyacınız olacaktır.

1. **Basit Hesap Makinesi:** İki sayı ve bir işlem (toplama, çıkarma, çarpma, bölme) seçimi alan bir `GET` formu oluşturun; PHP tarafında `switch` yapısıyla seçilen işlemi uygulayan bir fonksiyon yazıp sonucu ekrana yazdırın. Sıfıra bölme durumunu ayrıca kontrol edin.
2. **Ziyaretçi Defteri:** Ad ve mesaj alanları olan bir `POST` formuyla gönderilen verileri PDO kullanarak bir MySQL tablosuna kaydedin; sayfanın altında, tablodaki tüm mesajları en yeniden en eskiye doğru listelein.
3. **Veritabanı Destekli Giriş Sistemi:** Bölümdeki `giris.php` örneğini genişletin: kullanıcı adı ve şifreyi sabit kodlamak yerine, PDO ile bir `kullanicilar` tablosunda arayın (şifreleri `password_hash()` ile saklayıp `password_verify()` ile doğrulayın) ve başarılı girişte oturumu (`$_SESSION`) başlatın.
4. **Ürün Kataloğu Yönetim Paneli:** Bir MySQL tablosundaki ürünleri (ad, fiyat) listeleyen, yeni ürün eklemeye (`POST`) ve mevcut bir ürünü silmeye (bağlantı üzerinden `GET` ile ürün `id`'sini göndererek) imkan tanıyan basit bir yönetim sayfası oluşturun.
5. **AJAX ile Anlık Arama:** Bir arama kutusuna her tuş vuruşunda (`keyup` olayı), `fetch()` ile bir PHP betiğine

istek gönderin; bu betik, gelen anahtar kelimeyi PDO ile veritabanında arayıp sonuçları JSON olarak döndürsün ve tarayıcı bu sonuçları sayfa yenilenmeden listelesin.

§8.8.1 Ziyaretçi Defteri Örneği

```
<?php
// ziyaretci-defteri.php
// Not: Çalıştırmadan önce aşağıdaki veritabanı bilgilerini kendi
// XAMPP/Apache/MySQL ortamınıza göre düzenleyin ve 'web_programlama' veritabanında 'ziyaretler'
// tablosunu oluşturun:
//
// CREATE TABLE ziyaretler (
//     id INT AUTO_INCREMENT PRIMARY KEY,
//     ad VARCHAR(100) NOT NULL,
//     mesaj TEXT NOT NULL,
//     tarih DATETIME DEFAULT CURRENT_TIMESTAMP
// );

$sunucu = 'localhost';
$kullanici = 'devuser';
$sifre = 'YourStrongPassword123!';
$veritabani = 'web_programlama';

try {
    $pdo = new PDO("mysql:host=$sunucu;dbname=$veritabani;charset=utf8", $kullanici, $sifre);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $hata) {
    die("Veritabanı bağlantı hatası: " . $hata->getMessage());
}

$hataMesaji = "";

// Form POST ile gönderildiyse yeni mesajı kaydet
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $ad = trim($_POST['ad'] ?? '');
    $mesaj = trim($_POST['mesaj'] ?? '');

    if ($ad != "" && $mesaj != "") {
        // Prepared Statement kullanarak SQL Injection riskini ortadan kaldırıyoruz
        $sorgu = $pdo->prepare("INSERT INTO ziyaretler (ad, mesaj) VALUES (:ad, :mesaj)");
        $sorgu->execute([
            ':ad' => $ad,
            ':mesaj' => $mesaj
        ]);

        // Post/Redirect/Get deseni: kaydettikten sonra aynı sayfaya yönlendir
        header("Location: " . $_SERVER['PHP_SELF']);
        exit();
    } else {
        $hataMesaji = "Lütfen adınızı ve mesajınızı giriniz.";
    }
}

// Tüm mesajları en yenisinden en eskiye doğru çekiyoruz
$sonuc = $pdo->query("SELECT ad, mesaj, tarih FROM ziyaretler ORDER BY tarih DESC");
$mesajlar = $sonuc->fetchAll(PDO::FETCH_ASSOC);
?>

<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <title>Ziyaretçi Defteri</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 500px; margin: 40px auto; }
        form { margin-bottom: 30px; }
        input, textarea { width: 100%; padding: 8px; margin-bottom: 10px; box-sizing: border-box; }
        .mesaj-kutusu { border-bottom: 1px solid #ddd; padding: 10px 0; }
```

```
.hata { color: red; }
</style>
</head>
<body>

<h1>Ziyaretçi Defteri</h1>

<?php if ($hataMesaji): ?>
<p class="hata"><?php echo htmlspecialchars($hataMesaji); ?></p>
<?php endif; ?>

<form action="ziyaretci-defteri.php" method="POST">
<input type="text" name="ad" placeholder="Adınız">
<textarea name="mesaj" rows="3" placeholder="Mesajınız"></textarea>
<button type="submit">Gönder</button>
</form>

<h2>Gelen Mesajlar</h2>

<?php if (count($mesajlar) === 0): ?>
<p>Henüz mesaj yok. İlk mesajı siz bırakın!</p>
<?php endif; ?>

<?php foreach ($mesajlar as $satir): ?>
<div class="mesaj-kutusu">
<strong><?php echo htmlspecialchars($satir['ad']); ?></strong>
<small> - <?php echo $satir['tarih']; ?></small>
<p><?php echo htmlspecialchars($satir['mesaj']); ?></p>
</div>
<?php endforeach; ?>

</body>
</html>
```

9. Bölüm

RESTful Mimari ve Web Servis Tasarımı

Modern web ve mobil uygulamaların birbiriyle iletişim kurabilmesi, veri alışverişinde bulunabilmesi ve ölçeklenebilir sistemler inşa edilebilmesi için web servisleri kritik bir role sahiptir. Bu bölümde, *Roy Fielding* tarafından 2000 yılında doktora tezinde tanımlanan REST (*Representational State Transfer*) mimari stilini, bu mimarinin temel ilkelerini ve standartlarını inceleyeceğiz. Ardından, mimarinin dilden bağımsız yapısını göstermek adına hem PHP hem de ASP.NET Core platformları ile örnek bir RESTful API geliştireceğiz.

§9.0.1 RESTful Mimariye Giriş

REST, istemci ve sunucu arasındaki veri alışverişini yönlendiren bir mimari stil ve tasarım prensipleri kümesidir. Katı kurallara ve protokollere dayanan SOAP (*Simple Object Access Protocol*) yapısının aksine REST, web'in mevcut altyapısını ve HTTP protokolünün doğal yeteneklerini kullanır.

REST'in Temel İlkeleri

Bir sistemin "RESTful" olarak adlandırılabilmesi için aşağıdaki altı temel mimari kısıtlamaya (*Constraints*) uyması beklenir:

- ♦ **İstemci-Sunucu (*Client-Server*):** Kullanıcı arayüzü (istemci) ile veri depolama ve iş mantığı (sunucu) birbirinden tamamen ayrılmalıdır. Bu ayrım, bileşenlerin bağımsız geliştirilebilmesini ve ölçeklenebilmesini sağlar.
- ♦ **Durumsuzluk (*Stateless*):** Sunucu, istemciye ait oturum durumunu (*state*) kaydetmez. İstemciden gelen her bir istek (request), o isteğin işlenebilmesi için gerekli tüm bilgileri (kimlik doğrulama verileri dahil) içermelidir.
- ♦ **Önbelleklenebilirlik (*Cacheable*):** Yanıtlar (*responses*), açık bir şekilde önbelleklenebilir veya önbelleklenebilir olarak etiketlenmelidir. Bu sayede gereksiz istemci-sunucu etkileşimleri engellenir.
- ♦ **Tekdüze Arayüz (*Uniform Interface*):** Sistem bileşenleri arasında basitleştirilmiş ve standartlaştırılmış bir iletişim arayüzü bulunmalıdır. Kaynakların URI ile tanımlanması ve standart HTTP yöntemlerinin kullanımı bu ilkenin gereğidir.
- ♦ **Katmanlı Sistem (*Layered System*):** İstemci, doğrudan son sunucuya mı yoksa aradaki bir proxy veya yük dengeleyiciye (*load balancer*) mi bağlandığını bilmek zorunda değildir.
- ♦ **İsteğe Bağlı Kod (*Code on Demand*):** Sunucu, istemciye çalıştırılabilir kod parçacıkları (örneğin JavaScript) göndererek istemci yeteneklerini geçici olarak genişletebilir.

§9.0.2 RESTful Mimarinin Yapı Taşları

RESTful mimaride her şey bir **Kaynak** (*Resource*) olarak ele alınır. Bir kaynak; bir kullanıcı, bir ürün, bir sipariş veya bir dosya olabilir.

URI Tasarımı ve Kaynak İsimlendirme

Kaynaklar benzersiz URI (Uniform Resource Identifier) değerleri ile temsil edilir. İyi bir RESTful API tasarımında isim soyutlamaları kullanılmalı ve fiil kullanımından kaçınılmalıdır:

- ♦ **Hatalı Yaklaşım:** GET /getUsers, POST /createUser/12
- ♦ **Doğru Yaklaşım:** GET /api/v1/users, POST /api/v1/users

HTTP yöntemleri

HTTP yöntemleri, kaynaklar üzerinde gerçekleştirilecek eylemleri ifade eder:

İlk bölümde (Başlık ??) sunucudan veri okuma yöntemi olan GET yöntemine örnek verilmiştir. Bu yöntemler, istemcinin sunucudaki hedef kaynak üzerinde gerçekleştirmek istediği eylemin amacını belirtir.

RESTful mimarinin temel özelliği HTTP yöntemleri standart amaçları dışında kullanılır; sıklıkla tüm **CRUD** (*Create, Read, Update, Delete*) işlemleri tek bir yöntem (GET veya POST) üzerinden, URL içerisindeki eylem isimleriyle yürütülür.

Bu yöntemleri, en sık kullanılanlardan başlayarak inceleyelim. Her yöntemi, bir önceki bölümde öğrendiğiniz

Tablo 9.1: HTTP yöntemleri ve Özellikleri

yöntem	Eylem	Safe (Güvenli)	Idempotent (Etkisiz)
GET	Kaynağı Okuma	Evet	Evet
POST	Yeni Kaynak Oluşturma	Hayır	Hayır
PUT	Kaynağı Tamamen Güncelleme	Hayır	Evet
PATCH	Kaynağı Kısmi Güncelleme	Hayır	Hayır
DELETE	Kaynağı Silme	Hayır	Evet

istek/yanıt paket yapısını kullanan somut örneklerle pekiştireceğiz.

GET İstemcinin belirtilen kaynaktan veri okumak için kullandığı yöntemdir.

- ◊ **Güvenli (Safe)**: Sunucu durumunda hiçbir değişiklik yapmaz.
- ◊ **Etkisiz Eleman (Idempotent)**: Aynı GET isteği defalarca tekrarlınsa dahi sunucudaki nihai durum değişmez.

Örnek İstek:

```
GET /api/urunler/42 HTTP/1.1
Host: www.ornek.com
Accept: application/json
```

[Boş Satır -- Gövde Yok]

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 58
```

```
{"id": 42, "ad": "Kablosuz Klavye", "fiyat": 349.90}
```

HEAD GET ile aynıdır; fakat sunucu yanıt olarak **yalnızca HTTP başlıklarını** döner, mesaj gövdesini (*body*) göndermez. Dosya boyutunu veya kaynağın varlığını kontrol etmek için kullanılır.

Örnek İstek:

```
HEAD /images/logo.png HTTP/1.1
Host: www.ornek.com
```

[Boş Satır -- Gövde Yok]

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: image/png
Content-Length: 24560
```

[Boş Satır -- Gövde Yine Yok]

POST Sunucuya veri göndererek yeni bir kaynak oluşturmak veya bir süreci tetiklemek için kullanılır. Güvenli değildir ve etkisiz eleman (*Idempotent*) değildir (üst üste atılan istekler mükerrer kayıtlara yol açabilir).

Örnek İstek:

```
POST /api/kullanici HTTP/1.1
Host: www.ornek.com
Content-Type: application/json
Content-Length: 46
```

```
{"ad": "Ayşe Yılmaz", "email": "ayse@ornek.com"}
```

Örnek Yanıt:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/kullanici/107
```

```
{"id": 107, "ad": "Ayşe Yılmaz", "email": "ayse@ornek.com"}
```

Burada dikkat edilmesi gereken nokta, sunucunun yeni oluşturulan kaynağın adresini **Location** başlığıyla istemciye bildirmesidir.

PUT Hedef URL'deki kaynağın **tamamını** verilen verilerle değiştirmek (*replace*) için kullanılır. Etkisiz elemandır (*idempotent*); aynı verilerle tekrarlanan istekler sonucu değiştirmez.

Örnek İstek:

```
PUT /api/urunler/42 HTTP/1.1
Host: www.ornek.com
Content-Type: application/json
Content-Length: 54
```

```
{"id": 42, "ad": "Kablosuz Klavye Pro", "fiyat": 399.90}
```

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"id": 42, "ad": "Kablosuz Klavye Pro", "fiyat": 399.90}
```

DELETE Belirtilen URL'deki kaynağın silinmesini talep eder. Etkisiz elemandır (*idempotent*).

Örnek İstek:

```
DELETE /api/urunler/42 HTTP/1.1
Host: www.ornek.com
```

[Boş Satır -- Gövde Yok]

Örnek Yanıt:

```
HTTP/1.1 204 No Content
```

[Boş Satır -- Gövde Yok]

Silme işlemi başarılı olduğunda sunucunun geri döndürecek bir içeriği kalmadığı için tipik olarak **204 No Content** durum kodu kullanılır.

PATCH Hedef kaynağın **kısmi olarak** (*partial update*) güncellenmesi için kullanılır (**PUT**'tan farkı, yalnızca değişen alanı gönderebilmesidir).

Örnek İstek:

```
PATCH /api/kullanici/107 HTTP/1.1
Host: www.ornek.com
Content-Type: application/json
Content-Length: 30
```

```
{"email": "yeni@ornek.com"}
```

Örnek Yanıt:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"id": 107, "ad": "Ayşe Yılmaz", "email": "yeni@ornek.com"}
```

Dikkat edilirse istek gövdesinde yalnızca değiştirilecek alan (*email*) gönderilmiştir; **PUT**'ta olduğu gibi kaynağın tamamının tekrar gönderilmesine gerek yoktur.

Diğer Yöntemler

- ◇ **OPTIONS:** Sunucunun desteklediği yöntemleri ve **Çapraz Kaynak Paylaşımı CORS** (*Cross-Origin Resource Sharing*) güvenlik kurallarını sorgular. Bu kurallar, bir web tarayıcısının farklı bir etki alanından (*domain*), porttan veya şemadan gelen kaynaklara kontrollü bir şekilde erişmesine izin veren HTTP tabanlı bir güvenlik mekanizmasıdır.
 - ◇ **CONNECT:** Vekil sunucular üzerinden şifreli tünel (HTTPS) kurmak için kullanılır.
 - ◇ **TRACE:** İstek paketinin izlediği yolu test eden teşhis yöntemidir.
- Yukarıdaki örneklerde sürekli karşımıza çıkan **200 OK**, **201 Created** ve **204 No Content** gibi ifadeler, aslında sunucunun isteğe ne şekilde karşılık verdiğini özetleyen büyük bir sınıflandırma sisteminin yalnızca birer üyesidir.

HTTP Durum Kodları

RESTful web servisleri, yapılan isteğin sonucunu standart HTTP durum kodları (*Status Codes*) ile bildirir:

- ◇ **2xx Başarı:** **200 OK** (Başarılı), **201 Created** (Kaynak Oluşturuldu), **204 No Content** (İçerik Yok/Silindi).

- ◇ **4xx İstemci Hataları:** 400 Bad Request (Geçersiz İstek), 401 Unauthorized (Yetkisiz), 403 Forbidden (Erişim Yasak), 404 Not Found (Bulunamadı).
- ◇ **5xx Sunucu Hataları:** 500 Internal Server Error (Sunucu İçi Hata).

§9.0.3 PHP (Laravel) ile RESTful API Geliştirme Örneği

Modern PHP ekosisteminde Laravel, sunduğu routing ve controller mekanizmaları ile RESTful servis geliştirmeyi oldukça kolaylaştırır. Aşağıdaki örnekte bir ürün (**Product**) kaynağı için endpoint tanımları yer almaktadır.

Laravel Rotaları (routes/api.php)

```
use App\Http\Controllers\ProductController;

// RESTful kaynak rotalarını tek satırda tanımlama
Route::apiResource('products', ProductController::class);
```

Laravel Controller Yapısı

```
namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    // GET /api/products
    public function index()
    {
        return response()->json(Product::all(), 200);
    }

    // POST /api/products
    public function store(Request $request)
    {
        $product = Product::create($request->all());
        return response()->json($product, 201);
    }

    // GET /api/products/{id}
    public function show($id)
    {
        $product = Product::find($id);
        if (!$product) {
            return response()->json(['message' => 'Bulunamadi'], 404);
        }
        return response()->json($product, 200);
    }
}
```

§9.0.4 ASP.NET Core ile RESTful API Geliştirme Örneği

C# ve .NET ekosisteminde ASP.NET Core Web API, tip güvenliği yüksek ve performanslı RESTful mimariler oluşturmak için kullanılır.

ASP.NET Core Controller Yapısı

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Linq;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private static readonly List<Product> Products = new List<Product>();

    // GET: api/products
    [HttpGet]
```

```
public IActionResult GetProducts()
{
    return Ok(Products);
}

// GET: api/products/1
[HttpGet("{id}")]
public IActionResult GetProduct(int id)
{
    var product = Products.FirstOrDefault(p => p.Id == id);
    if (product == null)
    {
        return NotFound(new { message = "Ürün bulunamadı" });
    }
    return Ok(product);
}

// POST: api/products
[HttpPost]
public IActionResult CreateProduct([FromBody] Product product)
{
    Products.Add(product);
    return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
}
}
```

§9.0.5 Özet ve Karşılaştırma

Bu bölümde RESTful mimarinin temel prensiplerini ve standartlarını ele aldık. Görüldüğü üzere, ister dinamik bir dil olan PHP kullanılsın ister tip güvenli bir dil olan C# (ASP.NET Core), RESTful servislerin temel mantığı tamamen aynıdır: **Standart HTTP yöntemleri, açık URI yapıları ve durum kodları üzerinden veri yönetimi.** Teknolojiden bağımsız bu standartlar, modern yazılım mimarilerinin en önemli yapı taşıdır.

10. Bölüm

Modern Web Mimarisi

Modern web yazılım geliştirme süreçleri, uygulamanın farklı sorumluluklarını üstlenen iki ana katmana ayrılmıştır: Kullanıcı ile doğrudan etkileşime geçen **Ön Yüz (Front-End)** ve arka planda iş mantığını, veritabanı işlemlerini ve güvenliği yöneten **Arka Yüz (Back-End)**. Bu bölümde, bu iki kavramın temel rollerini, kullanılan güncel teknolojileri, birbirlerine karşı avantaj ve dezavantajlarını ve bu iki katmanın RESTful API'ler üzerinden nasıl entegre edildiğini inceleyeceğiz.

§10.0.1 Ön Yüz ve Arka Yüz Kavramlarına Giriş

Geleneksel web uygulamalarında sunucu, hem veritabanı işlemlerini yürütür hem de kullanıcıya gösterilecek HTML kodunu üretip istemciye gönderirdi. Modern web mimarisinde ise bu yapı iki ayrı uzmanlık ve çalışma alanına bölünmüştür.

İstemci ve Sunucu Rollerinin Ayrışması

Web ekosisteminde istemci (*client*) ve sunucu (*server*) rollerinin belirgin sınırları vardır:

- ◊ **Front-End (İstemci Tarafı):** Kullanıcının web tarayıcısı (*browser*) üzerinde çalışan tüm görsel öğeleri, animasyonları, form alanlarını ve arayüz mantığını kapsar.
- ◊ **Back-End (Sunucu Tarafı):** Kullanıcının doğrudan görmediği; sunucuda çalışan veritabanı sorgularını, iş kurallarını (*business logic*), yetkilendirmeyi ve sistem entegrasyonlarını yönetir.

Önceki bölümde incelediğimiz RESTful web servisleri, istemci ve sunucu arasındaki bu veri iletişimini sağlayan standart bir protokol köprüsü görevi görür.

Sorumlulukların Ayrılması İlkesi

İş mantığı ile arayüz tasarımının birbirinden ayrılması (*Separation of Concerns*) yazılım yaşam döngüsüne önemli katkılar sağlar:

- ◊ **Bağımsız Geliştirme:** Front-End ve Back-End ekipleri aynı anda, birbirlerini engellemeden çalışabilir.
- ◊ **Yeniden Kullanılabilirlik:** Aynı Back-End servisi (API), hem web ön yüzü hem de mobil uygulama (iOS/Android) tarafından tüketilebilir.
- ◊ **Bakım Kolaylığı:** Arayüzde yapılan bir tasarım değişikliği arka yüzdeki iş mantığını etkilemez.

§10.0.2 Ön Yüz Teknolojileri ve Mimarisi

Ön Yüz (*Front-End*) geliştirme, tarayıcının anlayabildiği temel diller üzerine kuruludur.

Temel Yapı Taşları: HTML, CSS ve JavaScript

Modern web arayüzlerinin temeli üç ana bileşenden oluşur:

- ◊ **HTML5:** Sayfanın iskeletini ve semantik yapısını belirler.
- ◊ **CSS3:** Sayfanın görsel tasarımını, renklerini, yazı tiplerini ve ekran boyutlarına uyumunu (*Responsive Design*) düzenler.
- ◊ **JavaScript (ES6+):** Sayfaya dinamizm katar, kullanıcı etkileşimlerini dinler ve asenkron veri transferi gerçekleştirir.

Modern Front-End Framework ve Kütüphaneleri

Karmaşılaşan web uygulamalarını modüler şekilde geliştirebilmek için modern araçlar kullanılmaktadır:

- ◊ **React:** Meta tarafından geliştirilen, bileşen (*component*) tabanlı ve *Virtual DOM* teknolojisini kullanan popüler bir JavaScript kütüphanesidir.
- ◊ **Vue.js:** Kolay öğrenme eğrisi, reaktif (*reactive*) veri bağlama yapısı ve esnek mimarisi ile öne çıkan bir kütüphanedir.
- ◊ **Angular:** Google tarafından desteklenen, TypeScript tabanlı, büyük ölçekli kurumsal uygulamalar için ihtiyaç duyulan tüm araçları bünyesinde barındıran (*opinionated*) tam teşekküllü bir framework'tür.

Ön Yüz Teknolojilerinin Avantaj ve Dezavantajları

- ◊ **Avantajları:**
 - **Zengin Kullanıcı Deneyimi (UX):** Sayfa tamamen yenilenmeden verilerin güncellenmesini sağlar.
 - **Sunucu Yükünün Azalması:** Arayüz çizim (*rendering*) işlemleri kullanıcının işlemcisinde yapılır.
- ◊ **Dezavantajları:**
 - **Arama Motoru Optimizasyonu (SEO) Zorlukları:** İstemci taraflı gösterim yani **CSR** (*Client Side Rendering*) sitelerde arama motoru botlarının içeriği taraması zorlaşabilir.
 - **Güvenlik Riskleri:** Tüm istemci kodları açık bir şekilde tarayıcıya indiği için kritik iş mantığı Front-End'de tutulamaz.

§10.0.3 Arka Yüz Teknolojileri ve Mimarisi

Arka Yüz (*Back-End*) katmanı, uygulamanın beyni olarak çalışır. Güvenli veri depolama, veri işleme ve yetkilendirme süreçlerini üstlenir.

Öne Çıkan Arka Yüz Teknolojileri

- ◊ **Node.js (Express.js):** JavaScript'in V8 motoru sayesinde sunucu tarafında çalışmasını sağlayan, olay güdümlü (*event-driven*) ve asenkron bir çalışma zamanıdır (*runtime*).
- ◊ **PHP (Laravel):** Önceki bölümlerde ele aldığımız, web için özel olarak doğmuş ve Laravel framework'ü ile modern bir nesne yönelimli mimariye kavuşmuş dinamik dildir.
- ◊ **C# (ASP.NET Core):** Tip güvenliği yüksek, derlenen, özellikle kurumsal sistemlerde yüksek performans ve ölçeklenebilirlik sunan Microsoft ekosistemi teknolojisidir. Konu ile ilgili ¹itabımı inceleyebilirsiniz <https://github.com/HoydaBre/csharp>.
- ◊ **Python (Django / FastAPI):** Temiz sözdizimi, zengin kütüphane desteği ve özellikle veri bilimi/yapay zeka entegrasyonlu projelerde tercih edilen ekosistemdir.

Arka Yüz Teknolojilerinin Avantaj ve Dezavantajları

- ◊ **Avantajları:**
 - **Güvenlik:** İş mantığı ve veritabanı erişim şifreleri sunucuda gizli kalır.
 - **Veri Bütünlüğü:** Veritabanı işlemleri merkezi olarak doğrulanır ve saklanır.
- ◊ **Dezavantajları:**
 - **Sunucu Maliyetleri:** İşlem gücü sunucuda harcandığı için ölçekleme maliyeti yüksektir.
 - **Ağ Gecikmesi (Latency):** İstemci ile sunucu arasındaki coğrafi uzaklık istek sürelerini etkileyebilir.

§10.0.4 Ön Yüz ve Arka Yüz Entegrasyonu

Önceki bölümde geliştirdiğimiz RESTful web servislerini istemci tarafında tüketmek (*consume*) için modern JavaScript'in sunduğu *fetch* API yapısı kullanılır.

Asenkron API İsteği Örneği

Aşağıdaki JavaScript kodu, sunucudaki `/api/products` endpoint'ine asenkron bir *GET* isteği atarak gelen JSON verisini HTML arayüzüne dinamik olarak basmaktadır:

```
// API'den ürün listesini çeken asenkron fonksiyon
async function urunleriGetir() {
  try {
    const response = await fetch('https://api.ornek.com/api/products', {
      method: 'GET',
      headers: {
        'Accept': 'application/json'
      }
    });

    if (!response.ok) {
      throw new Error(`HTTP hatası! Durum: ${response.status}`);
    }

    const urunler = await response.json();

    // DOM Üzerinde Verileri Gösterme
    const listeElementi = document.getElementById('urun-listesi');
    listeElementi.innerHTML = '';
  }
}
```

¹k

```

    urunler.forEach(urun => {
      const li = document.createElement('li');
      li.textContent = `${urun.ad} - ${urun.fiyat} TL`;
      listeElementi.appendChild(li);
    });

  } catch (error) {
    console.error("Veri çekme hatası:", error);
  }
}

// Fonksiyonu tetikle
urunleriGetir();

```

§10.0.5 Bölüm Özeti ve Değerlendirme Soruları

Bölüm Özeti

Modern web mimarisi, sorumlulukların ayrılması ilkesi gereğince Front-End ve Back-End olarak ikiye ayrılmıştır. Front-End tarafı kullanıcı deneyimine ve arayüz dinamizmine odaklanırken; Back-End tarafı iş kuralları, veri güvenliği ve sistem performansından sorumludur. Bu iki yapı, standart HTTP yöntemleri ve JSON veri formatı üzerinden RESTful API'ler vasıtasıyla haberleşir.

Kriter	Front-End (İstemci Tarafı)	
Çalışma Ortamı	Web Tarayıcısı (Browser)	Web Sunucusu / Çalışma Zamanı (Runtime)
Ana Odak	Kullanıcı Arayüzü (UI) ve Deneyimi (UX)	Veri Yönetimi, İş Mantığı, Güvenlik
Temel Teknolojiler	HTML, CSS, JS, React, Vue, Angular	Node.js, Python, C#, Java, PHP, SQL/NoSQL
Güvenlik Yaklaşımı	İstemci tarafı doğrulama (Arayüz seviyesi)	Sunucu tarafı doğrulama ve yetkilendirme (Kritik seviye)

Tablo 10.1: ÖnYüz Arka Yüz Karşılaştırması

Düşün ve Çöz

1. Bir E-Ticaret uygulamasında ödeme adımı gerçekleştirilirken kredi kartı doğrulama mantığının Front-End yerine Back-End tarafında çalıştırılmasının güvenlik gerekçelerini açıklayınız.
2. İstemci Taraflı Gösterim (*Client-Side Rendering*) ile Sunucu Taraflı Gösterim (*Server-Side Rendering*) arasındaki temel farklar nelerdir? Arama motoru optimizasyonu (SEO) açısından hangisi tercih edilmelidir?
3. Önceki bölümde öğrendiğiniz *OPTIONS* HTTP yönteminin Front-End entegrasyonundaki *CORS* (*Cross-Origin Resource Sharing*) mekanizması ile ilişkisini araştırınız.

11. Bölüm

Node.js (Express.js) - React ile Öğrenci Bilgi Sistemi

Bölüm 8’de RESTful mimarinin ilkelerini, Bölüm 9’da ise Front-End ve Back-End katmanlarının nasıl ayrıştığını öğrendik. Bu bölümde artık teoriden pratiğe geçiyor ve uçtan uca çalışan gerçek bir uygulama kuruyoruz: bir **Öğrenci Bilgi Sistemi**. Bu sistemde öğrencileri listeleyecek ve bilgilerini güncelleyecek bir REST API’yi **Node.js (Express.js)** ile, bu API’yi tüketen bir arayüzü ise **React** ile geliştireceğiz. Tüm kurulum adımları bir Linux (Ubuntu tabanlı) sistem üzerinden anlatılacaktır.

11.1 Proje Mimarisi

Uygulamamız, Bölüm 9’da öğrendiğimiz Front-End/Back-End ayrımını birebir izleyen üç katmandan oluşur:

1. **Veritabanı (MySQL):** Öğrenci kayıtlarının kalıcı olarak saklandığı katman.
2. **Back-End (Node.js + Express.js):** Veritabanıyla konuşan, `/api/ogrenciler` uç noktası altında RESTful bir API sunan katman.
3. **Front-End (React):** Bu API’den veri çeken, kullanıcıya bir tablo ve düzenleme formu gösteren katman.

Neden İki Ayrı Sunucu?

Geliştirme sırasında Back-End (varsayılan olarak `localhost:3001`) ve Front-End (varsayılan olarak `localhost:5173`) **iki farklı portta** çalışacaktır. Bu, Bölüm 9’da bahsettiğimiz Front-End/Back-End ayrımının somut bir yansımasıdır ve Bölüm 8’de öğrendiğimiz CORS (*Cross-Origin Resource Sharing*) mekanizmasının neden gerekli olduğunu gösterir.

11.2 Linux Üzerinde Geliştirme Ortamının Kurulumu

§11.2.1 Node.js Kurulumu

Node.js’i Ubuntu/Debian tabanlı bir sistemde NodeSource deposu üzerinden kurmak, güncel bir sürüm elde etmenin en güvenilir yoludur:

```
# Sistem paket listesini güncelle
sudo apt update

# NodeSource kurulum betiğini indir ve çalıştır (Node.js 20 LTS)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Node.js ve npm'i kur
sudo apt install -y nodejs

# Kurulumu doğrula
node -v
npm -v
```

§11.2.2 MySQL Kurulumu

Bölüm 7’de tanıştığımız MySQL’i sunucumuza kuralım:

```
sudo apt install -y mysql-server
sudo systemctl start mysql
sudo systemctl enable mysql

# Güvenlik ayarlarını yapılandır (kök şifresi vb.)
sudo mysql_secure_installation
```

11.3 Veritabanı Hazırlığı

MySQL'e bağlanıp veritabanımızı ve tablomuzu oluşturalım:

```
sudo mysql -u root -p
CREATE DATABASE ogrenci_sistemi;
USE ogrenci_sistemi;

CREATE TABLE ogrenciler (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ad VARCHAR(50) NOT NULL,
  soyad VARCHAR(50) NOT NULL,
  ogrenci_no VARCHAR(20) NOT NULL UNIQUE,
  bolum VARCHAR(100) NOT NULL
);

-- Örnek veriler
INSERT INTO ogrenciler (ad, soyad, ogrenci_no, bolum) VALUES
('Ayşe', 'Yılmaz', '2024001', 'Bilgisayar Mühendisliği'),
('Mehmet', 'Kaya', '2024002', 'Elektrik-Elektronik Mühendisliği'),
('Zeynep', 'Demir', '2024003', 'Endüstri Mühendisliği');

-- API'nin bu veritabanına bağlanabilmesi için ayrı bir kullanıcı oluşturalım
CREATE USER 'ogrenci_api'@'localhost' IDENTIFIED BY 'GucluBirSifre123!';
GRANT ALL PRIVILEGES ON ogrenci_sistemi.* TO 'ogrenci_api'@'localhost';
FLUSH PRIVILEGES;
```

Güvenlik Notu

Uygulamanın veritabanına `root` kullanıcısıyla değil, yalnızca gerekli izinlere sahip ayrı bir kullanıcıyla (`ogrenci_api`) bağlanması, üretim ortamındaki en temel güvenlik alışkanlıklarından biridir.

11.4 Back-End: Node.js ve Express.js ile REST API Geliştirme

§11.4.1 Proje Kurulumu ve Bağımlılıklar

```
mkdir ogrenci-api && cd ogrenci-api
npm init -y

# Express.js (web sunucusu çatısı), mysql2 (veritabanı sürücüsü),
# cors (Front-End'in farklı porttan erişebilmesi için)
npm install express mysql2 cors
```

§11.4.2 Veritabanı Bağlantısı (db.js)

```
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'ogrenci_api',
  password: 'GucluBirSifre123!',
  database: 'ogrenci_sistemi'
});

module.exports = pool;
```

§11.4.3 RESTful API (server.js)

Bölüm 8'de öğrendiğimiz RESTful ilkelere uygun olarak, kaynak odaklı (`/api/ogrenciler`) ve standart HTTP yöntemlerini kullanan bir API tasarlıyoruz:

```
const express = require('express');
const cors = require('cors');
const db = require('./db');

const app = express();
app.use(cors()); // Front-End'in farklı porttan istek atabilmesi için
app.use(express.json()); // Gelen JSON gövdelerini otomatik olarak ayrıştırır

// GET /api/ogrenciler -> Tüm öğrencileri listele
```

```

app.get('/api/ogrenciler', async (req, res) => {
  try {
    const [satirlar] = await db.query('SELECT * FROM ogrenciler ORDER BY id');
    res.json(satirlar);
  } catch (hata) {
    res.status(500).json({ mesaj: 'Sunucu hatası', detay: hata.message });
  }
});

// GET /api/ogrenciler/:id -> Tekil öğrenci getir
app.get('/api/ogrenciler/:id', async (req, res) => {
  const [satirlar] = await db.query(
    'SELECT * FROM ogrenciler WHERE id = ?',
    [req.params.id]
  );
  if (satirlar.length === 0) {
    return res.status(404).json({ mesaj: 'Öğrenci bulunamadı' });
  }
  res.json(satirlar[0]);
});

// PUT /api/ogrenciler/:id -> Öğrenci bilgilerini güncelle
app.put('/api/ogrenciler/:id', async (req, res) => {
  const { ad, soyad, ogrenci_no, bolum } = req.body;

  const [sonuc] = await db.query(
    'UPDATE ogrenciler SET ad = ?, soyad = ?, ogrenci_no = ?, bolum = ? WHERE id = ?',
    [ad, soyad, ogrenci_no, bolum, req.params.id]
  );
  if (sonuc.affectedRows === 0) {
    return res.status(404).json({ mesaj: 'Öğrenci bulunamadı' });
  }
  res.json({ mesaj: 'Öğrenci başarıyla güncellendi' });
});

const PORT = 3001;
app.listen(PORT, () => {
  console.log(`API http://localhost:${PORT} adresinde çalışıyor`);
});

```

§11.4.4 API'yi Çalıştırma ve Test Etme

node server.js

Başka bir terminalde `curl` ile API'yi test edelim (Bölüm 8'de gördüğümüz gibi):

Tüm öğrencileri listele

curl http://localhost:3001/api/ogrenciler

Bir öğrenciyi güncelle

```

curl -X PUT http://localhost:3001/api/ogrenciler/1 \
  -H "Content-Type: application/json" \
  -d '{"ad":"Ayşe","soyad":"Yılmaz","ogrenci_no":"2024001","bolum":"Yazılım Mühendisliği"}'

```

11.5 Front-End: React ile Arayüz Geliştirme

§11.5.1 Proje Kurulumu

Modern React projeleri için hızlı bir geliştirme aracı olan Vite'ı kullanacağız:

npm create vite@latest ogrenci-arayuz -- --template react

cd ogrenci-arayuz

npm install

npm install axios

§11.5.2 Öğrenci Listesi ve Güncelleme Bileşeni (src/App.jsx)

```

import { useState, useEffect } from 'react';
import axios from 'axios';

```

```

const API_URL = 'http://localhost:3001/api/ogrenciler';

function App() {
  const [ogrenciler, setOgrenciler] = useState([]);
  const [duzenlenen, setDuzenlenen] = useState(null); // Şu an düzenlenen öğrenci

  // Bileşen ilk yüklendiğinde öğrenci listesini çek
  useEffect(() => {
    ogrencileriGetir();
  }, []);

  async function ogrencileriGetir() {
    const yanıt = await axios.get(API_URL);
    setOgrenciler(yanıt.data);
  }

  function duzenlemeyeBasla(ogrenci) {
    setDuzenlenen({ ...ogrenci });
  }

  function alanDegisti(alan, deger) {
    setDuzenlenen({ ...duzenlenen, [alan]: deger });
  }

  async function guncellemeyiKaydet() {
    await axios.put(`${API_URL}/${duzenlenen.id}`, duzenlenen);
    setDuzenlenen(null);
    ogrencileriGetir(); // Listeyi güncel veriyle yenile
  }

  return (
    <div>
      <h1>Öğrenci Bilgi Sistemi</h1>

      <table border="1" cellPadding="8">
        <thead>
          <tr>
            <th>Ad</th>
            <th>Soyad</th>
            <th>Öğrenci No</th>
            <th>Bölüm</th>
            <th>İşlem</th>
          </tr>
        </thead>
        <tbody>
          {ogrenciler.map((ogrenci) => (
            <tr key={ogrenci.id}>
              <td>{ogrenci.ad}</td>
              <td>{ogrenci.soyad}</td>
              <td>{ogrenci.ogrenci_no}</td>
              <td>{ogrenci.bolum}</td>
              <td>
                <button onClick={() => duzenlemeyeBasla(ogrenci)}>
                  Düzenle
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>

      {duzenlenen && (
        <div>
          <h2>Öğrenciyi Güncelle</h2>
          <input
            value={duzenlenen.ad}

```

```

        onChange={e => alanDegisti('ad', e.target.value)}
        placeholder="Ad"
      />
      <input
        value={duzenlenen.soyad}
        onChange={e => alanDegisti('soyad', e.target.value)}
        placeholder="Soyad"
      />
      <input
        value={duzenlenen.bolum}
        onChange={e => alanDegisti('bolum', e.target.value)}
        placeholder="Bölüm"
      />
      <button onClick={guncellemeyiKaydet}>Kaydet</button>
      <button onClick={() => setDuzenlenen(null)}>İptal</button>
    </div>
  )}
</div>
);
}

export default App;

```

Pedagojik Not: useState ve useEffect

useState, bir bileşenin zaman içinde değişen verisini (öğrenci listesi, düzenlenen kayıt) tutmamızı sağlar; değer değiştiğinde React arayüzü otomatik olarak yeniden çizer. **useEffect** ise “bileşen ekrana ilk geldiğinde çalıştır” gibi yan etkileri (burada API’den veri çekme işlemini) yönetir – Bölüm 6’da gördüğümüz **DOMContentLoaded** olayının React’teki karşılığı gibi düşünebilirsiniz.

§11.5.3 Uygulamayı Çalıştırma

npm run dev

Tarayıcıda <http://localhost:5173> adresini açtığınızda, Back-End’den (localhost:3001) çekilen öğrenci listesini görmemiz gerekir. “Düzenle” butonuna bastığınızda açılan formda yaptığınız değişiklik, “Kaydet” butonuyla **PUT** isteği olarak API’ye gönderilir ve MySQL veritabanında kalıcı olarak güncellenir.

11.6 Uygulamayı Bütün Olarak Çalıştırma

Sistemin uçtan uca çalışması için iki terminalde eş zamanlı olarak şu iki sunucunun ayakta olması gerekir:

1. **Terminal 1 (Back-End):** `cd ogrenci-api && node server.js` → <http://localhost:3001>
2. **Terminal 2 (Front-End):** `cd ogrenci-arayuz && npm run dev` → <http://localhost:5173>

Bölüm Özeti

- ✦ Bir MySQL veritabanı, **ogrenciler** tablosunu barındıran kalıcı depolama katmanını oluşturdu.
- ✦ **Node.js** ve **Express.js** ile, Bölüm 8’in RESTful ilkelerine uygun bir API (**GET/PUT /api/ogrenciler**) geliştirdik.
- ✦ **React** ile, bu API’den **axios** kullanarak veri çeken, **useState/useEffect** ile durumu yöneten bir arayüz geliştirdik.
- ✦ **cors** paketi, Bölüm 9’da öğrendiğimiz Front-End/Back-End port ayrımının pratikte yarattığı tarayıcı kısıtlamasını çözdü.

11.7 Düşün ve Çöz

1. Bu sistemde **ogrenci_no** alanına **UNIQUE** kısıtlaması koyduk. Bir güncelleme isteğinde, kullanıcı zaten var olan bir numarayı girerse MySQL’den dönecek hatayı Express.js tarafında nasıl yakalar ve kullanıcıya anlamlı bir mesaj olarak gösterirsiniz?
2. `app.use(cors())` satırını kaldırırsak tarayıcıda ne tür bir hata alırız? Bu hatanın Bölüm 9’da öğrendiğimiz CORS mekanizmasıyla ilişkisini açıklayınız.
3. **useEffect** içindeki bağımlılık dizisini (`[]`) kaldırırsak (yani `useEffect(() => {...})` şeklinde yazarsak) uygulamada ne gibi bir soruna yol açar?

11.8 Uygulamalı Projeler

1. **Yeni Öğrenci Ekleme:** API'ye bir `POST /api/ogrenciler` uç noktası ekleyin ve React arayüzüne yeni öğrenci eklemek için bir form yerleştirin.
2. **Öğrenci Silme:** `DELETE /api/ogrenciler/:id` uç noktasını ve arayüzde bir "Sil" butonunu ekleyin.
3. **Arama Kutusu:** React arayüzüne, girilen metne göre öğrenci listesini anlık filtreleyen bir arama kutusu ekleyin.
4. **Doğrulama (Validation):** `ogrenci_no` alanının yalnızca rakamlardan oluşmasını hem Front-End'de (gönderim öncesi) hem Back-End'de (API içinde) doğrulayın.
5. **Ortam Değişkenleri:** Veritabanı bağlantı bilgilerini `db.js` içine sabit kodlamak yerine `dotenv` paketiyle bir `.env` dosyasından okuyacak şekilde yeniden düzenleyin.

12. Bölüm

C# (ASP.NET Core) - Vue.js ile Öğrenci Bilgi Sistemi

Bir önceki bölümde Node.js ve React ile dinamik tipli bir yığın (*stack*) kurmuştuk. Bu bölümde ise aynı **Öğrenci Bilgi Sistemini**, Bölüm 4'te tanıştığımız **tip güvenli, derlenen** bir dil olan **C#** ile, **ASP.NET Core** çatısı üzerinde yeniden inşa edeceğiz. Arayüz tarafında ise kademeli benimsemesi ve öğrenme eğrisinin düşük olmasıyla bilinen **Vue.js** kütüphanesini kullanacağız. Böylece aynı problemin, tamamen farklı bir dil ve tip felsefesiyle nasıl çözüldüğünü karşılaştırma imkânı bulacaksınız.

12.1 Proje Mimarisi

Mimari, bir önceki bölümdekiyle birebir aynı üç katmandan oluşur; yalnızca Back-End dili ve Front-End kütüphanesi değişir:

1. **Veritabanı (MySQL):** Bir önceki bölümdeki **ogrenciler** tablosunun aynısını kullanacağız.
2. **Back-End (C# / ASP.NET Core):** </api/ogrenciler> uç noktasını sunan, tip güvenli bir Web API. Konu ile ilgili ¹kitabımı inceleyebilirsiniz <https://github.com/HoydaBre/csharp>.
3. **Front-End (Vue.js):** Bu API'yi tüketen reaktif bir arayüz.

12.2 Linux Üzerinde Geliştirme Ortamının Kurulumu

§12.2.1 .NET SDK Kurulumu

Microsoft'un resmî paket deposunu ekleyerek .NET SDK'yı Ubuntu üzerine kuralım:

```
# Microsoft paket imzasını ve deposunu ekle
wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb -O
  → packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
rm packages-microsoft-prod.deb

# .NET SDK'yı kur
sudo apt update
sudo apt install -y dotnet-sdk-8.0

# Kurulumu doğrula
dotnet --version
```

§12.2.2 Node.js Kurulumu (Vue Araçları İçin)

Vue.js projelerini oluşturmak ve derlemek için Node.js gereklidir (bkz. bir önceki bölümdeki kurulum adımları):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

§12.2.3 MySQL Kurulumu

Sisteminizde MySQL zaten kuruluysa (bir önceki bölümden) bu adımı atlayabilirsiniz:

```
sudo apt install -y mysql-server
sudo systemctl start mysql
sudo systemctl enable mysql
```

¹k

12.3 Veritabanı Hazırlığı

Aynı öğrenci tablosunu (farklı bir veritabanı adıyla) yeniden oluşturalım:

```
CREATE DATABASE ogrenci_sistemi_dotnet;
USE ogrenci_sistemi_dotnet;

CREATE TABLE ogrenciler (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    soyad VARCHAR(50) NOT NULL,
    ogrenci_no VARCHAR(20) NOT NULL UNIQUE,
    bolum VARCHAR(100) NOT NULL
);

INSERT INTO ogrenciler (ad, soyad, ogrenci_no, bolum) VALUES
('Ayşe', 'Yılmaz', '2024001', 'Bilgisayar Mühendisliği'),
('Mehmet', 'Kaya', '2024002', 'Elektrik-Elektronik Mühendisliği'),
('Zeynep', 'Demir', '2024003', 'Endüstri Mühendisliği');

CREATE USER 'ogrenci_api_dotnet'@'localhost' IDENTIFIED BY 'GucluBirSifre456!';
GRANT ALL PRIVILEGES ON ogrenci_sistemi_dotnet.* TO 'ogrenci_api_dotnet'@'localhost';
FLUSH PRIVILEGES;
```

12.4 Back-End: ASP.NET Core ile REST API Geliştirme

§12.4.1 Proje Kurulumu ve Bağımlılıklar

```
dotnet new webapi -n OgrenciApi -controllers
cd OgrenciApi

# MySQL'e bağlanmak için hafif bir sürücü ekliyoruz
dotnet add package MySqlConnector
```

Neden Tam Bir ORM Değil?

ASP.NET Core projelerinde genellikle Entity Framework Core kullanılır; ancak bu, ek yapılandırma (migration, DbContext vb.) gerektirir. Bölüm 7'de PHP'de PDO ile öğrendiğimiz "ham SQL yazma" yaklaşımına daha yakın kalabilmek ve öğrenme eğrisini düşük tutabilmek için burada hafif bir sürücü olan `MySqlConnector`'ı doğrudan kullanacağız.

§12.4.2 Öğrenci Modeli (Models/Ogrenci.cs)

C#'ın Bölüm 4'te öğrendiğimiz statik tip sistemi, veri şeklini net bir sınıfla tanımlamamızı sağlar:

```
namespace OgrenciApi.Models;

public class Ogrenci
{
    public int Id { get; set; }
    public string Ad { get; set; } = string.Empty;
    public string Soyad { get; set; } = string.Empty;
    public string OgrenciNo { get; set; } = string.Empty;
    public string Bolum { get; set; } = string.Empty;
}
```

§12.4.3 Veritabanı Servisi (Services/OgrenciServisi.cs)

```
using MySqlConnector;
using OgrenciApi.Models;

namespace OgrenciApi.Services;

public class OgrenciServisi
{
    private readonly string _baglantiDizesi =
        "Server=localhost;Database=ogrenci_sistemi_dotnet;" +
        "User=ogrenci_api_dotnet;Password=GucluBirSifre456!";
```

```

public async Task<List<Ogrenci>> TumunuGetir()
{
    var liste = new List<Ogrenci>();
    using var baglanti = new MySqlConnection(_baglantiDizesi);
    await baglanti.OpenAsync();

    using var komut = new MySqlCommand("SELECT * FROM ogrenciler ORDER BY id", baglanti);
    using var okuyucu = await komut.ExecuteReaderAsync();

    while (await okuyucu.ReadAsync())
    {
        liste.Add(new Ogrenci
        {
            Id = okuyucu.GetInt32("id"),
            Ad = okuyucu.GetString("ad"),
            Soyad = okuyucu.GetString("soyad"),
            OgrenciNo = okuyucu.GetString("ogrenci_no"),
            Bolum = okuyucu.GetString("bolum")
        });
    }
    return liste;
}

public async Task<bool> Guncelle(int id, Ogrenci ogrenci)
{
    using var baglanti = new MySqlConnection(_baglantiDizesi);
    await baglanti.OpenAsync();

    using var komut = new MySqlCommand(
        "UPDATE ogrenciler SET ad=@ad, soyad=@soyad, ogrenci_no=@no, bolum=@bolum WHERE id=@id",
        baglanti);

    komut.Parameters.AddWithValue("@ad", ogrenci.Ad);
    komut.Parameters.AddWithValue("@soyad", ogrenci.Soyad);
    komut.Parameters.AddWithValue("@no", ogrenci.OgrenciNo);
    komut.Parameters.AddWithValue("@bolum", ogrenci.Bolum);
    komut.Parameters.AddWithValue("@id", id);

    int etkilenenSatir = await komut.ExecuteNonQueryAsync();
    return etkilenenSatir > 0;
}
}

```

§12.4.4 RESTful Denetleyici (Controllers/OgrencilerController.cs)

Bölüm 8’de gördüğümüz [HttpGet]/[HttpPut] örüntüsünü, bu kez gerçek bir veritabanına bağlayarak uyguluyoruz:

```

using Microsoft.AspNetCore.Mvc;
using OgrenciApi.Models;
using OgrenciApi.Services;

namespace OgrenciApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class OgrencilerController : ControllerBase
{
    private readonly OgrenciServisi _servis = new();

    // GET /api/ogrenciler
    [HttpGet]
    public async Task<ActionResult<List<Ogrenci>>> TumunuGetir()
    {
        return Ok(await _servis.TumunuGetir());
    }

    // PUT /api/ogrenciler/1

```

```
[HttpPut("{id}")]
public async Task<IActionResult> Guncelle(int id, [FromBody] Ogrenci ogrenci)
{
    bool basarili = await _servis.Guncelle(id, ogrenci);
    if (!basarili)
    {
        return NotFound(new { mesaj = "Öğrenci bulunamadı" });
    }
    return Ok(new { mesaj = "Öğrenci başarıyla güncellendi" });
}
```

§12.4.5 CORS Yapılandırması (Program.cs)

Bölüm 9'da öğrendiğimiz CORS ihtiyacını, ASP.NET Core'da uygulama başlangıcında tanımlarız:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Vue.js Front-End'inin (localhost:5173) API'ye erişebilmesi için
builder.Services.AddCors(options =>
{
    options.AddPolicy("VueArayuzIzni", policy =>
    {
        policy.WithOrigins("http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

app.UseCors("VueArayuzIzni");
app.MapControllers();

app.Run();
```

§12.4.6 API'yi Çalıştırma

dotnet run
Terminalde belirtilen adresten (genellikle <http://localhost:5116> veya benzeri) API'ye `curl` ile erişebilirsiniz:
`curl http://localhost:5116/api/ogrenciler`

12.5 Front-End: Vue.js ile Arayüz Geliştirme

§12.5.1 Proje Kurulumu

```
npm create vue@latest ogrenci-arayuz-vue
# Sorulan sorularda TypeScript, Router vb. için "Hayır" (No) diyerek
# sade bir başlangıç projesi oluşturabilirsiniz.

cd ogrenci-arayuz-vue
npm install
npm install axios
```

§12.5.2 Öğrenci Listesi ve Güncelleme Bileşeni (src/App.vue)

Vue'nun *Composition API* yaklaşımıyla, React'teki `useState`'e karşılık gelen `ref` ve `useEffect`'e karşılık gelen `onMounted` kullanıyoruz:

```
<script setup>
import { ref, onMounted } from 'vue';
import axios from 'axios';

const API_URL = 'http://localhost:5116/api/ogrenciler';

const ogrenciler = ref([]);
const duzenlenen = ref(null);

async function ogrencileriGetir() {
```

```

    const yanit = await axios.get(API_URL);
    ogrenciler.value = yanit.data;
  }

  // Bileşen ekrana ilk geldiğinde çalışır
  onMounted(ogrencileriGetir);

  function duzenlemeyeBasla(ogrenci) {
    duzenlenen.value = { ...ogrenci };
  }

  async function guncellemeyiKaydet() {
    await axios.put(`${API_URL}/${duzenlenen.value.id}`, duzenlenen.value);
    duzenlenen.value = null;
    ogrencileriGetir();
  }
</script>

<template>
  <h1>Öğrenci Bilgi Sistemi</h1>

  <table border="1" cellpadding="8">
    <thead>
      <tr>
        <th>Ad</th>
        <th>Soyad</th>
        <th>Öğrenci No</th>
        <th>Bölüm</th>
        <th>İşlem</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="ogrenci in ogrenciler" :key="ogrenci.id">
        <td>{{ ogrenci.ad }}</td>
        <td>{{ ogrenci.soyad }}</td>
        <td>{{ ogrenci.ogrenciNo }}</td>
        <td>{{ ogrenci.bolum }}</td>
        <td>
          <button @click="duzenlemeyeBasla(ogrenci)">Düzenle</button>
        </td>
      </tr>
    </tbody>
  </table>

  <div v-if="duzenlenen">
    <h2>Öğrenciyi Güncelle</h2>
    <input v-model="duzenlenen.ad" placeholder="Ad">
    <input v-model="duzenlenen.soyad" placeholder="Soyad">
    <input v-model="duzenlenen.bolum" placeholder="Bölüm">
    <button @click="guncellemeyiKaydet">Kaydet</button>
    <button @click="duzenlenen = null">İptal</button>
  </div>
</template>

```

v-model ve v-for

v-model, bir **input** öğesini doğrudan bir JavaScript değişkenine bağlar (*iki yönlü veri bağlama*); kullanıcı kutuya yazdıkça değişken otomatik güncellenir. **v-for** ise Bölüm 10'daki React'in `.map()` kullanımına karşılık gelir: bir diziyi dolaşarak her öğe için bir HTML parçası üretir.

§12.5.3 Uygulamayı Çalıştırma

```
npm run dev
```

12.6 Uygulamayı Bütün Olarak Çalıştırma

1. **Terminal 1 (Back-End):** `cd OğrenciApi && dotnet run`
2. **Terminal 2 (Front-End):** `cd oğrenci-arayuz-vue && npm run dev` → <http://localhost:5173>

Bölüm Özeti

- ◇ Aynı Öğrenci Bilgi Sistemi problemini, bir önceki bölümdeki dinamik tipli Node.js/React yığınınından farklı olarak, **statik tip güvenli C#** ve **ASP.NET Core** ile çözdük.
- ◇ C#'ta **Oğrenci** sınıfı, verinin şeklini derleme zamanında (*compile-time*) garanti altına aldı; bu, Bölüm 4'te öğrendiğimiz derlenen dillerin bir avantajıdır.
- ◇ **Vue.js**'in **ref**, **onMounted**, **v-model** ve **v-for** yapıları, React'in **useState**, **useEffect** ve **.map()** yapılarına işlevsel olarak karşılık geldi.
- ◇ CORS politikası, PHP/Node.js'teki tek satırlık çözümlerin aksine, ASP.NET Core'da açık bir yapılandırma nesnesiyle (**AddCors**) tanımlandı.

12.7 Düşün ve Çöz

1. Node.js/Express örneğinde veritabanı sorguları **async/await** ile, ASP.NET Core örneğinde de **async Task** ile yazıldı. İki dildeki asenkron yaklaşımın ortak amacı nedir?
2. C#'taki **Oğrenci** sınıfının alanlarına yanlış tipte bir değer (örneğin **Id** alanına bir metin) atamaya çalışırsak ne olur? Bunu JavaScript'teki dinamik tipliklemeyle karşılaştırınız.
3. **OğrenciServisi** sınıfındaki SQL sorgularında parametreleri (**@ad**, **@id** vb.) doğrudan string birleştirme yerine **Parameters.AddWithValue** ile eklememizin güvenlik açısından nedeni nedir? (İpucu: Bölüm 7'deki Prepared Statement konusuna bakınız.)

12.8 Uygulamalı Projeler

1. **Yeni Öğrenci Ekleme:** **[HttpPost]** ile bir ekleme uç noktası ve Vue arayüzünde bir ekleme formu oluşturun.
2. **Öğrenci Silme:** **[HttpDelete("{id}")]** uç noktasını ve arayüzde bir silme butonu ekleyin.
3. **Model Doğrulama:** C# modelinde **[Required]** ve **[StringLength]** gibi veri açıklama (*Data Annotation*) öznitelikleri kullanarak sunucu tarafı doğrulama ekleyin.
4. **Bölüme Göre Filtreleme:** API'ye **?bolum=** sorgu parametresini destekleyen bir filtreleme özelliği, arayüze de bir açılır liste (**select**) ekleyin.
5. **Karşılaştırma Raporu:** Bu bölümdeki ASP.NET Core kodu ile bir önceki bölümdeki Express.js kodunu satır satır karşılaştıran bir tablo hazırlayın (rota tanımlama, hata yönetimi, asenkron işlemler).

13. Bölüm

Python (FastAPI) - Angular ile Öğrenci Bilgi Sistemi

Son olarak, aynı **Öğrenci Bilgi Sistemi**ni üçüncü bir dil ve çatı kombinasyonu ile inşa ediyoruz: Back-End tarafında Bölüm 9’da tanıştığımız, temiz sözdizimiyle bilinen **Python** dilini ve modern, hızlı bir API çatısı olan **FastAPI**’yi; Front-End tarafında ise büyük ölçekli kurumsal uygulamalar için tercih edilen, TypeScript tabanlı tam teşekküllü bir çatı olan **Angular**’ı kullanacağız.

Neden Django Değil de FastAPI?

Python ekosisteminde web geliştirme için en bilinen iki seçenek **Django** ve **FastAPI**’dir. Django, yönetici paneli ve ORM’i kendi içinde barındıran kapsamlı (*batteries-included*) bir çatıdır; büyük, çok özellikli siteler için idealdir. **FastAPI** ise yalnızca API geliştirmeye odaklanan, öğrenme eğrisi daha düşük, modern bir çatıdır ve tam da bu bölümdeki gibi “sade bir REST API” ihtiyacı için daha az kod ile aynı sonuca ulaştırır. Bu nedenle bu bölümde FastAPI’yi tercih ediyoruz.

13.1 Proje Mimarisi

1. **Veritabanı (MySQL):** Önceki iki bölümdeki ile birebir aynı **ogrenciler** tablosu.
2. **Back-End (Python / FastAPI):** `/api/ogrenciler` uç noktasını sunan, otomatik veri doğrulamalı bir API.
3. **Front-End (Angular):** Bu API’yi tüketen, TypeScript ile yazılmış bir arayüz.

13.2 Linux Üzerinde Geliştirme Ortamının Kurulumu

§13.2.1 Python ve Sanal Ortam Kurulumu

Ubuntu’da Python3 genellikle önceden kuruludur; yalnızca paket yöneticisini ve sanal ortam aracını kuralım:

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv

python3 --version
```

Pedagojik Not: Sanal Ortam (Virtual Environment) Nedir?

Bir sanal ortam, her Python projesinin kendi bağımsız paket kütüphanesine sahip olmasını sağlayan izole bir klasördür. Node.js’teki `node_modules` klasörünün Python dünyasındaki dengi gibi düşünebilirsiniz; bu sayede bir projenin paketleri, başka bir projenin paket sürümleriyle çakışmaz.

§13.2.2 Node.js ve Angular CLI Kurulumu

Angular projelerini oluşturmak için Node.js ve Angular’ın komut satırı aracı gereklidir:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

sudo npm install -g @angular/cli
ng version
```

§13.2.3 MySQL Kurulumu

Sisteminizde MySQL zaten kuruluysa bu adımı atlayabilirsiniz:

```
sudo apt install -y mysql-server
sudo systemctl start mysql
```

```
sudo systemctl enable mysql
```

13.3 Veritabanı Hazırlığı

```
CREATE DATABASE ogrenci_sistemi_python;
USE ogrenci_sistemi_python;
```

```
CREATE TABLE ogrenciler (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    soyad VARCHAR(50) NOT NULL,
    ogrenci_no VARCHAR(20) NOT NULL UNIQUE,
    bolum VARCHAR(100) NOT NULL
);
```

```
INSERT INTO ogrenciler (ad, soyad, ogrenci_no, bolum) VALUES
('Ayşe', 'Yılmaz', '2024001', 'Bilgisayar Mühendisliği'),
('Mehmet', 'Kaya', '2024002', 'Elektrik-Elektronik Mühendisliği'),
('Zeynep', 'Demir', '2024003', 'Endüstri Mühendisliği');
```

```
CREATE USER 'ogrenci_api_py'@'localhost' IDENTIFIED BY 'GucluBirSifre789!';
GRANT ALL PRIVILEGES ON ogrenci_sistemi_python.* TO 'ogrenci_api_py'@'localhost';
FLUSH PRIVILEGES;
```

13.4 Back-End: Python ve FastAPI ile REST API Geliştirme

§13.4.1 Proje Kurulumu ve Bağımlılıklar

```
mkdir ogrenci-api-python && cd ogrenci-api-python
```

```
# Sanal ortamı oluştur ve etkinleştir
python3 -m venv venv
source venv/bin/activate
```

```
# FastAPI, sunucu (uvicorn) ve MySQL sürücüsünü kur
pip install fastapi uvicorn mysql-connector-python
```

§13.4.2 Veri Modeli ve API (main.py)

FastAPI'nin en güçlü özelliklerinden biri, Python'ın tip belirteçlerini (*type hints*) kullanarak veri doğrulamasını otomatik yapan **Pydantic** kütüphanesidir:

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import mysql.connector
```

```
app = FastAPI()
```

```
# Angular Front-End'inin (localhost:4200) API'ye erişebilmesi için
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:4200"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
def veritabani_baglantisini():
    return mysql.connector.connect(
        host="localhost",
        user="ogrenci_api_py",
        password="GucluBirSifre789!",
        database="ogrenci_sistemi_python"
    )
```

```
# Pydantic modeli: gelen/giden verinin şeklini ve tipini tanımlar
class Ogrenci(BaseModel):
```

```

ad: str
soyad: str
ogrenci_no: str
bolum: str

# GET /api/ogrenciler -> Tüm öğrencileri listele
@app.get("/api/ogrenciler")
def ogrencileri_listele():
    baglanti = veritabani_baglantisi()
    imlec = baglanti.cursor(dictionary=True)
    imlec.execute("SELECT * FROM ogrenciler ORDER BY id")
    sonuc = imlec.fetchall()
    baglanti.close()
    return sonuc

# GET /api/ogrenciler/{id} -> Tekil öğrenci getir
@app.get("/api/ogrenciler/{ogrenci_id}")
def ogrenci_getir(ogrenci_id: int):
    baglanti = veritabani_baglantisi()
    imlec = baglanti.cursor(dictionary=True)
    imlec.execute("SELECT * FROM ogrenciler WHERE id = %s", (ogrenci_id,))
    ogrenci = imlec.fetchone()
    baglanti.close()

    if ogrenci is None:
        raise HTTPException(status_code=404, detail="Öğrenci bulunamadı")
    return ogrenci

# PUT /api/ogrenciler/{id} -> Öğrenci bilgilerini güncelle
@app.put("/api/ogrenciler/{ogrenci_id}")
def ogrenci_guncelle(ogrenci_id: int, ogrenci: Ogrenci):
    baglanti = veritabani_baglantisi()
    imlec = baglanti.cursor()
    imlec.execute(
        "UPDATE ogrenciler SET ad=%s, soyad=%s, ogrenci_no=%s, bolum=%s WHERE id=%s",
        (ogrenci.ad, ogrenci.soyad, ogrenci.ogrenci_no, ogrenci.bolum, ogrenci_id)
    )
    baglanti.commit()
    etkilenen = imlec.rowcount
    baglanti.close()

    if etkilenen == 0:
        raise HTTPException(status_code=404, detail="Öğrenci bulunamadı")
    return {"mesaj": "Öğrenci başarıyla güncellendi"}

```

Pedagojik Not: Pydantic ile Otomatik Doğrulama

Ogrenci(BaseModel) sınıfını bir fonksiyon parametresi olarak yazdığımızda, FastAPI gelen JSON gövdesini otomatik olarak bu şekle göre doğrular. Örneğin **ad** alanı gönderilmezse, elle bir **if** kontrolü yazmamıza gerek kalmadan FastAPI kendisi **422 Unprocessable Entity** durum koduyla anlamlı bir hata mesajı döner.

§13.4.3 API'yi Çalıştırma ve Test Etme

```
uvicorn main:app --reload --port 8000
```

FastAPI'nin sunduğu otomatik, interaktif dokümantasyon sayfasını tarayıcıda görüntüleyebilirsiniz:

```
http://localhost:8000/docs
```

```
curl http://localhost:8000/api/ogrenciler
```

13.5 Front-End: Angular ile Arayüz Geliştirme

§13.5.1 Proje Kurulumu

```
ng new ogrenci-arayuz-angular --routing=false --style=css
cd ogrenci-arayuz-angular
```

§13.5.2 API Servisi (src/app/ogrenci.service.ts)

Angular'da Bölüm 6'da öğrendiğimiz AJAX isteklerini, dahili `HttpClient` servisiyle ve TypeScript'in tip güvenliğiyle yaparız:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Ogresci {
  id: number;
  ad: string;
  soyad: string;
  ogrenci_no: string;
  bolum: string;
}

@Injectable({ providedIn: 'root' })
export class OgresciService {
  private apiUrl = 'http://localhost:8000/api/ogrenciler';

  constructor(private http: HttpClient) {}

  tumunuGetir(): Observable<Ogresci[]> {
    return this.http.get<Ogresci[]>(this.apiUrl);
  }

  guncelle(id: number, ogrenci: Ogresci): Observable<any> {
    return this.http.put(`${this.apiUrl}/${id}`, ogrenci);
  }
}
```

§13.5.3 Ana Bileşen (src/app/app.component.ts)

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { OgresciService, Ogresci } from '../ogrenci.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './app.component.html'
})
export class AppComponent implements OnInit {
  ogrenciler: Ogresci[] = [];
  duzenlenen: Ogresci | null = null;

  constructor(private ogrenciService: OgresciService) {}

  // Bileşen ekrana ilk geldiğinde çalışır (React'teki useEffect'in dengi)
  ngOnInit(): void {
    this.ogrencileriGetir();
  }

  ogrencileriGetir(): void {
    this.ogrenciService.tumunuGetir().subscribe(veri => {
      this.ogrenciler = veri;
    });
  }
}
```

```

duzenlemeyeBasla(ogrenci: Ogrenci): void {
  this.duzenlenen = { ...ogrenci };
}

guncellemeyiKaydet(): void {
  if (!this.duzenlenen) return;

  this.ogrenciService.guncelle(this.duzenlenen.id, this.duzenlenen)
    .subscribe(() => {
      this.duzenlenen = null;
      this.ogrencileriGetir();
    });
}

iptalEt(): void {
  this.duzenlenen = null;
}
}

```

§13.5.4 Şablon (src/app/app.component.html)

```

<h1>Öğrenci Bilgi Sistemi</h1>

<table border="1" cellpadding="8">
  <thead>
    <tr>
      <th>Ad</th>
      <th>Soyad</th>
      <th>Öğrenci No</th>
      <th>Bölüm</th>
      <th>İşlem</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngFor="let ogrenci of ogrenciler">
      <td>{{ ogrenci.ad }}</td>
      <td>{{ ogrenci.soyad }}</td>
      <td>{{ ogrenci.ogrenci_no }}</td>
      <td>{{ ogrenci.bolum }}</td>
      <td>
        <button (click)="duzenlemeyeBasla(ogrenci)">Düzenle</button>
      </td>
    </tr>
  </tbody>
</table>

<div *ngIf="duzenlenen">
  <h2>Öğrenciyi Güncelle</h2>
  <input [(ngModel)]="duzenlenen.ad" placeholder="Ad">
  <input [(ngModel)]="duzenlenen.soyad" placeholder="Soyad">
  <input [(ngModel)]="duzenlenen.bolum" placeholder="Bölüm">
  <button (click)="guncellemeyiKaydet()">Kaydet</button>
  <button (click)="iptalEt()">İptal</button>
</div>

```

Angular'ın şablon söz dizimi, Bölüm 11'deki Vue.js ile kavramsal olarak birebir örtüşür: `*ngFor` → `v-for` (listeyi dolaşma), `*ngIf` → `v-if` (koşullu gösterme), `[(ngModel)]` → `v-model` (iki yönlü veri bağlama). Üç farklı çatının da aynı temel ihtiyaçlara kendi söz dizimleriyle çözüm ürettiğini görmek, bir çatıdan diğerine geçişi kolaylaştırır.

§13.5.5 HttpClient'ı Etkinleştirme (src/app/app.config.ts)

Angular'ın HTTP istekleri atabilmesi için bu servisin uygulamaya tanıtılması gerekir:

```

import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()]
}

```

```
};
```

§13.5.6 Uygulamayı Çalıştırma

```
ng serve
```

Tarayıcıda <http://localhost:4200> adresini açtığınızda öğrenci listesini görmeniz gerekir.

13.6 Uygulamayı Bütün Olarak Çalıştırma

1. **Terminal 1 (Back-End):** `cd ogrenci-api-python && source venv/bin/activate && uvicorn main:app --reload --port 8000`
2. **Terminal 2 (Front-End):** `cd ogrenci-arayuz-angular && ng serve → http://localhost:4200`

Bölüm Özeti

- ◇ Aynı Öğrenci Bilgi Sistemini, üçüncü kez, Python'ın sade sözdizimi ile FastAPI çatısı kullanarak inşa ettik.
- ◇ **Pydantic** modelleri, PHP/Node.js'teki elle yazılan doğrulama kontrollerinin aksine, veri doğrulamasını Python'ın tip belirteçlerinden otomatik olarak türetti.
- ◇ **Angular**, TypeScript tabanlı yapısı ve `*ngFor`/`*ngIf`/`[(ngModel)]` direktifleriyle, önceki iki bölümdeki React ve Vue.js'e işlevsel olarak eşdeğer ama söz dizimi olarak farklı bir çözüm sundu.
- ◇ Üç bölüm boyunca aynı problemi (öğrenci listeleme/güncelleme) üç farklı dil ailesinde (JavaScript, C#, Python) ve üç farklı Front-End felsefesinde (React, Vue, Angular) çözerek, RESTful API kavramının teknoloji-den bağımsızlığını somut olarak gördük.

13.7 Düşün ve Çöz

1. FastAPI'deki `Ogrenci(BaseModel)` sınıfı ile bir önceki bölümdeki C#'taki `Ogrenci` sınıfının veri doğrulama açısından benzerliğini ve farkını açıklayınız.
2. <http://localhost:8000/docs> adresindeki otomatik dokümantasyon sayfası, geliştirme sürecinde size ne gibi bir kolaylık sağlar?
3. Bu üç bölümde geliştirdiğimiz üç Back-End'in (Express.js, ASP.NET Core, FastAPI) sunduğu `GET /api/ogrenciler` uç noktalarının HTTP yanıtlarını (durum kodu, `Content-Type` başlığı, gövde yapısı) karşılaştırınız. Ne kadar benzer, ne kadar farklıdır?

13.8 Uygulamalı Projeler

1. **Yeni Öğrenci Ekleme:** FastAPI'de bir `@app.post` uç noktası ve Angular arayüzünde bir ekleme formu oluşturun.
2. **Öğrenci Silme:** `@app.delete` uç noktasını ve arayüze bir silme butonu ekleyin.
3. **Pydantic ile Gelişmiş Doğrulama:** `Ogrenci` modelinde `ogrenci_no` alanının tam olarak 7 haneli bir sayı olmasını zorunlu kılan bir doğrulama kuralı ekleyin.
4. **Üç Sistemi Karşılaştıran Bir Rapor:** Bu üç bölümde geliştirdiğiniz Node.js/React, ASP.NET Core/Vue.js ve FastAPI/Angular sistemlerini; kod satır sayısı, kurulum adımı sayısı ve öğrenme eğrisi açısından karşılaştıran bir tablo hazırlayın.
5. **Docker ile Paketleme:** Bu üç sistemden birini, hem Back-End hem Front-End hem de MySQL'i tek bir `docker-compose.yml` dosyasıyla ayağa kaldıracak şekilde konteynerleştirin.